

Sitekraft - AI-Powered Website Builder

Complete Source Code

Author: **Sneha Chaudhary & Koustubh Kukreti**

Date: **22 April 2026**

Language: **Python 3.12, JavaScript (React 19)**

About this project:

Sitekraft is a web application built by Sneha Chaudhary and Koustubh Kukreti. It takes a business description as input and generates a complete, responsive React website. The backend is written in Python using FastAPI and integrates the Google Gemini API for code generation. Stock images are fetched via the Pexels API and the finished site is deployed to Vercel. The frontend is built with React 19 and Vite.

Technologies used:

- Python 3.12, FastAPI, Uvicorn
- Google Gemini API, Groq API, OpenRouter
- React 19, Vite, Axios
- Pexels API (stock images)
- Vercel (deployment), Render (backend hosting)
- Supabase (database)

Total source files included: 29

Table of Contents

1. Backend - Core AI Agent ? backend/autonomous_agent.py
2. Backend - React Agent Helper ? backend/autonomous_agent_react_helper.py
3. Backend - React Builder ? backend/react_builder.py
4. Backend - React Template Generator ? backend/react_template_generator.py
5. Backend - React API Routes ? backend/react_api_routes.py
6. Backend - Vercel Deployer ? backend/vercel_deployer.py
7. Backend - Professional Copywriter ? backend/professional_copywriter.py
8. Backend - Groq Helper ? backend/groq_helper.py
9. Backend - OpenRouter Helper ? backend/openrouter_helper.py
10. Backend - Pexels Helper ? backend/pexels_helper.py
11. Backend - Pixabay Helper ? backend/pixabay_helper.py
12. Backend - Image Proxy ? backend/image_proxy.py
13. Backend - Supabase Config ? backend/supabase_config.py
14. Backend - Requirements ? backend/requirements.txt
15. Backend - Dockerfile ? backend/Dockerfile
16. Backend - Render Config ? backend/render.yaml
17. Backend - Runtime ? backend/runtime.txt
18. Frontend - main.jsx ? frontend/src/main.jsx
19. Frontend - App.jsx ? frontend/src/App.jsx
20. Frontend - Onboarding.jsx ? frontend/src/Onboarding.jsx
21. Frontend - useTheme.js ? frontend/src/useTheme.js
22. Frontend - App.css ? frontend/src/App.css
23. Frontend - Onboarding.css ? frontend/src/Onboarding.css
24. Frontend - index.css ? frontend/src/index.css
25. Frontend - index.html ? frontend/index.html
26. Frontend - vite.config.js ? frontend/vite.config.js
27. Frontend - eslint.config.js ? frontend/eslint.config.js
28. Frontend - vercel.json ? frontend/vercel.json
29. Frontend - package.json ? frontend/package.json

File 1: backend/autonomous_agent.py

Backend - Core AI Agent

```
1
2     from dotenv import load_dotenv
3     load_dotenv()
4     """
5     AI WEBSITE BUILDER AGENT - Interactive Chat + Full Websites
6     Conversational chatbot that guides users to build production-ready websites
7     with e-commerce, booking, media uploads, and admin panels.
8     """
9
10    import sys
11    import os
12    import json
13    import uuid
14    import random
15    import requests
16
17    # Ensure stdout uses UTF-8 on Windows (avoids cp1252 UnicodeEncodeError for arrow chars in
18    # print)
19    if hasattr(sys.stdout, 'reconfigure'):
20        sys.stdout.reconfigure(encoding='utf-8', errors='replace')
21    import zipfile
22    import io
23    import shutil
24    import base64
25    from datetime import datetime
26    from typing import Dict, Optional, List
27    import google.generativeai as genai
28    from fastapi import FastAPI, HTTPException, UploadFile, File, Form
29    from fastapi.middleware.cors import CORSMiddleware
30    from fastapi.staticfiles import StaticFiles
31    from fastapi.responses import HTMLResponse, FileResponse, StreamingResponse, JSONResponse
32    from pydantic import BaseModel
33    from autonomous_agent_react_helper import build_react_website_from_chat
34    from professional_copywriter import copywriter
35    from pexels_helper import search_pexels_image, search_pexels_images
36
37    # Import React and Vercel modules
38    try:
39        from react_builder import ReactWebsiteBuilder
40        from vercel_deployer import VercelDeployer
41        REACT_VERCEL_AVAILABLE = True
42    except ImportError as e:
43        print(f"[WARNING] React/Vercel modules not available: {e}")
44        REACT_VERCEL_AVAILABLE = False
45
46    # Load environment variables
47    GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
48
49    if not GEMINI_API_KEY:
50        raise ValueError("GEMINI_API_KEY environment variable not set")
```

```
51     genai.configure(api_key=GEMINI_API_KEY)
52
53     # Initialize FastAPI
54     app = FastAPI(title="Sitekraft Agent")
55     app.add_middleware(
56         CORSMiddleware,
57         allow_origins=["*"],
58         allow_credentials=True,
59         allow_methods=["*"],
60         allow_headers=["*"],
61     )
62
63     # Create directories
64     os.makedirs("generated_websites", exist_ok=True)
65     os.makedirs("website_states", exist_ok=True)
66     os.makedirs("static/uploads", exist_ok=True)
67     os.makedirs("site_data", exist_ok=True)
68
69     # Mount static files for uploads (used by chat image uploads)
70     app.mount("/static", StaticFiles(directory="static"), name="static")
71
72     # =====
73     # IN-MEMORY STATE
74     # =====
75     CHAT_SESSIONS = {} # session_id -> conversation state
76     WEBSITE_STATES = {} # session_id -> website state
77
78
79     # =====
80     # REQUEST MODELS
81     # =====
82     class ChatMessage(BaseModel):
83         session_id: str
84         message: str
85         uploaded_files: Optional[List[str]] = []
86
87     class BuildRequest(BaseModel):
88         description: str
89         preferences: Optional[Dict] = None
90
91     class EditRequest(BaseModel):
92         session_id: str
93         edit_request: str
94
95     class BookingRequest(BaseModel):
96         name: str
97         email: str
98         phone: str
99         service: str
100         date: str
101         time: str
102         notes: Optional[str] = ""
103
104     class OrderRequest(BaseModel):
```

```
105     name: str
106     email: str
107     phone: str
108     address: str
109     items: List[Dict]
110     notes: Optional[str] = ""
111
112 class ContactRequest(BaseModel):
113     name: str
114     email: str
115     phone: Optional[str] = ""
116     message: str
117
118
119 # =====
120 # CONVERSATION STATE MACHINE
121 # =====
122 CONVERSATION_STAGES = [
123     "greeting",          # Ask business type
124     "business_details",  # Ask name, location, description
125     "services",          # Ask about services/products
126     "media",             # Ask for photos/videos
127     "features",          # Ask what features they need
128     "style",             # Ask about colors/vibe
129     "building",          # AI is building the website
130     "review",            # Website built, user can edit via chat
131 ]
132
133 BUSINESS_TYPE_QUESTIONS = {
134     "restaurant": {
135         "services": "What type of cuisine do you serve? List your signature dishes or menu categories.",
136         "media": "Do you have photos of your restaurant or dishes? Share them and I'll add them to your website! If not, I'll find beautiful stock photos.",
137         "features": "Which features do you need?",
138         "features_options": ["Online Menu", "Table Booking", "Online Ordering", "Delivery Tracking", "Reviews Section", "Photo Gallery"],
139     },
140     "salon": {
141         "services": "What services do you offer? (e.g., Haircuts, Coloring, Manicure, Facial, etc.)",
142         "media": "Do you have photos of your salon or your work? Share them! If not, I'll use professional stock images.",
143         "features": "Which features do you need?",
144         "features_options": ["Online Booking", "Service Menu with Prices", "Before/After Gallery", "Reviews", "Gift Cards"],
145     },
146     "gym": {
147         "services": "What fitness programs do you offer? (e.g., HIIT, Yoga, CrossFit, Personal Training)",
148         "media": "Do you have photos of your gym or training sessions? Share them!",
149         "features": "Which features do you need?",
150         "features_options": ["Membership Plans", "Class Schedule", "Online Booking", "Trainer Profiles", "Transformation Gallery"],
```

```
151     },
152     "shop": {
153         "services": "What products do you sell? List your main categories.",
154         "media": "Do you have product photos? Share them and I'll showcase them
beautifully!",
155         "features": "Which features do you need?",
156         "features_options": ["Product Catalog", "Online Ordering", "Shopping Cart",
"Reviews", "Size Guide"],
157     },
158     "clinic": {
159         "services": "What medical services do you provide? (e.g., General Practice, Dental,
Dermatology)",
160         "media": "Do you have photos of your clinic or team? Share them!",
161         "features": "Which features do you need?",
162         "features_options": ["Appointment Booking", "Doctor Profiles", "Service List",
"Patient Reviews", "Insurance Info"],
163     },
164     "cafe": {
165         "services": "What do you serve? (e.g., Specialty Coffee, Pastries, Breakfast,
Lunch)",
166         "media": "Do you have photos of your cafe or menu items? Share them!",
167         "features": "Which features do you need?",
168         "features_options": ["Menu Display", "Online Ordering", "Table Reservation", "Photo
Gallery", "Loyalty Program"],
169     },
170     "default": {
171         "services": "What are your main services or products?",
172         "media": "Do you have any photos or videos of your business? Share them and I'll add
them to your website!",
173         "features": "Which features do you need?",
174         "features_options": ["Contact Form", "Service Catalog", "Online Booking", "Photo
Gallery", "Testimonials", "Blog"],
175     }
176 }
177
178
179 def get_business_questions(business_type):
180     btype = business_type.lower()
181     for key in BUSINESS_TYPE_QUESTIONS:
182         if key in btype:
183             return BUSINESS_TYPE_QUESTIONS[key]
184     return BUSINESS_TYPE_QUESTIONS["default"]
185
186
187 # =====
188 # IMAGE SEARCH ENDPOINT
189 # =====
190
191 @app.get("/api/image-search")
192 async def image_search(query: str, orientation: str = ""):
193     """Search Pexels and return the direct image URL.
194     Direct Pexels URLs work fine in <img> tags and CSS url() on any page (blob, Vercel,
etc.).
195     Returning the URL directly keeps the HTML tiny, makes deployment reliable, and is ~10x
```

```
faster.
196     """
197     pexels_url = search_pexels_image(query, orientation)
198     print(f"[image-search] '{query}' -> {(pexels_url or 'none')[:80]}")
199     return {"url": pexels_url}
200
201
202 # =====
203 # CHAT ENDPOINTS
204 # =====
205
206 @app.post("/api/chat/start")
207 async def start_chat():
208     """Start a new chat session"""
209     session_id = str(uuid.uuid4())
210     CHAT_SESSIONS[session_id] = {
211         "stage": "greeting",
212         "business_type": None,
213         "business_name": None,
214         "location": None,
215         "description": None,
216         "services": [],
217         "uploaded_media": [],
218         "features": [],
219         "style_vibe": None,
220         "conversation_history": [],
221         "website_session_id": None,
222     }
223
224     return {
225         "session_id": session_id,
226         "message": "Hey there! I'm your AI website builder. I'll help you create a stunning,
227 fully functional website for your business.\n\nWhat kind of business do you have?",
228         "quick_replies": ["Restaurant", "Salon / Spa", "Gym / Fitness", "Online Shop",
229 "Clinic / Healthcare", "Cafe / Coffee", "Other"]
230     }
231
232 @app.post("/api/chat")
233 async def chat(request: ChatMessage):
234     """Process a chat message and return AI response"""
235     session_id = request.session_id
236     message = request.message.strip()
237     uploaded_files = request.uploaded_files or []
238
239     if session_id not in CHAT_SESSIONS:
240         # Auto-create session if missing
241         CHAT_SESSIONS[session_id] = {
242             "stage": "greeting",
243             "business_type": None,
244             "business_name": None,
245             "location": None,
246             "description": None,
247             "services": [],
```

```
247         "uploaded_media": [],
248         "features": [],
249         "style_vibe": None,
250         "conversation_history": [],
251         "website_session_id": None,
252     }
253
254     session = CHAT_SESSIONS[session_id]
255     stage = session["stage"]
256
257     # Store uploaded files
258     if uploaded_files:
259         session["uploaded_media"].extend(uploaded_files)
260
261     # Add to conversation history
262     session["conversation_history"].append({"role": "user", "content": message, "files":
uploaded_files})
263
264     response_data = {"session_id": session_id, "preview_ready": False, "website_code": "",
"quick_replies": []}
265
266     try:
267         # ---- GREETING STAGE: Identify business type ----
268         if stage == "greeting":
269             business_type = identify_business_type(message)
270             session["business_type"] = business_type
271             session["stage"] = "business_details"
272
273             response_data["message"] = (
274                 f"Great, a **{business_type}** business! Let me help you build an amazing
website.\n\n"
275                 f"Tell me a bit more:\n"
276                 f"- What's your business name?\n"
277                 f"- Where is it located (city)?\n"
278                 f"- A brief description of what makes it special"
279             )
280
281         # ---- BUSINESS DETAILS STAGE ----
282         elif stage == "business_details":
283             # Use AI to extract business details
284             details = extract_business_details(message, session["business_type"])
285             session["business_name"] = details.get("name", "My Business")
286             session["location"] = details.get("location", "")
287             session["description"] = message
288             session["stage"] = "services"
289
290             questions = get_business_questions(session["business_type"])
291             response_data["message"] = (
292                 f"**{session['business_name']}** sounds wonderful!\n\n"
293                 f"{questions['services']}"
294             )
295
296         # ---- SERVICES STAGE ----
297         elif stage == "services":
```

```
298         services = extract_services(message, session["business_type"])
299         session["services"] = services
300         session["stage"] = "media"
301
302         questions = get_business_questions(session["business_type"])
303         response_data["message"] = (
304             f"Noted! I'll feature these on your website.\n\n"
305             f"{questions['media']}\n\n"
306             f"You can upload photos using the **** button, or type **skip** to use stock
images."
307         )
308         response_data["quick_replies"] = ["Skip - use stock photos", "I'll upload some
photos"]
309
310         # ---- MEDIA STAGE ----
311         elif stage == "media":
312             if uploaded_files:
313                 count = len(session["uploaded_media"])
314                 response_data["message"] = (
315                     f"Got {count} photo(s)! They'll look great on your website.\n\n"
316                     f"Want to add more, or shall we move on to features?"
317                 )
318                 response_data["quick_replies"] = ["Add more photos", "Move on to features"]
319                 # Stay in media stage if they want to add more
320                 if "more" not in message.lower() and "add" not in message.lower():
321                     session["stage"] = "features"
322                     questions = get_business_questions(session["business_type"])
323                     response_data["message"] = (
324                         f"Got {count} photo(s)! They'll look great on your website.\n\n"
325                         f"{questions['features']}"
326                     )
327                     response_data["quick_replies"] = questions["features_options"]
328                     response_data["multi_select"] = True
329             else:
330                 if "skip" in message.lower() or "stock" in message.lower() or "no" in
message.lower() or "don't" in message.lower():
331                     session["stage"] = "features"
332                     questions = get_business_questions(session["business_type"])
333                     response_data["message"] = (
334                         f"No worries! I'll find beautiful professional photos for you.\n\n"
335                         f"{questions['features']}"
336                     )
337                     response_data["quick_replies"] = questions["features_options"]
338                     response_data["multi_select"] = True
339                 elif "upload" in message.lower() or "photo" in message.lower() or "i'll" in
message.lower():
340                     response_data["message"] = "Go ahead! Click the **** button to upload
your photos. You can upload multiple at once."
341                 else:
342                     session["stage"] = "features"
343                     questions = get_business_questions(session["business_type"])
344                     response_data["message"] = (
345                         f"Got it! Moving on.\n\n{questions['features']}"
346                     )
```

```
347         response_data["quick_replies"] = questions["features_options"]
348         response_data["multi_select"] = True
349
350     # ---- FEATURES STAGE ----
351     elif stage == "features":
352         features = extract_features(message, session["business_type"])
353         session["features"] = features
354         session["stage"] = "style"
355
356         response_data["message"] = (
357             f"Great choices! Last question - what style/vibe do you want for your
website?"
358         )
359         response_data["quick_replies"] = ["Modern & Clean", "Luxury & Elegant", "Warm &
Cozy", "Bold & Energetic", "Minimalist", "Let AI decide"]
360
361     # ---- STYLE STAGE ----
362     elif stage == "style":
363         session["style_vibe"] = message.lower().replace("let ai decide", "modern")
364         session["stage"] = "building"
365
366         response_data["message"] = (
367             f"Perfect! I have everything I need. Let me build your website now...\n\n"
368             f"Building your {session['business_type']} website for
{session['business_name']}*\n\n"
369             f"Hold on while I create something amazing..."
370         )
371
372     # Actually build the React website
373     try:
374         import uuid
375         session["website_session_id"] = str(uuid.uuid4())[:8]
376         build_result = await build_react_website_from_chat(session)
377         website_session_id = build_result["session_id"]
378         session["website_session_id"] = website_session_id
379         session["stage"] = "review"
380
381         response_data["session_id"] = website_session_id
382         response_data["preview_ready"] = True
383         response_data["website_code"] = build_result["html"]
384
385         # Get Vercel URL from build result
386         vercel_url = build_result.get("vercel_url")
387         if vercel_url:
388             session["vercel_url"] = vercel_url
389             print(f"[OK] Deployed to Vercel: {vercel_url}")
390
391         message_text = f"\n\nYour website is ready! Check the preview on the
right.\n\n"
392         if vercel_url:
393             message_text += f"Live Link: {vercel_url}\n(Your website is live and
shareable!)\n\n"
394         message_text += "Want to make any changes? Just tell me what you'd like to
modify - colors, text, layout, anything!"
```

```
395
396         response_data["message"] += message_text
397         response_data["vercel_url"] = vercel_url
398         response_data["quick_replies"] = ["Change colors", "Edit text", "Add more
sections", "Looks perfect!", "Download it"]
399
400     except Exception as e:
401         print(f"Build error: {e}")
402         session["stage"] = "style"
403         response_data["message"] = f"Sorry, I hit an issue building your website. Let
me try again. Error: {str(e)[:100]}"
404
405     # ---- REVIEW/EDIT STAGE ----
406     elif stage == "review":
407         if "download" in message.lower():
408             response_data["message"] = "You can download your website using the
**Download** button in the preview panel!"
409             response_data["quick_replies"] = ["Change colors", "Edit text", "Add more
sections", "Start over"]
410             elif "perfect" in message.lower() or "great" in message.lower() or "love it" in
message.lower() or "looks good" in message.lower():
411                 response_data["message"] = (
412                     "Awesome! Your website is ready to go.\n\n"
413                     "You can:\n"
414                     "- **Download** it using the button in the preview panel\n"
415                     "- **Open in new tab** to see the full version\n"
416                     "- Keep chatting with me to make more changes\n\n"
417                     "Need anything else?"
418                 )
419                 response_data["quick_replies"] = ["Make more changes", "Download", "Start a
new website"]
420             elif "start over" in message.lower() or "new website" in message.lower():
421                 # Reset session
422                 new_session = await start_chat()
423                 return new_session
424             else:
425                 # Apply edit via AI
426                 try:
427                     edit_result = edit_website_from_chat(session, message)
428                     session["website_session_id"] = edit_result["session_id"]
429
430                     response_data["session_id"] = edit_result["session_id"]
431                     response_data["preview_ready"] = True
432                     response_data["website_code"] = edit_result["html"]
433                     response_data["message"] = (
434                         f"Done! I've applied your changes. Check the updated preview.\n\n"
435                         f"Want to change anything else?"
436                     )
437                     response_data["quick_replies"] = ["Change colors", "Edit text", "Add
sections", "Looks perfect!", "Download"]
438
439                 except Exception as e:
440                     print(f"Edit error: {e}")
441                     response_data["message"] = f"Sorry, I couldn't apply that change. Could
```

```
        you try rephrasing? Error: {str(e)[:100]}"
442
443         # ---- BUILDING STAGE (shouldn't normally reach here) ----
444         elif stage == "building":
445             response_data["message"] = "I'm still building your website. Please wait a
moment..."
446
447         except Exception as e:
448             print(f"Chat error: {e}")
449             response_data["message"] = "Sorry, something went wrong. Could you try again?"
450
451         # Store AI response in history
452         session["conversation_history"].append({"role": "assistant", "content":
response_data["message"]})
453
454         return response_data
455
456
457     # =====
458     # AI HELPER FUNCTIONS
459     # =====
460
461     def identify_business_type(message):
462         """Identify business type from user message"""
463         msg = message.lower()
464         type_map = {
465             "restaurant": ["restaurant", "food", "dining", "eatery", "bistro", "diner"],
466             "salon": ["salon", "spa", "beauty", "hair", "nail", "barber"],
467             "gym": ["gym", "fitness", "workout", "training", "crossfit", "yoga"],
468             "shop": ["shop", "store", "ecommerce", "e-commerce", "retail", "boutique",
"clothing", "fashion"],
469             "clinic": ["clinic", "hospital", "doctor", "medical", "dental", "healthcare",
"health"],
470             "cafe": ["cafe", "coffee", "bakery", "tea", "pastry"],
471         }
472         for btype, keywords in type_map.items():
473             if any(kw in msg for kw in keywords):
474                 return btype
475
476         # Use AI for ambiguous cases
477         try:
478             model = genai.GenerativeModel('gemini-2.5-flash')
479             resp = model.generate_content(
480                 f'What type of business is this? "{message}". '
481                 f'Reply with ONLY one word from: restaurant, salon, gym, shop, clinic, cafe,
business'
482             )
483             return resp.text.strip().lower().replace('.', '')
484         except:
485             return "business"
486
487
488     def extract_business_details(message, business_type):
489         """Extract business name and location from message"""
```

```
490     try:
491         model = genai.GenerativeModel('gemini-2.5-flash')
492         resp = model.generate_content(
493             f'Extract the business name and city/location from this text: "{message}". '
494             f'The business type is: {business_type}. '
495             f'Return ONLY valid JSON with no extra text: {{"name": "exact business name from
text", "location": "city, country"}}'
496         )
497         text = resp.text.strip()
498         json_start = text.find('{')
499         json_end = text.rfind('}') + 1
500         if json_start >= 0:
501             result = json.loads(text[json_start:json_end])
502             print(f"Extracted details: {result}")
503             return result
504     except Exception as e:
505         print(f"Extract details error: {e}")
506
507     # Fallback: try to find name in the message itself
508     msg_lower = message.lower()
509     name = "My Business"
510     # Simple heuristic: look for "name is X" or "called X"
511     for pattern in ["name is ", "name's ", "called ", "named "]:
512         if pattern in msg_lower:
513             idx = msg_lower.index(pattern) + len(pattern)
514             rest = message[idx:].strip()
515             # Take until next punctuation or common word
516             for end_word in [" i ", " we ", " in ", " at ", " located", " based", ",", ".", ":"]:
517                 if end_word in rest.lower():
518                     name = rest[:rest.lower().index(end_word)].strip()
519                     break
520             else:
521                 name = rest.split()[0] if rest else "My Business"
522             break
523     return {"name": name, "location": ""}
524
525
526 def extract_services(message, business_type):
527     """Extract services/products from message"""
528     try:
529         model = genai.GenerativeModel('gemini-2.5-flash')
530         resp = model.generate_content(
531             f'Extract the specific product categories or services mentioned in this text:
"{message}". '
532             f'Business type: {business_type}. '
533             f'Return ONLY a JSON array of the items mentioned, keep the exact names: ["Item
1", "Item 2", ...]'
534         )
535         text = resp.text.strip()
536         json_start = text.find('[')
537         json_end = text.rfind(']') + 1
538         if json_start >= 0:
539             result = json.loads(text[json_start:json_end])
540             if result and len(result) > 0:
```

```
541         print(f"Extracted services: {result}")
542         return result
543     except Exception as e:
544         print(f"Extract services error: {e}")
545
546     # Fallback: split by commas and common separators
547     parts = []
548     for sep in [",", " - ", "- ", " and ", "\n", "/"]:
549         if sep in message:
550             parts = [p.strip() for p in message.split(sep) if p.strip() and len(p.strip()) >
1]
551             break
552     if parts:
553         return parts[:8]
554     return [message.strip()] if message.strip() else ["Products"]
555
556
557 def extract_features(message, business_type):
558     """Extract requested features from message - captures ALL selected features"""
559     features = []
560     msg = message.lower()
561
562     # Comprehensive feature mapping - maps user selections to internal feature names
563     feature_mappings = {
564         # Booking related
565         "booking": ["booking", "appointment", "schedule", "reserve", "online booking", "table
booking"],
566
567         # Ordering/Shopping related
568         "ordering": ["order", "cart", "buy", "purchase", "delivery", "online ordering",
"shopping cart"],
569
570         # Menu/Catalog related
571         "menu": ["menu", "catalog", "product", "items", "service menu", "product catalog",
"class schedule"],
572
573         # Gallery related
574         "gallery": ["gallery", "photos", "portfolio", "before/after", "photo gallery",
"transformation gallery"],
575
576         # Reviews related
577         "reviews": ["review", "testimonial", "feedback", "patient reviews"],
578
579         # Contact related
580         "contact": ["contact", "form", "inquiry", "enquiry", "contact form"],
581
582         # Blog/Content related
583         "blog": ["blog", "news", "articles"],
584
585         # Membership/Plans
586         "membership": ["membership", "plans", "pricing", "packages", "membership plans"],
587
588         # Profiles
589         "profiles": ["profiles", "team", "trainers", "doctors", "trainer profiles", "doctor
```

```
        profiles"],
590
591        # Tracking
592        "tracking": ["tracking", "delivery tracking", "order tracking"],
593
594        # Size guide
595        "size_guide": ["size", "sizing", "size guide", "size chart"],
596
597        # Gift cards
598        "gift_cards": ["gift", "cards", "gift cards", "vouchers"],
599
600        # Loyalty
601        "loyalty": ["loyalty", "rewards", "loyalty program"],
602
603        # Insurance
604        "insurance": ["insurance", "insurance info"],
605    }
606
607    # Check each feature mapping
608    for feature, keywords in feature_mappings.items():
609        if any(kw in msg for kw in keywords):
610            if feature not in features: # Avoid duplicates
611                features.append(feature)
612
613    # If still no features found, use defaults
614    if not features:
615        defaults = {
616            "restaurant": ["menu", "ordering", "reviews", "gallery", "contact"],
617            "salon": ["booking", "gallery", "reviews", "contact"],
618            "gym": ["booking", "membership", "gallery", "reviews", "contact"],
619            "shop": ["menu", "ordering", "reviews", "gallery", "contact"],
620            "store": ["menu", "ordering", "reviews", "gallery", "contact"],
621            "ecommerce": ["menu", "ordering", "reviews", "gallery", "contact"],
622            "boutique": ["menu", "ordering", "reviews", "gallery", "contact"],
623            "clinic": ["booking", "profiles", "reviews", "contact"],
624            "cafe": ["menu", "ordering", "gallery", "reviews", "contact"],
625        }
626        features = defaults.get(business_type, ["contact", "gallery", "reviews"])
627
628    print(f"[Features] Extracted from '{message}': {features}")
629    return features
630
631
632    # =====
633    # PEXELS IMAGE FETCHING
634    # =====
635
636    def fetch_pexels_images(query, count=1):
637        """Fetch images from Pexels API"""
638        urls = search_pexels_images(query, count=count, orientation="landscape")
639        if urls:
640            return urls
641        # picsum fallback if Pexels fails
642        print(f"[Pexels] Using fallback images for '{query}'")
```

```
643         return [f"https://picsum.photos/seed/{query.replace(' ', '')}/1920/1080" for _ in
range(count)]
644
645
646 def get_images_for_website(session):
647     """Get images - use uploaded ones first, fill with Pexels (all fetches run in
parallel)"""
648     from concurrent.futures import ThreadPoolExecutor, as_completed
649
650     uploaded = session.get("uploaded_media", [])
651     business_type = session.get("business_type", "business")
652     business_name = session.get("business_name", "")
653     services = session.get("services", [])
654
655     images = {"hero": "", "about": "", "services": [], "gallery": [], "products": {}}
656
657     # Uploaded images become gallery/service images
658     uploaded_urls = [f"{u}" if u.startswith("http") else u for u in uploaded]
659
660     # Specific image queries for better results across different business types
661     service_queries = {
662         # E-commerce / Fashion - Western
663         "jeans": "women jeans denim fashion stylish",
664         "tops": "women tops blouse fashion trendy",
665         "top": "women top blouse fashion casual",
666         "dresses": "women dress fashion elegant",
667         "dress": "women dress fashion elegant",
668         "skorts": "women skort fashion active trendy",
669         "skort": "women skort fashion active trendy",
670         "skirts": "women skirt fashion midi maxi",
671         "skirt": "women skirt fashion midi maxi",
672         "bell bottom": "women bell bottom pants flare jeans retro",
673         "bell-bottom": "women bell bottom pants flare jeans retro",
674         "palazzo": "women palazzo pants wide leg fashion",
675         "trousers": "women trousers formal pants fashion",
676         "shorts": "women shorts casual fashion summer",
677         "co-ord": "women co-ord set matching outfit",
678         "coord": "women co-ord set matching outfit",
679         "handbags": "women handbag purse fashion luxury",
680         "accessories": "women fashion accessories jewelry",
681         "shoes": "women shoes heels fashion footwear",
682         "jackets": "women jacket fashion outerwear blazer",
683         "jacket": "women jacket fashion outerwear",
684         "sweaters": "women sweater knitwear fashion cozy",
685         "sweater": "women sweater knitwear fashion",
686         "activewear": "women activewear gym fitness sportswear",
687         "lingerie": "women lingerie innerwear",
688         "swimwear": "women swimwear beach fashion",
689
690         # E-commerce / Fashion - Indian Ethnic
691         "kurta": "women kurta kurti ethnic indian fashion",
692         "kurti": "women kurta kurti ethnic indian fashion",
693         "saree": "women saree sari indian ethnic fashion",
694         "sarree": "women saree sari indian ethnic fashion",
```

```
695     "sari": "women saree sari indian ethnic fashion",
696     "lehenga": "women lehenga bridal indian wedding fashion",
697     "lehnga": "women lehenga bridal indian wedding fashion",
698     "suits": "women salwar suit punjabi ethnic indian",
699     "suit": "women salwar suit punjabi ethnic indian",
700     "salwar": "women salwar kameez suit indian ethnic",
701     "anarkali": "women anarkali suit indian ethnic fashion",
702     "indo-western": "women indo western fusion indian fashion",
703     "indo western": "women indo western fusion indian fashion",
704     "dupatta": "women dupatta scarf indian ethnic",
705     "churidar": "women churidar pants indian ethnic",
706     "palazzo suit": "women palazzo suit indian ethnic fusion",
707
708     # Salon / Spa / Beauty
709     "facial": "woman facial spa treatment skincare glowing",
710     "massage": "spa massage therapy relaxation wellness luxury",
711     "body massage": "spa body massage therapy wellness relax",
712     "hair styling": "hairstylist salon hair beauty styling",
713     "hair cut": "hairstylist salon haircut styling modern",
714     "haircut": "hairstylist salon haircut styling",
715     "bridal makeup": "bridal makeup wedding beauty bride",
716     "makeup": "makeup artist beauty cosmetics professional",
717     "manicure": "manicure nails beauty salon polish",
718     "pedicure": "pedicure feet spa treatment nails",
719     "waxing": "beauty salon spa waxing treatment",
720     "hair color": "hair coloring salon hairstylist dye",
721     "spa": "luxury spa wellness relaxation massage",
722     "beauty": "beauty salon treatment skincare cosmetics",
723     "skincare": "skincare facial beauty treatment serum",
724     "micro facial": "facial skincare microdermabrasion beauty spa",
725     "threading": "eyebrow threading beauty salon",
726     "bleach": "facial bleach beauty salon treatment",
727     "cleanup": "facial cleanup beauty salon skincare",
728     "tan removal": "tan removal beauty treatment skincare",
729
730     # Fitness / Gym
731     "hiit": "HIIT workout fitness gym training intense",
732     "yoga": "yoga meditation fitness wellness peaceful",
733     "crossfit": "crossfit gym workout training athletes",
734     "personal training": "personal trainer gym fitness coaching",
735     "gym": "modern fitness gym workout equipment",
736     "fitness": "fitness training workout gym active",
737     "cardio": "cardio workout fitness gym running",
738     "strength": "strength training weights gym muscle",
739     "strength training": "strength training weights gym dumbbells",
740     "pilates": "pilates workout fitness studio mat",
741     "zumba": "zumba dance fitness class energetic",
742     "aerobics": "aerobics fitness class workout group",
743     "spinning": "spinning bike cycling fitness gym",
744     "weight training": "weight training gym dumbbells fitness",
745     "bodybuilding": "bodybuilding gym muscles fitness",
746     "martial arts": "martial arts karate fitness training",
747     "boxing": "boxing gym fitness training punching",
748     "dance": "dance fitness class studio energetic",
```

```
749
750     # Restaurant / Cafe / Food
751     "pizza": "pizza italian food restaurant delicious",
752     "pasta": "pasta italian cuisine food restaurant",
753     "burger": "burger food restaurant gourmet delicious",
754     "sushi": "sushi japanese food restaurant fresh",
755     "dessert": "dessert pastry cake bakery sweet",
756     "coffee": "coffee cafe espresso latte barista",
757     "salad": "fresh salad healthy food bowl colorful",
758     "steak": "steak meat grill restaurant juicy",
759     "seafood": "seafood fish restaurant cuisine fresh",
760     "wine": "wine bottle glass restaurant elegant",
761     "cocktail": "cocktail drink bar mixology colorful",
762     "breakfast": "breakfast food cafe morning pancakes",
763     "lunch": "lunch food restaurant meal delicious",
764     "dinner": "dinner food restaurant meal elegant",
765     "brunch": "brunch food cafe breakfast lunch",
766     "appetizers": "appetizers food restaurant starter",
767     "sandwiches": "sandwich food cafe fresh deli",
768     "wraps": "wraps food healthy lunch fresh",
769     "smoothies": "smoothie bowl healthy fruit fresh",
770     "juice": "fresh juice fruit healthy colorful",
771     "ice cream": "ice cream dessert sweet colorful",
772     "donuts": "donuts pastry bakery sweet glazed",
773     "cupcakes": "cupcakes bakery sweet frosting colorful",
774     "cookies": "cookies bakery sweet homemade",
775     "waffles": "waffles breakfast cafe sweet syrup",
776     "crepes": "crepes french thin pancake sweet cafe",
777     "pancakes": "pancakes breakfast fluffy syrup stack",
778     "french fries": "french fries crispy fast food snack",
779     "fries": "french fries crispy fast food snack",
780     "nachos": "nachos cheese snack mexican food",
781     "noodles": "noodles bowl asian food restaurant",
782     "fried rice": "fried rice asian food restaurant wok",
783     "rice": "rice dish food restaurant bowl",
784     "soup": "soup bowl hot food restaurant",
785     "chicken": "chicken dish grilled food restaurant",
786     "mutton": "mutton curry dish food restaurant",
787     "lamb": "lamb chops grilled food restaurant",
788     "fish": "fish fillet grilled food restaurant",
789     "prawns": "prawns shrimp seafood dish food",
790     "tacos": "tacos mexican food restaurant fresh",
791     "sandwich": "sandwich food cafe fresh deli",
792     "pasta salad": "pasta salad food fresh healthy",
793     "milkshake": "milkshake creamy thick drink sweet",
794     "mocktail": "mocktail colorful drink fresh fruit",
795     "lassi": "lassi yogurt drink indian refreshing",
796     "tea": "tea cup hot drink cafe",
797     "chai": "masala chai tea indian hot drink",
798
799     # Indian Food
800     "biryani": "biryani rice aromatic indian food",
801     "curry": "curry indian food spicy bowl",
802     "dal": "dal lentil soup indian food",
```

```
803     "dal makhani": "dal makhani indian food creamy",
804     "naan": "naan bread indian restaurant flatbread",
805     "roti": "roti chapati indian bread food",
806     "chapati": "chapati roti indian bread food",
807     "tandoori": "tandoori chicken indian food grill",
808     "tikka": "chicken tikka indian food grill spicy",
809     "tikka masala": "chicken tikka masala indian curry food",
810     "butter chicken": "butter chicken makhani indian curry",
811     "paneer": "paneer tikka indian food restaurant",
812     "dosa": "dosa south indian food crispy",
813     "idli": "idli sambar south indian food",
814     "samosa": "samosa indian snack fried crispy",
815     "chaat": "chaat indian street food snack",
816     "pav bhaji": "pav bhaji indian street food",
817     "chole": "chole bhature indian food chickpea",
818     "rajma": "rajma chawal indian food kidney beans",
819     "gulab jamun": "gulab jamun indian sweet dessert",
820     "rasgulla": "rasgulla indian sweet dessert syrup",
821     "kheer": "kheer rice pudding indian dessert",
822     "halwa": "halwa indian sweet dessert",
823     "kulfi": "kulfi indian ice cream dessert",
824     "ladoo": "ladoo indian sweet dessert ball",
825     "barfi": "barfi indian sweet dessert milk",
826     "jalebi": "jalebi indian sweet dessert crispy",
827
828     # Bakery / Pastry
829     "bread": "fresh bread bakery artisan loaves",
830     "croissant": "croissant pastry bakery french flaky",
831     "croissants": "croissants pastry bakery french",
832     "pastries": "pastries bakery sweet danish croissant",
833     "pastry": "pastry bakery sweet delicate",
834     "cakes": "cakes bakery celebration birthday wedding",
835     "cake": "cake bakery celebration frosting layers",
836     "cheesecake": "cheesecake dessert creamy slice",
837     "cheese cake": "cheesecake dessert creamy slice",
838     "brownies": "brownies chocolate bakery fudge",
839     "muffins": "muffins bakery breakfast blueberry",
840     "tarts": "tarts pastry bakery fruit elegant",
841     "pies": "pies bakery fruit dessert homemade",
842     "macarons": "macarons pastry bakery french colorful",
843     "eclairs": "eclairs pastry bakery french chocolate",
844     "baguette": "baguette bread bakery french artisan",
845     "custom cakes": "custom cake bakery celebration decorated",
846     "wedding cakes": "wedding cake bakery elegant tiered",
847     "birthday cakes": "birthday cake bakery celebration colorful",
848
849     # Salon / Spa extras
850     "nail extensions": "nail extensions acrylic nails beauty salon",
851     "gel nails": "gel nails manicure beauty salon",
852     "nail art": "nail art design beauty salon colorful",
853     "eyelash extensions": "eyelash extensions beauty salon glam",
854     "keratin treatment": "keratin hair treatment salon smooth",
855     "hair treatment": "hair treatment salon spa deep conditioning",
856     "hair spa": "hair spa treatment salon nourishing",
```

```
857     "balayage": "balayage hair color salon highlights",
858     "highlights": "hair highlights color salon styling",
859     "eyebrow shaping": "eyebrow shaping beauty salon grooming",
860     "pre-bridal": "pre-bridal package beauty salon bride",
861     "bridal package": "bridal makeup package salon bride",
862     "detan": "tan removal detan beauty treatment face",
863
864     # Fitness extras
865     "swimming": "swimming pool fitness workout laps",
866     "kickboxing": "kickboxing martial arts fitness training",
867     "meditation": "meditation yoga peaceful mindfulness wellness",
868     "stretching": "stretching flexibility yoga fitness",
869     "nutrition": "nutrition healthy food diet wellness",
870     "nutrition counseling": "nutrition counseling diet healthy food",
871     "functional training": "functional training workout fitness gym",
872     "group fitness": "group fitness class gym workout energetic",
873     "cycling": "cycling bike fitness workout gym",
874
875     # Clinic / Healthcare
876     "physiotherapy": "physiotherapy physical therapy rehabilitation exercise",
877     "dermatology": "dermatology skin treatment clinic doctor",
878     "cardiology": "cardiology heart care clinic doctor",
879     "pediatrics": "pediatrics children health clinic doctor",
880     "gynecology": "gynecology women health clinic doctor",
881     "orthopedics": "orthopedics bone joint clinic doctor",
882     "ophthalmology": "ophthalmology eye care clinic doctor",
883     "ayurveda": "ayurveda herbal medicine treatment wellness",
884     "homeopathy": "homeopathy medicine treatment clinic",
885     "dentist": "dentist dental clinic teeth care professional",
886     "doctor": "doctor clinic healthcare professional stethoscope",
887     "general practice": "doctor general practice clinic healthcare",
888     "blood test": "blood test laboratory clinic healthcare",
889     "x-ray": "x-ray radiology clinic medical imaging",
890     "ultrasound": "ultrasound scan clinic medical",
891
892     # Men's Fashion / Clothing
893     "shirts": "men shirts fashion formal casual",
894     "shirt": "men shirt fashion formal casual",
895     "t-shirts": "men t-shirt casual fashion streetwear",
896     "t-shirt": "men t-shirt casual fashion streetwear",
897     "men jeans": "men jeans denim fashion casual",
898     "men kurta": "men kurta ethnic indian fashion",
899     "sherwani": "men sherwani wedding indian ethnic fashion",
900     "blazer": "men blazer suit formal fashion",
901     "formal wear": "men formal wear suit professional",
902     "ethnic wear": "men ethnic wear indian fashion kurta",
903
904     # Jewelry / Accessories
905     "necklace": "necklace jewelry fashion gold silver",
906     "earrings": "earrings jewelry fashion gold silver",
907     "bangles": "bangles jewelry indian fashion gold",
908     "rings": "rings jewelry fashion gold diamond",
909     "bracelet": "bracelet jewelry fashion wrist",
910     "anklet": "anklet jewelry fashion gold",
```

```
911     "jewelry": "jewelry fashion gold silver elegant",
912     "watch": "watch wristwatch fashion luxury timepiece",
913     "sunglasses": "sunglasses eyewear fashion trendy",
914     "bag": "handbag purse fashion women leather",
915     "clutch": "clutch bag fashion women evening",
916     "wallet": "wallet leather fashion men women",
917
918     # Footwear
919     "heels": "women heels shoes fashion elegant",
920     "sandals": "sandals shoes fashion summer casual",
921     "boots": "boots shoes fashion leather",
922     "sneakers": "sneakers shoes fashion casual sporty",
923     "flats": "flat shoes women fashion casual",
924     "footwear": "footwear shoes fashion collection",
925
926     # Real Estate
927     "apartment": "apartment interior modern living room",
928     "villa": "villa luxury house exterior garden",
929     "house": "house exterior modern architecture",
930     "property": "property real estate modern building",
931     "plot": "land plot real estate",
932     "commercial space": "commercial office space interior modern",
933     "office space": "office space interior modern professional",
934     "flat": "apartment flat interior modern living",
935
936     # Education / Coaching
937     "coaching": "coaching education class students learning",
938     "online coaching": "online coaching education laptop learning",
939     "courses": "courses education learning books classroom",
940     "tuition": "tuition education students classroom learning",
941     "workshop": "workshop training education professional",
942     "classes": "classes education students learning",
943     "python": "python programming code laptop developer",
944     "java": "java programming software developer laptop",
945     "web development": "web development coding laptop developer",
946     "data science": "data science analytics laptop charts",
947     "machine learning": "machine learning artificial intelligence technology",
948     "ielts": "ielts english test preparation study books",
949     "spoken english": "english speaking class students learning",
950     "public speaking": "public speaking presentation confident stage",
951     "personality development": "personality development coaching professional confident",
952     "interview preparation": "job interview preparation coaching professional",
953     "soft skills": "soft skills training workshop professional team",
954     "leadership": "leadership training workshop professional business",
955     "career counseling": "career counseling guidance professional mentor",
956     "aptitude training": "aptitude test preparation study mathematics",
957     "gate coaching": "engineering gate exam preparation study",
958     "neet coaching": "medical neet exam preparation study books",
959     "jee coaching": "engineering jee exam preparation study",
960     "mba coaching": "mba business school preparation study professional",
961
962     # General Services
963     "consulting": "business consulting professional meeting office",
964     "photography": "photography camera professional portrait studio",
```

```
965         "videography": "videography video camera production",
966         "graphic design": "graphic design creative work computer",
967         "web design": "web design website development computer",
968         "marketing": "digital marketing strategy business creative",
969         "accounting": "accounting finance calculator business",
970         "legal": "legal law office lawyer professional",
971         "tutoring": "tutoring education student learning books",
972         "music lessons": "music lessons guitar piano teaching",
973         "art classes": "art class painting creative studio",
974         "pet grooming": "pet grooming dog salon care",
975         "veterinary": "veterinary clinic pet dog cat care",
976         "dental": "dental clinic dentist teeth care",
977         "medical": "medical clinic doctor healthcare professional",
978         "cleaning": "house cleaning service professional home",
979         "plumbing": "plumber plumbing repair tools professional",
980         "electrical": "electrician electrical work tools professional",
981         "gardening": "gardening landscaping plants flowers garden",
982         "catering": "catering food service event professional",
983         "event planning": "event planning wedding party decoration",
984         "interior design": "interior design home decor modern elegant",
985         "architecture": "architecture building design modern exterior",
986         "real estate": "real estate property house modern",
987         "bungalow": "bungalow house exterior architecture garden",
988         "penthouse": "luxury penthouse interior view modern",
989         "studio apartment": "studio apartment modern interior compact",
990         "warehouse": "industrial warehouse space interior large",
991         "shop space": "retail shop space interior modern",
992         "property management": "property management real estate professional",
993         "home loan": "home loan mortgage financial planning professional",
994         "travel": "travel destination landscape beautiful scenic",
995         "hotel": "hotel luxury room interior elegant",
996         "car rental": "car rental vehicle luxury automobile",
997         "car wash": "car wash cleaning service vehicle",
998         "repair": "repair service tools professional workshop",
999         "printing": "printing service design professional",
1000        "tailoring": "tailoring sewing clothes fashion custom",
1001        "pharmacy": "pharmacy medicine healthcare professional",
1002        "optician": "optician eyewear glasses vision care",
1003    }
1004
1005    def get_smart_query(service, btype):
1006        """Get a specific image search query based on service/product category"""
1007        svc_lower = service.lower()
1008
1009        # Try to find a specific match first
1010        for key, query in service_queries.items():
1011            if key in svc_lower:
1012                return query
1013
1014        # Business type specific fallbacks
1015        if btype in ["shop", "store", "ecommerce", "boutique"]:
1016            return f"women {service} fashion clothing"
1017        elif btype in ["salon", "spa", "beauty"]:
1018            return f"{service} beauty spa treatment"
```

```
1019     elif btype in ["gym", "fitness", "studio"]:
1020         return f"{service} fitness workout gym"
1021     elif btype in ["restaurant", "cafe", "food"]:
1022         return f"{service} food restaurant cuisine"
1023
1024     # Generic fallback
1025     return f"{service} professional high quality"
1026
1027 fallback = "https://picsum.photos/seed/business/1920/1080"
1028
1029 hero_queries = {
1030     "shop": "fashion model women clothing store",
1031     "store": "fashion model women clothing store",
1032     "ecommerce": "fashion model women clothing store",
1033     "boutique": "fashion model women clothing store",
1034     "salon": "luxury salon interior beauty spa",
1035     "spa": "luxury spa wellness relaxation interior",
1036     "beauty": "beauty salon interior elegant",
1037     "gym": "modern gym fitness equipment interior",
1038     "fitness": "fitness studio workout space",
1039     "studio": "fitness yoga studio interior",
1040     "restaurant": "elegant restaurant interior dining",
1041     "cafe": "modern cafe interior coffee shop",
1042     "food": "restaurant food interior ambiance",
1043 }
1044 about_queries = {
1045     "shop": "fashion designer workspace boutique",
1046     "store": "fashion designer workspace boutique",
1047     "ecommerce": "fashion designer workspace boutique",
1048     "boutique": "fashion designer workspace boutique",
1049     "salon": "beauty salon professional team staff",
1050     "spa": "spa therapist massage wellness professional",
1051     "beauty": "beauty professional makeup artist",
1052     "gym": "fitness trainer gym coach professional",
1053     "fitness": "personal trainer fitness professional",
1054     "studio": "fitness instructor yoga teacher",
1055     "restaurant": "chef kitchen restaurant professional",
1056     "cafe": "barista coffee cafe professional",
1057     "food": "chef cooking restaurant kitchen",
1058 }
1059 gallery_queries = {
1060     "shop": "fashion lookbook editorial style",
1061     "store": "fashion lookbook editorial style",
1062     "ecommerce": "fashion lookbook editorial style",
1063     "boutique": "fashion lookbook editorial style",
1064     "salon": "beauty salon hair makeup before after",
1065     "spa": "spa wellness relaxation massage treatment",
1066     "beauty": "beauty makeup skincare treatment",
1067     "gym": "fitness workout gym training results",
1068     "fitness": "fitness training workout motivation",
1069     "studio": "fitness yoga studio class workout",
1070     "restaurant": "restaurant food plating delicious",
1071     "cafe": "cafe coffee latte food dessert",
1072     "food": "food plating restaurant delicious meal",
```

```
1073     }
1074
1075     is_ecommerce = business_type in ["shop", "store", "ecommerce", "boutique"]
1076
1077     # Build the list of Pexels tasks (only for slots not covered by uploads)
1078     # Each task: (task_key, query, count)
1079     tasks = []
1080     if len(uploaded_urls) < 1:
1081         tasks.append(("hero", hero_queries.get(business_type.lower(), f"{business_type}
interior modern professional"), 1))
1082     if len(uploaded_urls) < 2:
1083         tasks.append(("about", about_queries.get(business_type.lower(), f"{business_type}
professional team workspace"), 1))
1084
1085     for i, service in enumerate(services[:8]):
1086         if i + 2 >= len(uploaded_urls):
1087             query = get_smart_query(service, business_type)
1088             # For ecommerce, fetch 5 images per service: [0] for card, [1:5] for product grid
1089             count = 5 if (is_ecommerce and i < 6) else 1
1090             tasks.append((f"service_{i}", query, count))
1091
1092     remaining_uploads = uploaded_urls[2 + len(services):]
1093     if not remaining_uploads:
1094         gallery_query = gallery_queries.get(business_type.lower(), f"{business_type} gallery
showcase")
1095         tasks.append(("gallery", gallery_query, 3))
1096
1097     # Run ALL Pexels fetches in parallel
1098     results = {}
1099     if tasks:
1100         print(f"[Images] Fetching {len(tasks)} image sets in parallel...")
1101         with ThreadPoolExecutor(max_workers=min(len(tasks), 12)) as executor:
1102             future_to_key = {
1103                 executor.submit(fetch_pexels_images, query, count): key
1104                 for key, query, count in tasks
1105             }
1106             for future in as_completed(future_to_key):
1107                 key = future_to_key[future]
1108                 try:
1109                     results[key] = future.result(timeout=15)
1110                 except Exception as e:
1111                     print(f"[Images] Failed for {key}: {e}")
1112                     results[key] = []
1113             print(f"[Images] All parallel fetches done.")
1114
1115     # Assign results
1116     images["hero"] = uploaded_urls[0] if uploaded_urls else (results.get("hero") or
[fallback])[0]
1117     images["about"] = uploaded_urls[1] if len(uploaded_urls) > 1 else (results.get("about")
or [fallback])[0]
1118
1119     for i, service in enumerate(services[:8]):
1120         if i + 2 < len(uploaded_urls):
1121             svc_img = uploaded_urls[i + 2]
```

```
1122         images["services"].append(svc_img)
1123         if is_ecommerce and i < 6:
1124             images["products"][service] = [svc_img] * 4
1125         else:
1126             imgs = results.get(f"service_{i}") or [fallback]
1127             images["services"].append(imgs[0])
1128             if is_ecommerce and i < 6:
1129                 # Reuse extra images fetched in the same call (no extra API call)
1130                 product_imgs = imgs[1:5] if len(imgs) > 1 else [imgs[0]] * 4
1131                 while len(product_imgs) < 4:
1132                     product_imgs.append(imgs[0])
1133                 images["products"][service] = product_imgs
1134
1135     if remaining_uploads:
1136         images["gallery"] = remaining_uploads[:6]
1137     else:
1138         images["gallery"] = results.get("gallery") or [fallback] * 3
1139
1140     return images
1141
1142
1143 # =====
1144 # WEBSITE BUILDING FROM CHAT
1145 # =====
1146
1147 def build_website_from_chat(session):
1148     """Build a complete website from chat session data"""
1149     business_type = session["business_type"]
1150     business_name = session["business_name"] or "My Business"
1151     location = session["location"] or ""
1152     description = session["description"] or ""
1153     services = session["services"] or ["Service 1", "Service 2", "Service 3"]
1154     features = session["features"] or ["contact"]
1155     style_vibe = session.get("style_vibe", "modern")
1156
1157     # Run image fetching and copywriting IN PARALLEL ? they are fully independent
1158     from concurrent.futures import ThreadPoolExecutor
1159     print("[BUILD] Fetching images + generating copy in parallel...")
1160     with ThreadPoolExecutor(max_workers=2) as executor:
1161         images_future = executor.submit(get_images_for_website, session)
1162         copy_future = executor.submit(
1163             copywriter.generate_website_copy,
1164             business_name=business_name,
1165             business_type=business_type,
1166             business_description=description,
1167             services=services,
1168             location=location,
1169             style_vibe=style_vibe,
1170         )
1171     images = images_future.result()
1172     copy_data = copy_future.result()
1173     print("[BUILD] Images + copy ready.")
1174
1175     # Build complete analysis with professional copy
```

```
1176     analysis = {
1177         "business_name": business_name,
1178         "business_type": business_type,
1179         "location": {"city": location.split(",")[0] if location else "City", "country":
location.split(",")[-1].strip() if "," in location else "Country"},
1180         "vibe": style_vibe.split()[0].lower() if style_vibe else "modern",
1181         "target_audience": "everyone",
1182         "services": services[:6],
1183         "hero_headline": copy_data.get("hero_headline", f"Welcome to {business_name}"),
1184         "hero_subtext": copy_data.get("hero_subtext", f"Experience exceptional
{business_type} services"),
1185         "tagline": copy_data.get("tagline", "Excellence in Every Detail"),
1186         "about_text": copy_data.get("about_text", f"{business_name} provides exceptional
{business_type} services."),
1187         "cta_text": copy_data.get("cta_text", "Get Started")
1188     }
1189
1190     print("[SUCCESS] Professional copy generated!")
1191
1192     # Color scheme
1193     vibe = analysis.get("vibe", "modern")
1194     color_schemes = {
1195         "luxury": {"primary": "#C9A96E", "secondary": "#1A1A1A", "accent": "#F5F5F5", "text":
"#1A1A1A"},
1196         "modern": {"primary": "#6366F1", "secondary": "#0F172A", "accent": "#F59E0B", "text":
"#1E293B"},
1197         "cozy": {"primary": "#D97706", "secondary": "#78350F", "accent": "#FEF3C7", "text":
"#451A03"},
1198         "energetic": {"primary": "#EF4444", "secondary": "#991B1B", "accent": "#FEE2E2",
"text": "#7F1D1D"},
1199         "elegant": {"primary": "#8B5CF6", "secondary": "#4C1D95", "accent": "#F3E8FF",
"text": "#581C87"},
1200         "minimalist": {"primary": "#000000", "secondary": "#374151", "accent": "#F3F4F6",
"text": "#111827"},
1201         "warm": {"primary": "#D97706", "secondary": "#78350F", "accent": "#FEF3C7", "text":
"#451A03"},
1202         "bold": {"primary": "#EF4444", "secondary": "#991B1B", "accent": "#FEE2E2", "text":
"#7F1D1D"},
1203         "clean": {"primary": "#6366F1", "secondary": "#0F172A", "accent": "#F59E0B", "text":
"#1E293B"},
1204     }
1205     colors = color_schemes.get(vibe, color_schemes["modern"])
1206
1207     # Generate HTML with functional features
1208     html = generate_full_website(analysis, colors, images, features, session)
1209
1210     # Save
1211     website_session_id = session.get("website_session_id") or str(uuid.uuid4())
1212     html_path = f"generated_websites/{website_session_id}.html"
1213     state_path = f"website_states/{website_session_id}.json"
1214
1215     with open(html_path, "w", encoding="utf-8") as f:
1216         f.write(html)
1217
```

```
1218     state = {
1219         "version": 1,
1220         "created_at": datetime.now().isoformat(),
1221         "business_description": description,
1222         "analysis": analysis,
1223         "colors": colors,
1224         "images": images,
1225         "features": features,
1226         "chat_session": session.get("conversation_history", [])[-5:], # Last 5 messages
1227     }
1228
1229     with open(state_path, "w", encoding="utf-8") as f:
1230         json.dump(state, f, indent=2)
1231
1232     WEBSITE_STATES[website_session_id] = state
1233
1234     return {"session_id": website_session_id, "html": html, "analysis": analysis}
1235
1236
1237 def edit_website_from_chat(session, edit_request):
1238     """Edit an existing website from chat"""
1239     website_session_id = session["website_session_id"]
1240     if not website_session_id:
1241         raise Exception("No website to edit")
1242
1243     state_path = f"website_states/{website_session_id}.json"
1244     if not os.path.exists(state_path):
1245         raise Exception("Website state not found")
1246
1247     with open(state_path, "r", encoding="utf-8") as f:
1248         state = json.load(f)
1249
1250     original_desc = state.get("business_description", "")
1251     analysis = state.get("analysis", {})
1252     features = state.get("features", session.get("features", ["contact"]))
1253
1254     # Create modified description
1255     modified_desc = (
1256         f"{original_desc} (Business: {analysis.get('business_name', '')}), "
1257         f"Type: {analysis.get('business_type', '')}). "
1258         f"Apply these changes: {edit_request}"
1259     )
1260
1261     # Rebuild
1262     session["description"] = modified_desc
1263     result = build_website_from_chat(session)
1264
1265     # Update version
1266     new_version = state.get("version", 1) + 1
1267     updated_state = WEBSITE_STATES.get(result["session_id"], {})
1268     updated_state["version"] = new_version
1269     WEBSITE_STATES[result["session_id"]] = updated_state
1270
1271     state_path = f"website_states/{result['session_id']}.json"
```

```
1272     with open(state_path, "w", encoding="utf-8") as f:
1273         json.dump(updated_state, f, indent=2)
1274
1275     return result
1276
1277
1278     # =====
1279     # SPECIALIZED E-COMMERCE WEBSITE GENERATOR
1280     # =====
1281
1282     def generate_ecommerce_website(analysis, colors, images, features, session):
1283         """Generate a premium e-commerce website like Zara/H&M/Forever21"""
1284
1285         name = analysis.get('business_name', 'My Store')
1286         services = analysis.get('services', []) # These are product categories
1287         hero_headline = analysis.get('hero_headline', f'Welcome to {name}')
1288         hero_subtext = analysis.get('hero_subtext', 'Discover your style')
1289         tagline = analysis.get('tagline', 'Style Redefined')
1290         about_text = analysis.get('about_text', f'{name} brings you the latest fashion.')
1291         cta_text = analysis.get('cta_text', 'Shop Now')
1292         location = analysis.get('location', {})
1293         city = location.get('city', 'City')
1294         country = location.get('country', 'Country')
1295         website_session_id = session.get("website_session_id") or "preview"
1296
1297         # Generate realistic product names using AI
1298         product_name_map = {}
1299         try:
1300             model = genai.GenerativeModel('gemini-2.5-flash')
1301             cats_str = ", ".join(services[:6])
1302             resp = model.generate_content(
1303                 f'Generate 3 trendy product names for each of these fashion categories:
1304 {cats_str}. '
1305                 f'The brand is "{name}". Make names catchy and realistic like real fashion brands
1306 use. '
1307                 f'Return ONLY valid JSON like: {"category1": ["Name 1", "Name 2", "Name 3"],
1308 "category2": [...]}')
1309             )
1310             text = resp.text.strip()
1311             if '```' in text:
1312                 text = text.split('```')[1].replace('json', '').strip()
1313             product_name_map = json.loads(text)
1314         except Exception as e:
1315             print(f"Product name generation error: {e}")
1316
1317         # Build collections grid
1318         collections_html = ""
1319         for i, category in enumerate(services[:8]):
1320             img = images['services'][i] if i < len(images['services']) else images['services'][0]
1321             if images['services'] else ""
1322             delay = i * 80
1323             size_class = "collection-large" if i < 2 else "collection-small"
1324             collections_html += f'''
1325                 <div    class="collection-card    {size_class}"    data-aos="fade-up"
```

```

        data-aos-delay="{delay}">
1322         
1323         <div class="collection-overlay">
1324             <h3>{category}</h3>
1325             <span class="collection-count">{random.randint(12, 80)}+ Products</span>
1326             <a href="#products" class="shop-collection-btn">Shop Now</a>
1327         </div>
1328     </div>'''
1329
1330     # Build product cards with UNIQUE images per product
1331     products_html = ""
1332     product_idx = 0
1333     for cat_idx, category in enumerate(services[:6]):
1334         # Get unique images for this category's products
1335         cat_product_imgs = images.get('products', {}).get(category, [])
1336         if not cat_product_imgs:
1337             cat_product_imgs = [images['services'][cat_idx]] if cat_idx <
len(images['services']) else []
1338
1339         # Get realistic names or fallback
1340         cat_names = product_name_map.get(category, [])
1341         if not cat_names:
1342             # Try case-insensitive match
1343             for k, v in product_name_map.items():
1344                 if k.lower() == category.lower():
1345                     cat_names = v
1346                     break
1347         if not cat_names:
1348             cat_names = [f"{category} Classic", f"{category} Premium", f"{category} Trendy"]
1349
1350         for j in range(min(3, len(cat_names))):
1351             # Use a DIFFERENT image for each product
1352             img = cat_product_imgs[j % len(cat_product_imgs)] if cat_product_imgs else ""
1353             pname = cat_names[j] if j < len(cat_names) else f"{category} Style {j+1}"
1354             price = random.randint(499, 2999)
1355             old_price = price + random.randint(200, 800)
1356             discount = int((1 - price/old_price) * 100)
1357             badge_html = f'<span class="product-badge">--{discount}%</span>' if j % 3 != 2
else '<span class="product-badge" style="background:#22c55e">NEW</span>'
1358             product_idx += 1
1359             products_html += f'''
1360                 <div class="product-card" data-category="{category.lower()}" data-aos="fade-up"
data-aos-delay="{(product_idx % 4) * 60}">
1361                     <div class="product-image">
1362                         
1363                         {badge_html}
1364                     <div class="product-actions">
1365                         <button class="product-action-btn" onclick="addToCart(this)"
data-name="{pname}" data-price="{price}">Add to Cart</button>
1366                         <button class="product-action-btn wishlist-btn">?</button>
1367                     </div>
1368                 </div>
1369                 <div class="product-info">
1370                     <span class="product-category">{category}</span>

```

```

1371         <h4>{pname}</h4>
1372         <div class="product-pricing">
1373             <span class="current-price">?{price}</span>
1374             <span class="old-price">?{old_price}</span>
1375         </div>
1376         <div class="product-sizes">
1377             <span class="size">XS</span><span class="size">S</span><span
class="size">M</span><span class="size">L</span><span class="size">XL</span>
1378         </div>
1379     </div>
1380 </div>'''
1381
1382     # Category filter buttons
1383     filter_btns = '<button class="filter-btn active" onclick="filterProducts(\'all\',
this)">All</button>'
1384     for cat in services[:6]:
1385         filter_btns += f'<button class="filter-btn"
onclick="filterProducts(\'{cat.lower()}\', this)">{cat}</button>'
1386
1387     # Gallery / lookbook - use unique gallery images
1388     gallery_html = ""
1389     for i, img in enumerate(images.get('gallery', [])[:4]):
1390         delay = i * 100
1391         gallery_html += f'''
1392         <div class="lookbook-item" data-aos="fade-up" data-aos-delay="{delay}">
1393             
1394             <div class="lookbook-overlay">
1395                 <span>View Look</span>
1396             </div>
1397         </div>'''
1398
1399     html = f'''<!DOCTYPE html>
1400 <html lang="en">
1401 <head>
1402     <meta charset="UTF-8">
1403     <meta name="viewport" content="width=device-width, initial-scale=1.0">
1404     <title>{name} | {tagline}</title>
1405     <meta name="description" content="{about_text[:150]}">
1406     <link rel="preconnect" href="https://fonts.googleapis.com">
1407                                     <link
href="https://fonts.googleapis.com/css2?family=Playfair+Display:wght@400;500;600;700;800;900&
family=Inter:wght@300;400;500;600;700;800&family=DM+Sans:wght@300;400;500;600;700&display=swa
p" rel="stylesheet">
1408     <link href="https://unpkg.com/aos@2.3.1/dist/aos.css" rel="stylesheet">
1409     <style>
1410         * {{ margin: 0; padding: 0; box-sizing: border-box; }}
1411         :root {{
1412             --primary: {colors['primary']};
1413             --secondary: {colors['secondary']};
1414             --accent: {colors['accent']};
1415             --text: {colors['text']};
1416             --bg: #FAFAF9;
1417             --card-bg: #FFFFFF;
1418             --border: #E8E8E5;

```

```
1419     }}
1420     html {{ scroll-behavior: smooth; }}
1421     body {{ font-family: 'DM Sans', 'Inter', sans-serif; color: var(--text); line-height:
1.6; background: var(--bg); }}
1422
1423     /* === ANNOUNCEMENT BAR === */
1424     .announcement {{ background: var(--secondary); color: white; text-align: center;
padding: 0.6rem 1rem; font-size: 0.82rem; font-weight: 500; letter-spacing: 1px;
text-transform: uppercase; }}
1425
1426     /* === NAVBAR === */
1427     .navbar {{ position: sticky; top: 0; z-index: 1000; background: white; border-bottom:
1px solid var(--border); }}
1428     .nav-container {{ max-width: 1400px; margin: 0 auto; padding: 0 2rem; display: flex;
justify-content: space-between; align-items: center; height: 70px; }}
1429     .nav-logo {{ font-family: 'Playfair Display', serif; font-size: 1.8rem; font-weight:
800; color: var(--text); text-decoration: none; letter-spacing: -0.5px; }}
1430     .nav-links {{ display: flex; gap: 2rem; align-items: center; list-style: none; }}
1431     .nav-links a {{ text-decoration: none; color: var(--text); font-weight: 500;
font-size: 0.9rem; text-transform: uppercase; letter-spacing: 1px; transition: color 0.2s;
position: relative; }}
1432     .nav-links a::after {{ content: ''; position: absolute; bottom: -4px; left: 0; width:
0; height: 1.5px; background: var(--primary); transition: width 0.3s; }}
1433     .nav-links a:hover::after {{ width: 100%; }}
1434     .nav-links a:hover {{ color: var(--primary); }}
1435     .nav-icons {{ display: flex; gap: 1.2rem; align-items: center; }}
1436     .nav-icon {{ background: none; border: none; font-size: 1.2rem; cursor: pointer;
color: var(--text); transition: color 0.2s; position: relative; }}
1437     .nav-icon:hover {{ color: var(--primary); }}
1438     .cart-badge {{ position: absolute; top: -6px; right: -8px; background:
var(--primary); color: white; font-size: 0.65rem; width: 18px; height: 18px; border-radius:
50%; display: flex; align-items: center; justify-content: center; font-weight: 700; }}
1439     .mobile-toggle {{ display: none; background: none; border: none; font-size: 1.5rem;
cursor: pointer; }}
1440
1441     /* === HERO === */
1442     .hero {{ position: relative; height: 85vh; min-height: 500px; overflow: hidden;
display: flex; align-items: center; }}
1443     .hero-bg {{ position: absolute; inset: 0; }}
1444     .hero-bg img {{ width: 100%; height: 100%; object-fit: cover; }}
1445     .hero-bg::after {{ content: ''; position: absolute; inset: 0; background:
linear-gradient(90deg, rgba(0,0,0,0.55) 0%, rgba(0,0,0,0.1) 60%); }}
1446     .hero-content {{ position: relative; z-index: 2; max-width: 1400px; margin: 0 auto;
padding: 0 4rem; width: 100%; }}
1447     .hero-tag {{ display: inline-block; color: rgba(255,255,255,0.9); font-size: 0.85rem;
font-weight: 600; text-transform: uppercase; letter-spacing: 3px; margin-bottom: 1.5rem;
border-left: 3px solid var(--primary); padding-left: 1rem; }}
1448     .hero h1 {{ font-family: 'Playfair Display', serif; font-size: clamp(3rem, 6vw,
5.5rem); font-weight: 800; color: white; line-height: 1.08; margin-bottom: 1.5rem;
letter-spacing: -1px; max-width: 650px; }}
1449     .hero p {{ color: rgba(255,255,255,0.85); font-size: 1.15rem; max-width: 480px;
margin-bottom: 2.5rem; line-height: 1.7; }}
1450     .hero-btns {{ display: flex; gap: 1rem; flex-wrap: wrap; }}
1451     .btn-shop {{ padding: 1rem 2.5rem; background: white; color: var(--text); border:
```

```
    none; font-weight: 700; font-size: 0.9rem; text-transform: uppercase; letter-spacing: 2px;
    cursor: pointer; transition: all 0.3s; text-decoration: none; }}
1452     .btn-shop:hover {{ background: var(--primary); color: white; }}
1453     .btn-outline {{ padding: 1rem 2.5rem; background: transparent; color: white; border:
2px solid white; font-weight: 700; font-size: 0.9rem; text-transform: uppercase;
letter-spacing: 2px; cursor: pointer; transition: all 0.3s; text-decoration: none; }}
1454     .btn-outline:hover {{ background: white; color: var(--text); }}
1455
1456     /* === MARQUEE === */
1457     .marquee {{ background: var(--secondary); color: white; padding: 0.8rem 0; overflow:
hidden; white-space: nowrap; }}
1458     .marquee-track {{ display: inline-block; animation: marquee 20s linear infinite; }}
1459     .marquee-track span {{ display: inline-block; padding: 0 3rem; font-size: 0.85rem;
font-weight: 500; text-transform: uppercase; letter-spacing: 2px; }}
1460     @keyframes marquee {{ 0% {{ transform: translateX(0); }} 100% {{ transform:
translateX(-50%); }} }}
1461
1462     /* === COLLECTIONS === */
1463     .collections {{ padding: 5rem 2rem; max-width: 1400px; margin: 0 auto; }}
1464     .section-header {{ text-align: center; margin-bottom: 3.5rem; }}
1465     .section-header h2 {{ font-family: 'Playfair Display', serif; font-size: clamp(2rem,
4vw, 3rem); font-weight: 700; margin-bottom: 0.8rem; }}
1466     .section-header p {{ color: #888; font-size: 1rem; max-width: 500px; margin: 0 auto;
}}
1467     .collections-grid {{ display: grid; grid-template-columns: repeat(auto-fit,
minmax(280px, 1fr)); gap: 1.5rem; }}
1468     .collection-card {{ position: relative; border-radius: 12px; overflow: hidden;
cursor: pointer; height: 380px; }}
1469     .collection-large {{ height: 450px; }}
1470     .collection-card img {{ width: 100%; height: 100%; object-fit: cover; transition:
transform 0.6s ease; }}
1471     .collection-card:hover img {{ transform: scale(1.08); }}
1472     .collection-overlay {{ position: absolute; inset: 0; background:
linear-gradient(180deg, transparent 30%, rgba(0,0,0,0.7) 100%); display: flex;
flex-direction: column; justify-content: flex-end; padding: 2rem; }}
1473     .collection-overlay h3 {{ color: white; font-family: 'Playfair Display', serif;
font-size: 1.6rem; font-weight: 700; margin-bottom: 0.3rem; }}
1474     .collection-count {{ color: rgba(255,255,255,0.7); font-size: 0.85rem; margin-bottom:
1rem; }}
1475     .shop-collection-btn {{ display: inline-block; padding: 0.6rem 1.5rem; background:
white; color: var(--text); text-decoration: none; font-weight: 600; font-size: 0.82rem;
text-transform: uppercase; letter-spacing: 1px; transition: all 0.3s; border: none; cursor:
pointer; }}
1476     .shop-collection-btn:hover {{ background: var(--primary); color: white; }}
1477
1478     /* === PRODUCTS === */
1479     .products-section {{ padding: 5rem 2rem; background: white; }}
1480     .products-container {{ max-width: 1400px; margin: 0 auto; }}
1481     .filter-bar {{ display: flex; gap: 0.8rem; justify-content: center; flex-wrap: wrap;
margin-bottom: 3rem; }}
1482     .filter-btn {{ padding: 0.6rem 1.5rem; border: 1.5px solid var(--border); background:
white; color: var(--text); font-family: inherit; font-size: 0.82rem; font-weight: 600;
text-transform: uppercase; letter-spacing: 1px; cursor: pointer; border-radius: 30px;
transition: all 0.2s; }}
```

```
1483         .filter-btn.active, .filter-btn:hover {{ background: var(--text); color: white;
border-color: var(--text); }}
1484         .products-grid {{ display: grid; grid-template-columns: repeat(auto-fill,
minmax(260px, 1fr)); gap: 2rem; }}
1485         .product-card {{ background: var(--card-bg); border-radius: 12px; overflow: hidden;
transition: transform 0.3s, box-shadow 0.3s; }}
1486         .product-card:hover {{ transform: translateY(-5px); box-shadow: 0 15px 40px
rgba(0,0,0,0.1); }}
1487         .product-image {{ position: relative; height: 340px; overflow: hidden; background:
#f5f5f3; }}
1488         .product-image img {{ width: 100%; height: 100%; object-fit: cover; transition:
transform 0.5s; }}
1489         .product-card:hover .product-image img {{ transform: scale(1.05); }}
1490         .product-badge {{ position: absolute; top: 12px; left: 12px; background: #E53935;
color: white; padding: 0.3rem 0.8rem; border-radius: 4px; font-size: 0.75rem; font-weight:
700; }}
1491         .product-actions {{ position: absolute; bottom: 0; left: 0; right: 0; padding: 1rem;
display: flex; gap: 0.5rem; transform: translateY(100%); transition: transform 0.3s; }}
1492         .product-card:hover .product-actions {{ transform: translateY(0); }}
1493         .product-action-btn {{ flex: 1; padding: 0.7rem; background: white; border: none;
font-family: inherit; font-weight: 600; font-size: 0.82rem; cursor: pointer; transition: all
0.2s; text-transform: uppercase; letter-spacing: 0.5px; }}
1494         .product-action-btn:hover {{ background: var(--text); color: white; }}
1495         .wishlist-btn {{ flex: 0 0 44px; font-size: 1.1rem; }}
1496         .product-info {{ padding: 1.2rem 1rem; }}
1497         .product-category {{ font-size: 0.75rem; color: #999; text-transform: uppercase;
letter-spacing: 1px; }}
1498         .product-info h4 {{ font-size: 1rem; font-weight: 600; margin: 0.3rem 0 0.5rem; }}
1499         .product-pricing {{ display: flex; gap: 0.8rem; align-items: center; }}
1500         .current-price {{ font-weight: 800; font-size: 1.1rem; color: var(--text); }}
1501         .old-price {{ font-size: 0.9rem; color: #bbb; text-decoration: line-through; }}
1502         .product-sizes {{ display: flex; gap: 0.4rem; margin-top: 0.8rem; }}
1503         .size {{ width: 32px; height: 28px; display: flex; align-items: center;
justify-content: center; border: 1px solid var(--border); border-radius: 4px; font-size:
0.7rem; font-weight: 600; color: #666; cursor: pointer; transition: all 0.2s; }}
1504         .size:hover {{ border-color: var(--text); color: var(--text); }}
1505
1506         /* === LOOKBOOK === */
1507         .lookbook {{ padding: 5rem 2rem; max-width: 1400px; margin: 0 auto; }}
1508         .lookbook-grid {{ display: grid; grid-template-columns: repeat(auto-fill,
minmax(350px, 1fr)); gap: 1.5rem; }}
1509         .lookbook-item {{ position: relative; height: 450px; border-radius: 12px; overflow:
hidden; cursor: pointer; }}
1510         .lookbook-item img {{ width: 100%; height: 100%; object-fit: cover; transition:
transform 0.6s; }}
1511         .lookbook-item:hover img {{ transform: scale(1.06); }}
1512         .lookbook-overlay {{ position: absolute; inset: 0; background: rgba(0,0,0,0.3);
display: flex; align-items: center; justify-content: center; opacity: 0; transition: opacity
0.3s; }}
1513         .lookbook-item:hover .lookbook-overlay {{ opacity: 1; }}
1514         .lookbook-overlay span {{ background: white; color: var(--text); padding: 0.8rem
2rem; font-weight: 700; font-size: 0.85rem; text-transform: uppercase; letter-spacing: 2px;
}}
1515
```

```
1516     /* === FEATURES STRIP === */
1517     .features-strip {{ display: grid; grid-template-columns: repeat(auto-fit,
minmax(220px, 1fr)); gap: 0; border-top: 1px solid var(--border); border-bottom: 1px solid
var(--border); }}
1518     .feature-item {{ text-align: center; padding: 2.5rem 1.5rem; border-right: 1px solid
var(--border); }}
1519     .feature-item:last-child {{ border-right: none; }}
1520     .feature-icon {{ font-size: 1.8rem; margin-bottom: 0.8rem; }}
1521     .feature-item h4 {{ font-size: 0.9rem; font-weight: 700; text-transform: uppercase;
letter-spacing: 1px; margin-bottom: 0.3rem; }}
1522     .feature-item p {{ font-size: 0.82rem; color: #888; }}
1523
1524     /* === NEWSLETTER === */
1525     .newsletter {{ padding: 5rem 2rem; background: var(--secondary); color: white;
text-align: center; }}
1526     .newsletter h2 {{ font-family: 'Playfair Display', serif; font-size: clamp(1.8rem,
3vw, 2.5rem); margin-bottom: 0.8rem; }}
1527     .newsletter p {{ color: rgba(255,255,255,0.7); margin-bottom: 2rem; max-width: 450px;
margin-left: auto; margin-right: auto; }}
1528     .newsletter-form {{ display: flex; gap: 0; max-width: 500px; margin: 0 auto; }}
1529     .newsletter-form input {{ flex: 1; padding: 1rem 1.5rem; border: none; font-family:
inherit; font-size: 0.95rem; outline: none; }}
1530     .newsletter-form button {{ padding: 1rem 2rem; background: var(--primary); color:
white; border: none; font-family: inherit; font-weight: 700; font-size: 0.85rem;
text-transform: uppercase; letter-spacing: 1px; cursor: pointer; transition: background 0.3s;
}}
1531     .newsletter-form button:hover {{ background: white; color: var(--text); }}
1532
1533     /* === FOOTER === */
1534     .footer {{ padding: 4rem 2rem 2rem; background: #0a0a0a; color: white; }}
1535     .footer-grid {{ max-width: 1400px; margin: 0 auto; display: grid;
grid-template-columns: 2fr repeat(3, 1fr); gap: 3rem; margin-bottom: 3rem; }}
1536     .footer-brand {{ font-family: 'Playfair Display', serif; font-size: 1.6rem;
font-weight: 800; margin-bottom: 1rem; }}
1537     .footer-section h4 {{ font-size: 0.85rem; text-transform: uppercase; letter-spacing:
2px; margin-bottom: 1.2rem; font-weight: 700; }}
1538     .footer-section p, .footer-section a {{ display: block; color: rgba(255,255,255,0.6);
text-decoration: none; margin-bottom: 0.6rem; font-size: 0.9rem; transition: color 0.2s; }}
1539     .footer-section a:hover {{ color: white; }}
1540     .footer-bottom {{ max-width: 1400px; margin: 0 auto; padding-top: 2rem; border-top:
1px solid rgba(255,255,255,0.1); display: flex; justify-content: space-between; align-items:
center; font-size: 0.82rem; color: rgba(255,255,255,0.4); }}
1541     .social-links {{ display: flex; gap: 1rem; }}
1542     .social-links a {{ color: rgba(255,255,255,0.5); text-decoration: none; transition:
color 0.2s; }}
1543     .social-links a:hover {{ color: white; }}
1544
1545     /* === CART SIDEBAR === */
1546     .cart-sidebar {{ position: fixed; top: 0; right: -450px; width: 420px; height: 100vh;
background: white; box-shadow: -10px 0 50px rgba(0,0,0,0.15); z-index: 10001; transition:
right 0.35s ease; display: flex; flex-direction: column; }}
1547     .cart-sidebar.open {{ right: 0; }}
1548     .cart-overlay {{ position: fixed; inset: 0; background: rgba(0,0,0,0.4); z-index:
10000; display: none; }}
```

```
1549     .cart-overlay.open {{ display: block; }}
1550     .cart-header {{ display: flex; justify-content: space-between; align-items: center;
padding: 1.8rem 2rem; border-bottom: 1px solid #eee; }}
1551     .cart-header h3 {{ font-family: 'Playfair Display', serif; font-size: 1.3rem; }}
1552     .cart-close {{ background: none; border: none; font-size: 1.5rem; cursor: pointer;
color: #999; }}
1553     .cart-items {{ flex: 1; overflow-y: auto; padding: 1.5rem 2rem; }}
1554     .cart-item {{ display: flex; justify-content: space-between; align-items: center;
padding: 1rem 0; border-bottom: 1px solid #f0f0f0; }}
1555     .cart-item strong {{ font-size: 0.95rem; }}
1556     .cart-item span {{ font-size: 0.85rem; color: #888; }}
1557     .cart-item-actions {{ display: flex; align-items: center; gap: 0.5rem; }}
1558     .cart-item-actions button {{ width: 30px; height: 30px; border: 1px solid #e0e0e0;
border-radius: 50%; background: white; cursor: pointer; font-weight: 600; font-size: 0.9rem;
transition: all 0.2s; }}
1559     .cart-item-actions button:hover {{ background: var(--text); color: white;
border-color: var(--text); }}
1560     .cart-footer {{ padding: 1.5rem 2rem; border-top: 1px solid #eee; }}
1561     .cart-total {{ font-size: 1.2rem; font-weight: 800; margin-bottom: 1rem; display:
flex; justify-content: space-between; }}
1562     .checkout-btn {{ width: 100%; padding: 1rem; background: var(--text); color: white;
border: none; font-family: inherit; font-size: 0.9rem; font-weight: 700; text-transform:
uppercase; letter-spacing: 2px; cursor: pointer; transition: background 0.3s; }}
1563     .checkout-btn:hover {{ background: var(--primary); }}
1564     .cart-empty {{ text-align: center; color: #bbb; padding: 3rem 1rem; font-size:
0.95rem; }}
1565
1566     /* === CHECKOUT MODAL === */
1567     .checkout-modal {{ position: fixed; inset: 0; background: rgba(0,0,0,0.5); z-index:
10002; display: none; align-items: center; justify-content: center; }}
1568     .checkout-modal.open {{ display: flex; }}
1569     .checkout-content {{ background: white; border-radius: 16px; padding: 2.5rem; width:
90%; max-width: 500px; max-height: 90vh; overflow-y: auto; position: relative; }}
1570     .checkout-content h3 {{ font-family: 'Playfair Display', serif; font-size: 1.4rem;
margin-bottom: 1.5rem; }}
1571     .modal-close {{ position: absolute; top: 1.2rem; right: 1.5rem; background: none;
border: none; font-size: 1.3rem; cursor: pointer; color: #999; }}
1572     .checkout-content input, .checkout-content textarea {{ width: 100%; padding: 0.9rem
1rem; border: 1.5px solid #e0e0e0; border-radius: 8px; margin-bottom: 0.8rem; font-family:
inherit; font-size: 0.9rem; }}
1573     .checkout-content input:focus, .checkout-content textarea:focus {{ outline: none;
border-color: var(--text); }}
1574     .checkout-content .submit-btn {{ width: 100%; padding: 1rem; background: var(--text);
color: white; border: none; font-family: inherit; font-size: 0.9rem; font-weight: 700;
text-transform: uppercase; letter-spacing: 1px; cursor: pointer; border-radius: 8px;
margin-top: 0.5rem; }}
1575     .checkout-content .submit-btn:hover {{ background: var(--primary); }}
1576     .form-message {{ margin-top: 0.5rem; }}
1577
1578     /* === CONTACT === */
1579     .contact-section {{ padding: 5rem 2rem; background: #f8f8f6; }}
1580     .contact-container {{ max-width: 600px; margin: 0 auto; text-align: center; }}
1581     .contact-form {{ display: flex; flex-direction: column; gap: 1rem; margin-top: 2rem;
}}
```

```
1582     .contact-form input, .contact-form textarea {{ padding: 1rem 1.2rem; border: 1.5px
solid var(--border); border-radius: 8px; font-family: inherit; font-size: 0.95rem;
background: white; }}
1583     .contact-form input:focus, .contact-form textarea:focus {{ outline: none;
border-color: var(--text); }}
1584     .contact-form .submit-btn {{ padding: 1rem 3rem; background: var(--text); color:
white; border: none; font-family: inherit; font-weight: 700; font-size: 0.9rem;
text-transform: uppercase; letter-spacing: 2px; cursor: pointer; border-radius: 8px;
transition: background 0.3s; align-self: center; }}
1585     .contact-form .submit-btn:hover {{ background: var(--primary); }}
1586
1587     /* === MOBILE NAV === */
1588     .nav-links.mobile-open {{
1589         display: flex !important;
1590         flex-direction: column;
1591         position: absolute;
1592         top: 70px;
1593         left: 0;
1594         right: 0;
1595         background: white;
1596         padding: 1.5rem 2rem;
1597         box-shadow: 0 10px 30px rgba(0,0,0,0.1);
1598         gap: 1rem;
1599         z-index: 999;
1600     }}
1601
1602     /* === RESPONSIVE === */
1603     @media (max-width: 968px) {{
1604         .nav-links {{ display: none; }}
1605         .mobile-toggle {{ display: block; }}
1606         .hero-content {{ padding: 0 2rem; }}
1607         .footer-grid {{ grid-template-columns: 1fr 1fr; }}
1608         .lookbook-grid {{ grid-template-columns: 1fr 1fr; }}
1609         .about-grid {{ grid-template-columns: 1fr !important; }}
1610         .about-grid > div:first-child img {{ height: 300px !important; }}
1611     }}
1612     @media (max-width: 640px) {{
1613         .hero {{ height: 70vh; }}
1614         .hero h1 {{ font-size: 2.2rem; }}
1615         .hero p {{ font-size: 0.95rem; }}
1616         .products-grid {{ grid-template-columns: repeat(2, 1fr); gap: 0.8rem; }}
1617         .product-image {{ height: 220px; }}
1618         .product-info {{ padding: 0.8rem 0.6rem; }}
1619         .product-info h4 {{ font-size: 0.85rem; }}
1620         .product-sizes {{ display: none; }}
1621         .collections-grid {{ grid-template-columns: 1fr 1fr; }}
1622         .collection-card, .collection-large {{ height: 250px; }}
1623         .collection-overlay h3 {{ font-size: 1.1rem; }}
1624         .footer-grid {{ grid-template-columns: 1fr; }}
1625         .newsletter-form {{ flex-direction: column; }}
1626         .cart-sidebar {{ width: 100%; }}
1627         .features-strip {{ grid-template-columns: 1fr 1fr; }}
1628         .feature-item {{ border-right: none; border-bottom: 1px solid var(--border); }}
1629         .lookbook-grid {{ grid-template-columns: 1fr; }}
```

```
1630         .lookbook-item {{ height: 300px; }}
1631         .section-header h2 {{ font-size: 1.8rem; }}
1632         .filter-bar {{ gap: 0.4rem; }}
1633         .filter-btn {{ padding: 0.4rem 1rem; font-size: 0.75rem; }}
1634     }}
1635 </style>
1636 </head>
1637 <body>
1638     <!-- ANNOUNCEMENT -->
1639     <div class="announcement">Free Shipping on Orders Above ₹999 | Use Code FIRST10 for 10%
1640     Off</div>
1641
1642     <!-- NAVBAR -->
1643     <nav class="navbar">
1644         <div class="nav-container">
1645             <button class="mobile-toggle"
1646             onclick="document.querySelector('.nav-links').classList.toggle('mobile-open')">?</button>
1647             <a href="#" class="nav-logo">{name}</a>
1648             <ul class="nav-links">
1649                 <li><a href="#collections">Collections</a></li>
1650                 <li><a href="#products">Shop</a></li>
1651                 <li><a href="#lookbook">Lookbook</a></li>
1652                 <li><a href="#about">About</a></li>
1653                 <li><a href="#contact">Contact</a></li>
1654             </ul>
1655             <div class="nav-icons">
1656                 <button class="nav-icon" onclick="toggleCart()">? <span class="cart-badge"
1657                 id="cartBadge" style="display:none">0</span></button>
1658             </div>
1659         </div>
1660     </nav>
1661
1662     <!-- HERO -->
1663     <section class="hero">
1664         <div class="hero-bg">
1665             
1666         </div>
1667         <div class="hero-content">
1668             <span class="hero-tag">{tagline}</span>
1669             <h1>{hero_headline}</h1>
1670             <p>{hero_subtext}</p>
1671             <div class="hero-btns">
1672                 <a href="#products" class="btn-shop">{cta_text}</a>
1673                 <a href="#collections" class="btn-outline">View Collections</a>
1674             </div>
1675         </div>
1676     </section>
1677
1678     <!-- MARQUEE -->
1679     <div class="marquee">
1680         <div class="marquee-track">
1681             <span>New Arrivals</span><span>Trending Now</span><span>Free
1682             Returns</span><span>COD Available</span><span>Exclusive Online Deals</span><span>Pan India
1683             Shipping</span>
```

```
1679             <span>New Arrivals</span><span>Trending Now</span><span>Free
Returns</span><span>COD Available</span><span>Exclusive Online Deals</span><span>Pan India
Shipping</span>
1680         </div>
1681     </div>
1682
1683     <!-- COLLECTIONS -->
1684     <section class="collections" id="collections">
1685         <div class="section-header" data-aos="fade-up">
1686             <h2>Shop by Collection</h2>
1687             <p>Curated categories for every mood and occasion</p>
1688         </div>
1689         <div class="collections-grid">{collections_html}</div>
1690     </section>
1691
1692     <!-- PRODUCTS -->
1693     <section class="products-section" id="products">
1694         <div class="products-container">
1695             <div class="section-header" data-aos="fade-up">
1696                 <h2>Our Bestsellers</h2>
1697                 <p>The pieces our customers can't stop wearing</p>
1698             </div>
1699             <div class="filter-bar" data-aos="fade-up">{filter_btns}</div>
1700             <div class="products-grid" id="productsGrid">{products_html}</div>
1701         </div>
1702     </section>
1703
1704     <!-- LOOKBOOK -->
1705     <section class="lookbook" id="lookbook">
1706         <div class="section-header" data-aos="fade-up">
1707             <h2>Lookbook</h2>
1708             <p>Get inspired by our latest styled looks</p>
1709         </div>
1710         <div class="lookbook-grid">{gallery_html}</div>
1711     </section>
1712
1713     <!-- FEATURES -->
1714     <div class="features-strip">
1715         <div class="feature-item" data-aos="fade-up">
1716             <div class="feature-icon">?</div>
1717             <h4>Free Shipping</h4>
1718             <p>On orders above ?999</p>
1719         </div>
1720         <div class="feature-item" data-aos="fade-up" data-aos-delay="100">
1721             <div class="feature-icon">??</div>
1722             <h4>Easy Returns</h4>
1723             <p>7-day return policy</p>
1724         </div>
1725         <div class="feature-item" data-aos="fade-up" data-aos-delay="200">
1726             <div class="feature-icon">?</div>
1727             <h4>Secure Payment</h4>
1728             <p>100% secure checkout</p>
1729         </div>
1730         <div class="feature-item" data-aos="fade-up" data-aos-delay="300">
```

```
1731         <div class="feature-icon">?</div>
1732         <h4>24/7 Support</h4>
1733         <p>Chat or call anytime</p>
1734     </div>
1735 </div>
1736
1737 <!-- ABOUT -->
1738 <section style="padding:5rem 2rem;max-width:1400px;margin:0 auto" id="about">
1739     <div class="about-grid" style="display:grid;grid-template-columns:1fr
1740 1fr;gap:4rem;align-items:center">
1741         <div data-aos="fade-right">
1742             
1742         </div>
1743         <div data-aos="fade-left">
1744             <h2 style="font-family:'Playfair
1745             Display',serif;font-size:2.5rem;margin-bottom:1.5rem">About {name}</h2>
1745             <p
1746             style="color:#666;line-height:1.9;margin-bottom:1rem;font-size:1.05rem">{about_text}</p>
1747             <p style="color:#666;line-height:1.9;font-size:1.05rem">Based in {city},
1748             {country} ? shipping across the country with love and care.</p>
1747         </div>
1748     </div>
1749 </section>
1750
1751 <!-- CONTACT -->
1752 <section class="contact-section" id="contact">
1753     <div class="contact-container">
1754         <div class="section-header" data-aos="fade-up">
1755             <h2>Get in Touch</h2>
1756             <p>We'd love to hear from you</p>
1757         </div>
1758         <form class="contact-form" id="contactForm" data-aos="fade-up">
1759             <input type="text" name="name" placeholder="Your Name" required>
1760             <input type="email" name="email" placeholder="Email Address" required>
1761             <input type="tel" name="phone" placeholder="Phone Number">
1762             <textarea name="message" placeholder="Your message..." required
1763             rows="4"></textarea>
1764             <button type="submit" class="submit-btn">Send Message</button>
1765             <div class="form-message" id="contactMessage"></div>
1766         </form>
1767     </div>
1768 </section>
1769
1770 <!-- NEWSLETTER -->
1771 <section class="newsletter">
1772     <h2 data-aos="fade-up">Stay in the Loop</h2>
1773     <p data-aos="fade-up" data-aos-delay="100">Subscribe for exclusive drops, styling
1774     tips & 15% off your first order</p>
1775     <form class="newsletter-form" data-aos="fade-up" data-aos-delay="200"
1776     onsubmit="event.preventDefault(); this.innerHTML='<p style=\\'padding:1rem;color:white\\'?
1777     You\\'re in! Check your inbox.</p>'>
1778         <input type="email" placeholder="Enter your email" required>
1779         <button type="submit">Subscribe</button>
```

```
1776         </form>
1777     </section>
1778
1779     <!-- FOOTER -->
1780     <footer class="footer">
1781         <div class="footer-grid">
1782             <div class="footer-section">
1783                 <div class="footer-brand">{name}</div>
1784                 <p>{tagline}</p>
1785                 <p style="margin-top:1rem">{city}, {country}</p>
1786             </div>
1787             <div class="footer-section">
1788                 <h4>Shop</h4>
1789                 {''.join(f'<a href="#products">{s}</a>' for s in services[:5])}
1790             </div>
1791             <div class="footer-section">
1792                 <h4>Help</h4>
1793                 <a href="#contact">Contact Us</a>
1794                 <a href="#">Shipping & Returns</a>
1795                 <a href="#">Size Guide</a>
1796                 <a href="#">FAQ</a>
1797             </div>
1798             <div class="footer-section">
1799                 <h4>Contact</h4>
1800                 <p>info@{name.lower().replace(' ', '')}.com</p>
1801                 <p>+91 98765 43210</p>
1802             </div>
1803         </div>
1804         <div class="footer-bottom">
1805             <p>© 2025 {name}. All rights reserved.</p>
1806             <div class="social-links">
1807                 <a href="#">Instagram</a>
1808                 <a href="#">Facebook</a>
1809                 <a href="#">Pinterest</a>
1810             </div>
1811         </div>
1812     </footer>
1813
1814     <!-- CART SIDEBAR -->
1815     <div class="cart-overlay" id="cartOverlay" onclick="toggleCart()"></div>
1816     <div class="cart-sidebar" id="cartSidebar">
1817         <div class="cart-header">
1818             <h3>Shopping Bag</h3>
1819             <button class="cart-close" onclick="toggleCart()">?</button>
1820         </div>
1821         <div class="cart-items" id="cartItems">
1822             <p class="cart-empty">Your bag is empty</p>
1823         </div>
1824         <div class="cart-footer">
1825             <div class="cart-total"><span>Total</span> <span>?</span>
1826             id="cartTotal">0</span></span></div>
1827             <button class="checkout-btn" onclick="showCheckout()">Checkout</button>
1828         </div>
1829     </div>
```

```
1829
1830     <!-- CHECKOUT MODAL -->
1831     <div class="checkout-modal" id="checkoutModal">
1832         <div class="checkout-content">
1833             <h3>Checkout</h3>
1834             <button class="modal-close" onclick="closeCheckout()">?</button>
1835             <form id="orderForm" onsubmit="submitOrder(event)">
1836                 <input type="text" name="name" placeholder="Full Name" required>
1837                 <input type="email" name="email" placeholder="Email" required>
1838                 <input type="tel" name="phone" placeholder="Phone" required>
1839                 <textarea name="address" placeholder="Delivery Address" required
rows="3"></textarea>
1840                 <textarea name="notes" placeholder="Order notes (optional)"
rows="2"></textarea>
1841                 <div id="orderSummary"
style="padding:1rem;background:#f8f8f6;border-radius:8px;margin:0.5rem
0;font-size:0.9rem"></div>
1842                 <button type="submit" class="submit-btn">Place Order</button>
1843                 <div class="form-message" id="orderMessage"></div>
1844             </form>
1845         </div>
1846     </div>
1847
1848     <script src="https://unpkg.com/aos@2.3.1/dist/aos.js"></script>
1849     <script>
1850         AOS.init({{ duration: 700, easing: 'ease-out', once: true, offset: 80 }});
1851
1852         const API = window.location.origin || 'http://localhost:8000';
1853         const SESSION = '{website_session_id}';
1854
1855         // Filter products
1856         function filterProducts(category, btn) {{
1857             document.querySelectorAll('.filter-btn').forEach(b =>
b.classList.remove('active'));
1858             btn.classList.add('active');
1859             document.querySelectorAll('.product-card').forEach(card => {{
1860                 if (category === 'all' || card.dataset.category === category) {{
1861                     card.style.display = '';
1862                 }} else {{
1863                     card.style.display = 'none';
1864                 }}
1865             }});
1866         }}
1867
1868         // Cart
1869         let cart = [];
1870         function addToCart(btn) {{
1871             const name = btn.dataset.name;
1872             const price = parseFloat(btn.dataset.price);
1873             const existing = cart.find(i => i.name === name);
1874             if (existing) {{ existing.qty++; }}
1875             else {{ cart.push({{ name, price, qty: 1 }}); }}
1876             updateCart();
1877             btn.textContent = '? Added';
```

```

1878         setTimeout(() => {{ btn.textContent = 'Add to Cart'; }}, 1200);
1879     }}
1880
1881     function updateCart() {{
1882         const itemsEl = document.getElementById('cartItems');
1883         const totalEl = document.getElementById('cartTotal');
1884         const badge = document.getElementById('cartBadge');
1885         const count = cart.reduce((s, i) => s + i.qty, 0);
1886         badge.style.display = count > 0 ? 'flex' : 'none';
1887         badge.textContent = count;
1888
1889         if (cart.length === 0) {{
1890             itemsEl.innerHTML = '<p class="cart-empty">Your bag is empty</p>';
1891             totalEl.textContent = '0';
1892         }} else {{
1893             let total = 0;
1894             itemsEl.innerHTML = cart.map((item, idx) => {{
1895                 total += item.price * item.qty;
1896
1897                 return `<div
1898 class="cart-item"><div><strong>${{item.name}}</strong><br><span>?${{item.price}}
1899 ${{item.qty}}</span></div><div
1900 class="cart-item-actions"><button
1901 onclick="changeQty(${{idx}}, -1)">?</button><span>${{item.qty}}</span><button
1902 onclick="changeQty(${{idx}}, 1)">+</button><button
1903 onclick="removeItem(${{idx}})"
1904 style="color:#e53935;border-color:#e53935">?</button></div></div>`;
1905             }}).join('');
1906             totalEl.textContent = total.toLocaleString();
1907         }}
1908     }}
1909
1910     function changeQty(idx, d) {{ cart[idx].qty += d; if (cart[idx].qty <= 0)
1911 cart.splice(idx, 1); updateCart(); }}
1912     function removeItem(idx) {{ cart.splice(idx, 1); updateCart(); }}
1913
1914     function toggleCart() {{
1915         document.getElementById('cartSidebar').classList.toggle('open');
1916         document.getElementById('cartOverlay').classList.toggle('open');
1917     }}
1918
1919     function showCheckout() {{
1920         if (!cart.length) return;
1921         const summary = cart.map(i => `${{i.name}} x${{i.qty}} =
1922 ?${{(i.price*i.qty).toLocaleString()}}`).join('<br>');
1923         const total = cart.reduce((s, i) => s + i.price * i.qty, 0);
1924         document.getElementById('orderSummary').innerHTML = `${{summary}}<br><strong
1925 style="margin-top:0.5rem;display:block">Total: ?${{total.toLocaleString()}}</strong>`;
1926         document.getElementById('checkoutModal').classList.add('open');
1927         toggleCart();
1928     }}
1929
1930     function closeCheckout() {{
1931         document.getElementById('checkoutModal').classList.remove('open'); }}
1932
1933     async function submitOrder(e) {{
1934         e.preventDefault();

```

```
1923         const form = e.target;
1924         const fd = Object.fromEntries(new FormData(form));
1925         fd.items = cart.map(i => ({name: i.name, price: i.price, qty: i.qty}));
1926         const msg = document.getElementById('orderMessage');
1927         try {{
1928             const res = await fetch(`${API}/api/site/${SESSION}/order`, {{
1929                 method: 'POST', headers: {{'Content-Type': 'application/json'}}, body:
JSON.stringify(fd)
1930             }});
1931             if (res.ok) {{
1932                 msg.innerHTML = '<div
style="color:#22c55e;padding:1rem;text-align:center">? Order placed! We will contact you
shortly.</div>';
1933                 cart = []; updateCart(); form.reset();
1934                 setTimeout(closeCheckout, 3000);
1935             }} else throw new Error();
1936             }} catch {{ msg.innerHTML = '<div
style="color:#e53935;padding:1rem;text-align:center">Could not place order. Try
again.</div>'; }}
1937         }}
1938
1939         // Contact form
1940         document.getElementById('contactForm')?.addEventListener('submit', async (e) => {{
1941             e.preventDefault();
1942             const form = e.target;
1943             const data = Object.fromEntries(new FormData(form));
1944             const msg = document.getElementById('contactMessage');
1945             try {{
1946                 const res = await fetch(`${API}/api/site/${SESSION}/contact`, {{
1947                     method: 'POST', headers: {{'Content-Type': 'application/json'}}, body:
JSON.stringify(data)
1948                 }});
1949                 if (res.ok) {{
1950                     msg.innerHTML = '<div
style="color:#22c55e;padding:1rem;background:#f0fdf4;border-radius:8px;margin-top:1rem;text-a
lign:center">? Message sent! We\\'ll get back to you soon.</div>';
1951                     form.reset();
1952                 }} else throw new Error();
1953             }} catch {{ msg.innerHTML = '<div style="color:#e53935;padding:1rem">Could not
send message.</div>'; }}
1954             }});
1955         </script>
1956     </body>
1957 </html>'
1958
1959     return html
1960
1961
1962 def generate_restaurant_website(analysis, colors, images, features, session):
1963     """Placeholder - falls back to generic template for now"""
1964     # Will be specialized later, use generic for now
1965     return None
1966
1967
```

```
1968 # =====
1969 # FULL WEBSITE GENERATOR (with functional features)
1970 # =====
1971
1972 def generate_full_website(analysis, colors, images, features, session):
1973     """Generate a complete, functional website with booking/ordering/contact"""
1974
1975     name = analysis.get('business_name', 'My Business')
1976     business_type = analysis.get('business_type', 'business')
1977     services = analysis.get('services', [])
1978     hero_headline = analysis.get('hero_headline', f'Welcome to {name}')
1979     hero_subtext = analysis.get('hero_subtext', 'Quality service you can trust')
1980     tagline = analysis.get('tagline', 'Excellence in Every Detail')
1981     about_text = analysis.get('about_text', f'{name} provides exceptional services.')
1982     cta_text = analysis.get('cta_text', 'Get Started')
1983     location = analysis.get('location', {})
1984     city = location.get('city', 'City')
1985     country = location.get('country', 'Country')
1986
1987     website_session_id = session.get("website_session_id") or "preview"
1988
1989     # Use specialized template for shop/e-commerce
1990     if business_type in ["shop", "store", "ecommerce", "boutique"]:
1991         return generate_ecommerce_website(analysis, colors, images, features, session)
1992
1993     # Build service cards HTML
1994     service_cards = ""
1995     for i, service in enumerate(services[:6]):
1996         img = images['services'][i] if i < len(images['services']) else images['services'][0]
1997         if images['services'] else ""
1998         delay = i * 100
1999         service_cards += f'''
2000             <div class="service-card" data-aos="fade-up" data-aos-delay="{delay}">
2001                 <div class="service-image-wrapper">
2002                     
2003                     <div class="service-overlay"></div>
2004                 </div>
2005                 <div class="service-content">
2006                     <h3>{service}</h3>
2007                     <p>Professional {service.lower()} delivered with excellence and care</p>
2008                     <a href="#contact" class="service-link">Learn More ?</a>
2009                 </div>
2010             </div>'''
2011
2012     # Build gallery HTML
2013     gallery_items = ""
2014     for i, img in enumerate(images.get('gallery', [])):
2015         delay = i * 150
2016         gallery_items += f'''
2017             <div class="gallery-item" data-aos="zoom-in" data-aos-delay="{delay}">
2018                 
2019                 <div class="gallery-overlay"><span>?</span></div>
2020             </div>'''
```

```
2021     # Build booking section if needed
2022     booking_section = ""
2023     if "booking" in features:
2024         booking_section = f'''
2025         <section class="booking-section" id="booking">
2026             <div class="section-container">
2027                 <h2 class="section-title" data-aos="fade-up">Book an Appointment</h2>
2028                 <p class="section-subtitle" data-aos="fade-up" data-aos-delay="100">Choose your
service and preferred time</p>
2029                 <form class="booking-form" id="bookingForm" data-aos="fade-up"
data-aos-delay="200">
2030                     <div class="form-grid">
2031                         <input type="text" name="name" placeholder="Your Name" required>
2032                         <input type="email" name="email" placeholder="Email Address" required>
2033                         <input type="tel" name="phone" placeholder="Phone Number" required>
2034                         <select name="service" required>
2035                             <option value="">Select Service</option>
2036                             { ''.join(f'<option value="{s}">{s}</option>' for s in services)}
2037                         </select>
2038                         <input type="date" name="date" required>
2039                         <select name="time" required>
2040                             <option value="">Select Time</option>
2041                             <option value="09:00">9:00 AM</option>
2042                             <option value="10:00">10:00 AM</option>
2043                             <option value="11:00">11:00 AM</option>
2044                             <option value="12:00">12:00 PM</option>
2045                             <option value="13:00">1:00 PM</option>
2046                             <option value="14:00">2:00 PM</option>
2047                             <option value="15:00">3:00 PM</option>
2048                             <option value="16:00">4:00 PM</option>
2049                             <option value="17:00">5:00 PM</option>
2050                         </select>
2051                     </div>
2052                     <textarea name="notes" placeholder="Any special requests?"
rows="3"></textarea>
2053                     <button type="submit" class="submit-btn">Book Now</button>
2054                     <div class="form-message" id="bookingMessage"></div>
2055                 </form>
2056             </div>
2057         </section>'''
2058
2059     # Build ordering/menu section if needed
2060     menu_section = ""
2061     if "menu" in features or "ordering" in features:
2062         menu_items_html = ""
2063         for i, service in enumerate(services[:8]):
2064             price = random.randint(5, 50) if business_type in ["restaurant", "cafe"] else
random.randint(20, 200)
2065             menu_items_html += f'''
2066             <div class="menu-item" data-aos="fade-up" data-aos-delay="{i*80}">
2067                 
2068                 <div class="menu-item-content">
2069                     <h4>{service}</h4>
```

```

2070         <p>Premium quality {service.lower()}</p>
2071         <div class="menu-item-footer">
2072             <span class="price">${price}</span>
2073             {"<button class='add-to-cart-btn' onclick='addToCart(this)'
data-name='" + service + "' data-price='" + str(price) + "'>Add to Cart</button>" if
"ordering" in features else ""}
2074         </div>
2075     </div>
2076 </div>'''
2077
2078     menu_section = f'''
2079     <section class="menu-section" id="menu">
2080         <div class="section-container">
2081             <h2 class="section-title" data-aos="fade-up">{"Our Menu" if business_type in
["restaurant", "cafe"] else "Our Products"></h2>
2082             <p class="section-subtitle" data-aos="fade-up" data-aos-delay="100">{tagline}</p>
2083             <div class="menu-grid">{menu_items_html}</div>
2084         </div>
2085     </section>'''
2086
2087     # Cart section for ordering
2088     cart_section = ""
2089     if "ordering" in features:
2090         cart_section = f'''
2091         <div class="cart-sidebar" id="cartSidebar">
2092             <div class="cart-header">
2093                 <h3>Your Cart</h3>
2094                 <button class="cart-close" onclick="toggleCart()">?</button>
2095             </div>
2096             <div class="cart-items" id="cartItems">
2097                 <p class="cart-empty">Your cart is empty</p>
2098             </div>
2099             <div class="cart-footer">
2100                 <div class="cart-total">Total: $<span id="cartTotal">0</span></div>
2101                 <button class="checkout-btn" onclick="showCheckout()">Proceed to
Checkout</button>
2102             </div>
2103         </div>
2104         <button class="cart-fab" id="cartFab" onclick="toggleCart()" style="display:none">
? <span id="cartCount">0</span>
2105         </button>
2106
2107         <div class="checkout-modal" id="checkoutModal">
2108             <div class="checkout-content">
2109                 <h3>Checkout</h3>
2110                 <button class="modal-close" onclick="closeCheckout()">?</button>
2111                 <form id="orderForm" onsubmit="submitOrder(event)">
2112                     <input type="text" name="name" placeholder="Your Name" required>
2113                     <input type="email" name="email" placeholder="Email" required>
2114                     <input type="tel" name="phone" placeholder="Phone" required>
2115                     <textarea name="address" placeholder="Delivery Address" required></textarea>
2116                     <textarea name="notes" placeholder="Special instructions
(optional)"></textarea>
2117                 <div class="order-summary" id="orderSummary"></div>

```

```
2119         <button type="submit" class="submit-btn">Place Order</button>
2120         <div class="form-message" id="orderMessage"></div>
2121     </form>
2122 </div>
2123 </div>'''
2124
2125 # Contact section
2126 contact_section = f'''
2127 <section class="contact" id="contact">
2128     <div class="section-container">
2129         <h2 data-aos="fade-up">Get In Touch</h2>
2130         <p data-aos="fade-up" data-aos-delay="100">We'd love to hear from you!</p>
2131         <form class="contact-form" id="contactForm" data-aos="fade-up"
data-aos-delay="200">
2132             <input type="text" name="name" placeholder="Your Name" required>
2133             <input type="email" name="email" placeholder="Email Address" required>
2134             <input type="tel" name="phone" placeholder="Phone Number">
2135             <textarea name="message" placeholder="Your message..." required
rows="4"></textarea>
2136             <button type="submit" class="submit-btn">{cta_text}</button>
2137             <div class="form-message" id="contactMessage"></div>
2138         </form>
2139     </div>
2140 </section>'''
2141
2142 # Navigation links
2143 nav_links = '<a href="#home">Home</a>'
2144 nav_links += '<a href="#services">Services</a>'
2145 if menu_section:
2146     nav_links += '<a href="#menu">Menu</a>'
2147 if booking_section:
2148     nav_links += '<a href="#booking">Book Now</a>'
2149 nav_links += '<a href="#about">About</a>'
2150 nav_links += '<a href="#gallery">Gallery</a>'
2151 nav_links += f'<a href="#contact" class="cta-btn">{cta_text}</a>'
2152
2153 # JavaScript for functional features
2154 js_code = f'''
2155 <script>
2156     // AOS Init
2157     AOS.init({{ duration: 800, easing: 'ease-out-cubic', once: true, offset: 100 }});
2158
2159     // Navbar scroll
2160     window.addEventListener('scroll', () => {{
2161         const nav = document.getElementById('navbar');
2162         if (window.scrollY > 50) nav?.classList.add('scrolled');
2163         else nav?.classList.remove('scrolled');
2164     }});
2165
2166     // Mobile menu
2167     function toggleMenu() {{
2168         document.querySelector('.hamburger').classList.toggle('active');
2169         document.getElementById('navLinks').classList.toggle('active');
2170     }}
```

```
2171     document.querySelectorAll('.nav-links a').forEach(link => {{
2172         link.addEventListener('click', () => {{
2173             document.getElementById('navLinks').classList.remove('active');
2174             document.querySelector('.hamburger')?.classList.remove('active');
2175         }});
2176     }});
2177
2178     const API = window.location.origin || 'http://localhost:8000';
2179     const SESSION = '{website_session_id}';
2180     '''
2181
2182     # Booking JS
2183     if "booking" in features:
2184         js_code += '''
2185         // Booking
2186         document.getElementById('bookingForm')?.addEventListener('submit', async (e) => {
2187             e.preventDefault();
2188             const form = e.target;
2189             const data = Object.fromEntries(new FormData(form));
2190             const msgEl = document.getElementById('bookingMessage');
2191             try {
2192                 const res = await fetch(`${API}/api/site/${SESSION}/booking`, {
2193                     method: 'POST',
2194                     headers: {'Content-Type': 'application/json'},
2195                     body: JSON.stringify(data)
2196                 });
2197                 if (res.ok) {
2198
2199                                     msgEl.innerHTML = '<div
2200                                     style="color:#22c55e;padding:1rem;background:rgba(34,197,94,0.1);border-radius:10px;margin-to
2201                                     p:1rem;">? Booking confirmed! We\\'ll contact you shortly.</div>';
2202                                     form.reset();
2203                                 } else {
2204                                     throw new Error('Failed');
2205                                 }
2206                             } catch (err) {
2207
2208                                     msgEl.innerHTML = '<div
2209                                     style="color:#ef4444;padding:1rem;background:rgba(239,68,68,0.1);border-radius:10px;margin-to
2210                                     p:1rem;">? Could not submit booking. Please try again.</div>';
2211                                 }
2212                             });
2213             '''
2214
2215     # Ordering/Cart JS
2216     if "ordering" in features:
2217         js_code += '''
2218         // Cart
2219         let cart = [];
2220
2221         function addToCart(btn) {
2222             const name = btn.dataset.name;
2223             const price = parseFloat(btn.dataset.price);
2224             const existing = cart.find(i => i.name === name);
2225             if (existing) { existing.qty++; }
2226             else { cart.push({ name, price, qty: 1 }); }
```

```
2221         updateCart();
2222         btn.textContent = '? Added';
2223         setTimeout(() => { btn.textContent = 'Add to Cart'; }, 1000);
2224     }
2225
2226     function updateCart() {
2227         const itemsEl = document.getElementById('cartItems');
2228         const totalEl = document.getElementById('cartTotal');
2229         const countEl = document.getElementById('cartCount');
2230         const fab = document.getElementById('cartFab');
2231
2232         if (cart.length === 0) {
2233             itemsEl.innerHTML = '<p class="cart-empty">Your cart is empty</p>';
2234             fab.style.display = 'none';
2235         } else {
2236             fab.style.display = 'flex';
2237             countEl.textContent = cart.reduce((sum, i) => sum + i.qty, 0);
2238             let total = 0;
2239             itemsEl.innerHTML = cart.map((item, idx) => {
2240                 total += item.price * item.qty;
2241                 return `<div class="cart-item">
2242                     <div>
2243                         <strong>${item.name}</strong>
2244                         <span>${item.price} x ${item.qty}</span>
2245                     </div>
2246                     <div class="cart-item-actions">
2247                         <button onclick="changeQty(${idx}, -1)">-</button>
2248                         <span>${item.qty}</span>
2249                         <button onclick="changeQty(${idx}, 1)">+</button>
2250                         <button onclick="removeFromCart(${idx})"
2251                             style="color:#ef4444">?</button>
2252                     </div>
2253                 `;
2254             }).join('');
2255             totalEl.textContent = total.toFixed(2);
2256         }
2257
2258         function changeQty(idx, delta) {
2259             cart[idx].qty += delta;
2260             if (cart[idx].qty <= 0) cart.splice(idx, 1);
2261             updateCart();
2262         }
2263
2264         function removeFromCart(idx) {
2265             cart.splice(idx, 1);
2266             updateCart();
2267         }
2268
2269         function toggleCart() {
2270             document.getElementById('cartSidebar').classList.toggle('open');
2271         }
2272
2273         function showCheckout() {
```

```
2274         if (cart.length === 0) return alert('Cart is empty!');
2275         const summary = cart.map(i => `${i.name} x${i.qty} =
    `${(i.price*i.qty).toFixed(2)}`).join('<br>');
2276         const total = cart.reduce((sum, i) => sum + i.price * i.qty, 0);
2277         document.getElementById('orderSummary').innerHTML = `<div
    style="padding:1rem;background:rgba(255,255,255,0.05);border-radius:10px;margin:1rem
    0;">${summary}<br><strong>Total: ${total.toFixed(2)}</strong></div>`;
2278         document.getElementById('checkoutModal').classList.add('open');
2279         document.getElementById('cartSidebar').classList.remove('open');
2280     }
2281
2282     function closeCheckout() {
2283         document.getElementById('checkoutModal').classList.remove('open');
2284     }
2285
2286     async function submitOrder(e) {
2287         e.preventDefault();
2288         const form = e.target;
2289         const formData = Object.fromEntries(new FormData(form));
2290         formData.items = cart.map(i => ({name: i.name, price: i.price, qty: i.qty}));
2291         const msgEl = document.getElementById('orderMessage');
2292         try {
2293             const res = await fetch(`${API}/api/site/${SESSION}/order`, {
2294                 method: 'POST',
2295                 headers: {'Content-Type': 'application/json'},
2296                 body: JSON.stringify(formData)
2297             });
2298             if (res.ok) {
2299                 msgEl.innerHTML = '<div style="color:#22c55e;padding:1rem;">? Order
    placed successfully! We\\'ll contact you with details.</div>';
2300                 cart = [];
2301                 updateCart();
2302                 form.reset();
2303                 setTimeout(closeCheckout, 3000);
2304             } else { throw new Error('Failed'); }
2305         } catch (err) {
2306             msgEl.innerHTML = '<div style="color:#ef4444;padding:1rem;">? Could not place
    order. Please try again.</div>';
2307         }
2308     }
2309     '''
2310
2311     # Contact JS
2312     js_code += '''
2313         // Contact
2314         document.getElementById('contactForm')?.addEventListener('submit', async (e) => {
2315             e.preventDefault();
2316             const form = e.target;
2317             const data = Object.fromEntries(new FormData(form));
2318             const msgEl = document.getElementById('contactMessage');
2319             try {
2320                 const res = await fetch(`${API}/api/site/${SESSION}/contact`, {
2321                     method: 'POST',
2322                     headers: {'Content-Type': 'application/json'},
```

```

2323         body: JSON.stringify(data)
2324     });
2325     if (res.ok) {
2326         msgEl.innerHTML = '<div
style="color:#22c55e;padding:1rem;background:rgba(34,197,94,0.1);border-radius:10px;margin-to
p:1rem;">? Message sent! We\\'ll get back to you soon.</div>';
2327         form.reset();
2328     } else { throw new Error('Failed'); }
2329 } catch (err) {
2330     msgEl.innerHTML = '<div
style="color:#ef4444;padding:1rem;background:rgba(239,68,68,0.1);border-radius:10px;margin-to
p:1rem;">? Could not send message. Please try again.</div>';
2331 }
2332 });
2333 </script>'''
2334
2335 # ===== FULL HTML =====
2336 html = f'''<!DOCTYPE html>
2337 <html lang="en">
2338 <head>
2339     <meta charset="UTF-8">
2340     <meta name="viewport" content="width=device-width, initial-scale=1.0">
2341     <title>{name} | {tagline}</title>
2342     <meta name="description" content="{about_text[:150]}">
2343     <link rel="preconnect" href="https://fonts.googleapis.com">
2344         <link
href="https://fonts.googleapis.com/css2?family=Inter:wght@300;400;500;600;700;800;900&family=
Playfair+Display:wght@700;800;900&display=swap" rel="stylesheet">
2345     <link href="https://unpkg.com/aos@2.3.1/dist/aos.css" rel="stylesheet">
2346     <style>
2347         * {{ margin: 0; padding: 0; box-sizing: border-box; }}
2348         :root {{
2349             --primary: {colors['primary']};
2350             --secondary: {colors['secondary']};
2351             --accent: {colors['accent']};
2352             --text: {colors['text']};
2353             --gradient-1: linear-gradient(135deg, {colors['primary']} 0%,
{colors['secondary']} 100%);
2354             --shadow-sm: 0 2px 10px rgba(0,0,0,0.05);
2355             --shadow-md: 0 8px 30px rgba(0,0,0,0.12);
2356             --shadow-lg: 0 20px 60px rgba(0,0,0,0.15);
2357             --shadow-xl: 0 30px 90px rgba(0,0,0,0.2);
2358         }}
2359         html {{ scroll-behavior: smooth; scroll-padding-top: 100px; }}
2360         body {{ font-family: 'Inter', -apple-system, sans-serif; color: var(--text);
line-height: 1.7; overflow-x: hidden; background: #fff; }}
2361
2362         /* NAV */
2363         .nav {{ position: fixed; top: 0; width: 100%; background: rgba(255,255,255,0.95);
backdrop-filter: blur(20px); padding: 1.2rem 5%; display: flex; justify-content:
space-between; align-items: center; z-index: 10000; box-shadow: 0 4px 30px rgba(0,0,0,0.08);
transition: all 0.3s; }}
2364         .nav.scrolled {{ padding: 0.8rem 5%; }}
2365         .logo {{ font-size: 1.6rem; font-weight: 900; background: var(--gradient-1);

```

```
-webkit-background-clip: text; -webkit-text-fill-color: transparent; background-clip: text;
}}
2366     .nav-links {{ display: flex; gap: 2.5rem; align-items: center; }}
2367     .nav-links a {{ text-decoration: none; color: var(--text); font-weight: 600;
font-size: 0.95rem; transition: color 0.3s; }}
2368     .nav-links a:hover {{ color: var(--primary); }}
2369     .cta-btn {{ background: var(--primary); color: white !important; padding: 0.75rem
2rem; border-radius: 50px; font-weight: 700; transition: all 0.3s; box-shadow: 0 4px 15px
rgba(0,0,0,0.1); }}
2370     .cta-btn:hover {{ transform: translateY(-2px); box-shadow: 0 8px 25px
rgba(0,0,0,0.2); }}
2371     .hamburger {{ display: none; flex-direction: column; gap: 5px; cursor: pointer;
z-index: 1001; }}
2372     .hamburger span {{ width: 25px; height: 3px; background: var(--text); border-radius:
3px; transition: all 0.3s; }}
2373     .hamburger.active span:nth-child(1) {{ transform: rotate(45deg) translate(8px, 8px);
}}
2374     .hamburger.active span:nth-child(2) {{ opacity: 0; }}
2375     .hamburger.active span:nth-child(3) {{ transform: rotate(-45deg) translate(7px,
-7px); }}
2376
2377     /* HERO */
2378     .hero {{ position: relative; min-height: 100vh; display: flex; align-items: center;
justify-content: center; text-align: center; color: white; padding: 120px 5% 2rem; overflow:
hidden; }}
2379     .hero::before {{ content: ''; position: absolute; inset: 0; background:
linear-gradient(135deg, rgba(0,0,0,0.5), rgba(0,0,0,0.3)); z-index: 1; }}
2380     .hero-bg {{ position: absolute; inset: 0; width: 100%; height: 100%; object-fit:
cover; z-index: 0; }}
2381     .hero-content {{ position: relative; z-index: 2; max-width: 1000px; }}
2382     .hero h1 {{ font-family: 'Playfair Display', serif; font-size: clamp(2.5rem, 8vw,
5.5rem); font-weight: 900; margin-bottom: 1.5rem; line-height: 1.1; letter-spacing: -2px;
animation: fadeInUp 0.8s ease-out; }}
2383     .hero p {{ font-size: clamp(1.1rem, 2.5vw, 1.6rem); margin-bottom: 3rem; opacity:
0.95; max-width: 700px; margin-left: auto; margin-right: auto; animation: fadeInUp 0.8s
ease-out 0.2s both; }}
2384     .hero-cta {{ display: flex; gap: 1.5rem; justify-content: center; flex-wrap: wrap;
animation: fadeInUp 0.8s ease-out 0.4s both; }}
2385     @keyframes fadeInUp {{ from {{ opacity: 0; transform: translateY(30px); }} to {{
opacity: 1; transform: translateY(0); }} }}
2386     .btn {{ padding: 1.2rem 3rem; border-radius: 50px; font-weight: 700; font-size:
1.05rem; text-decoration: none; transition: all 0.3s; cursor: pointer; border: none; display:
inline-block; }}
2387     .btn-primary {{ background: white; color: var(--primary); box-shadow: 0 8px 30px
rgba(255,255,255,0.3); }}
2388     .btn-primary:hover {{ transform: translateY(-3px); box-shadow: 0 15px 50px
rgba(255,255,255,0.4); }}
2389     .btn-secondary {{ background: rgba(255,255,255,0.1); color: white; border: 2px solid
rgba(255,255,255,0.8); backdrop-filter: blur(10px); }}
2390     .btn-secondary:hover {{ background: white; color: var(--primary); }}
2391
2392     /* SECTIONS */
2393     .section-container {{ max-width: 1400px; margin: 0 auto; }}
2394     .section-title {{ text-align: center; font-size: clamp(2.5rem, 6vw, 4rem);
```

```
font-weight: 900; margin-bottom: 1rem; letter-spacing: -1px; background: var(--gradient-1);
-webkit-background-clip: text; -webkit-text-fill-color: transparent; background-clip: text;
}}
2395     .section-subtitle {{ text-align: center; font-size: clamp(1.1rem, 2vw, 1.4rem);
color: #64748b; margin-bottom: 4rem; font-weight: 500; }}
2396
2397     /* SERVICES */
2398     .services {{ padding: 100px 5%; background: linear-gradient(180deg, #fff, #f8f9fb,
#fff); }}
2399     .services-grid {{ display: grid; grid-template-columns: repeat(auto-fit,
minmax(300px, 1fr)); gap: 2.5rem; max-width: 1400px; margin: 0 auto; }}
2400     .service-card {{ background: white; border-radius: 24px; overflow: hidden;
box-shadow: var(--shadow-md); transition: all 0.4s; border: 1px solid rgba(0,0,0,0.05); }}
2401     .service-card:hover {{ transform: translateY(-12px); box-shadow: var(--shadow-xl); }}
2402     .service-image-wrapper {{ position: relative; height: 240px; overflow: hidden; }}
2403     .service-image {{ width: 100%; height: 100%; object-fit: cover; transition: transform
0.5s; }}
2404     .service-card:hover .service-image {{ transform: scale(1.1); }}
2405     .service-overlay {{ position: absolute; inset: 0; background: linear-gradient(180deg,
transparent, rgba(0,0,0,0.3)); }}
2406     .service-content {{ padding: 2rem; }}
2407     .service-content h3 {{ font-size: 1.4rem; margin-bottom: 0.8rem; font-weight: 800; }}
2408     .service-content p {{ color: #64748b; font-size: 1rem; line-height: 1.7;
margin-bottom: 1.2rem; }}
2409     .service-link {{ color: var(--primary); text-decoration: none; font-weight: 700;
transition: all 0.3s; }}
2410
2411     /* MENU */
2412     .menu-section {{ padding: 100px 5%; background: #f8f9fb; }}
2413     .menu-grid {{ display: grid; grid-template-columns: repeat(auto-fit, minmax(280px,
1fr)); gap: 2rem; max-width: 1400px; margin: 0 auto; }}
2414     .menu-item {{ background: white; border-radius: 20px; overflow: hidden; box-shadow:
var(--shadow-sm); transition: all 0.3s; }}
2415     .menu-item:hover {{ transform: translateY(-5px); box-shadow: var(--shadow-md); }}
2416     .menu-item img {{ width: 100%; height: 200px; object-fit: cover; }}
2417     .menu-item-content {{ padding: 1.5rem; }}
2418     .menu-item-content h4 {{ font-size: 1.2rem; margin-bottom: 0.5rem; }}
2419     .menu-item-content p {{ color: #64748b; font-size: 0.9rem; margin-bottom: 1rem; }}
2420     .menu-item-footer {{ display: flex; justify-content: space-between; align-items:
center; }}
2421     .price {{ font-size: 1.4rem; font-weight: 800; color: var(--primary); }}
2422     .add-to-cart-btn {{ padding: 0.5rem 1.2rem; background: var(--primary); color: white;
border: none; border-radius: 25px; font-weight: 600; cursor: pointer; transition: all 0.2s;
font-size: 0.85rem; }}
2423     .add-to-cart-btn:hover {{ transform: scale(1.05); }}
2424
2425     /* BOOKING */
2426     .booking-section {{ padding: 100px 5%; background: white; }}
2427     .booking-form {{ max-width: 700px; margin: 0 auto; }}
2428     .form-grid {{ display: grid; grid-template-columns: 1fr 1fr; gap: 1rem;
margin-bottom: 1rem; }}
2429     .booking-form input, .booking-form select, .booking-form textarea {{ padding: 1rem
1.5rem; border: 2px solid #e0e0e0; border-radius: 12px; font-size: 1rem; font-family:
inherit; transition: border-color 0.3s; width: 100%; }}
```

```
2430     .booking-form input:focus, .booking-form select:focus, .booking-form textarea:focus
    {{ outline: none; border-color: var(--primary); }}

2431
2432     /* ABOUT */
2433     .about {{ padding: 100px 5%; background: var(--gradient-1); color: white; position:
    relative; overflow: hidden; }}
2434     .about::before {{ content: ''; position: absolute; width: 500px; height: 500px;
    background: rgba(255,255,255,0.05); border-radius: 50%; top: -250px; right: -250px; }}
2435     .about-grid {{ display: grid; grid-template-columns: 1fr 1fr; gap: 5rem; max-width:
    1400px; margin: 0 auto; align-items: center; position: relative; z-index: 1; }}
2436     .about-content h2 {{ font-family: 'Playfair Display', serif; font-size: clamp(2.5rem,
    5vw, 4rem); margin-bottom: 2rem; font-weight: 900; line-height: 1.2; }}
2437     .about-content p {{ font-size: 1.15rem; line-height: 1.9; opacity: 0.95;
    margin-bottom: 1.5rem; }}
2438     .about-image {{ height: 500px; border-radius: 30px; overflow: hidden; box-shadow: 0
    30px 90px rgba(0,0,0,0.3); }}
2439     .about-image img {{ width: 100%; height: 100%; object-fit: cover; }}
2440
2441     /* GALLERY */
2442     .gallery {{ padding: 100px 5%; background: #fff; }}
2443     .gallery-grid {{ display: grid; grid-template-columns: repeat(auto-fit, minmax(320px,
    1fr)); gap: 2rem; max-width: 1400px; margin: 3rem auto 0; }}
2444     .gallery-item {{ position: relative; height: 350px; border-radius: 24px; overflow:
    hidden; cursor: pointer; box-shadow: var(--shadow-md); transition: all 0.4s; }}
2445     .gallery-item img {{ width: 100%; height: 100%; object-fit: cover; transition:
    transform 0.5s; }}
2446     .gallery-item:hover img {{ transform: scale(1.1); }}
2447     .gallery-overlay {{ position: absolute; inset: 0; background: rgba(0,0,0,0.6);
    display: flex; align-items: center; justify-content: center; opacity: 0; transition: opacity
    0.3s; font-size: 3rem; }}
2448     .gallery-item:hover .gallery-overlay {{ opacity: 1; }}
2449
2450     /* CONTACT */
2451     .contact {{ padding: 100px 5%; background: linear-gradient(135deg, #1e293b, #0f172a);
    color: white; text-align: center; }}
2452     .contact h2 {{ font-size: clamp(2.5rem, 5vw, 4rem); margin-bottom: 1.5rem;
    font-weight: 900; }}
2453     .contact > .section-container > p {{ font-size: 1.3rem; margin-bottom: 3rem; opacity:
    0.9; }}
2454     .contact-form {{ max-width: 600px; margin: 0 auto; display: flex; flex-direction:
    column; gap: 1.2rem; }}
2455     .contact-form input, .contact-form textarea {{ padding: 1.2rem 1.5rem; border: 2px
    solid rgba(255,255,255,0.1); border-radius: 14px; font-size: 1rem; font-family: inherit;
    background: rgba(255,255,255,0.05); color: white; transition: all 0.3s; }}
2456     .contact-form input:focus, .contact-form textarea:focus {{ outline: none;
    border-color: var(--primary); background: rgba(255,255,255,0.1); }}
2457     .contact-form input::placeholder, .contact-form textarea::placeholder {{ color:
    rgba(255,255,255,0.4); }}
2458     .submit-btn {{ background: white; color: var(--primary); padding: 1.2rem 3rem;
    border: none; border-radius: 50px; font-size: 1.1rem; font-weight: 800; cursor: pointer;
    transition: all 0.3s; box-shadow: 0 10px 40px rgba(255,255,255,0.2); }}
2459     .submit-btn:hover {{ transform: translateY(-3px); box-shadow: 0 15px 50px
    rgba(255,255,255,0.3); }}
2460
```

```
2461      /* FOOTER */
2462      .footer {{ background: #0a0a0a; color: white; padding: 4rem 5% 2rem; }}
2463      .footer-content {{ display: grid; grid-template-columns: repeat(auto-fit,
minmax(250px, 1fr)); gap: 3rem; max-width: 1400px; margin: 0 auto 2rem; }}
2464      .footer-section h4 {{ font-size: 1.3rem; margin-bottom: 1rem; font-weight: 800; }}
2465      .footer-section p {{ opacity: 0.7; margin: 0.5rem 0; line-height: 1.8; }}
2466      .footer-section a {{ color: inherit; text-decoration: none; }}
2467      .footer-bottom {{ text-align: center; border-top: 1px solid rgba(255,255,255,0.1);
padding-top: 2rem; opacity: 0.5; }}
2468
2469      /* CART */
2470      .cart-sidebar {{ position: fixed; top: 0; right: -400px; width: 380px; height: 100vh;
background: white; box-shadow: -10px 0 40px rgba(0,0,0,0.15); z-index: 10001; transition:
right 0.3s; display: flex; flex-direction: column; color: #333; }}
2471      .cart-sidebar.open {{ right: 0; }}
2472      .cart-header {{ display: flex; justify-content: space-between; align-items: center;
padding: 1.5rem; border-bottom: 1px solid #eee; }}
2473      .cart-header h3 {{ font-size: 1.3rem; }}
2474      .cart-close {{ background: none; border: none; font-size: 1.5rem; cursor: pointer; }}
2475      .cart-items {{ flex: 1; overflow-y: auto; padding: 1rem; }}
2476      .cart-item {{ display: flex; justify-content: space-between; align-items: center;
padding: 1rem 0; border-bottom: 1px solid #f0f0f0; }}
2477      .cart-item-actions {{ display: flex; align-items: center; gap: 0.5rem; }}
2478      .cart-item-actions button {{ width: 28px; height: 28px; border: 1px solid #ddd;
border-radius: 50%; background: white; cursor: pointer; font-weight: bold; }}
2479      .cart-footer {{ padding: 1.5rem; border-top: 1px solid #eee; }}
2480      .cart-total {{ font-size: 1.3rem; font-weight: 800; margin-bottom: 1rem; }}
2481      .checkout-btn {{ width: 100%; padding: 1rem; background: var(--primary); color:
white; border: none; border-radius: 50px; font-size: 1rem; font-weight: 700; cursor: pointer;
}}
2482      .cart-fab {{ position: fixed; bottom: 2rem; right: 2rem; background: var(--primary);
color: white; border: none; border-radius: 50px; padding: 1rem 1.5rem; font-size: 1rem;
cursor: pointer; box-shadow: var(--shadow-lg); z-index: 10000; display: flex; align-items:
center; gap: 0.5rem; font-weight: 700; }}
2483      .cart-empty {{ text-align: center; color: #999; padding: 2rem; }}
2484
2485      /* CHECKOUT MODAL */
2486      .checkout-modal {{ position: fixed; inset: 0; background: rgba(0,0,0,0.5); z-index:
10002; display: none; align-items: center; justify-content: center; }}
2487      .checkout-modal.open {{ display: flex; }}
2488      .checkout-content {{ background: white; border-radius: 24px; padding: 2rem; width:
90%; max-width: 500px; max-height: 90vh; overflow-y: auto; color: #333; position: relative;
}}
2489      .checkout-content h3 {{ margin-bottom: 1.5rem; font-size: 1.4rem; }}
2490      .modal-close {{ position: absolute; top: 1rem; right: 1rem; background: none; border:
none; font-size: 1.3rem; cursor: pointer; }}
2491      .checkout-content input, .checkout-content textarea {{ width: 100%; padding: 0.8rem
1rem; border: 2px solid #e0e0e0; border-radius: 10px; margin-bottom: 0.8rem; font-family:
inherit; font-size: 0.95rem; }}
2492      .checkout-content input:focus, .checkout-content textarea:focus {{ outline: none;
border-color: var(--primary); }}
2493      .checkout-content .submit-btn {{ width: 100%; margin-top: 0.5rem; color: white;
background: var(--primary); }}
2494
```

```
2495     /* RESPONSIVE */
2496     @media (max-width: 968px) {{
2497         .about-grid {{ grid-template-columns: 1fr; gap: 3rem; }}
2498         .form-grid {{ grid-template-columns: 1fr; }}
2499     }}
2500     @media (max-width: 768px) {{
2501         .nav-links {{ position: fixed; top: 0; right: -100%; height: 100vh; width: 70%;
max-width: 400px; background: rgba(255,255,255,0.98); backdrop-filter: blur(20px);
flex-direction: column; padding: 6rem 2rem 2rem; box-shadow: -5px 0 30px rgba(0,0,0,0.1);
transition: right 0.4s; justify-content: flex-start; gap: 2rem; }}
2502         .nav-links.active {{ right: 0; }}
2503         .nav-links a {{ font-size: 1.2rem; }}
2504         .hamburger {{ display: flex; }}
2505         .cart-sidebar {{ width: 100%; }}
2506     }}
2507 </style>
2508 </head>
2509 <body>
2510     <!-- NAV -->
2511     <nav class="nav" id="navbar">
2512         <div class="logo">{name}</div>
2513                                     <div class="hamburger"
onclick="toggleMenu()"><span></span><span></span><span></span><span></span></div>
2514         <div class="nav-links" id="navLinks">{nav_links}</div>
2515     </nav>
2516
2517     <!-- HERO -->
2518     <section class="hero" id="home">
2519         
2520         <div class="hero-content">
2521             <h1>{hero_headline}</h1>
2522             <p>{hero_subtext}</p>
2523             <div class="hero-cta">
2524                 <a href="#{'booking' if 'booking' in features else 'contact'}" class="btn
btn-primary">{cta_text}</a>
2525                 <a href="#services" class="btn btn-secondary">Explore Services</a>
2526             </div>
2527         </div>
2528     </section>
2529
2530     <!-- SERVICES -->
2531     <section class="services" id="services">
2532         <div class="section-container">
2533             <h2 class="section-title" data-aos="fade-up">Our Services</h2>
2534             <p class="section-subtitle" data-aos="fade-up" data-aos-delay="100">{tagline}</p>
2535             <div class="services-grid">{service_cards}</div>
2536         </div>
2537     </section>
2538
2539     {menu_section}
2540
2541     {booking_section}
2542
2543     <!-- ABOUT -->
```

```
2544     <section class="about" id="about">
2545         <div class="about-grid">
2546             <div class="about-content" data-aos="fade-right">
2547                 <h2>About {name}</h2>
2548                 <p>{about_text}</p>
2549                 <p>Located in {city}, {country}, we bring excellence to everything we do.</p>
2550             </div>
2551             <div class="about-image" data-aos="fade-left">
2552                 
2553             </div>
2554         </div>
2555     </section>
2556
2557     <!-- GALLERY -->
2558     <section class="gallery" id="gallery">
2559         <div class="section-container">
2560             <h2 class="section-title" data-aos="fade-up">Gallery</h2>
2561             <p class="section-subtitle" data-aos="fade-up" data-aos-delay="100">See what
we've accomplished</p>
2562             <div class="gallery-grid">{gallery_items}</div>
2563         </div>
2564     </section>
2565
2566     {contact_section}
2567
2568     <!-- FOOTER -->
2569     <footer class="footer">
2570         <div class="footer-content">
2571             <div class="footer-section">
2572                 <h4>{name}</h4>
2573                 <p>{tagline}</p>
2574                 <p>{city}, {country}</p>
2575             </div>
2576             <div class="footer-section">
2577                 <h4>Quick Links</h4>
2578                 <p><a href="#home">Home</a></p>
2579                 <p><a href="#services">Services</a></p>
2580                 <p><a href="#about">About</a></p>
2581                 <p><a href="#contact">Contact</a></p>
2582             </div>
2583             <div class="footer-section">
2584                 <h4>Contact</h4>
2585                 <p>Email: info@{name.lower().replace(' ', '')}.com</p>
2586                 <p>Phone: +1 (555) 123-4567</p>
2587             </div>
2588         </div>
2589         <div class="footer-bottom">
2590             <p>&copy; 2025 {name}. All rights reserved.</p>
2591         </div>
2592     </footer>
2593
2594     {cart_section}
2595
2596     <script src="https://unpkg.com/aos@2.3.1/dist/aos.js"></script>
```

```
2597         {js_code}
2598     </body>
2599 </html>'''
2600
2601     return html
2602
2603
2604 # =====
2605 # FILE UPLOAD ENDPOINT
2606 # =====
2607
2608 @app.post("/api/upload")
2609 async def upload_file(file: UploadFile = File(...), session_id: str = Form(...)):
2610     """Upload a photo or video"""
2611     # Validate file type
2612     allowed = ["image/jpeg", "image/png", "image/webp", "image/gif", "video/mp4",
2613 "video/webm"]
2614     if file.content_type not in allowed:
2615         raise HTTPException(status_code=400, detail=f"File type {file.content_type} not
2616 allowed")
2617
2618     # Create session upload directory
2619     upload_dir = f"static/uploads/{session_id}"
2620     os.makedirs(upload_dir, exist_ok=True)
2621
2622     # Save file
2623     filename = f"{uuid.uuid4().hex[:8]}_{file.filename}"
2624     filepath = os.path.join(upload_dir, filename)
2625
2626     with open(filepath, "wb") as f:
2627         content = await file.read()
2628         if len(content) > 10 * 1024 * 1024: # 10MB limit
2629             raise HTTPException(status_code=400, detail="File too large (max 10MB)")
2630         f.write(content)
2631
2632     url = f"/static/uploads/{session_id}/{filename}"
2633     return {"url": url, "filename": filename}
2634
2635 # =====
2636 # SITE-SPECIFIC API ENDPOINTS (for generated websites)
2637 # =====
2638
2639 @app.post("/api/site/{session_id}/booking")
2640 async def handle_booking(session_id: str, request: BookingRequest):
2641     """Handle booking submission from generated website"""
2642     data_dir = f"site_data/{session_id}"
2643     os.makedirs(data_dir, exist_ok=True)
2644
2645     bookings_file = os.path.join(data_dir, "bookings.json")
2646     bookings = []
2647     if os.path.exists(bookings_file):
2648         with open(bookings_file, "r") as f:
2649             bookings = json.load(f)
```

```
2649
2650     booking = {
2651         "id": str(uuid.uuid4())[0:8],
2652         "timestamp": datetime.now().isoformat(),
2653         **request.dict()
2654     }
2655     bookings.append(booking)
2656
2657     with open(bookings_file, "w") as f:
2658         json.dump(bookings, f, indent=2)
2659
2660     return {"status": "success", "booking_id": booking["id"]}
2661
2662
2663 @app.post("/api/site/{session_id}/order")
2664 async def handle_order(session_id: str, request: OrderRequest):
2665     """Handle order submission from generated website"""
2666     data_dir = f"site_data/{session_id}"
2667     os.makedirs(data_dir, exist_ok=True)
2668
2669     orders_file = os.path.join(data_dir, "orders.json")
2670     orders = []
2671     if os.path.exists(orders_file):
2672         with open(orders_file, "r") as f:
2673             orders = json.load(f)
2674
2675     order = {
2676         "id": str(uuid.uuid4())[0:8],
2677         "timestamp": datetime.now().isoformat(),
2678         **request.dict()
2679     }
2680     orders.append(order)
2681
2682     with open(orders_file, "w") as f:
2683         json.dump(orders, f, indent=2)
2684
2685     return {"status": "success", "order_id": order["id"]}
2686
2687
2688 @app.post("/api/site/{session_id}/contact")
2689 async def handle_contact(session_id: str, request: ContactRequest):
2690     """Handle contact form submission from generated website"""
2691     data_dir = f"site_data/{session_id}"
2692     os.makedirs(data_dir, exist_ok=True)
2693
2694     contacts_file = os.path.join(data_dir, "contacts.json")
2695     contacts = []
2696     if os.path.exists(contacts_file):
2697         with open(contacts_file, "r") as f:
2698             contacts = json.load(f)
2699
2700     contact = {
2701         "id": str(uuid.uuid4())[0:8],
2702         "timestamp": datetime.now().isoformat(),
```

```
2703         **request.dict()
2704     }
2705     contacts.append(contact)
2706
2707     with open(contacts_file, "w") as f:
2708         json.dump(contacts, f, indent=2)
2709
2710     return {"status": "success", "message_id": contact["id"]}
2711
2712
2713 @app.get("/api/site/{session_id}/admin")
2714 async def admin_panel(session_id: str):
2715     """Simple admin panel to view bookings/orders/contacts"""
2716     data_dir = f"site_data/{session_id}"
2717
2718     bookings = []
2719     orders = []
2720     contacts = []
2721
2722     if os.path.exists(f"{data_dir}/bookings.json"):
2723         with open(f"{data_dir}/bookings.json") as f:
2724             bookings = json.load(f)
2725     if os.path.exists(f"{data_dir}/orders.json"):
2726         with open(f"{data_dir}/orders.json") as f:
2727             orders = json.load(f)
2728     if os.path.exists(f"{data_dir}/contacts.json"):
2729         with open(f"{data_dir}/contacts.json") as f:
2730             contacts = json.load(f)
2731
2732     html = f'''<!DOCTYPE html>
2733 <html><head><title>Admin Panel</title>
2734 <style>
2735     body {{ font-family: 'Inter', sans-serif; padding: 2rem; background: #f5f5f5; }}
2736     h1 {{ color: #6366F1; margin-bottom: 2rem; }}
2737     h2 {{ color: #333; margin: 2rem 0 1rem; padding-bottom: 0.5rem; border-bottom: 2px solid
2738 #6366F1; }}
2739     table {{ width: 100%; border-collapse: collapse; background: white; border-radius: 12px;
2740 overflow: hidden; box-shadow: 0 2px 10px rgba(0,0,0,0.1); margin-bottom: 2rem; }}
2741     th {{ background: #6366F1; color: white; padding: 1rem; text-align: left; }}
2742     td {{ padding: 0.8rem 1rem; border-bottom: 1px solid #eee; }}
2743     tr:hover td {{ background: #f8f9ff; }}
2744     .badge {{ padding: 0.3rem 0.8rem; border-radius: 20px; font-size: 0.8rem; font-weight:
2745 600; }}
2746     .count {{ background: #6366F1; color: white; padding: 0.2rem 0.6rem; border-radius: 10px;
2747 font-size: 0.85rem; }}
2748     .empty {{ text-align: center; color: #999; padding: 2rem; }}
2749 </style></head>
2750 <body>
2751     <h1>Admin Panel</h1>
2752
2753     <h2>Bookings <span class="count">{len(bookings)}</span></h2>
2754
2755     {'<table><tr><th>Date</th><th>Name</th><th>Email</th><th>Phone</th><th>Service</th><th>Appt
2756 Date</th><th>Time</th></tr>'
2757
2758
2759
2760
2761
2762
2763
2764
2765
2766
2767
2768
2769
2770
2771
2772
2773
2774
2775
2776
2777
2778
2779
2780
2781
2782
2783
2784
2785
2786
2787
2788
2789
2790
2791
2792
2793
2794
2795
2796
2797
2798
2799
2800
2801
2802
2803
2804
2805
2806
2807
2808
2809
2810
2811
2812
2813
2814
2815
2816
2817
2818
2819
2820
2821
2822
2823
2824
2825
2826
2827
2828
2829
2830
2831
2832
2833
2834
2835
2836
2837
2838
2839
2840
2841
2842
2843
2844
2845
2846
2847
2848
2849
2850
2851
2852
2853
2854
2855
2856
2857
2858
2859
2860
2861
2862
2863
2864
2865
2866
2867
2868
2869
2870
2871
2872
2873
2874
2875
2876
2877
2878
2879
2880
2881
2882
2883
2884
2885
2886
2887
2888
2889
2890
2891
2892
2893
2894
2895
2896
2897
2898
2899
2900
2901
2902
2903
2904
2905
2906
2907
2908
2909
2910
2911
2912
2913
2914
2915
2916
2917
2918
2919
2920
2921
2922
2923
2924
2925
2926
2927
2928
2929
2930
2931
2932
2933
2934
2935
2936
2937
2938
2939
2940
2941
2942
2943
2944
2945
2946
2947
2948
2949
2950
2951
2952
2953
2954
2955
2956
2957
2958
2959
2960
2961
2962
2963
2964
2965
2966
2967
2968
2969
2970
2971
2972
2973
2974
2975
2976
2977
2978
2979
2980
2981
2982
2983
2984
2985
2986
2987
2988
2989
2990
2991
2992
2993
2994
2995
2996
2997
2998
2999
3000
```

+

```

        ''.join(f'<tr><td>{b.get("timestamp", "")[:10]}</td><td>{b.get("name", "")}</td><td>{b.get("email", "")}</td><td>{b.get("phone", "")}</td><td>{b.get("service", "")}</td><td>{b.get("date", "")}</td><td>{b.get("time", "")}</td></tr>' for b in bookings) + '</table>' if bookings else '<p class="empty">No bookings yet</p>'}

2751
2752     <h2>Orders <span class="count">{len(orders)}</span></h2>
2753
    {f'<table><tr><th>Date</th><th>Name</th><th>Email</th><th>Phone</th><th>Items</th><th>Address</th></tr>'
      +
      f'<tr><td>{o.get("timestamp", "")[:10]}</td><td>{o.get("name", "")}</td><td>{o.get("email", "")}</td><td>{o.get("phone", "")}</td><td>{o.get("items", [])}</td><td>{o.get("address", "")}</td></tr>' for o in orders) + '</table>' if
      orders else '<p class="empty">No orders yet</p>'}

2754
2755     <h2>Contact Messages <span class="count">{len(contacts)}</span></h2>
2756     {f'<table><tr><th>Date</th><th>Name</th><th>Email</th><th>Phone</th><th>Message</th></tr>'
      +
      f'<tr><td>{c.get("timestamp", "")[:10]}</td><td>{c.get("name", "")}</td><td>{c.get("email", "")}</td><td>{c.get("phone", "")}</td><td>{c.get("message", "")[:100]}</td></tr>' for c in
      contacts) + '</table>' if contacts else '<p class="empty">No messages yet</p>'}

2757 </body></html>'
2758
2759     return HTMLResponse(content=html)
2760
2761
2762 # =====
2763 # LEGACY API ENDPOINTS (keep for backwards compatibility)
2764 # =====
2765
2766 @app.post("/api/build")
2767 async def build_website(request: BuildRequest):
2768     """Legacy BUILD MODE"""
2769     try:
2770         session_id = str(uuid.uuid4())
2771         session = {
2772             "stage": "building",
2773             "business_type": "business",
2774             "business_name": None,
2775             "location": None,
2776             "description": request.description,
2777             "services": [],
2778             "uploaded_media": [],
2779             "features": ["contact", "gallery"],
2780             "style_vibe": "modern",
2781             "conversation_history": [],
2782             "website_session_id": session_id,
2783         }
2784         CHAT_SESSIONS[session_id] = session
2785
2786         result = build_website_from_chat(session)
2787
2788         return {
2789             "session_id": result["session_id"],
2790             "status": "success",

```

```
2791         "mode": "BUILD",
2792         "website_code": result["html"],
2793         "website_state": WEBSITE_STATES.get(result["session_id"], {}),
2794         "business_name": result["analysis"]["business_name"],
2795         "explanation": f"Generated website for {result['analysis']['business_name']}",
2796         "ai_model": "gemini-2.5-flash",
2797         "model_reasoning": "Fast and efficient for website generation"
2798     }
2799     except Exception as e:
2800         raise HTTPException(status_code=500, detail=str(e))
2801
2802
2803 @app.post("/api/edit")
2804 async def edit_website(request: EditRequest):
2805     """Legacy EDIT MODE"""
2806     try:
2807         if request.session_id not in CHAT_SESSIONS:
2808             # Load from state
2809             state_path = f"website_states/{request.session_id}.json"
2810             if not os.path.exists(state_path):
2811                 raise HTTPException(status_code=404, detail="Website not found")
2812             with open(state_path) as f:
2813                 state = json.load(f)
2814             CHAT_SESSIONS[request.session_id] = {
2815                 "stage": "review",
2816                 "business_type": state.get("analysis", {}).get("business_type", "business"),
2817                 "business_name": state.get("analysis", {}).get("business_name"),
2818                 "location": None,
2819                 "description": state.get("business_description", ""),
2820                 "services": state.get("analysis", {}).get("services", []),
2821                 "uploaded_media": [],
2822                 "features": state.get("features", ["contact"]),
2823                 "style_vibe": state.get("analysis", {}).get("vibe", "modern"),
2824                 "conversation_history": [],
2825                 "website_session_id": request.session_id,
2826             }
2827
2828             session = CHAT_SESSIONS[request.session_id]
2829             result = edit_website_from_chat(session, request.edit_request)
2830
2831             return {
2832                 "session_id": result["session_id"],
2833                 "status": "success",
2834                 "mode": "EDIT",
2835                 "website_code": result["html"],
2836                 "website_state": WEBSITE_STATES.get(result["session_id"], {}),
2837                 "business_name": result["analysis"]["business_name"],
2838                 "explanation": f"Updated website: {request.edit_request}",
2839                 "ai_model": "gemini-2.5-flash",
2840                 "model_reasoning": "Fast editing"
2841             }
2842         except HTTPException:
2843             raise
2844     except Exception as e:
```

```
2845         raise HTTPException(status_code=500, detail=str(e))
2846
2847
2848 @app.get("/api/preview/{session_id}")
2849 async def preview_website(session_id: str):
2850     """Preview the generated website"""
2851     html_path = f"generated_websites/{session_id}.html"
2852     if not os.path.exists(html_path):
2853         raise HTTPException(status_code=404, detail="Website not found")
2854     with open(html_path, "r", encoding="utf-8") as f:
2855         return HTMLResponse(content=f.read())
2856
2857
2858 @app.get("/api/website/{session_id}/html")
2859 async def get_website_html(session_id: str):
2860     """Get HTML preview for website (supports both legacy HTML and React)"""
2861     # First check if it's a React website in CHAT_SESSIONS
2862     for chat_id, chat_session in CHAT_SESSIONS.items():
2863         if chat_session.get("website_session_id") == session_id:
2864             # This is a React website from chat flow
2865             vercel_url = chat_session.get("vercel_url")
2866             html_content = f"""
2867 <!DOCTYPE html>
2868 <html lang="en">
2869 <head>
2870     <meta charset="UTF-8">
2871     <meta name="viewport" content="width=device-width, initial-scale=1.0">
2872     <title>Your React Website is Ready!</title>
2873     <style>
2874         body {{
2875             font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', sans-serif;
2876             display: flex;
2877             justify-content: center;
2878             align-items: center;
2879             min-height: 100vh;
2880             margin: 0;
2881             background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
2882             padding: 20px;
2883         }}
2884         .container {{
2885             background: white;
2886             border-radius: 12px;
2887             padding: 40px;
2888             max-width: 600px;
2889             text-align: center;
2890             box-shadow: 0 20px 60px rgba(0,0,0,0.3);
2891         }}
2892         h1 {{
2893             color: #333;
2894             margin: 0 0 10px 0;
2895             font-size: 32px;
2896         }}
2897         .checkmark {{
2898             font-size: 64px;
```

```
2899         color: #4CAF50;
2900         margin: 20px 0;
2901     }}
2902     p {{
2903         color: #666;
2904         line-height: 1.6;
2905         margin: 15px 0;
2906     }}
2907     .btn {{
2908         display: inline-block;
2909         padding: 12px 30px;
2910         margin: 10px 5px;
2911         background: #667eea;
2912         color: white;
2913         text-decoration: none;
2914         border-radius: 6px;
2915         font-weight: 600;
2916         transition: transform 0.2s;
2917     }}
2918     .btn:hover {{
2919         transform: translateY(-2px);
2920         background: #5568d3;
2921     }}
2922     .info {{
2923         background: #f5f5f5;
2924         padding: 20px;
2925         border-radius: 8px;
2926         margin: 20px 0;
2927     }}
2928     .vercel-link {{
2929         color: #667eea;
2930         word-break: break-all;
2931         font-weight: 600;
2932     }}
2933 </style>
2934 </head>
2935 <body>
2936     <div class="container">
2937         <div class="checkmark">?</div>
2938         <h1>Your React Website is Ready!</h1>
2939         <p>Your professional React website has been generated successfully.</p>
2940
2941         {f'<div class="info"><p>? <strong>Live Website:</strong><br><a href="{vercel_url}"
2942         target="_blank" class="vercel-link">{vercel_url}</a></p></div>' if vercel_url else '<div
2943         class="info"><p>?? Deployment is in progress. Your website will be available
2944         shortly.</p></div>'}
2945
2946         <p><strong>What's Next?</strong></p>
2947         <p>Go back to the chat to:</p>
2948         <ul style="text-align: left; display: inline-block;">
2949             <li>Download the React source code</li>
2950             <li>Make changes to colors, text, or layout</li>
2951             <li>Add more sections and features</li>
2952         </ul>
```

```
2950
2951         {f'<a href="{vercel_url}" target="_blank" class="btn">Visit Live Website ?</a>' if
vercel_url else ''}
2952     </div>
2953 </body>
2954 </html>
2955 """
2956     return HTMLResponse(content=html_content)
2957
2958     # Otherwise check for legacy HTML file
2959     html_path = f"generated_websites/{session_id}.html"
2960     if os.path.exists(html_path):
2961         with open(html_path, "r", encoding="utf-8") as f:
2962             return HTMLResponse(content=f.read())
2963
2964     # Not found
2965     raise HTTPException(status_code=404, detail="Website not found. Please build a website
first.")
2966
2967
2968 @app.get("/api/download/{session_id}")
2969 async def download_website(session_id: str, format: str = "html"):
2970     """Download website"""
2971     html_path = f"generated_websites/{session_id}.html"
2972     if not os.path.exists(html_path):
2973         raise HTTPException(status_code=404, detail="Website not found")
2974
2975     if format.lower() == "html":
2976         return FileResponse(html_path, media_type="text/html",
filename=f"website-{session_id}.html")
2977     elif format.lower() == "zip":
2978         # Create zip with separate HTML, CSS, JS files
2979         import re
2980
2981         with open(html_path, "r", encoding="utf-8") as f:
2982             html_content = f.read()
2983
2984         # Extract CSS from <style> tags
2985         css_match = re.search(r'<style>(.*?)</style>', html_content, re.DOTALL)
2986         css_content = css_match.group(1).strip() if css_match else ""
2987
2988         # Extract JS from <script> tags (exclude external scripts)
2989         js_matches = re.findall(r'<script>(.*?)</script>', html_content, re.DOTALL)
2990         js_content = "\n\n".join(match.strip() for match in js_matches if match.strip())
2991
2992         # Replace style and script tags with links to external files
2993         html_clean = html_content
2994         if css_match:
2995             html_clean = html_clean.replace(css_match.group(0), '<link rel="stylesheet"
href="styles.css">')
2996
2997         # Remove all inline scripts and add external script tag before </body>
2998         html_clean = re.sub(r'<script>.*?</script>', '', html_clean, flags=re.DOTALL)
2999         html_clean = html_clean.replace('</body>', '<script'

```

```
        src="script.js"></script>\n</body>')
3000
3001     # Create zip file in memory
3002     zip_buffer = io.BytesIO()
3003     with zipfile.ZipFile(zip_buffer, 'w', zipfile.ZIP_DEFLATED) as zip_file:
3004         # Add index.html
3005         zip_file.writestr('index.html', html_clean)
3006         # Add styles.css
3007         if css_content:
3008             zip_file.writestr('styles.css', css_content)
3009         # Add script.js
3010         if js_content:
3011             zip_file.writestr('script.js', js_content)
3012         # Add README
3013         readme = f"""Website Package
3014 =====
3015 Generated: {datetime.now().strftime('%Y-%m-%d %H:%M:%S')}
3016
3017 Files included:
3018 - index.html - Main HTML file
3019 - styles.css - Stylesheet
3020 - script.js - JavaScript functionality
3021
3022 To use:
3023 1. Extract this zip file
3024 2. Open index.html in your web browser
3025
3026 Note: This is a standalone website. All functionality is self-contained.
3027 """
3028         zip_file.writestr('README.txt', readme)
3029
3030     zip_buffer.seek(0)
3031     return StreamingResponse(
3032         zip_buffer,
3033         media_type="application/zip",
3034         headers={"Content-Disposition": f"attachment; filename=website-{session_id}.zip"}
3035     )
3036     else:
3037         raise HTTPException(status_code=400, detail="Use 'html' or 'zip' format")
3038
3039
3040 @app.get("/")
3041 async def health_check():
3042     """Backend health check"""
3043     return {"status": "ok", "service": "Sitekraft API"}
3044
3045
3046
3047 # =====
3048 # REACT & DEPLOYMENT ROUTES (Auto-integrated)
3049 # =====
3050 from react_api_routes import router as react_router
3051 app.include_router(react_router)
3052
```

```
3053
3054 # =====
3055 # STATIC HTML DEPLOYMENT (inline fallback for hot-reload)
3056 # =====
3057 class _StaticHtmlDeployRequest(BaseModel):
3058     project_name: str
3059     html_content: str
3060
3061 @app.post("/api/react/deploy-html")
3062 async def _deploy_html_inline(request: _StaticHtmlDeployRequest):
3063     """Deploy raw HTML string as a static Vercel site (identical to preview)."""
3064     from vercel_deployer import vercel_deployer as _vd
3065     if not _vd.is_enabled():
3066         raise HTTPException(status_code=400, detail="Vercel not configured")
3067     result = await _vd.deploy_static_html(
3068         project_name=request.project_name,
3069         html_content=request.html_content
3070     )
3071     if not result:
3072         raise HTTPException(status_code=500, detail="Deployment failed")
3073     return {"success": True, "deployment_url": result["url"]}
3074
3075
3076 if __name__ == "__main__":
3077     import uvicorn
3078     import sys
3079
3080     if sys.platform == "win32":
3081         sys.stdout = io.TextIOWrapper(sys.stdout.buffer, encoding='utf-8')
3082
3083     print("\n" + "="*60)
3084     print("AI WEBSITE BUILDER AGENT")
3085     print("="*60)
3086     print("\nServer: http://localhost:8000")
3087     print("API Docs: http://localhost:8000/docs")
3088     print("="*60 + "\n")
3089
3090     port = int(os.getenv("PORT", 8000))
3091     uvicorn.run(app, host="0.0.0.0", port=port)
3092
```

```

1 """
2 Helper to integrate React builder with chat workflow
3 """
4 import asyncio
5 import json
6 import hashlib
7 from typing import Dict
8 from react_builder import ReactWebsiteBuilder
9 from vercel_deployer import VercelDeployer
10 from professional_copywriter import copywriter
11 import os
12 import requests
13 import google.generativeai as genai
14
15
16 # Gemini
17 GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
18 if GEMINI_API_KEY:
19     genai.configure(api_key=GEMINI_API_KEY)
20
21 # ?? Image Cache (in-memory + disk) ??????????????????????????????????????????
22 _IMAGE_CACHE: dict = {}
23 _CACHE_FILE = os.path.join(os.path.dirname(os.path.abspath(__file__)), "image_cache.json")
24
25 def _load_disk_cache():
26     global _IMAGE_CACHE
27     try:
28         with open(_CACHE_FILE, "r") as f:
29             _IMAGE_CACHE = json.load(f)
30         print(f"[ImageCache] Loaded {len(_IMAGE_CACHE)} cached entries from disk")
31     except Exception:
32         _IMAGE_CACHE = {}
33
34 def _save_disk_cache():
35     try:
36         with open(_CACHE_FILE, "w") as f:
37             json.dump(_IMAGE_CACHE, f, indent=2)
38     except Exception as e:
39         print(f"[ImageCache] Save failed: {e}")
40
41 def _cache_key(query: str, decision: str) -> str:
42     """Stable hash key for (query, image_type) pair."""
43     normalized = f"{decision}:{query.lower().strip()}"
44     return hashlib.md5(normalized.encode()).hexdigest()[ :16]
45
46 # Load cache at import time
47 _load_disk_cache()
48
49
50 def get_smart_query_for_service(service, business_type):
51     """Get specific image query for a service/product"""

```

```
52     svc_lower = service.lower()
53
54     # COMPREHENSIVE service-to-query mapping for ALL business types
55     service_queries = {
56         # E-commerce / Fashion - Western
57         "jeans": "women jeans denim fashion stylish",
58         "tops": "women tops blouse fashion trendy",
59         "top": "women top blouse fashion casual",
60         "dresses": "women dress fashion elegant",
61         "dress": "women dress fashion elegant",
62         "skorts": "women casual fashion shorts summer",
63         "skort": "women casual fashion shorts summer",
64         "skirts": "women skirt fashion midi maxi",
65         "skirt": "women skirt fashion midi maxi",
66         "bell bottom": "women bell bottom pants flare jeans retro",
67         "bell-bottom": "women bell bottom pants flare jeans retro",
68         "palazzo": "women palazzo pants wide leg fashion",
69         "trousers": "women trousers formal pants fashion",
70         "shorts": "women shorts casual fashion summer",
71         "co-ord": "women co-ord set matching outfit",
72         "coord": "women co-ord set matching outfit",
73         "handbags": "women handbag purse fashion luxury",
74         "accessories": "women fashion accessories jewelry",
75         "shoes": "women shoes heels fashion footwear",
76         "jackets": "women jacket fashion outerwear blazer",
77         "jacket": "women jacket fashion outerwear",
78         "sweaters": "women sweater knitwear fashion cozy",
79         "sweater": "women sweater knitwear fashion",
80         "activewear": "women activewear gym fitness sportswear",
81         "lingerie": "women lingerie innerwear",
82         "swimwear": "women swimwear beach fashion",
83         "western wear": "women western fashion clothing",
84
85         # E-commerce / Fashion - Indian Ethnic
86         "kurta": "women kurta kurti ethnic indian fashion",
87         "kurti": "women kurta kurti ethnic indian fashion",
88         "saree": "women saree sari indian ethnic fashion",
89         "sarree": "women saree sari indian ethnic fashion",
90         "sari": "women saree sari indian ethnic fashion",
91         "lehenga": "women lehenga bridal indian wedding fashion",
92         "lehnga": "women lehenga bridal indian wedding fashion",
93         "suits": "women salwar suit punjabi ethnic indian",
94         "suit": "women salwar suit punjabi ethnic indian",
95         "salwar": "women salwar kameez suit indian ethnic",
96         "anarkali": "women anarkali suit indian ethnic fashion",
97         "indo-western": "women indo western fusion indian fashion",
98         "indo western": "women indo western fusion indian fashion",
99         "dupatta": "women dupatta scarf indian ethnic",
100        "churidar": "women churidar pants indian ethnic",
101        "palazzo suit": "women palazzo suit indian ethnic fusion",
102        "indian wear": "women indian ethnic fashion",
103
104        # Salon / Spa / Beauty
105        "facial": "woman facial spa treatment skincare glowing",
```

```
106     "massage": "spa massage therapy relaxation wellness luxury",
107     "body massage": "spa body massage therapy wellness relax",
108     "hair styling": "hairstylist salon hair beauty styling",
109     "hair cut": "hairstylist salon haircut styling modern",
110     "haircut": "hairstylist salon haircut styling",
111     "bridal makeup": "bridal makeup wedding beauty bride",
112     "makeup": "makeup artist beauty cosmetics professional",
113     "manicure": "manicure nails beauty salon polish",
114     "pedicure": "pedicure feet spa treatment nails",
115     "waxing": "beauty salon spa waxing treatment",
116     "hair color": "hair coloring salon hairstylist dye",
117     "spa": "luxury spa wellness relaxation massage",
118     "beauty": "beauty salon treatment skincare cosmetics",
119     "skincare": "skincare facial beauty treatment serum",
120     "threading": "eyebrow threading beauty salon",
121     "bleach": "facial bleach beauty salon treatment",
122     "cleanup": "facial cleanup beauty salon skincare",
123
124     # Fitness / Gym
125     "hiit": "HIIT workout fitness gym training intense",
126     "yoga": "yoga meditation fitness wellness peaceful",
127     "crossfit": "crossfit gym workout training athletes",
128     "personal training": "personal trainer gym fitness coaching",
129     "gym": "modern fitness gym workout equipment",
130     "fitness": "fitness training workout gym active",
131     "cardio": "cardio workout fitness gym running",
132     "strength": "strength training weights gym muscle",
133     "strength training": "strength training weights gym dumbbells",
134     "pilates": "pilates workout fitness studio mat",
135     "zumba": "zumba dance fitness class energetic",
136     "aerobics": "aerobics fitness class workout group",
137     "spinning": "spinning bike cycling fitness gym",
138     "weight training": "weight training gym dumbbells fitness",
139     "bodybuilding": "bodybuilding gym muscles fitness",
140     "martial arts": "martial arts karate fitness training",
141     "boxing": "boxing gym fitness training punching",
142     "dance": "dance fitness class studio energetic",
143
144     # Indian Restaurant / Food
145     "paneer tikka": "paneer tikka indian food restaurant plate",
146     "paneer": "paneer indian cheese curry food restaurant",
147     "chicken tikka": "chicken tikka indian food restaurant plate",
148     "chicken 65": "chicken 65 indian food restaurant spicy",
149     "butter chicken": "butter chicken curry indian food restaurant",
150     "dal makhani": "dal makhani lentil curry indian food restaurant",
151     "dal": "dal lentil curry indian food restaurant",
152     "biryani": "biryani rice indian food restaurant",
153     "pulao": "pulao rice indian food restaurant",
154     "naan": "naan bread indian food restaurant",
155     "roti": "roti chapati indian bread food",
156     "paratha": "paratha indian bread breakfast food",
157     "dosa": "dosa south indian food crispy restaurant",
158     "idli": "idli south indian food restaurant",
159     "samosa": "samosa indian snack food crispy",
```

```
160      "tikka masala": "tikka masala curry indian food restaurant",
161      "palak paneer": "palak paneer spinach curry indian food",
162      "chole": "chole chickpea curry indian food restaurant",
163      "rajma": "rajma kidney bean curry indian food",
164      "gulab jamun": "gulab jamun indian sweet dessert syrup",
165      "jalebi": "jalebi indian sweet dessert crispy",
166      "rasgulla": "rasgulla indian sweet dessert white",
167      "kheer": "kheer rice pudding indian dessert sweet",
168      "halwa": "halwa indian sweet dessert",
169      "kulfi": "kulfi indian ice cream dessert",
170      "mango lassi": "mango lassi yogurt drink indian beverage",
171      "lassi": "lassi yogurt drink indian beverage glass",
172      "chaas": "chaas buttermilk indian drink",
173      "cold coffee": "cold coffee iced latte cafe drink glass",
174      "iced coffee": "iced coffee cold brew cafe drink",
175      "fresh lime soda": "lime soda fizzy drink refreshing glass",
176      "lemonade": "lemonade fresh lemon drink glass refreshing",
177      "mocktail": "mocktail drink colorful glass refreshing bar",
178      "mocktails": "mocktails drinks colorful glasses refreshing bar",
179      "milkshake": "milkshake creamy drink glass cafe",
180      "shakes": "milkshake creamy drink glass sweet",
181
182      # Restaurant / Cafe / Food
183      "pizza": "pizza italian food restaurant delicious",
184      "pasta": "pasta italian cuisine food restaurant",
185      "burger": "burger food restaurant gourmet delicious",
186      "sushi": "sushi japanese food restaurant fresh",
187      "dessert": "dessert pastry cake bakery sweet",
188      "coffee": "coffee cafe espresso latte barista",
189      "salad": "fresh salad healthy food bowl colorful",
190      "steak": "steak meat grill restaurant juicy",
191      "seafood": "seafood fish restaurant cuisine fresh",
192      "wine": "wine bottle glass restaurant elegant",
193      "cocktail": "cocktail drink bar mixology colorful",
194      "breakfast": "breakfast food cafe morning pancakes",
195      "lunch": "lunch food restaurant meal delicious",
196      "dinner": "dinner food restaurant meal elegant",
197      "brunch": "brunch food cafe breakfast lunch",
198      "appetizers": "appetizers food restaurant starter",
199      "sandwiches": "sandwich food cafe fresh deli",
200      "wraps": "wraps food healthy lunch fresh",
201      "smoothies": "smoothie bowl healthy fruit fresh",
202      "juice": "fresh juice fruit healthy colorful",
203      "ice cream": "ice cream dessert sweet colorful",
204      "donuts": "donuts pastry bakery sweet glazed",
205      "cupcakes": "cupcakes bakery sweet frosting colorful",
206      "cookies": "cookies bakery sweet homemade",
207      "waffles": "waffles breakfast cafe sweet syrup",
208      "crepes": "crepes french thin pancake sweet cafe",
209      "pancakes": "pancakes breakfast fluffy syrup stack",
210      "french fries": "french fries crispy fast food snack",
211      "fries": "french fries crispy fast food snack",
212      "nachos": "nachos cheese snack mexican food",
213      "noodles": "noodles bowl asian food restaurant",
```

```
214     "fried rice": "fried rice asian food restaurant wok",
215     "rice": "rice dish food restaurant bowl",
216     "soup": "soup bowl hot food restaurant",
217     "chicken": "chicken dish grilled food restaurant",
218     "mutton": "mutton curry dish food restaurant",
219     "lamb": "lamb chops grilled food restaurant",
220     "fish": "fish fillet grilled food restaurant",
221     "prawns": "prawns shrimp seafood dish food",
222     "tacos": "tacos mexican food restaurant fresh",
223     "sandwich": "sandwich food cafe fresh deli",
224     "milkshake": "milkshake creamy thick drink sweet",
225     "mocktail": "mocktail colorful drink fresh fruit",
226     "lassi": "lassi yogurt drink indian refreshing",
227     "tea": "tea cup hot drink cafe",
228     "chai": "masala chai tea indian hot drink",
229
230     # Indian Food
231     "biryani": "biryani rice aromatic indian food",
232     "curry": "curry indian food spicy bowl",
233     "dal": "dal lentil soup indian food",
234     "dal makhani": "dal makhani indian food creamy",
235     "naan": "naan bread indian restaurant flatbread",
236     "roti": "roti chapati indian bread food",
237     "chapati": "chapati roti indian bread food",
238     "tandoori": "tandoori chicken indian food grill",
239     "tikka": "chicken tikka indian food grill spicy",
240     "tikka masala": "chicken tikka masala indian curry food",
241     "butter chicken": "butter chicken makhani indian curry",
242     "paneer": "paneer tikka indian food restaurant",
243     "dosa": "dosa south indian food crispy",
244     "idli": "idli sambar south indian food",
245     "samosa": "samosa indian snack fried crispy",
246     "chaat": "chaat indian street food snack",
247     "pav bhaji": "pav bhaji indian street food",
248     "chole": "chole bhature indian food chickpea",
249     "rajma": "rajma chawal indian food kidney beans",
250     "gulab jamun": "gulab jamun indian sweet dessert",
251     "rasgulla": "rasgulla indian sweet dessert syrup",
252     "kheer": "kheer rice pudding indian dessert",
253     "halwa": "halwa indian sweet dessert",
254     "kulfi": "kulfi indian ice cream dessert",
255     "ladoo": "ladoo indian sweet dessert ball",
256     "barfi": "barfi indian sweet dessert milk",
257     "jalebi": "jalebi indian sweet dessert crispy",
258
259     # Bakery / Pastry
260     "bread": "fresh bread bakery artisan loaves",
261     "croissant": "croissant pastry bakery french flaky",
262     "croissants": "croissants pastry bakery french",
263     "pastries": "pastries bakery sweet danish croissant",
264     "pastry": "pastry bakery sweet delicate",
265     "cakes": "cakes bakery celebration birthday wedding",
266     "cake": "cake bakery celebration frosting layers",
267     "cheesecake": "cheesecake dessert creamy slice",
```

```
268     "cheese cake": "cheesecake dessert creamy slice",
269     "brownies": "brownies chocolate bakery fudge",
270     "muffins": "muffins bakery breakfast blueberry",
271     "tarts": "tarts pastry bakery fruit elegant",
272     "pies": "pies bakery fruit dessert homemade",
273     "macarons": "macarons pastry bakery french colorful",
274     "eclairs": "eclairs pastry bakery french chocolate",
275     "baguette": "baguette bread bakery french artisan",
276     "custom cakes": "custom cake bakery celebration decorated",
277     "wedding cakes": "wedding cake bakery elegant tiered",
278     "birthday cakes": "birthday cake bakery celebration colorful",
279
280     # Salon / Spa extras
281     "nail extensions": "nail extensions acrylic nails beauty salon",
282     "gel nails": "gel nails manicure beauty salon",
283     "nail art": "nail art design beauty salon colorful",
284     "eyelash extensions": "eyelash extensions beauty salon glam",
285     "keratin treatment": "keratin hair treatment salon smooth",
286     "hair treatment": "hair treatment salon spa deep conditioning",
287     "hair spa": "hair spa treatment salon nourishing",
288     "balayage": "balayage hair color salon highlights",
289     "highlights": "hair highlights color salon styling",
290     "eyebrow shaping": "eyebrow shaping beauty salon grooming",
291     "pre-bridal": "pre-bridal package beauty salon bride",
292     "bridal package": "bridal makeup package salon bride",
293     "detan": "tan removal detan beauty treatment face",
294
295     # Fitness extras
296     "swimming": "swimming pool fitness workout laps",
297     "kickboxing": "kickboxing martial arts fitness training",
298     "meditation": "meditation yoga peaceful mindfulness wellness",
299     "stretching": "stretching flexibility yoga fitness",
300     "nutrition": "nutrition healthy food diet wellness",
301     "nutrition counseling": "nutrition counseling diet healthy food",
302     "functional training": "functional training workout fitness gym",
303     "group fitness": "group fitness class gym workout energetic",
304     "cycling": "cycling bike fitness workout gym",
305
306     # Clinic / Healthcare
307     "physiotherapy": "physiotherapy physical therapy rehabilitation exercise",
308     "dermatology": "dermatology skin treatment clinic doctor",
309     "cardiology": "cardiology heart care clinic doctor",
310     "pediatrics": "pediatrics children health clinic doctor",
311     "gynecology": "gynecology women health clinic doctor",
312     "orthopedics": "orthopedics bone joint clinic doctor",
313     "ophthalmology": "ophthalmology eye care clinic doctor",
314     "ayurveda": "ayurveda herbal medicine treatment wellness",
315     "homeopathy": "homeopathy medicine treatment clinic",
316     "dentist": "dentist dental clinic teeth care professional",
317     "doctor": "doctor clinic healthcare professional stethoscope",
318     "blood test": "blood test laboratory clinic healthcare",
319     "x-ray": "x-ray radiology clinic medical imaging",
320     "ultrasound": "ultrasound scan clinic medical",
321
```

```
322     # Men's Fashion
323     "shirts": "men shirts fashion formal casual",
324     "shirt": "men shirt fashion formal casual",
325     "t-shirts": "men t-shirt casual fashion streetwear",
326     "t-shirt": "men t-shirt casual fashion streetwear",
327     "men jeans": "men jeans denim fashion casual",
328     "men kurta": "men kurta ethnic indian fashion",
329     "sherwani": "men sherwani wedding indian ethnic fashion",
330     "blazer": "men blazer suit formal fashion",
331     "formal wear": "men formal wear suit professional",
332     "ethnic wear": "men ethnic wear indian fashion kurta",
333
334     # Jewelry / Accessories
335     "necklace": "necklace jewelry fashion gold silver",
336     "earrings": "earrings jewelry fashion gold silver",
337     "bangles": "bangles jewelry indian fashion gold",
338     "rings": "rings jewelry fashion gold diamond",
339     "bracelet": "bracelet jewelry fashion wrist",
340     "anklet": "anklet jewelry fashion gold",
341     "jewelry": "jewelry fashion gold silver elegant",
342     "watch": "watch wristwatch fashion luxury timepiece",
343     "sunglasses": "sunglasses eyewear fashion trendy",
344     "bag": "handbag purse fashion women leather",
345     "clutch": "clutch bag fashion women evening",
346     "wallet": "wallet leather fashion men women",
347
348     # Footwear
349     "heels": "women heels shoes fashion elegant",
350     "sandals": "sandals shoes fashion summer casual",
351     "boots": "boots shoes fashion leather",
352     "sneakers": "sneakers shoes fashion casual sporty",
353     "flats": "flat shoes women fashion casual",
354     "footwear": "footwear shoes fashion collection",
355
356     # General Services
357     "consulting": "business consulting professional meeting office",
358     "photography": "photography camera professional portrait studio",
359     "videography": "videography video camera production",
360     "graphic design": "graphic design creative work computer",
361     "web design": "web design website development computer",
362     "marketing": "digital marketing strategy business creative",
363     "accounting": "accounting finance calculator business",
364     "legal": "legal law office lawyer professional",
365     "tutoring": "tutoring education student learning books",
366     "music lessons": "music lessons guitar piano teaching",
367     "art classes": "art class painting creative studio",
368     "pet grooming": "pet grooming dog salon care",
369     "veterinary": "veterinary clinic pet dog cat care",
370     "dental": "dental clinic dentist teeth care",
371     "medical": "medical clinic doctor healthcare professional",
372     "cleaning": "house cleaning service professional home",
373     "plumbing": "plumber plumbing repair tools professional",
374     "electrical": "electrician electrical work tools professional",
375     "gardening": "gardening landscaping plants flowers garden",
```

```
376         "catering": "catering food service event professional",
377         "event planning": "event planning wedding party decoration",
378         "tailoring": "tailoring sewing clothes fashion custom",
379         "pharmacy": "pharmacy medicine healthcare professional",
380         "optician": "optician eyewear glasses vision care",
381         "car rental": "car rental vehicle luxury automobile",
382         "car wash": "car wash cleaning service vehicle",
383         "repair": "repair service tools professional workshop",
384         "printing": "printing service design professional",
385         "travel": "travel destination landscape beautiful scenic",
386         "hotel": "hotel luxury room interior elegant",
387         "architecture": "architecture building design modern exterior",
388
389         # Coaching / Education
390         "python": "python programming code laptop developer",
391         "java": "java programming software developer laptop",
392         "web development": "web development coding laptop developer",
393         "data science": "data science analytics laptop charts",
394         "machine learning": "machine learning artificial intelligence technology",
395         "ielts": "ielts english test preparation study books",
396         "spoken english": "english speaking class students learning",
397         "public speaking": "public speaking presentation confident stage",
398         "personality development": "personality development coaching professional confident",
399         "interview preparation": "job interview preparation coaching professional",
400         "soft skills": "soft skills training workshop professional team",
401         "leadership": "leadership training workshop professional business",
402         "career counseling": "career counseling guidance professional mentor",
403         "aptitude training": "aptitude test preparation study mathematics",
404         "gate coaching": "engineering gate exam preparation study",
405         "neet coaching": "medical neet exam preparation study books",
406         "jee coaching": "engineering jee exam preparation study",
407         "mba coaching": "mba business school preparation study professional",
408
409         # Real Estate
410         "apartment": "modern apartment interior living room elegant",
411         "villa": "luxury villa exterior architecture swimming pool",
412         "flat": "modern flat apartment interior bright",
413         "plot": "land plot aerial view residential",
414         "commercial space": "commercial office space modern interior",
415         "office space": "modern office space interior professional bright",
416         "warehouse": "industrial warehouse space interior large",
417         "shop space": "retail shop space interior modern",
418         "penthouse": "luxury penthouse interior view modern",
419         "studio apartment": "studio apartment modern interior compact",
420         "bungalow": "bungalow house exterior architecture garden",
421         "property management": "property management real estate professional",
422         "home loan": "home loan mortgage financial planning professional",
423         "interior design": "interior design modern home elegant beautiful",
424     }
425
426     # Try exact or partial match
427     for key, query in service_queries.items():
428         if key in svc_lower:
429             return query
```

```
430
431     # Business type fallbacks
432     if business_type in ["shop", "store", "ecommerce", "boutique", "clothing",
"online-store"]:
433         return f"{service} fashion clothing wear"
434     elif business_type in ["salon", "spa", "beauty"]:
435         return f"{service} beauty spa treatment"
436     elif business_type in ["gym", "fitness", "studio"]:
437         return f"{service} fitness workout gym"
438     elif business_type in ["restaurant", "cafe", "food"]:
439         return f"{service} food plated dish restaurant"
440     elif business_type in ["bakery", "pastry"]:
441         return f"{service} bakery pastry fresh"
442     elif business_type in ["coaching", "education", "tutoring", "academy"]:
443         return f"{service} education learning class professional"
444     elif business_type in ["real-estate", "property", "realty"]:
445         return f"{service} property real estate home interior"
446     elif business_type in ["clinic", "medical", "dental", "health"]:
447         return f"{service} medical clinic healthcare professional"
448     elif business_type in ["photography", "videography"]:
449         return f"{service} photography studio camera professional"
450     elif business_type in ["hotel", "resort", "hospitality"]:
451         return f"{service} hotel luxury interior elegant"
452
453     return f"{service} professional"
454
455
456 # ?? Image Fetch with Cache ?????????????????????????????????????????????
457
458 def smart_fetch_image(query: str, slot_key: str) -> dict:
459     """
460     Fetch a single image via Pexels with in-memory + disk caching.
461     Returns: {"url": "<image URL>"}
462     """
463     from pexels_helper import search_pexels_image
464
465     key = _cache_key(query, "realistic")
466
467     if key in _IMAGE_CACHE:
468         print(f"[ImageCache] HIT '{query}' ? {_IMAGE_CACHE[key][:60]}...")
469         return {"url": _IMAGE_CACHE[key]}
470
471     print(f"[Images] Fetching '{query}' via Pexels")
472     url = search_pexels_image(query) or f"https://picsum.photos/seed/{slot_key}/1920/1080"
473
474     _IMAGE_CACHE[key] = url
475     _save_disk_cache()
476     return {"url": url}
477
478
479 async def build_react_website_from_chat(session: Dict) -> Dict:
480     """
481     Build a React website from chat session data
482
```

```
483     Args:
484         session: Chat session with business details
485
486     Returns:
487         Dict with website files and Vercel URL
488     """
489     # Prepare analysis data
490     business_name = session.get("business_name", "My Business")
491     business_type = session.get("business_type", "business")
492     description = session.get("description", "")
493     location = session.get("location", "")
494     services_list = session.get("services", [])
495
496     # Create professional copy based on business type - COMPREHENSIVE FOR ALL BUSINESSES
497     hero_headlines = {
498         # Fashion/Retail
499         "shop": f"Discover Your Perfect Style at {business_name}",
500         "store": f"Your Premium Shopping Destination - {business_name}",
501         "ecommerce": f"Shop the Latest Trends at {business_name}",
502         "boutique": f"Exclusive Fashion Finds at {business_name}",
503
504         # Beauty/Wellness
505         "salon": f"Transform Your Look at {business_name}",
506         "spa": f"Experience Ultimate Relaxation at {business_name}",
507         "beauty": f"Enhance Your Natural Beauty at {business_name}",
508
509         # Fitness/Health
510         "gym": f"Transform Your Body at {business_name}",
511         "fitness": f"Your Fitness Journey Starts at {business_name}",
512         "studio": f"Where Wellness Meets Excellence - {business_name}",
513         "clinic": f"Trusted Healthcare at {business_name}",
514         "dental": f"Your Smile is Our Priority - {business_name}",
515         "medical": f"Comprehensive Care at {business_name}",
516
517         # Food/Beverage
518         "restaurant": f"Exquisite Dining Experience at {business_name}",
519         "cafe": f"Your Favorite Coffee Destination - {business_name}",
520         "bakery": f"Freshly Baked Goodness from {business_name}",
521         "food": f"Delicious Cuisine at {business_name}",
522         "pastry": f"Sweet Perfection from {business_name}",
523
524         # Professional Services
525         "photography": f"Capturing Life's Precious Moments - {business_name}",
526         "videography": f"Professional Video Production by {business_name}",
527         "consulting": f"Expert Business Solutions from {business_name}",
528         "legal": f"Trusted Legal Expertise - {business_name}",
529         "accounting": f"Professional Financial Services - {business_name}",
530
531         # Education/Training
532         "tutoring": f"Empowering Students to Excel - {business_name}",
533         "education": f"Quality Education at {business_name}",
534         "music lessons": f"Discover Your Musical Talent at {business_name}",
535         "art classes": f"Unleash Your Creativity at {business_name}",
536
```

```
537         # Pet Services
538         "pet grooming": f"Pamper Your Furry Friends at {business_name}",
539         "veterinary": f"Compassionate Pet Care at {business_name}",
540
541         # Home Services
542         "cleaning": f"Professional Cleaning Services by {business_name}",
543         "plumbing": f"Reliable Plumbing Solutions - {business_name}",
544         "electrical": f"Expert Electrical Services - {business_name}",
545         "gardening": f"Transform Your Outdoor Space - {business_name}",
546
547         # Events
548         "event planning": f"Making Your Events Unforgettable - {business_name}",
549         "catering": f"Exceptional Catering Services by {business_name}",
550     }
551
552     hero_subtexts = {
553         # Fashion/Retail
554         "shop": f"Premium fashion and style in {location}. Discover our curated collection of {', '.join(services_list[:3])} and more.",
555         "store": f"Located in {location}. Explore our exclusive range of {', '.join(services_list[:3])} crafted for you.",
556         "ecommerce": f"Online shopping made easy from {location}. Browse our {', '.join(services_list[:3])} collection.",
557         "boutique": f"Luxury fashion boutique in {location}. Featuring {', '.join(services_list[:3])} and exclusive pieces.",
558
559         # Beauty/Wellness
560         "salon": f"Professional beauty services in {location}. We offer {', '.join(services_list[:3])} and more.",
561         "spa": f"Luxury wellness in {location}. Indulge in our {', '.join(services_list[:3])} treatments.",
562         "beauty": f"Expert beauty care in {location}. Specializing in {', '.join(services_list[:3])} services.",
563
564         # Fitness/Health
565         "gym": f"Modern fitness facility in {location}. Join us for {', '.join(services_list[:3])} and transform your life.",
566         "fitness": f"Expert training in {location}. Specializing in {', '.join(services_list[:3])} programs.",
567         "studio": f"Boutique fitness studio in {location}. Offering {', '.join(services_list[:3])} classes.",
568         "clinic": f"Professional medical care in {location}. Providing {', '.join(services_list[:3])} services.",
569         "dental": f"Advanced dental care in {location}. Expert {', '.join(services_list[:3])} treatments.",
570         "medical": f"Comprehensive healthcare in {location}. Trusted for {', '.join(services_list[:3])} care.",
571
572         # Food/Beverage
573         "restaurant": f"Fine dining in {location}. Savor our signature {', '.join(services_list[:3])} dishes.",
574         "cafe": f"Artisan cafe in {location}. Enjoy our {', '.join(services_list[:3])} and cozy ambiance.",
575         "bakery": f"Artisan bakery in {location}. Fresh {', '.join(services_list[:3])} baked
```

```
        daily.",
576             "food": f"Delicious cuisine in {location}. Famous for our {'',
'.join(services_list[:3])}'.",
577             "pastry": f"Gourmet pastries in {location}. Handcrafted {'',
'.join(services_list[:3])} daily.",
578
579         # Professional Services
580         "photography": f"Professional photography in {location}. Specializing in {'',
'.join(services_list[:3])} photography.",
581         "videography": f"Expert video production in {location}. Creating stunning {'',
'.join(services_list[:3])} videos.",
582         "consulting": f"Business consulting in {location}. Expert guidance in {'',
'.join(services_list[:3])}'.",
583         "legal": f"Legal services in {location}. Experienced in {'',
'.join(services_list[:3])} law.",
584         "accounting": f"Professional accounting in {location}. Trusted for {'',
'.join(services_list[:3])} services.",
585
586         # Education/Training
587         "tutoring": f"Quality education in {location}. Expert tutoring in {'',
'.join(services_list[:3])}'.",
588         "education": f"Learning center in {location}. Offering {'', '.join(services_list[:3])}
courses.",
589         "music lessons": f"Music instruction in {location}. Teaching {'',
'.join(services_list[:3])} and more.",
590         "art classes": f"Art education in {location}. Classes in {'',
'.join(services_list[:3])}'.",
591
592         # Pet Services
593         "pet grooming": f"Pet care in {location}. Expert {'', '.join(services_list[:3])}
services for your pets.",
594         "veterinary": f"Veterinary care in {location}. Providing {'',
'.join(services_list[:3])} treatment.",
595
596         # Home Services
597         "cleaning": f"Cleaning services in {location}. Professional {'',
'.join(services_list[:3])} solutions.",
598         "plumbing": f"Plumbing services in {location}. Expert {'', '.join(services_list[:3])}
repairs.",
599         "electrical": f"Electrical services in {location}. Licensed {'',
'.join(services_list[:3])} experts.",
600         "gardening": f"Landscaping services in {location}. Beautiful {'',
'.join(services_list[:3])} designs.",
601
602         # Events
603         "event planning": f"Event services in {location}. Specializing in {'',
'.join(services_list[:3])} planning.",
604         "catering": f"Catering services in {location}. Delicious {'',
'.join(services_list[:3])} menus.",
605     }
606
607     # Simple color scheme (set before parallel tasks)
608     colors = {
609         "primary": "#667eea",
```

```
610         "secondary": "#764ba2",
611         "accent": "#f093fb"
612     }
613
614     # Build image query list: hero + one per service (all fetched in parallel)
615     hero_queries = {
616         "shop": "fashion model women clothing store boutique",
617         "store": "fashion boutique women clothing interior",
618         "ecommerce": "fashion model women online shopping",
619         "boutique": "luxury fashion boutique interior clothing",
620         "salon": "luxury salon interior beauty spa elegant",
621         "spa": "luxury spa wellness relaxation interior",
622         "beauty": "beauty salon interior elegant modern",
623         "gym": "modern gym fitness equipment interior",
624         "fitness": "fitness studio workout space equipment",
625         "studio": "fitness yoga studio interior peaceful",
626         "clinic": "medical clinic interior professional modern",
627         "dental": "dental clinic modern interior professional",
628         "restaurant": "elegant restaurant interior dining ambiance",
629         "cafe": "modern cafe interior coffee shop cozy",
630         "food": "restaurant food interior ambiance",
631         "bakery": "bakery interior pastries display artisan",
632         "pastry": "pastry shop bakery interior french",
633         "photography": "photography studio professional camera equipment",
634         "consulting": "modern office meeting professional business",
635         "legal": "law office professional interior modern",
636         "accounting": "professional office business interior",
637         "tutoring": "education classroom tutoring learning",
638         "coaching": "modern learning center classroom professional bright",
639         "education": "education classroom modern school learning center",
640         "real-estate": "luxury modern property architecture interior home",
641         "property": "luxury property interior architecture modern",
642         "online-store": "ecommerce warehouse modern shopping lifestyle",
643         "ecommerce": "ecommerce lifestyle shopping modern bright",
644         "clothing": "fashion boutique clothing store elegant interior",
645         "boutique": "fashion boutique clothing luxury elegant store",
646         "other": "modern professional business office interior bright",
647         "veterinary": "veterinary clinic interior modern pet",
648         "pet grooming": "pet grooming salon interior professional",
649         "cleaning": "professional cleaning service home",
650         "event planning": "event planning decoration wedding elegant",
651     }
652     hero_query = hero_queries.get(business_type, f"{business_type} professional modern
interior")
653
654     # All image slots to fetch: (slot_key, query)
655     image_slots = [("hero", hero_query)] + [
656         (f"service_{svc}", get_smart_query_for_service(svc, business_type))
657         for svc in services_list[:8]
658     ]
659
660     # Run copywriter + all image fetches in parallel
661     from concurrent.futures import ThreadPoolExecutor, as_completed
662     images = [None] * len(image_slots)
```

```
663     print(f"[Build] Fetching {len(image_slots)} images + copy in parallel...")
664
665     with ThreadPoolExecutor(max_workers=min(len(image_slots) + 1, 12)) as executor:
666         # Submit copywriter
667         copy_future = executor.submit(
668             copywriter.generate_website_copy,
669             business_name=business_name,
670             business_type=business_type,
671             business_description=description,
672             services=services_list,
673             location=location,
674             style_vibe="modern",
675         )
676         # Submit all image fetches
677         future_to_idx = {
678             executor.submit(smart_fetch_image, query, slot_key): idx
679             for idx, (slot_key, query) in enumerate(image_slots)
680         }
681         # Collect image results
682         for future in as_completed(future_to_idx):
683             idx = future_to_idx[future]
684             try:
685                 images[idx] = future.result(timeout=15)
686             except Exception as e:
687                 slot_key = image_slots[idx][0]
688                 print(f"[Images] Failed for {slot_key}: {e}")
689                 images[idx] = {"url": f"https://picsum.photos/seed/{slot_key}/1920/1080"}
690         # Collect copy result
691         copy_data = copy_future.result()
692
693     print(f"[Build] {len(images)} images + copy ready.")
694
695     hero_headline = copy_data.get("hero_headline", hero_headlines.get(business_type,
696 f"Welcome to {business_name}"))
697
698     hero_subtext = copy_data.get("hero_subtext", hero_subtexts.get(business_type, f"Your
699 trusted {business_type} in {location}."))
700
701     analysis = {
702         "business_name": business_name,
703         "business_type": business_type,
704         "tagline": copy_data.get("tagline", hero_subtext),
705         "hero_headline": hero_headline,
706         "hero_subtext": hero_subtext,
707         "about_text": copy_data.get("about_text", f"At {business_name}, we're passionate
708 about delivering exceptional {business_type} services in {location}."),
709         "services": services_list,
710         "cta_text": copy_data.get("cta_text", "Get Started"),
711         "sections": ["hero", "about", "services", "contact"],
712         "cta": "Contact Us",
713         "service_categories": session.get("service_categories", []),
714     }
715
716     # Features
717     features = session.get("features", ["contact"])
```

```
714
715     # Business description
716     business_description = f"{session['description']}. Located in {session.get('location',
'our city')}. We offer: {'', '.join(session.get('services', []))}"
717
718     # Build React website
719     builder = ReactWebsiteBuilder()
720     react_result = await builder.generate_react_website(
721         session_id=session.get("website_session_id", ""),
722         analysis=analysis,
723         colors=colors,
724         images=images,
725         features=features,
726         business_description=business_description,
727         deploy_to_vercel=False # We'll deploy separately
728     )
729
730     # Deploy to Vercel
731     deployer = VercelDeployer()
732     deployment = await deployer.create_deployment(
733         project_name=f"{session['business_name'].lower().replace(' ', '-')}",
734         files=react_result['files']
735     )
736
737     vercel_url = None
738     if deployment and 'url' in deployment:
739         vercel_url = deployment['url'] # Already includes https://
740
741     return {
742         "session_id": session.get("website_session_id", ""),
743         "files": react_result['files'],
744         "vercel_url": vercel_url,
745         "html": None # React websites don't have standalone HTML preview
746     }
747
748 def build_react_sync(session: Dict) -> Dict:
749     """Synchronous wrapper"""
750     return asyncio.run(build_react_website_from_chat(session))
751
```

File 3: backend/react_builder.pyBackend - React Builder

```
1      """
2      React Website Builder
3      Generates complete React/Vite websites from business descriptions
4      """
5
6      import json
7      import os
8      import re
9      import traceback
10     from typing import Dict, List, Optional
11     import google.generativeai as genai
12     from react_template_generator import react_template
13     from vercel_deployer import vercel_deployer
14     from supabase_config import supabase_manager
15
16     # Initialize Gemini
17     GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
18     if GEMINI_API_KEY:
19         genai.configure(api_key=GEMINI_API_KEY)
20
21
22     class ReactWebsiteBuilder:
23         """Builds complete React websites with AI"""
24
25         def __init__(self):
26             self.model = genai.GenerativeModel('gemini-2.5-flash')
27
28         async def generate_react_website(
29             self,
30             session_id: str,
31             analysis: Dict,
32             colors: Dict,
33             images: List[Dict],
34             features: List[str],
35             business_description: str,
36             deploy_to_vercel: bool = False
37         ) -> Dict:
38             """
39             Generate a complete React/Vite website
40
41             Args:
42                 session_id: Unique session identifier
43                 analysis: Business analysis data
44                 colors: Color scheme
45                 images: List of images to use
46                 features: List of features to include
47                 business_description: Full business description
48                 deploy_to_vercel: Whether to deploy to Vercel
49
50             Returns:
51                 Dict with website files and deployment info
```

```
52         """
53
54         business_name = analysis.get("business_name", "My Business")
55         business_type = analysis.get("business_type", "business")
56
57         print(f" Generating React website for: {business_name}")
58
59         # Step 1: Generate React components using AI
60         components = await self._generate_components(analysis, colors, features, images)
61
62         # Step 2: Extract Supabase features
63         supabase_features = self._extract_supabase_features(features)
64
65         # Step 3: Create theme configuration
66         theme = {
67             "colors": colors,
68             "fonts": {
69                 "heading": "Poppins",
70                 "body": "Inter"
71             }
72         }
73
74         # Step 4: Generate all project files
75         project_files = react_template.generate_complete_project(
76             project_name=business_name,
77             description=business_description,
78             components=components,
79             theme=theme,
80             features=supabase_features + ["routing", "animation", "forms", "icons"],
81             supabase_config={
82                 "url": os.getenv("SUPABASE_URL", ""),
83                 "key": os.getenv("SUPABASE_KEY", "")
84             }
85         )
86
87         # Step 5: Save to Supabase
88         if supabase_manager.is_enabled():
89             await supabase_manager.save_website_metadata({
90                 "session_id": session_id,
91                 "title": business_name,
92                 "description": business_description,
93                 "framework": "react-vite",
94                 "features": supabase_features,
95                 "status": "generated",
96                 "metadata": {
97                     "business_type": business_type,
98                     "colors": colors,
99                     "component_count": len(components)
100                 }
101             })
102
103         result = {
104             "session_id": session_id,
105             "files": project_files,
```

```
110         "components": [c["name"] for c in components],
111         "framework": "react-vite",
112         "deployment_url": None,
113         "deployment_id": None
114     }
115
116     # Step 6: Deploy to Vercel if requested
117     if deploy_to_vercel and vercel_deployer.is_enabled():
118         print(f" Deploying {business_name} to Vercel...")
119         deployment = await vercel_deployer.create_deployment(
120             project_name=business_name,
121             files=project_files,
122             env_vars={
123                 "VITE_SUPABASE_URL": os.getenv("SUPABASE_URL", ""),
124                 "VITE_SUPABASE_ANON_KEY": os.getenv("SUPABASE_KEY", "")
125             }
126         )
127
128         if deployment:
129             result["deployment_url"] = deployment.get("url")
130             result["deployment_id"] = deployment.get("id")
131
132             # Update Supabase with deployment URL
133             if supabase_manager.is_enabled():
134                 await supabase_manager.update_website_metadata(
135                     session_id,
136                     {
137                         "deployment_url": deployment.get("url"),
138                         "status": "deployed"
139                     }
140                 )
141
142             print(f"[OK] Deployed successfully: {deployment.get('url')}")
143         else:
144             print(f"[ERROR] Deployment failed")
145
146     return result
147
148 async def _generate_components(
149     self,
150     analysis: Dict,
151     colors: Dict,
152     features: List[str],
153     images: List[Dict]
154 ) -> List[Dict]:
155     """Generate React components using AI"""
156
157     business_name = analysis.get("business_name", "My Business")
158     business_type = analysis.get("business_type", "business")
159     services = analysis.get("services", [])
160     hero_headline = analysis.get("hero_headline", f"Welcome to {business_name}")
161     hero_subtext = analysis.get("hero_subtext", "")
162     about_text = analysis.get("about_text", "")
163     cta_text = analysis.get("cta_text", "Get Started")
```

```
160
161         # Hero image
162         hero_image = images[0].get("url") if images else
163         "https://picsum.photos/seed/business/1920/1080"
164
165         prompt = f"""Generate modern React components for a {business_type} website.
166
167         Business: {business_name}
168         Style: Modern, responsive, using Tailwind CSS
169         Colors: Primary {colors.get('primary')}, Secondary {colors.get('secondary')}
170
171         Generate these components:
172
173         1. **Navbar** - Fixed navigation with logo, links, mobile menu
174         2. **Hero** - Eye-catching hero section with headline: "{hero_headline}"
175         3. **Services** - Grid showcasing these services: {' '.join(services[:6])}
176         4. **About** - About section with text: "{about_text}"
177         5. **Contact** - Contact form with Supabase integration
178
179         For EACH component, return VALID JSX code using:
180         - Tailwind CSS classes
181         - Modern React hooks (useState, useEffect)
182         - Responsive design (mobile-first)
183         - Smooth animations
184         - Clean, production-ready code
185
186         Return ONLY this JSON structure:
187         {{
188             "components": [
189                 {{
190                     "name": "Navbar",
191                     "description": "Navigation bar",
192                     "code": "full JSX component code here"
193                 }},
194                 {{
195                     "name": "Hero",
196                     "description": "Hero section",
197                     "code": "full JSX component code here"
198                 }},
199                 ...
200             ]
201         }}"""
202
203         # NOTE: Skipping Gemini AI generation due to consistent timeouts
204         # Using high-quality fallback components instead (faster & more reliable)
205         print(f"[INFO] Generating components using production-ready templates...")
206         try:
207             return self._generate_fallback_components(analysis, colors, images, features)
208         except Exception as e:
209             print(f"[ERROR] Component generation failed:")
210             print(traceback.format_exc())
211             raise
212
213     def _generate_fallback_components(
```

```
213         self,
214         analysis: Dict,
215         colors: Dict,
216         images: List[Dict],
217         features: List[str]
218     ) -> List[Dict]:
219         """Generate comprehensive components based on selected features"""
220
221         business_name = analysis.get("business_name", "My Business")
222         hero_headline = analysis.get("hero_headline", f"Welcome to {business_name}")
223         hero_subtext = analysis.get("hero_subtext", "")
224         services = analysis.get("services", [])
225         service_descriptions = analysis.get("service_descriptions", {})
226
227         # Build category/subcategory/price lookup per service
228         service_categories_data = analysis.get("service_categories", [])
229         service_meta = {}
230         for item in service_categories_data:
231             name = item.get("name", "").strip()
232             if name:
233                 service_meta[name] = {
234                     "category": item.get("category", "").strip().title(),
235                     "subcategory": item.get("subcategory", "").strip().title(),
236                     "price": item.get("price", "").strip(),
237                 }
238
239         hero_image = images[0].get("url") if images else
240         "https://picsum.photos/seed/business/1920/1080"
241         # Use dedicated about image if provided, otherwise reuse hero image
242         about_image = analysis.get("about_image") or hero_image
243         # Service images start from index 1 (index 0 is the hero image slot)
244         service_images_list = images[1:] if len(images) > 1 else images
245
246         navbar_code = f"""import React, {{ useState, useEffect }} from 'react'
247
248         function Navbar() {{
249             const [isOpen, setIsOpen] = useState(false)
250             const [scrolled, setScrolled] = useState(false)
251
252             useEffect(() => {{
253                 const handleScroll = () => {{
254                     setScrolled(window.scrollY > 50)
255                 }}
256                 window.addEventListener('scroll', handleScroll)
257                 return () => window.removeEventListener('scroll', handleScroll)
258             }}, [])
259
260             const scrollToSection = (e, sectionId) => {{
261                 e.preventDefault()
262                 const element = document.getElementById(sectionId)
263                 if (element) {{
264                     const offsetTop = element.offsetTop - 80
265                     window.scrollTo({{
266                         top: offsetTop,
```

```

266         behavior: 'smooth'
267     })
268     setIsOpen(false)
269 }
270 }
271
272 return (
273     <nav className={`fixed w-full z-50 transition-all duration-300 ${scrolled ?
'bg-white/95 backdrop-blur-lg shadow-lg' : 'bg-white/80 backdrop-blur-sm'}}`>
274         <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8">
275             <div className="flex justify-between items-center h-20">
276                 {/* Logo */}
277                 <div className="flex items-center cursor-pointer" onClick={{(e) =>
scrollToSection(e, 'hero')}}>
278                     <div className="w-10 h-10 bg-gradient-to-br from-primary to-secondary rounded-xl
mr-3 flex items-center justify-center">
279                         <span className="text-white font-bold text-xl">{{business_name[0]}}</span>
280                     </div>
281                     <span className="text-2xl font-bold bg-gradient-to-r from-primary to-secondary
bg-clip-text text-transparent">
282                         {business_name}
283                     </span>
284                 </div>
285
286                 {/* Desktop Menu */}
287                 <div className="hidden md:flex items-center space-x-8">
288                     <a href="#hero" onClick={{(e) => scrollToSection(e, 'hero')}}
className="text-gray-700 hover:text-primary font-semibold transition-colors
cursor-pointer">Home</a>
289                     <a href="#about" onClick={{(e) => scrollToSection(e, 'about')}}
className="text-gray-700 hover:text-primary font-semibold transition-colors
cursor-pointer">About</a>
290                     <a href="#services" onClick={{(e) => scrollToSection(e, 'services')}}
className="text-gray-700 hover:text-primary font-semibold transition-colors
cursor-pointer">Services</a>
291                     <a href="#contact" onClick={{(e) => scrollToSection(e, 'contact')}}
className="px-6 py-3 bg-gradient-to-r from-primary to-secondary text-white font-bold
rounded-xl hover:shadow-xl hover:scale-105 transition-all duration-300 cursor-pointer">
292                         Contact Us
293                     </a>
294                 </div>
295
296                 {/* Mobile Menu Button */}
297                 <button
298                     className="md:hidden p-2 rounded-lg hover:bg-gray-100 transition-colors"
299                     onClick={{() => setIsOpen(!isOpen)}}
300                 >
301                     <svg className="w-6 h-6" fill="none" stroke="currentColor" viewBox="0 0 24 24">
302                         {{isOpen ? (
303                             <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={{2}} d="M6
18L18 6M6 12L12 12" />
304                         ) : (
305                             <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={{2}} d="M4
6h16M4 12h16M4 18h16" />

```

```

306         )}}
307     </svg>
308 </button>
309 </div>
310
311     {{/* Mobile Menu */}}
312     {{isOpen && (
313         <div className="md:hidden py-4 space-y-3 border-t border-gray-200">
314             <a href="#hero" onClick={{(e) => scrollToSection(e, 'hero')}} className="block
py-3 px-4 rounded-lg hover:bg-gray-100 font-semibold transition-colors
cursor-pointer">Home</a>
315             <a href="#about" onClick={{(e) => scrollToSection(e, 'about')}} className="block
py-3 px-4 rounded-lg hover:bg-gray-100 font-semibold transition-colors
cursor-pointer">About</a>
316             <a href="#services" onClick={{(e) => scrollToSection(e, 'services')}}
className="block py-3 px-4 rounded-lg hover:bg-gray-100 font-semibold transition-colors
cursor-pointer">Services</a>
317             <a href="#products" onClick={{(e) => scrollToSection(e, 'products')}}
className="block py-3 px-4 rounded-lg hover:bg-gray-100 font-semibold transition-colors
cursor-pointer">Products</a>
318             <a href="#contact" onClick={{(e) => scrollToSection(e, 'contact')}}
className="block py-3 px-4 bg-gradient-to-r from-primary to-secondary text-white font-bold
rounded-lg text-center cursor-pointer">Contact Us</a>
319         </div>
320     )}}
321 </div>
322 </nav>
323 )
324 }}
325
326 export default Navbar
327 ""
328
329     hero_code = f""import React from 'react'
330
331     function Hero() {{
332         return (
333             <section id="hero" className="relative min-h-[600px] md:min-h-[720px] flex items-center
justify-center text-white overflow-hidden pt-20">
334                 {{/* Background Image with Overlay */}}
335                 <div
336                     className="absolute inset-0 bg-cover bg-center scale-105 animate-slow-zoom"
337                     style={{{{{{backgroundImage: 'url({hero_image})'}}}}}
338                 >
339                     <div className="absolute inset-0 bg-gradient-to-br from-primary/80 via-secondary/70
to-black/60"></div>
340                 </div>
341
342                 {{/* Animated Gradient Orbs */}}
343                 <div className="absolute top-20 left-10 w-72 h-72 bg-primary/30 rounded-full blur-3xl
animate-pulse"></div>
344                 <div className="absolute bottom-20 right-10 w-96 h-96 bg-secondary/30 rounded-full
blur-3xl animate-pulse delay-1000"></div>
345

```

```

346     {{/* Content */}}
347     <div className="relative z-10 text-center px-4 max-w-5xl mx-auto">
348         <div className="mb-4 md:mb-6 inline-block px-4 md:px-6 py-2 md:py-3 bg-white/10
backdrop-blur-md rounded-full border border-white/20">
349             <span className="text-xs md:text-sm font-semibold tracking-wide">Welcome to
Excellence</span>
350         </div>
351
352         <h1 className="text-3xl sm:text-4xl md:text-6xl lg:text-7xl xl:text-8xl font-bold
mb-6 md:mb-8 leading-tight animate-fade-in-up">
353             {hero_headline}
354         </h1>
355
356         <p className="text-base sm:text-lg md:text-xl lg:text-2xl xl:text-3xl mb-8 md:mb-12
opacity-95 max-w-3xl mx-auto leading-relaxed animate-fade-in-up animation-delay-200">
357             {hero_subtext}
358         </p>
359
360         <div className="flex gap-3 md:gap-6 justify-center flex-wrap animate-fade-in-up
animation-delay-400">
361             <a
362                 href="#contact"
363                 className="px-6 md:px-10 py-3 md:py-5 bg-white text-primary font-bold text-sm
md:text-lg rounded-xl md:rounded-2xl hover:shadow-2xl hover:scale-105 transition-all
duration-300 transform"
364             >
365                 Get Started Today
366             </a>
367             <a
368                 href="#services"
369                 className="px-6 md:px-10 py-3 md:py-5 bg-white/10 backdrop-blur-md border-2
border-white/30 text-white font-bold text-sm md:text-lg rounded-xl md:rounded-2xl
hover:bg-white/20 hover:scale-105 transition-all duration-300 transform"
370             >
371                 Explore Services
372             </a>
373         </div>
374
375     {{/* Scroll Indicator */}}
376     <div className="absolute bottom-10 left-1/2 transform -translate-x-1/2
animate-bounce">
377         <svg className="w-6 h-6 text-white" fill="none" stroke="currentColor" viewBox="0 0
24 24">
378             <path strokeLinecap="round" strokeLinejoin="round" strokeWidth={{2}} d="M19 14l-7
7m0 0l-7-7m7 7V3" />
379         </svg>
380     </div>
381 </div>
382 </section>
383 )
384 }}
385
386 export default Hero
387 ""

```

```
388
389     # Generate proper image URLs for services using dynamic Pixabay images
390     _MARKETING_PREFIX = re.compile(
391         r'^(premium|special|classic|deluxe|signature|house|royal|chef\'?s?|fresh|homemade)\s+',
392         re.IGNORECASE
393     )
394
395     def _clean_name(name: str) -> str:
396         """Strip common marketing prefixes from service display names."""
397         return _MARKETING_PREFIX.sub('', name).strip()
398
399     services_with_images = []
400     for idx, service in enumerate(services[:6]):
401         # Use pre-fetched service images (offset by 1 since images[0] is the hero)
402         if idx < len(service_images_list) and service_images_list[idx].get("url"):
403             img_url = service_images_list[idx].get("url")
404         else:
405             # Fallback: unique seed per service so cards don't all look the same
406             seed = service.lower().replace(' ', '').replace("'", '')
407             img_url = f"https://picsum.photos/seed/{seed}/640/400"
408
409         meta = service_meta.get(service, {})
410         services_with_images.append({
411             "name": _clean_name(service),
412             "image": img_url,
413             "description": service_descriptions.get(service, f"Elevate your
414             {_clean_name(service).lower()} experience with unmatched quality and precision."),
415             "category": meta.get("category", ""),
416             "subcategory": meta.get("subcategory", ""),
417             "price": meta.get("price", ""),
418         })
419
420     services_code = f"""import React, {{ { useState } }} from 'react'
421
422     const services = {json.dumps(services_with_images)}
423
424     // Build case-insensitive deduplicated category list, preserving first-seen casing
425     const _catMap = new Map()
426     services.filter(s => s.category).forEach(s => {{
427         const key = s.category.toLowerCase()
428         if (!_catMap.has(key)) _catMap.set(key, s.category)
429     }})
430     const allCategories = ['All', ...Array.from(_catMap.values())]
431     const hasCategories = allCategories.length > 1
432
433     function Services() {{
434         const [activeCategory, setActiveCategory] = useState('All')
435         const [activeSubcategory, setActiveSubcategory] = useState('All')
436
437         const filteredByCategory = activeCategory === 'All'
438             ? services
439             : services.filter(s => s.category?.toLowerCase() === activeCategory.toLowerCase())
```

```

440     const _subMap = new Map()
441     filteredByCategory.filter(s => s.subcategory).forEach(s => {{
442         const key = s.subcategory.toLowerCase()
443         if (!_subMap.has(key)) _subMap.set(key, s.subcategory)
444     }})
445     const subcategories = activeCategory === 'All' ? [] : ['All',
...Array.from(_subMap.values())]
446     const hasSubcategories = subcategories.length > 1
447
448     const filtered = (hasSubcategories && activeSubcategory !== 'All')
449         ? filteredByCategory.filter(s => s.subcategory?.toLowerCase() ===
activeSubcategory.toLowerCase())
450       : filteredByCategory
451
452     const handleCategoryChange = (cat) => {{
453         setActiveCategory(cat)
454         setActiveSubcategory('All')
455     }}
456
457     return (
458         <section id="services" className="py-20 bg-gradient-to-br from-gray-50 to-white">
459             <div className="container-custom">
460                 <div className="text-center mb-10 md:mb-12">
461                     <h2 className="text-3xl md:text-5xl lg:text-6xl font-bold mb-3 md:mb-4
bg-gradient-to-r from-primary to-secondary bg-clip-text text-transparent">
462                         Our Services
463                     </h2>
464                     <p className="text-base md:text-xl text-gray-600 max-w-2xl mx-auto px-4">
465                         Discover our premium offerings designed to exceed your expectations
466                     </p>
467                 </div>
468
469                 {{/* Category filter pills ? only shown when categories exist */}}
470                 {{hasCategories && (
471                     <div className="flex flex-wrap gap-2 justify-center mb-4">
472                         {{allCategories.map(cat => (
473                             <button
474                                 key={{cat}}
475                                 onClick={{() => handleCategoryChange(cat)}}
476                                 className={{`px-5 py-2 rounded-full font-semibold text-sm transition-all
duration-300 ${{
477                                     activeCategory === cat
478                                     ? 'bg-gradient-to-r from-primary to-secondary text-white shadow-lg
scale-105'
479                                     : 'bg-white text-gray-600 border border-gray-200 hover:border-primary
hover:text-primary'
480                                 }}`}}
481                             >
482                                 {{cat}}
483                             </button>
484                         )}}
485                     </div>
486                 )}}
487

```

```

488         {{/* Subcategory filter pills ? only shown when a category with multiple
subcategories is selected */}}
489         {{hasSubcategories && (
490         <div className="flex flex-wrap gap-2 justify-center mb-8">
491             {{subcategories.map(sub => (
492                 <button
493                     key={{sub}}
494                     onClick={{() => setActiveSubcategory(sub)}}
495                     className={{`px-4 py-1.5 rounded-full text-xs font-medium transition-all
duration-300 ${{
496                         activeSubcategory === sub
497                         ? 'bg-secondary text-white shadow-md'
498                         : 'bg-gray-100 text-gray-500 hover:bg-gray-200'
499                     }}`}}
500                 >
501                     {{sub}}
502                 </button>
503             )}}
504         </div>
505     )}}
506
507     {{!hasCategories && <div className="mb-8" />}}
508
509     <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-6 md:gap-8">
510         {{filtered.map((service, index) => (
511             <div
512                 key={{index}}
513                 className="group bg-white rounded-2xl md:rounded-3xl shadow-xl hover:shadow-2xl
transition-all duration-500 hover:-translate-y-2 overflow-hidden border border-gray-100"
514             >
515                 {{/* Service Image */}}
516                 <div className="relative h-48 md:h-56 overflow-hidden">
517                     <img
518                         src={{service.image}}
519                         alt={{service.name}}
520                         className="w-full h-full object-cover transition-transform duration-500
group-hover:scale-110"
521                     >
522                         {{e => {{
523                             e.target.src = 'https://picsum.photos/seed/service/640/400'
524                         }}}}}
525                     />
526                     {{service.category && (
527                         <span className="absolute top-3 left-3 bg-white/90 backdrop-blur-sm
text-primary text-xs font-semibold px-3 py-1 rounded-full shadow-sm">
528                             {{service.category}}
529                         </span>
530                     )}}
531                     <div className="absolute inset-0 bg-gradient-to-t from-black/60
to-transparent opacity-0 group-hover:opacity-100 transition-opacity duration-300"></div>
532                 </div>
533                 {{/* Service Content */}}
534                 <div className="p-5 md:p-8">
535                     <h3 className="text-xl md:text-2xl font-bold mb-2 text-gray-900

```

```
        group-hover:text-primary transition-colors">
536             {{service.name}}
537         </h3>
538         {{service.price && (
539             <p className="text-primary font-bold text-lg mb-2">{{service.price}}</p>
540         )}}
541         <p className="text-sm md:text-base text-gray-600 mb-4 md:mb-6
leading-relaxed">
542             {{service.description}}
543         </p>
544         <button className="px-5 md:px-6 py-2.5 md:py-3 bg-gradient-to-r from-primary
to-secondary text-white rounded-lg md:rounded-xl font-semibold hover:shadow-lg transition-all
duration-300 transform hover:scale-105 text-sm md:text-base w-full md:w-auto">
545             {{service.price ? 'Order Now' : 'Learn More'}}
546         </button>
547     </div>
548 </div>
549     )}}
550 </div>
551 </div>
552 </section>
553 )
554 }}
555
556 export default Services
557 ""
558
559     contact_code = f""import React, {{ useState }} from 'react'
560
561 function Contact() {{
562     const [formData, setFormData] = useState({{
563         name: '',
564         email: '',
565         message: ''
566     }})
567     const [status, setStatus] = useState('')
568
569     const handleSubmit = async (e) => {{
570         e.preventDefault()
571         setStatus('sending')
572
573         // TODO: Integrate with Supabase or backend API
574         setTimeout(() => {{
575             setStatus('success')
576             setFormData({{ name: '', email: '', message: '' }})
577             setTimeout(() => setStatus(''), 3000)
578         }}, 1000)
579     }}
580
581     return (
582         <section id="contact" className="relative py-24 overflow-hidden">
583             {{/* Gradient Background */}}
584             <div className="absolute inset-0 bg-gradient-to-br from-primary via-secondary
to-purple-900"></div>
```

```
585
586     {{/* Animated Background Elements */}}
587     <div className="absolute top-0 left-0 w-96 h-96 bg-white/10 rounded-full blur-3xl
animate-pulse"></div>
588     <div className="absolute bottom-0 right-0 w-96 h-96 bg-white/10 rounded-full blur-3xl
animate-pulse delay-1000"></div>
589
590     <div className="container-custom max-w-4xl relative z-10">
591         <div className="text-center mb-16">
592             <h2 className="text-5xl md:text-6xl font-bold text-white mb-6">
593                 Let's Start a Conversation
594             </h2>
595             <p className="text-xl text-white/90 max-w-2xl mx-auto">
596                 Have a question or ready to get started? We'd love to hear from you.
597             </p>
598         </div>
599
600         <div className="bg-white/10 backdrop-blur-xl rounded-3xl p-8 md:p-12 border
border-white/20 shadow-2xl">
601             <form onSubmit={{handleSubmit}} className="space-y-6">
602                 <div className="grid md:grid-cols-2 gap-6">
603                     <div>
604                         <label className="block text-white/90 font-semibold mb-2">Full Name</label>
605                         <input
606                             type="text"
607                             placeholder="John Doe"
608                             required
609                             value={{formData.name}}
610                             onChange={{(e) => setFormData({...formData, name: e.target.value}})}}
611                             className="w-full px-6 py-4 rounded-xl bg-white/10 backdrop-blur-sm border
border-white/30 text-white placeholder-white/50 focus:outline-none focus:border-white/60
focus:bg-white/15 transition-all"
612                         />
613                     </div>
614                     <div>
615                         <label className="block text-white/90 font-semibold mb-2">Email
Address</label>
616                         <input
617                             type="email"
618                             placeholder="john@example.com"
619                             required
620                             value={{formData.email}}
621                             onChange={{(e) => setFormData({...formData, email: e.target.value}})}}
622                             className="w-full px-6 py-4 rounded-xl bg-white/10 backdrop-blur-sm border
border-white/30 text-white placeholder-white/50 focus:outline-none focus:border-white/60
focus:bg-white/15 transition-all"
623                         />
624                     </div>
625                 </div>
626                 <div>
627                     <label className="block text-white/90 font-semibold mb-2">Your Message</label>
628                     <textarea
629                         placeholder="Tell us about your project..."
630                         required
```

```
631         rows={{6}}
632         value={{formData.message}}
633         onChange={{(e) => setFormData({...formData, message: e.target.value})}}
634         className="w-full px-6 py-4 rounded-xl bg-white/10 backdrop-blur-sm border
border-white/30 text-white placeholder-white/50 focus:outline-none focus:border-white/60
focus:bg-white/15 transition-all resize-none"
635     ></textarea>
636 </div>
637 <button
638     type="submit"
639     disabled={{status === 'sending'}}
640     className="w-full bg-white text-primary font-bold py-5 px-8 rounded-xl
hover:shadow-2xl hover:scale-105 transition-all duration-300 transform disabled:opacity-70
disabled:cursor-not-allowed text-lg"
641 >
642     {{status === 'sending' ? 'Sending...' : 'Send Message'}}
643 </button>
644     {{status === 'success' && (
645         <div className="text-center p-4 bg-green-500/20 border border-green-400/30
rounded-xl">
646             <p className="text-white font-semibold">Message sent successfully! We'll get
back to you soon.</p>
647         </div>
648     )}}
649 </form>
650 </div>
651
652     {{/* Contact Info */}}
653     <div className="mt-12 grid md:grid-cols-3 gap-6 text-center text-white">
654         <div className="bg-white/10 backdrop-blur-sm rounded-2xl p-6 border
border-white/20">
655             <div className="text-3xl mb-3">?</div>
656             <h3 className="font-bold mb-2">Email</h3>
657             <p className="text-white/80">contact@{business_name.lower().replace(' ',
'' )}.com</p>
658         </div>
659         <div className="bg-white/10 backdrop-blur-sm rounded-2xl p-6 border
border-white/20">
660             <div className="text-3xl mb-3">?</div>
661             <h3 className="font-bold mb-2">Phone</h3>
662             <p className="text-white/80">+1 (555) 123-4567</p>
663         </div>
664         <div className="bg-white/10 backdrop-blur-sm rounded-2xl p-6 border
border-white/20">
665             <div className="text-3xl mb-3">?</div>
666             <h3 className="font-bold mb-2">Location</h3>
667             <p className="text-white/80">Your City, State</p>
668         </div>
669     </div>
670 </div>
671 </section>
672 )
673 }}
674
```

```
675 export default Contact
676 ""
677
678 # About Component - Always include
679 about_text = analysis.get("about_text", f"At {business_name}, we're dedicated to
excellence")
680 about_code = f""import React from 'react'
681
682 function About() {{
683   return (
684     <section id="about" className="py-24 bg-white">
685       <div className="container-custom">
686         <div className="grid md:grid-cols-2 gap-8 md:gap-12 items-center">
687           {{/* About Image */}}
688           <div className="relative order-2 md:order-1">
689             <div className="relative rounded-2xl md:rounded-3xl overflow-hidden shadow-2xl">
690               
695               <div className="absolute inset-0 bg-gradient-to-t from-black/50
to-transparent"></div>
696             </div>
697             {{/* Decorative element - hidden on mobile */}}
698             <div className="hidden md:block absolute -bottom-6 -right-6 w-64 h-64
bg-gradient-to-br from-primary to-secondary rounded-3xl opacity-20 -z-10"></div>
699           </div>
700
701           {{/* About Content */}}
702           <div className="order-1 md:order-2">
703             <div className="inline-block px-4 py-2 bg-primary/10 rounded-full mb-4 md:mb-6">
704               <span className="text-primary font-semibold text-sm md:text-base">About
Us</span>
705             </div>
706             <h2 className="text-3xl md:text-4xl lg:text-5xl font-bold mb-4 md:mb-6
text-gray-900">
707               Your Trusted Partner for Excellence
708             </h2>
709             <p className="text-base md:text-lg text-gray-600 mb-4 md:mb-6 leading-relaxed">
710               {about_text}
711             </p>
712             <p className="text-base md:text-lg text-gray-600 mb-6 md:mb-8 leading-relaxed">
713               With years of experience and a passion for quality, we bring you the best
service possible. Our team is dedicated to exceeding your expectations every single time.
714             </p>
715
716             {{/* Stats */}}
717             <div className="grid grid-cols-3 gap-3 md:gap-6 mt-6 md:mt-8">
718               <div className="text-center p-3 md:p-4 bg-gradient-to-br from-primary/10
to-secondary/10 rounded-xl md:rounded-2xl">
719                 <div className="text-2xl md:text-3xl font-bold text-primary mb-1">500+</div>
720                 <div className="text-xs md:text-sm text-gray-600">Happy Clients</div>
721               </div>
```

```

722         <div className="text-center p-3 md:p-4 bg-gradient-to-br from-primary/10
to-secondary/10 rounded-xl md:rounded-2xl">
723             <div className="text-2xl md:text-3xl font-bold text-primary mb-1">10+</div>
724             <div className="text-xs md:text-sm text-gray-600">Years Experience</div>
725         </div>
726         <div className="text-center p-3 md:p-4 bg-gradient-to-br from-primary/10
to-secondary/10 rounded-xl md:rounded-2xl">
727             <div className="text-2xl md:text-3xl font-bold text-primary mb-1">98%</div>
728             <div className="text-xs md:text-sm text-gray-600">Satisfaction</div>
729         </div>
730     </div>
731
732     <button className="mt-6 md:mt-8 px-6 md:px-8 py-3 md:py-4 bg-gradient-to-r
from-primary to-secondary text-white font-bold rounded-xl hover:shadow-2xl hover:scale-105
transition-all duration-300 text-sm md:text-base w-full md:w-auto">
733         Learn More About Us
734     </button>
735 </div>
736 </div>
737 </div>
738 </section>
739 )
740 }}
741
742 export default About
743 ""
744
745 # Gallery Component
746 # Use dedicated gallery images (appended at index 7+ by react_api_routes),
747 # then fill any remaining slots with hero/service images, then picsum fallback
748 dedicated_gallery = [img.get("url") for img in images[7:] if img.get("url")]
749 hero_service_pool = [img.get("url") for img in images[:7] if img.get("url")]
750 gallery_pool = dedicated_gallery + hero_service_pool
751 gallery_images_list = []
752 for i in range(6):
753     if i < len(gallery_pool):
754         gallery_images_list.append(gallery_pool[i])
755     else:
756         gallery_images_list.append(f"https://picsum.photos/seed/gallery{i}/600/400")
757
758 gallery_images_json = json.dumps(gallery_images_list)
759
760 gallery_code = f"""import React from 'react'
761
762 const galleryImages = {gallery_images_json}
763
764 function Gallery() {{
765     return (
766         <section id="gallery" className="py-24 bg-gradient-to-b from-gray-50 to-white">
767             <div className="container-custom">
768                 <div className="text-center mb-12 md:mb-16">
769                     <h2 className="text-3xl md:text-5xl lg:text-6xl font-bold mb-3 md:mb-4
bg-gradient-to-r from-primary to-secondary bg-clip-text text-transparent">
770                         Our Gallery

```

```
771         </h2>
772         <p className="text-base md:text-xl text-gray-600 max-w-2xl mx-auto px-4">
773             Take a look at our work and see why clients love us
774         </p>
775     </div>
776
777     <div className="grid grid-cols-1 sm:grid-cols-2 lg:grid-cols-3 gap-4 md:gap-6">
778         {{galleryImages.map((image, index) => (
779             <div
780                 key={{index}}
781                 className="group relative overflow-hidden rounded-xl md:rounded-2xl shadow-lg
782                 hover:shadow-2xl transition-all duration-500 aspect-video"
783             >
784                 <img
785                     src={{image}}
786                     alt={{`Gallery ${{index + 1}}`}}
787                     className="w-full h-full object-cover transform group-hover:scale-110
788                     transition-transform duration-500"
789                 />
790                 <div className="absolute inset-0 bg-gradient-to-t from-black/70 via-black/20
791                 to-transparent opacity-0 group-hover:opacity-100 transition-opacity duration-300 flex
792                 items-end p-4 md:p-6">
793                     <div className="text-white">
794                         <h3 className="text-lg md:text-xl font-bold">Image {{index + 1}}</h3>
795                         <p className="text-xs md:text-sm opacity-90">Click to view</p>
796                     </div>
797                 </div>
798             </div>
799         )}}
800     </div>
801 </section>
802 )
803 }}
804
805 export default Gallery
806
807
808 # Testimonials Component
809 testimonials_code = f"""import React from 'react'
810
811 const testimonials = [
812     {{
813         name: 'Sarah Johnson',
814         role: 'Happy Customer',
815         image: 'https://i.pravatar.cc/150?img=5',
816         text: 'Amazing service! Highly recommend {business_name} to everyone. The quality and
817         attention to detail are outstanding.',
818         rating: 5
819     }},
820     {{
821         name: 'Michael Chen',
822         role: 'Business Owner',
823         image: 'https://i.pravatar.cc/150?img=7',
```

```

820         text: 'Professional, reliable, and absolutely fantastic. They exceeded all my
expectations and delivered on time.',
821         rating: 5
822     }},
823     {{
824         name: 'Emily Davis',
825         role: 'Regular Client',
826         image: 'https://i.pravatar.cc/150?img=12',
827         text: 'Best decision ever! The team is incredible and the results speak for themselves.
Five stars!',
828         rating: 5
829     }},
830 ]
831
832 function Testimonials() {{
833     return (
834         <section id="testimonials" className="py-24 bg-gradient-to-br from-primary/5
to-secondary/5">
835             <div className="container-custom">
836                 <div className="text-center mb-12 md:mb-16">
837                     <h2 className="text-3xl md:text-5xl lg:text-6xl font-bold mb-3 md:mb-4
bg-gradient-to-r from-primary to-secondary bg-clip-text text-transparent">
838                         What Our Clients Say
839                     </h2>
840                     <p className="text-base md:text-xl text-gray-600 max-w-2xl mx-auto px-4">
841                         Don't just take our word for it - hear from our satisfied customers
842                     </p>
843                 </div>
844
845                 <div className="grid sm:grid-cols-2 lg:grid-cols-3 gap-6 md:gap-8">
846                     {{testimonials.map((testimonial, index) => (
847                         <div
848                             key={{index}}
849                             className="bg-white p-6 md:p-8 rounded-2xl md:rounded-3xl shadow-xl
hover:shadow-2xl transition-all duration-300 hover:-translate-y-2"
850                         >
851                             {{/* Stars */}}
852                             <div className="flex gap-1 mb-3 md:mb-4">
853                                 {{[...Array(testimonial.rating)].map((_, i) => (
854                                     <svg key={{i}} className="w-4 h-4 md:w-5 md:h-5 text-yellow-400
fill-current" viewBox="0 0 20 20">
855                                     <path d="M10 15l-5.878 3.09 1.123-6.545L4.89 6.91l6.572-.955L10 0l2.939
5.955 6.572.955-4.756 4.635 1.123 6.545z" />
856                                 </svg>
857                             )}}}}
858                             </div>
859
860                             {{/* Quote */}}
861                             <p className="text-sm md:text-base text-gray-600 mb-4 md:mb-6 leading-relaxed
italic">
862                                 "{{{testimonial.text}}}"
863                             </p>
864
865                             {{/* Author */}}

```

```
866         <div className="flex items-center gap-3 md:gap-4">
867             <img
868                 src={{testimonial.image}}
869                 alt={{testimonial.name}}
870                 className="w-10 h-10 md:w-12 md:h-12 rounded-full object-cover ring-2
ring-primary/20"
871             />
872             <div>
873                 <h4 className="font-bold text-sm md:text-base
text-gray-900">{{testimonial.name}}</h4>
874                 <p className="text-xs md:text-sm text-gray-500">{{testimonial.role}}</p>
875             </div>
876         </div>
877     </div>
878     )}}
879 </div>
880 </div>
881 </section>
882 )
883 }}
884
885 export default Testimonials
886 ""
887
888 # Features/Benefits Component
889 features_benefits_code = f""import React from 'react'
890
891 const features = [
892     {{
893         icon: '?',
894         title: 'Fast & Efficient',
895         description: 'Quick turnaround time without compromising on quality'
896     }},
897     {{
898         icon: '?',
899         title: 'Premium Quality',
900         description: 'Only the best materials and techniques used'
901     }},
902     {{
903         icon: '?',
904         title: 'Attention to Detail',
905         description: 'We pay attention to every little detail'
906     }},
907     {{
908         icon: '?',
909         title: '24/7 Support',
910         description: 'Always here when you need us'
911     }},
912     {{
913         icon: '?',
914         title: 'Award Winning',
915         description: 'Recognized for excellence in our industry'
916     }},
917     {{
```

```
918     icon: '?',
919     title: 'Best Value',
920     description: 'Competitive pricing with unmatched quality'
921   }},
922 ]
923
924 function Features() {{
925   return (
926     <section id="features" className="py-24 bg-white">
927       <div className="container-custom">
928         <div className="text-center mb-16">
929           <h2 className="text-4xl md:text-6xl font-bold mb-4 bg-gradient-to-r from-primary
to-secondary bg-clip-text text-transparent">
930             Why Choose Us
931           </h2>
932           <p className="text-xl text-gray-600 max-w-2xl mx-auto">
933             Discover what makes us the best choice for your needs
934           </p>
935         </div>
936
937         <div className="grid md:grid-cols-2 lg:grid-cols-3 gap-8">
938           {{features.map((feature, index) => (
939             <div
940               key={{index}}
941               className="group p-8 bg-gradient-to-br from-gray-50 to-white rounded-3xl border
border-gray-100 hover:border-primary/30 hover:shadow-xl transition-all duration-300"
942             >
943               <div className="text-5xl mb-4 group-hover:scale-110 transition-transform
duration-300">
944                 {{feature.icon}}
945               </div>
946               <h3 className="text-2xl font-bold mb-3 text-gray-900">
947                 {{feature.title}}
948               </h3>
949               <p className="text-gray-600 leading-relaxed">
950                 {{feature.description}}
951               </p>
952             </div>
953           )}})
954         </div>
955       </div>
956     </section>
957   )
958 }}
959
960 export default Features
961 ""
962
963 # Build components list dynamically based on features
964 components = [
965   {"name": "Navbar", "description": "Navigation bar", "code": navbar_code},
966   {"name": "Hero", "description": "Hero section", "code": hero_code},
967   {"name": "About", "description": "About section", "code": about_code},
968   {"name": "Services", "description": "Services grid", "code": services_code},
```

```
969     ]
970
971     # Add feature-based components
972     feature_list_lower = [f.lower() for f in features]
973     feature_string = " ".join(feature_list_lower)
974
975     # Gallery detection - includes transformation gallery, photos, etc.
976     if any(keyword in feature_string for keyword in ["gallery", "photos",
"transformation"]):
977         components.append({"name": "Gallery", "description": "Image gallery", "code":
gallery_code})
978
979     # Testimonials/Reviews detection
980     if any(keyword in feature_string for keyword in ["testimonials", "reviews",
"success"]):
981         components.append({"name": "Testimonials", "description": "Customer
testimonials", "code": testimonials_code})
982
983     # E-COMMERCE FEATURES - Product Catalog, Shopping Cart, Online Ordering
984     if any(keyword in feature_string for keyword in ["menu", "catalog", "product",
"ordering", "cart", "shop"]):
985         # Build products list with proper image mapping
986         products_list = []
987         for idx, service in enumerate(services[:6]):
988             product = {
989                 "id": idx + 1,
990                 "name": service,
991                 "price": (idx + 1) * 25 + 99,
992                 "category": service
993             }
994
995             # Get image from services_with_images or use fallback
996             if idx < len(services_with_images):
997                 product["image"] = services_with_images[idx]["image"]
998             else:
999                 product["image"] = f"https://picsum.photos/seed/{service.replace(' ',
''')}/640/400"
1000
1001             products_list.append(product)
1002
1003     # Product Catalog Component
1004     product_catalog_code = f"""import React, {{ {useState} }} from 'react'
1005
1006 const ProductCatalog = () => {{
1007     const products = {json.dumps(products_list, indent=2)}
1008
1009     const [selectedCategory, setSelectedCategory] = useState('All')
1010     const categories = ['All', ...new Set(products.map(p => p.category))]
1011
1012     const filteredProducts = selectedCategory === 'All'
1013         ? products
1014         : products.filter(p => p.category === selectedCategory)
1015
1016     return (
```

```

1017     <section id="products" className="py-24 bg-gradient-to-b from-gray-50 to-white">
1018         <div className="container-custom">
1019             <h2 className="text-4xl md:text-6xl font-bold text-center mb-4 bg-gradient-to-r
from-primary to-secondary bg-clip-text text-transparent">
1020                 Our Products
1021             </h2>
1022             <p className="text-xl text-gray-600 text-center mb-12 max-w-2xl mx-auto">
1023                 Explore our curated collection
1024             </p>
1025
1026             { {/* Category Filter */ }}
1027             <div className="flex flex-wrap justify-center gap-4 mb-12">
1028                 {{categories.map(category => (
1029                     <button
1030                         key={{category}}
1031                         onClick={{() => setSelectedCategory(category)}}
1032                         className={{`px-6 py-3 rounded-full font-semibold transition-all ${
1033                             selectedCategory === category
1034                                 ? 'bg-primary text-white shadow-lg'
1035                                 : 'bg-white text-gray-700 hover:bg-gray-100'
1036                         }}`}}
1037                     >
1038                         {{category}}
1039                     </button>
1040                 )}}
1041             </div>
1042
1043             { {/* Products Grid */ }}
1044             <div className="grid sm:grid-cols-2 lg:grid-cols-3 gap-8">
1045                 {{filteredProducts.map((product) => (
1046                     <div key={{product.id}} className="group bg-white rounded-3xl overflow-hidden
shadow-lg hover:shadow-2xl transition-all duration-300">
1047                         <div className="relative h-64 overflow-hidden">
1048                             <img
1049                                 src={{product.image}}
1050                                 alt={{product.name}}
1051                                 className="w-full h-full object-cover transition-transform duration-500
group-hover:scale-110"
1052                             />
1053                             <div className="absolute top-4 right-4 bg-white/90 backdrop-blur px-4 py-2
rounded-full font-bold text-primary">
1054                                 ?{{product.price}}
1055                             </div>
1056                         </div>
1057                         <div className="p-6">
1058                             <h3 className="text-xl font-bold mb-2">{{product.name}}</h3>
1059                             <p className="text-gray-600 mb-4">{{product.category}}</p>
1060                             <button className="w-full bg-gradient-to-r from-primary to-secondary
text-white font-semibold py-3 rounded-full hover:shadow-lg transition-all">
1061                                 Add to Cart
1062                             </button>
1063                         </div>
1064                     </div>
1065                 )}}

```

```
1066         </div>
1067     </div>
1068 </section>
1069 )
1070 }}
1071
1072 export default ProductCatalog
1073 ""
1074     components.append({"name": "ProductCatalog", "description": "Product catalog with
1075     cart", "code": product_catalog_code})
1076
1077     # Size Guide Component (for fashion/clothing stores)
1078     if any(keyword in feature_string for keyword in ["size_guide", "size", "sizing"]):
1079         size_guide_code = f""import React, {{ useState }} from 'react'
1080
1081     const SizeGuide = () => {{
1082         const [isOpen, setIsOpen] = useState(false)
1083
1084         const sizeChart = {{
1085             'XS': {{ chest: '30-32', waist: '24-26', hips: '34-36' }},
1086             'S': {{ chest: '32-34', waist: '26-28', hips: '36-38' }},
1087             'M': {{ chest: '34-36', waist: '28-30', hips: '38-40' }},
1088             'L': {{ chest: '36-38', waist: '30-32', hips: '40-42' }},
1089             'XL': {{ chest: '38-40', waist: '32-34', hips: '42-44' }},
1090         }}
1091
1092         return (
1093             <div>
1094                 <button
1095                     onClick={{() => setIsOpen(true)}}
1096                     className="fixed bottom-8 right-8 bg-primary text-white p-4 rounded-full shadow-2xl
1097                     hover:shadow-3xl transition-all z-40"
1098                 >
1099                     ? Size Guide
1100                 </button>
1101
1102                 {{isOpen && (
1103                     <div className="fixed inset-0 bg-black/50 backdrop-blur-sm z-50 flex items-center
1104                     justify-center p-4" onClick={{() => setIsOpen(false)}}>
1105                         <div className="bg-white rounded-3xl p-8 max-w-2xl w-full" onClick={{e =>
1106                         e.stopPropagation()}}>
1107                             <div className="flex justify-between items-center mb-6">
1108                                 <h3 className="text-3xl font-bold">Size Guide</h3>
1109                                 <button onClick={{() => setIsOpen(false)}}
1110                                 className="text-3xl">&times;</button>
1111                             </div>
1112                             <div className="overflow-x-auto">
1113                                 <table className="w-full">
1114                                     <thead>
1115                                         <tr className="bg-gradient-to-r from-primary to-secondary text-white">
1116                                             <th className="p-4 text-left">Size</th>
1117                                             <th className="p-4 text-left">Chest (in)</th>
1118                                             <th className="p-4 text-left">Waist (in)</th>
```

```
1115         <th className="p-4 text-left">Hips (in)</th>
1116     </tr>
1117 </thead>
1118 <tbody>
1119     {{Object.entries(sizeChart).map([size, measurements], idx) => (
1120         <tr key={{size}} className={{idx % 2 === 0 ? 'bg-gray-50' : 'bg-white'}}>
1121         <td className="p-4 font-bold">{{size}}</td>
1122         <td className="p-4">{{measurements.chest}}</td>
1123         <td className="p-4">{{measurements.waist}}</td>
1124         <td className="p-4">{{measurements.hips}}</td>
1125         </tr>
1126     )}}
1127 </tbody>
1128 </table>
1129 </div>
1130
1131     <p className="mt-6 text-sm text-gray-600">
1132         * All measurements are in inches. For best fit, please measure yourself and
compare with our size chart.
1133     </p>
1134 </div>
1135 </div>
1136     )}}
1137 </>
1138 )
1139 }}
1140
1141 export default SizeGuide
1142 ""
1143         components.append({"name": "SizeGuide", "description": "Size guide modal",
"code": size_guide_code})
1144
1145         # Always add Features/Benefits
1146         components.append({"name": "Features", "description": "Features and benefits",
"code": features_benefits_code})
1147
1148         # Contact is always last
1149         components.append({"name": "Contact", "description": "Contact form", "code":
contact_code})
1150
1151         return components
1152
1153     def _extract_supabase_features(self, features: List[str]) -> List[str]:
1154         """Extract which Supabase features to enable"""
1155         supabase_features = []
1156
1157         # Check if any feature requires database
1158         if any(f in ["booking", "order", "contact", "auth", "ecommerce"] for f in [f.lower()
for f in features]):
1159             supabase_features.append("database")
1160
1161         # Check for specific features
1162         feature_mapping = {
1163             "booking": "database",
```

```
1164         "order": "database",
1165         "ecommerce": "database",
1166         "auth": "auth",
1167         "login": "auth",
1168         "upload": "storage",
1169         "gallery": "storage",
1170         "real-time": "realtime",
1171         "chat": "realtime"
1172     }
1173
1174     for feature in features:
1175         feature_lower = feature.lower()
1176         if feature_lower in feature_mapping:
1177             mapped = feature_mapping[feature_lower]
1178             if mapped not in supabase_features:
1179                 supabase_features.append(mapped)
1180
1181     return supabase_features
1182
1183
1184 # Global React builder instance
1185 react_builder = ReactWebsiteBuilder()
1186
```

File 4: backend/react_template_generator.pyBackend - React Template Generator

```
1      """
2      React/Vite Template Generator
3      Creates complete React + Vite projects with Tailwind CSS and Supabase integration
4      """
5
6      import json
7      from typing import Dict, List, Optional
8
9      class ReactTemplateGenerator:
10         """Generates React/Vite project files with modern setup"""
11
12         def __init__(self):
13             self.supabase_enabled = True
14             self.tailwind_enabled = True
15
16         def generate_complete_project(
17             self,
18             project_name: str,
19             description: str,
20             components: List[Dict],
21             theme: Dict,
22             features: List[str],
23             supabase_config: Optional[Dict] = None
24         ) -> Dict[str, str]:
25             """
26             Generate all files for a complete React/Vite project
27
28             Args:
29                 project_name: Name of the project
30                 description: Project description
31                 components: List of React components to generate
32                 theme: Theme configuration (colors, fonts, etc.)
33                 features: List of features (auth, booking, ecommerce, etc.)
34                 supabase_config: Supabase configuration if enabled
35
36             Returns:
37                 Dictionary of {file_path: file_content}
38             """
39             files = {}
40
41             # Core configuration files
42             files["package.json"] = self.generate_package_json(project_name, description,
43             features)
44             files["vite.config.js"] = self.generate_vite_config()
45             files["index.html"] = self.generate_index_html(project_name)
46             files[".gitignore"] = self.generate_gitignore()
47             files["README.md"] = self.generate_readme(project_name, description)
48
49             # Tailwind CSS configuration
50             if self.tailwind_enabled:
51                 files["tailwind.config.js"] = self.generate_tailwind_config(theme)
```

```
51         files["postcss.config.js"] = self.generate_postcss_config()
52
53     # Main application files
54     files["src/main.jsx"] = self.generate_main_jsx()
55     files["src/App.jsx"] = self.generate_app_jsx(components, features)
56     files["src/index.css"] = self.generate_index_css()
57
58     # Supabase integration
59     if supabase_config and "database" in features:
60         files["src/lib/supabase.js"] = self.generate_supabase_client(supabase_config)
61
62     # Generate component files
63     for component in components:
64         comp_name = component.get("name")
65         comp_code = component.get("code", "")
66         files[f"src/components/{comp_name}.jsx"] = comp_code
67
68     # Generate pages
69     if "routing" in features:
70         files["src/pages/Home.jsx"] = self.generate_home_page(components)
71         files["src/pages/About.jsx"] = self.generate_about_page()
72
73     # Add utility files
74     files["src/utils/helpers.js"] = self.generate_helpers()
75
76     # Environment template
77     files[".env.example"] = self.generate_env_example(features)
78
79     return files
80
81     def generate_package_json(self, project_name: str, description: str, features: List[str])
-> str:
82         """Generate package.json with all dependencies"""
83         package_data = {
84             "name": project_name.lower().replace(" ", "-"),
85             "private": True,
86             "version": "0.1.0",
87             "type": "module",
88             "description": description,
89             "scripts": {
90                 "dev": "vite",
91                 "build": "vite build",
92                 "preview": "vite preview",
93                 "lint": "eslint . --ext js,jsx --report-unused-disable-directives
--max-warnings 0"
94             },
95             "dependencies": {
96                 "react": "^18.3.1",
97                 "react-dom": "^18.3.1",
98                 "react-router-dom": "^6.22.0"
99             },
100             "devDependencies": {
101                 "@vitejs/plugin-react": "^4.2.1",
102                 "vite": "^5.1.0",
```

```
103         "eslint": "^8.56.0",
104         "eslint-plugin-react": "^7.33.2",
105         "eslint-plugin-react-hooks": "^4.6.0",
106         "eslint-plugin-react-refresh": "^0.4.5"
107     }
108 }
109
110 # Add Tailwind CSS if enabled
111 if self.tailwind_enabled:
112     package_data["devDependencies"]["tailwindcss"] = "^3.4.1"
113     package_data["devDependencies"]["postcss"] = "^8.4.35"
114     package_data["devDependencies"]["autoprefixer"] = "^10.4.17"
115
116 # Add feature-specific dependencies
117 if "auth" in features or "database" in features or "storage" in features:
118     package_data["dependencies"]["@supabase/supabase-js"] = "^2.39.7"
119
120 if "animation" in features:
121     package_data["dependencies"]["framer-motion"] = "^11.0.5"
122
123 if "forms" in features:
124     package_data["dependencies"]["react-hook-form"] = "^7.50.1"
125
126 if "icons" in features:
127     package_data["dependencies"]["lucide-react"] = "^0.336.0"
128
129 if "charts" in features:
130     package_data["dependencies"]["recharts"] = "^2.12.0"
131
132 return json.dumps(package_data, indent=2)
133
134 def generate_vite_config(self) -> str:
135     """Generate vite.config.js"""
136     return """import { defineConfig } from 'vite'
137 import react from '@vitejs/plugin-react'
138
139 // https://vitejs.dev/config/
140 export default defineConfig({
141   plugins: [react()],
142   server: {
143     port: 3000,
144     open: true
145   },
146   build: {
147     outDir: 'dist',
148     sourcemap: false
149   }
150 })
151 """
152
153 def generate_index_html(self, project_name: str) -> str:
154     """Generate index.html"""
155     return f"""<!doctype html>
156 <html lang="en">
```

```
157     <head>
158         <meta charset="UTF-8" />
159         <link rel="icon" type="image/svg+xml" href="/vite.svg" />
160         <meta name="viewport" content="width=device-width, initial-scale=1.0" />
161         <title>{project_name}</title>
162     </head>
163     <body>
164         <div id="root"></div>
165         <script type="module" src="/src/main.jsx"></script>
166     </body>
167 </html>
168 """
169
170     def generate_gitignore(self) -> str:
171         """Generate .gitignore"""
172         return """# Logs
173 logs
174 *.log
175 npm-debug.log*
176 yarn-debug.log*
177 yarn-error.log*
178 pnpm-debug.log*
179 lerna-debug.log*
180
181 node_modules
182 dist
183 dist-ssr
184 *.local
185
186 # Editor directories and files
187 .vscode/*
188 !.vscode/extensions.json
189 .idea
190 .DS_Store
191 *.suo
192 *.ntvs*
193 *.njsproj
194 *.sln
195 *.sw?
196
197 # Environment variables
198 .env
199 .env.local
200 .env.*.local
201 """
202
203     def generate_readme(self, project_name: str, description: str) -> str:
204         """Generate README.md"""
205         return f"""# {project_name}
206
207 {description}
208
209 ## ? Getting Started
210
```

```
211   ### Prerequisites
212   - Node.js 18+
213   - npm or yarn
214
215   ### Installation
216
217   1. Install dependencies:
218   ```bash
219   npm install
220   ```
221
222   2. Copy environment variables:
223   ```bash
224   cp .env.example .env
225   ```
226
227   3. Add your Supabase credentials to `.env`
228
229   4. Start development server:
230   ```bash
231   npm run dev
232   ```
233
234   5. Build for production:
235   ```bash
236   npm run build
237   ```
238
239   ## ?? Built With
240
241   - **React 18** - UI library
242   - **Vite** - Build tool
243   - **Tailwind CSS** - Styling
244   - **Supabase** - Backend & Database
245   - **React Router** - Navigation
246
247   ## ? License
248
249   This project was generated by Sitekraft.
250
251   ---
252
253   Generated with ?? by Sitekraft
254   """
255
256   def generate_tailwind_config(self, theme: Dict) -> str:
257       """Generate tailwind.config.js with custom theme"""
258       colors = theme.get("colors", {})
259       fonts = theme.get("fonts", {})
260
261       return f"""/** @type {{import('tailwindcss').Config}} */
262   export default {{
263     content: [
264       "./index.html",
```

```
265     "./src/**/*.{{js,ts,jsx,tsx}}",
266 ],
267 theme: {{
268   extend: {{
269     colors: {{
270       primary: '{colors.get("primary", "#667eea")}',
271       secondary: '{colors.get("secondary", "#764ba2")}',
272       accent: '{colors.get("accent", "#f59e0b")}',
273     }},
274     fontFamily: {{
275       sans: ['{fonts.get("body", "Inter")}', 'system-ui', 'sans-serif'],
276       heading: ['{fonts.get("heading", "Poppins")}', 'system-ui', 'sans-serif'],
277     }},
278     animation: {{
279       'fade-in': 'fadeIn 0.8s ease-in-out',
280       'fade-in-up': 'fadeInUp 0.8s ease-out',
281       'slide-in': 'slideIn 0.6s ease-out',
282       'slow-zoom': 'slowZoom 20s ease-in-out infinite',
283       'bounce': 'bounce 2s infinite',
284     }},
285     keyframes: {{
286       fadeIn: {{
287         '0%': {{ opacity: '0' }},
288         '100%': {{ opacity: '1' }},
289       }},
290       fadeInUp: {{
291         '0%': {{ opacity: '0', transform: 'translateY(30px)' }},
292         '100%': {{ opacity: '1', transform: 'translateY(0)' }},
293       }},
294       slideIn: {{
295         '0%': {{ transform: 'translateX(-100%)' }},
296         '100%': {{ transform: 'translateX(0)' }},
297       }},
298       slowZoom: {{
299         '0%, 100%': {{ transform: 'scale(1)' }},
300         '50%': {{ transform: 'scale(1.1)' }},
301       }},
302     }},
303   }},
304 },
305 plugins: [],
306 }}
307 """
308
309 def generate_postcss_config(self) -> str:
310     """Generate postcss.config.js"""
311     return """export default {
312   plugins: {
313     tailwindcss: {},
314     autoprefixer: {},
315   },
316 }
317 """
318
```

```
319     def generate_main_jsx(self) -> str:
320         """Generate src/main.jsx"""
321         return """import React from 'react'
322 import ReactDOM from 'react-dom/client'
323 import App from './App.jsx'
324 import './index.css'
325
326 ReactDOM.createRoot(document.getElementById('root')).render(
327     <React.StrictMode>
328     <App />
329 </React.StrictMode>,
330 )
331 """
332
333     def generate_app_jsx(self, components: List[Dict], features: List[str]) -> str:
334         """Generate src/App.jsx"""
335         has_routing = "routing" in features
336
337         if has_routing:
338             return """import { BrowserRouter as Router, Routes, Route } from
'react-router-dom'
339 import Home from './pages/Home'
340 import About from './pages/About'
341 import './index.css'
342
343 function App() {
344     return (
345         <Router>
346         <Routes>
347             <Route path="/" element={<Home />} />
348             <Route path="/about" element={<About />} />
349         </Routes>
350     </Router>
351 )
352 }
353
354 export default App
355 """
356         else:
357             # Simple single-page app
358             comp_imports = "\n".join([f"import {c.get('name')} from
'./components/{c.get('name')}'" for c in components])
359             comp_renders = "\n    ".join([f"<{c.get('name')} />" for c in components])
360
361             return f"""import './index.css'
362 {comp_imports}
363
364 function App() {{
365     return (
366         <div className="app">
367             {comp_renders}
368         </div>
369     )
370 }}
```

```
371
372 export default App
373 ""
374
375     def generate_index_css(self) -> str:
376         """Generate src/index.css with Tailwind imports"""
377         return ""@tailwind base;
378 @tailwind components;
379 @tailwind utilities;
380
381 @layer base {
382     * {
383         @apply box-border;
384     }
385
386     body {
387         @apply antialiased overflow-x-hidden;
388     }
389
390     html {
391         scroll-behavior: smooth;
392         overflow-x: hidden;
393     }
394
395     /* Ensure images are responsive */
396     img {
397         @apply max-w-full h-auto;
398     }
399 }
400
401 @layer components {
402     .btn {
403         @apply px-4 py-2 sm:px-6 sm:py-3 rounded-lg font-semibold transition-all duration-200;
404     }
405
406     .btn-primary {
407         @apply bg-primary text-white hover:opacity-90;
408     }
409
410     .btn-secondary {
411         @apply bg-secondary text-white hover:opacity-90;
412     }
413
414     .container-custom {
415         @apply max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 w-full;
416     }
417
418     /* Mobile-optimized section spacing */
419     section {
420         @apply overflow-x-hidden;
421     }
422 }
423
424 @layer utilities {
```

```
425     /* Animation Delays */
426     .animation-delay-200 {
427         animation-delay: 200ms;
428     }
429
430     .animation-delay-400 {
431         animation-delay: 400ms;
432     }
433
434     .animation-delay-600 {
435         animation-delay: 600ms;
436     }
437
438     .delay-1000 {
439         animation-delay: 1000ms;
440     }
441
442     /* Fade in animations */
443     @keyframes fade-in-up {
444         from {
445             opacity: 0;
446             transform: translateY(20px);
447         }
448         to {
449             opacity: 1;
450             transform: translateY(0);
451         }
452     }
453
454     .animate-fade-in-up {
455         animation: fade-in-up 0.6s ease-out forwards;
456     }
457
458     @keyframes slow-zoom {
459         0%, 100% {
460             transform: scale(1);
461         }
462         50% {
463             transform: scale(1.05);
464         }
465     }
466
467     .animate-slow-zoom {
468         animation: slow-zoom 20s ease-in-out infinite;
469     }
470 }
471
472 /* Mobile-specific optimizations */
473 @media (max-width: 640px) {
474     h1 {
475         @apply text-3xl;
476     }
477
478     h2 {
```

```
479     @apply text-2xl;
480   }
481
482   h3 {
483     @apply text-xl;
484   }
485
486   /* Reduce padding on mobile */
487   .py-24 {
488     @apply py-12;
489   }
490
491   .py-20 {
492     @apply py-10;
493   }
494
495   /* Ensure content doesn't hide under fixed navbar */
496   body {
497     padding-top: 0;
498   }
499 }
500 """"
501
502     def generate_supabase_client(self, config: Dict) -> str:
503         """Generate Supabase client configuration"""
504         return f"""import {{ createClient }} from '@supabase/supabase-js'
505
506 const supabaseUrl = import.meta.env.VITE_SUPABASE_URL
507 const supabaseAnonKey = import.meta.env.VITE_SUPABASE_ANON_KEY
508
509 export const supabase = createClient(supabaseUrl, supabaseAnonKey)
510
511 // Helper functions
512 export const supabaseAuth = {{
513   signUp: async (email, password) => {{
514     return await supabase.auth.signUp({{ email, password }})
515   }},
516
517   signIn: async (email, password) => {{
518     return await supabase.auth.signInWithPassword({{ email, password }})
519   }},
520
521   signOut: async () => {{
522     return await supabase.auth.signOut()
523   }},
524
525   getUser: async () => {{
526     return await supabase.auth.getUser()
527   }}
528 }}
529
530 export const supabaseDB = {{
531   // Generic query helper
532   from: (table) => supabase.from(table),
```

```
533
534 // Insert data
535 insert: async (table, data) => {{
536   return await supabase.from(table).insert(data)
537 }},
538
539 // Update data
540 update: async (table, id, data) => {{
541   return await supabase.from(table).update(data).eq('id', id)
542 }},
543
544 // Delete data
545 delete: async (table, id) => {{
546   return await supabase.from(table).delete().eq('id', id)
547 }},
548
549 // Select all
550 selectAll: async (table) => {{
551   return await supabase.from(table).select('*')
552 }}
553 }}
554
555 export default supabase
556 ""
557
558   def generate_helpers(self) -> str:
559     """Generate utility helpers"""
560     return """/**
561  * Utility helper functions
562  */
563
564 // Format date
565 export const formatDate = (date) => {
566   return new Date(date).toLocaleDateString('en-US', {
567     year: 'numeric',
568     month: 'long',
569     day: 'numeric'
570   })
571 }
572
573 // Format currency
574 export const formatCurrency = (amount, currency = 'USD') => {
575   return new Intl.NumberFormat('en-US', {
576     style: 'currency',
577     currency: currency
578   }).format(amount)
579 }
580
581 // Truncate text
582 export const truncate = (text, length = 100) => {
583   if (text.length <= length) return text
584   return text.substring(0, length) + '...'
585 }
586
```

```
587 // Debounce function
588 export const debounce = (func, wait) => {
589   let timeout
590   return function executedFunction(...args) {
591     const later = () => {
592       clearTimeout(timeout)
593       func(...args)
594     }
595     clearTimeout(timeout)
596     timeout = setTimeout(later, wait)
597   }
598 }
599
600 // Generate random ID
601 export const generateId = () => {
602   return Math.random().toString(36).substring(2, 15) +
603   Math.random().toString(36).substring(2, 15)
604 }
605
606 def generate_env_example(self, features: List[str]) -> str:
607   """Generate .env.example"""
608   env_lines = ["# Environment Variables"]
609
610   if any(f in features for f in ["auth", "database", "storage"]):
611     env_lines.extend([
612       "",
613       "# Supabase Configuration",
614       "VITE_SUPABASE_URL=your_supabase_project_url",
615       "VITE_SUPABASE_ANON_KEY=your_supabase_anon_key"
616     ])
617
618   return "\n".join(env_lines)
619
620 def generate_home_page(self, components: List[Dict]) -> str:
621   """Generate Home page component using generated components"""
622   # Import all available components
623   imports = []
624   component_renders = []
625
626   for comp in components:
627     comp_name = comp.get("name", "")
628     if comp_name:
629       imports.append(f"import {comp_name} from '../components/{comp_name}'")
630       component_renders.append(f"<{comp_name} />")
631
632   imports_str = "\n".join(imports)
633   renders_str = "\n".join(component_renders)
634
635   return f"""import React from 'react'
636 {imports_str}
637
638 function Home() {{
639   return (
```

```
640     <div className="min-h-screen">
641   {renders_str}
642     </div>
643   )
644 }}
645
646 export default Home
647 ""
648
649   def generate_about_page(self) -> str:
650     """Generate beautiful About page component"""
651     return """import React from 'react'
652 import Navbar from '../components/Navbar'
653
654 function About() {
655   return (
656     <div className="min-h-screen bg-gradient-to-b from-white to-gray-50">
657       <Navbar />
658       {/* Hero Section */}
659       <div className="relative h-[400px] bg-gradient-to-br from-primary to-secondary flex
items-center justify-center overflow-hidden">
660         <div className="absolute inset-0 bg-black/20"></div>
661         <div className="relative z-10 text-center text-white px-4">
662           <h1 className="text-5xl md:text-7xl font-bold mb-4 animate-fade-in-up">
663             About Us
664           </h1>
665           <p className="text-xl md:text-2xl opacity-90 max-w-2xl mx-auto">
666             Discover our story, our mission, and what drives us every day
667           </p>
668         </div>
669         {/* Animated gradient orbs */}
670         <div className="absolute top-10 left-10 w-64 h-64 bg-white/10 rounded-full blur-3xl
animate-pulse"></div>
671         <div className="absolute bottom-10 right-10 w-96 h-96 bg-white/10 rounded-full
blur-3xl animate-pulse delay-1000"></div>
672       </div>
673
674       <div className="container-custom py-20">
675         {/* Our Story Section */}
676         <div className="grid md:grid-cols-2 gap-12 items-center mb-20">
677           <div>
678             <div className="inline-block px-4 py-2 bg-primary/10 rounded-full mb-6">
679               <span className="text-primary font-semibold">Our Story</span>
680             </div>
681             <h2 className="text-4xl font-bold mb-6 text-gray-900">
682               Building Excellence Since Day One
683             </h2>
684             <p className="text-lg text-gray-600 mb-4 leading-relaxed">
685               We started with a simple mission: to deliver exceptional service that exceeds
expectations. Over the years, we've grown into a trusted name in our industry, serving
hundreds of satisfied customers.
686             </p>
687             <p className="text-lg text-gray-600 leading-relaxed">
688               Our commitment to quality, innovation, and customer satisfaction drives
```

```
        everything we do. We're not just a business - we're your partners in success.
689         </p>
690     </div>
691     <div className="relative">
692         
697         <div className="absolute -bottom-6 -right-6 w-64 h-64 bg-gradient-to-br
from-primary to-secondary rounded-3xl opacity-20 -z-10"></div>
698     </div>
699 </div>
700
701     {/* Our Values */}
702     <div className="mb-20">
703         <div className="text-center mb-12">
704             <h2 className="text-4xl md:text-5xl font-bold mb-4 bg-gradient-to-r from-primary
to-secondary bg-clip-text text-transparent">
705                 Our Values
706             </h2>
707             <p className="text-xl text-gray-600 max-w-2xl mx-auto">
708                 The principles that guide everything we do
709             </p>
710         </div>
711         <div className="grid md:grid-cols-3 gap-8">
712             <div className="bg-white p-8 rounded-3xl shadow-lg hover:shadow-xl transition-all
duration-300 border border-gray-100">
713                 <div className="text-5xl mb-4">?</div>
714                 <h3 className="text-2xl font-bold mb-3">Excellence</h3>
715                 <p className="text-gray-600">We strive for excellence in everything we do,
never settling for anything less than the best.</p>
716             </div>
717             <div className="bg-white p-8 rounded-3xl shadow-lg hover:shadow-xl transition-all
duration-300 border border-gray-100">
718                 <div className="text-5xl mb-4">?</div>
719                 <h3 className="text-2xl font-bold mb-3">Innovation</h3>
720                 <p className="text-gray-600">We embrace innovation and continuously improve our
services to serve you better.</p>
721             </div>
722             <div className="bg-white p-8 rounded-3xl shadow-lg hover:shadow-xl transition-all
duration-300 border border-gray-100">
723                 <div className="text-5xl mb-4">??</div>
724                 <h3 className="text-2xl font-bold mb-3">Integrity</h3>
725                 <p className="text-gray-600">We operate with honesty, transparency, and respect
in all our relationships.</p>
726             </div>
727         </div>
728     </div>
729
730     {/* Team Section */}
731     <div className="text-center">
732         <h2 className="text-4xl md:text-5xl font-bold mb-4 bg-gradient-to-r from-primary
to-secondary bg-clip-text text-transparent">
```

```
733         Meet Our Team
734     </h2>
735     <p className="text-xl text-gray-600 mb-12 max-w-2xl mx-auto">
736         Talented professionals dedicated to your success
737     </p>
738     <div className="grid md:grid-cols-4 gap-8">
739         {[
740             { name: 'John Smith', role: 'Founder & CEO', image:
741               'https://i.pravatar.cc/150?img=3' },
742             { name: 'Sarah Johnson', role: 'Head of Operations', image:
743               'https://i.pravatar.cc/150?img=5' },
744             { name: 'Michael Chen', role: 'Lead Designer', image:
745               'https://i.pravatar.cc/150?img=7' },
746             { name: 'Emily Davis', role: 'Customer Success', image:
747               'https://i.pravatar.cc/150?img=12' }
748         ]}.map((member, index) => (
749             <div key={index} className="group">
750                 <div className="relative mb-4 overflow-hidden rounded-2xl">
751                     <img
752                         src={member.image}
753                         alt={member.name}
754                         className="w-full aspect-square object-cover transform
755                         group-hover:scale-110 transition-transform duration-500"
756                     />
757                     <div className="absolute inset-0 bg-gradient-to-t from-black/60
758                         to-transparent opacity-0 group-hover:opacity-100 transition-opacity duration-300"></div>
759                 </div>
760                 <h3 className="text-xl font-bold text-gray-900">{member.name}</h3>
761                 <p className="text-gray-600">{member.role}</p>
762             </div>
763         ))}
764     </div>
765     </div>
766     </div>
767     </div>
768     </div>
769     </div>
770     </div>
771     </div>
772     </div>
773     </div>
774     </div>
775     </div>
776     </div>
777     </div>
778     </div>
779     </div>
780     </div>
781     </div>
782     </div>
783     </div>
784     </div>
785     </div>
786     </div>
787     </div>
788     </div>
789     </div>
790     </div>
791     </div>
792     </div>
793     </div>
794     </div>
795     </div>
796     </div>
797     </div>
798     </div>
799     </div>
800     </div>
801     </div>
802     </div>
803     </div>
804     </div>
805     </div>
806     </div>
807     </div>
808     </div>
809     </div>
810     </div>
811     </div>
812     </div>
813     </div>
814     </div>
815     </div>
816     </div>
817     </div>
818     </div>
819     </div>
820     </div>
821     </div>
822     </div>
823     </div>
824     </div>
825     </div>
826     </div>
827     </div>
828     </div>
829     </div>
830     </div>
831     </div>
832     </div>
833     </div>
834     </div>
835     </div>
836     </div>
837     </div>
838     </div>
839     </div>
840     </div>
841     </div>
842     </div>
843     </div>
844     </div>
845     </div>
846     </div>
847     </div>
848     </div>
849     </div>
850     </div>
851     </div>
852     </div>
853     </div>
854     </div>
855     </div>
856     </div>
857     </div>
858     </div>
859     </div>
860     </div>
861     </div>
862     </div>
863     </div>
864     </div>
865     </div>
866     </div>
867     </div>
868     </div>
869     </div>
870     </div>
871     </div>
872     </div>
873     </div>
874     </div>
875     </div>
876     </div>
877     </div>
878     </div>
879     </div>
880     </div>
881     </div>
882     </div>
883     </div>
884     </div>
885     </div>
886     </div>
887     </div>
888     </div>
889     </div>
890     </div>
891     </div>
892     </div>
893     </div>
894     </div>
895     </div>
896     </div>
897     </div>
898     </div>
899     </div>
900     </div>
901     </div>
902     </div>
903     </div>
904     </div>
905     </div>
906     </div>
907     </div>
908     </div>
909     </div>
910     </div>
911     </div>
912     </div>
913     </div>
914     </div>
915     </div>
916     </div>
917     </div>
918     </div>
919     </div>
920     </div>
921     </div>
922     </div>
923     </div>
924     </div>
925     </div>
926     </div>
927     </div>
928     </div>
929     </div>
930     </div>
931     </div>
932     </div>
933     </div>
934     </div>
935     </div>
936     </div>
937     </div>
938     </div>
939     </div>
940     </div>
941     </div>
942     </div>
943     </div>
944     </div>
945     </div>
946     </div>
947     </div>
948     </div>
949     </div>
950     </div>
951     </div>
952     </div>
953     </div>
954     </div>
955     </div>
956     </div>
957     </div>
958     </div>
959     </div>
960     </div>
961     </div>
962     </div>
963     </div>
964     </div>
965     </div>
966     </div>
967     </div>
968     </div>
969     </div>
970     </div>
971     </div>
972     </div>
973     </div>
974     </div>
975     </div>
976     </div>
977     </div>
978     </div>
979     </div>
980     </div>
981     </div>
982     </div>
983     </div>
984     </div>
985     </div>
986     </div>
987     </div>
988     </div>
989     </div>
990     </div>
991     </div>
992     </div>
993     </div>
994     </div>
995     </div>
996     </div>
997     </div>
998     </div>
999     </div>
1000    </div>
```

File 5: backend/react_api_routes.pyBackend - React API Routes

```
1      """
2      React API Routes Extension
3      Additional API endpoints for React generation and Vercel deployment
4      """
5
6      from fastapi import APIRouter, HTTPException
7      from pydantic import BaseModel
8      from typing import Optional, Dict, List
9      import json
10     import os
11     from datetime import datetime
12
13     from react_builder import react_builder
14     from vercel_deployer import vercel_deployer
15     from supabase_config import supabase_manager
16     from pixabay_helper import get_service_images
17     from professional_copywriter import copywriter
18     from image_proxy import proxy_service_images
19     import openrouter_helper
20     import groq_helper
21
22     # Create router
23     router = APIRouter(prefix="/api/react", tags=["React & Deployment"])
24
25
26     # =====
27     # REQUEST MODELS
28     # =====
29
30     class ReactBuildRequest(BaseModel):
31         """Request to build a React website"""
32         session_id: str
33         business_name: str
34         business_type: str
35         business_description: str
36         services: List[str]
37         # Rich service data: [{name, category, subcategory}] ? optional, sent from frontend
38         service_categories: Optional[List[Dict]] = None
39         location: Optional[str] = ""
40         style_vibe: Optional[str] = "modern"
41         features: Optional[List[str]] = ["contact"]
42         deploy: Optional[bool] = False
43         # Images pre-fetched from Pexels during onboarding (or base64 data URLs from user
44         uploads)
45         hero_image: Optional[str] = None
46         about_image: Optional[str] = None
47         logo_image: Optional[str] = None # base64 data URL if user uploaded a logo
48         service_images: Optional[List[Optional[str]]] = None
49         gallery_images: Optional[List[Optional[str]]] = None
50         # Branding & contact
51         brand_color: Optional[str] = None
```

```
51     contact: Optional[Dict] = None
52     use_own_photos: Optional[bool] = False
53
54
55     class DeployRequest(BaseModel):
56         """Request to deploy an existing React website"""
57         session_id: str
58         deploy: bool = True
59
60
61     class StaticHtmlDeployRequest(BaseModel):
62         """Deploy a raw HTML string as a static Vercel site"""
63         project_name: str
64         html_content: str
65
66
67     class EditReactRequest(BaseModel):
68         """Request to edit a React website"""
69         session_id: str
70         edit_request: str
71         deploy: Optional[bool] = False
72
73
74     # =====
75     # ENDPOINTS
76     # =====
77
78     @router.post("/build")
79     async def build_react_website(request: ReactBuildRequest):
80         """
81         Build a complete React/Vite website with optional Vercel deployment
82
83         Returns:
84             - files: Dictionary of all project files
85             - components: List of generated components
86             - deployment_url: Vercel URL (if deployed)
87             - session_id: Website session ID
88         """
89         try:
90             # Prepare business analysis
91             from react_builder import ReactWebsiteBuilder
92             import google.generativeai as genai
93
94             model = genai.GenerativeModel('gemini-2.5-flash')
95
96             analysis_prompt = f"""You are an expert business analyst for a website builder.
97
98             Business type: {request.business_type}
99             Business name: {request.business_name}
100             Location: {request.location}
101             Description: {request.business_description}
102             Services: {'', '.join(request.services)}
103             Style: {request.style_vibe}
104
```

```
105 Return ONLY valid JSON:
106 {{
107     "business_name": "{request.business_name}",
108     "business_type": "{request.business_type}",
109     "location": {"city": "city", "country": "country"}},
110     "vibe": "{request.style_vibe}",
111     "target_audience": "target audience",
112     "services": {json.dumps(request.services[:6])},
113     "hero_headline": "powerful headline (max 8 words)",
114     "hero_subtext": "supporting text (1-2 sentences)",
115     "tagline": "memorable 3-5 word tagline",
116     "about_text": "compelling 2-3 sentence story",
117     "cta_text": "action button text"
118 }}""
119
120     resp = model.generate_content(analysis_prompt)
121     text = resp.text.strip()
122     json_start = text.find('{')
123     json_end = text.rfind('}') + 1
124     analysis = json.loads(text[json_start:json_end])
125
126     # Generate professional copy (OpenRouter ? Gemini fallback)
127     print("[COPYWRITER] Generating professional copy...")
128     copy_data = copywriter.generate_website_copy(
129         business_name=request.business_name,
130         business_type=request.business_type,
131         business_description=request.business_description,
132         services=request.services,
133         location=request.location,
134         style_vibe=request.style_vibe,
135         service_categories=request.service_categories or [],
136     )
137     print("[SUCCESS] Professional copy generated!")
138
139     # Generate per-service AI descriptions (OpenRouter ? Groq fallback)
140     service_descriptions = copy_data.get("service_descriptions", {})
141     if not service_descriptions:
142         if openrouter_helper.is_available():
143             print("[OPENROUTER] Generating per-service descriptions...")
144             service_descriptions = openrouter_helper.generate_service_descriptions(
145                 services=request.services,
146                 business_name=request.business_name,
147                 business_type=request.business_type,
148                 location=request.location or "",
149                 service_categories=request.service_categories or [],
150             )
151         if not service_descriptions and groq_helper.is_available():
152             print("[GROQ] Generating per-service descriptions...")
153             service_descriptions = groq_helper.generate_service_descriptions(
154                 services=request.services,
155                 business_name=request.business_name,
156                 business_type=request.business_type,
157                 location=request.location or "",
158                 service_categories=request.service_categories or [],
```

```
159         )
160         print("[SUCCESS] Service descriptions generated!")
161
162         # Normalize service_descriptions keys to exactly match request.services names
163         # (AI may return lowercased or slightly different keys)
164         if service_descriptions and request.services:
165             normalized = {}
166             sd_lower = {k.lower(): v for k, v in service_descriptions.items()}
167             for svc in request.services:
168                 val = service_descriptions.get(svc) or sd_lower.get(svc.lower())
169                 if val:
170                     normalized[svc] = val
171             service_descriptions = normalized
172
173         # Use Pexels images from onboarding if provided, otherwise fetch from Pixabay
174         pexels_service_urls = [url for url in (request.service_images or []) if url]
175         if pexels_service_urls:
176             print("[PEXELS] Using pre-fetched Pexels images from onboarding...")
177             images = [{"url": url} for url in pexels_service_urls[:6]]
178             service_images_cdn = {svc: pexels_service_urls[i] for i, svc in
167 enumerate(request.services[:len(pexels_service_urls)])}
179             service_images = service_images_cdn # already permanent URLs, no proxy needed
180         else:
181             print("[PIXABAY] Fetching dynamic images from Pixabay...")
182             service_images_cdn = get_service_images(request.services, request.business_type)
183             print("[SUCCESS] Got service images!")
184             print("[PROXY] Converting Pixabay URLs to permanent data URLs...")
185             service_images = proxy_service_images(service_images_cdn)
186             print("[SUCCESS] Images proxied successfully!")
187             images = [
188                 {"url": service_images_cdn[svc]}
189                 for svc in request.services[:6]
190                 if service_images_cdn.get(svc)
191             ]
192             if not images:
193                 images = [{"url": "https://picsum.photos/seed/business/1920/1080"}]
194
195         # If a dedicated hero image was fetched, use it as images[0] for the hero section
196         hero_image_url = request.hero_image
197         if hero_image_url:
198             images = [{"url": hero_image_url}] + [img for img in images if img.get("url") !=
167 hero_image_url]
199
200         # Append dedicated gallery images (indices 7+) so react_builder can use them in the
201         gallery section
202         gallery_image_urls = [url for url in (request.gallery_images or []) if url]
203         if gallery_image_urls:
204             images = images + [{"url": url} for url in gallery_image_urls]
205
206         # Update analysis with AI-generated copy and service images
207         analysis["hero_headline"] = copy_data.get("hero_headline",
167 analysis.get("hero_headline"))
208         analysis["hero_subtext"] = copy_data.get("hero_subtext",
167 analysis.get("hero_subtext"))
```

```
208         analysis["tagline"] = copy_data.get("tagline", analysis.get("tagline"))
209         analysis["about_text"] = copy_data.get("about_text", analysis.get("about_text"))
210         analysis["cta_text"] = copy_data.get("cta_text", analysis.get("cta_text"))
211         analysis["service_images"] = service_images
212         analysis["about_image"] = request.about_image # dedicated about section image
(Pexels or base64)
213         analysis["logo_image"] = request.logo_image # user-uploaded logo (base64 data URL
or None)
214         analysis["service_descriptions"] = service_descriptions # AI descriptions per
service
215         analysis["service_categories"] = request.service_categories or [] #
category/subcategory/price per service
216         # Always use the user's exact service names ? Gemini may invent/alter them in its
JSON
217         analysis["services"] = request.services
218
219         # Color schemes ? use user brand color if provided
220         vibe = analysis.get("vibe", "modern").lower()
221         color_schemes = {
222             "luxury": {"primary": "#C9A96E", "secondary": "#1A1A1A", "accent": "#F5F5F5"},
223             "modern": {"primary": "#6366F1", "secondary": "#0F172A", "accent": "#F59E0B"},
224             "cozy": {"primary": "#D97706", "secondary": "#78350F", "accent": "#FEF3C7"},
225             "energetic": {"primary": "#EF4444", "secondary": "#991B1B", "accent": "#FEE2E2"},
226         }
227         colors = color_schemes.get(vibe, color_schemes["modern"])
228         if request.brand_color:
229             colors["primary"] = request.brand_color
230
231         # Generate React website
232         result = await react_builder.generate_react_website(
233             session_id=request.session_id,
234             analysis=analysis,
235             colors=colors,
236             images=images,
237             features=request.features,
238             business_description=request.business_description,
239             deploy_to_vercel=request.deploy
240         )
241
242         return {
243             "success": True,
244             "message": "React website generated successfully!",
245             "session_id": result["session_id"],
246             "files": result["files"],
247             "components": result["components"],
248             "framework": result["framework"],
249             "deployment_url": result.get("deployment_url"),
250             "deployment_id": result.get("deployment_id"),
251             "deployed": result.get("deployment_url") is not None,
252             # Include AI-generated copy and dynamic images for frontend preview
253             "hero_headline": analysis.get("hero_headline"),
254             "hero_subtext": analysis.get("hero_subtext"),
255             "tagline": analysis.get("tagline"),
256             "about_text": analysis.get("about_text"),
```

```
257         "cta_text": analysis.get("cta_text"),
258         "service_images": service_images,
259         "service_descriptions": service_descriptions # AI-generated per-service copy
260     }
261
262     except Exception as e:
263         raise HTTPException(status_code=500, detail=str(e))
264
265
266 @router.post("/deploy")
267 async def deploy_existing_website(request: DeployRequest):
268     """
269     Deploy an existing React website to Vercel
270
271     Reads the website files from session storage and deploys
272     """
273     try:
274         if not vercel_deployer.is_enabled():
275             raise HTTPException(status_code=400, detail="Vercel not configured")
276
277         # Load website state
278         state_path = f"website_states/{request.session_id}.json"
279         if not os.path.exists(state_path):
280             raise HTTPException(status_code=404, detail="Website not found")
281
282         with open(state_path, "r") as f:
283             state = json.load(f)
284
285         # Get website files (assuming they're stored)
286         # This is a simplified version - in production, you'd load from storage
287         files = state.get("files", {})
288
289         if not files:
290             raise HTTPException(status_code=400, detail="No files to deploy")
291
292         analysis = state.get("analysis", {})
293         business_name = analysis.get("business_name", "website")
294
295         # Deploy to Vercel
296         deployment = await vercel_deployer.create_deployment(
297             project_name=business_name,
298             files=files,
299             env_vars={
300                 "VITE_SUPABASE_URL": os.getenv("SUPABASE_URL", ""),
301                 "VITE_SUPABASE_ANON_KEY": os.getenv("SUPABASE_KEY", "")
302             }
303         )
304
305         if not deployment:
306             raise HTTPException(status_code=500, detail="Deployment failed")
307
308         # Update Supabase
309         if supabase_manager.is_enabled():
310             await supabase_manager.update_website_metadata(
```

```
311         request.session_id,
312         {
313             "deployment_url": deployment.get("url"),
314             "deployment_id": deployment.get("id"),
315             "status": "deployed",
316             "deployed_at": datetime.now().isoformat()
317         }
318     )
319
320     return {
321         "success": True,
322         "message": "Website deployed successfully!",
323         "deployment_url": deployment.get("url"),
324         "deployment_id": deployment.get("id"),
325         "status": deployment.get("status")
326     }
327
328     except HTTPException:
329         raise
330     except Exception as e:
331         raise HTTPException(status_code=500, detail=str(e))
332
333
334 @router.get("/deployment/{deployment_id}")
335 async def get_deployment_status(deployment_id: str):
336     """Get deployment status and details"""
337     try:
338         if not vercel_deployer.is_enabled():
339             raise HTTPException(status_code=400, detail="Vercel not configured")
340
341         deployment = await vercel_deployer.get_deployment(deployment_id)
342
343         if not deployment:
344             raise HTTPException(status_code=404, detail="Deployment not found")
345
346         return {
347             "success": True,
348             "deployment": {
349                 "id": deployment.get("id"),
350                 "url": f"https://{deployment.get('url')}",
351                 "status": deployment.get("readyState"),
352                 "created": deployment.get("createdAt"),
353             }
354         }
355
356     except HTTPException:
357         raise
358     except Exception as e:
359         raise HTTPException(status_code=500, detail=str(e))
360
361
362 @router.get("/deployments")
363 async def list_deployments():
364     """List all deployments"""
```

```
365     try:
366         if not vercel_deployer.is_enabled():
367             raise HTTPException(status_code=400, detail="Vercel not configured")
368
369         deployments = await vercel_deployer.list_deployments()
370
371         return {
372             "success": True,
373             "count": len(deployments),
374             "deployments": [
375                 {
376                     "id": d.get("uid"),
377                     "name": d.get("name"),
378                     "url": f"https://{d.get('url')}",
379                     "status": d.get("state"),
380                     "created": d.get("createdAt")
381                 }
382                 for d in deployments
383             ]
384         }
385
386     except HTTPException:
387         raise
388     except Exception as e:
389         raise HTTPException(status_code=500, detail=str(e))
390
391
392 @router.delete("/deployment/{deployment_id}")
393 async def delete_deployment(deployment_id: str):
394     """Delete a Vercel deployment"""
395     try:
396         if not vercel_deployer.is_enabled():
397             raise HTTPException(status_code=400, detail="Vercel not configured")
398
399         success = await vercel_deployer.delete_deployment(deployment_id)
400
401         if not success:
402             raise HTTPException(status_code=500, detail="Failed to delete deployment")
403
404         return {
405             "success": True,
406             "message": "Deployment deleted successfully"
407         }
408
409     except HTTPException:
410         raise
411     except Exception as e:
412         raise HTTPException(status_code=500, detail=str(e))
413
414
415 @router.get("/website/{session_id}/data")
416 async def get_website_data(session_id: str):
417     """Get all data for a website (bookings, orders, contacts)"""
418     try:
```

```
419         if not supabase_manager.is_enabled():
420             raise HTTPException(status_code=400, detail="Supabase not configured")
421
422         # Get website metadata
423         website = await supabase_manager.get_website_metadata(session_id)
424
425         # Get all related data
426         bookings = await supabase_manager.get_bookings(session_id)
427         orders = await supabase_manager.get_orders(session_id)
428         contacts = await supabase_manager.get_contacts(session_id)
429
430         return {
431             "success": True,
432             "website": website,
433             "data": {
434                 "bookings": bookings,
435                 "orders": orders,
436                 "contacts": contacts
437             },
438             "stats": {
439                 "total_bookings": len(bookings),
440                 "total_orders": len(orders),
441                 "total_contacts": len(contacts)
442             }
443         }
444
445     except HTTPException:
446         raise
447     except Exception as e:
448         raise HTTPException(status_code=500, detail=str(e))
449
450
451 @router.post("/deploy-html")
452 async def deploy_html_as_static(request: StaticHtmlDeployRequest):
453     """
454     Deploy a raw HTML string as a static Vercel site.
455     The deployed site will look exactly like the in-app preview.
456     """
457     if not vercel_deployer.is_enabled():
458         raise HTTPException(status_code=400, detail="Vercel not configured")
459
460     result = await vercel_deployer.deploy_static_html(
461         project_name=request.project_name,
462         html_content=request.html_content
463     )
464     if not result:
465         raise HTTPException(status_code=500, detail="Deployment failed")
466
467     return {"success": True, "deployment_url": result["url"]}
468
469
470 @router.get("/status")
471 async def get_system_status():
472     """Get status of all integrated services"""
```

```
473     return {
474         "success": True,
475         "services": {
476             "vercel": {
477                 "enabled": vercel_deployer.is_enabled(),
478                 "status": "ready" if vercel_deployer.is_enabled() else "not configured"
479             },
480             "supabase": {
481                 "enabled": supabase_manager.is_enabled(),
482                 "status": "ready" if supabase_manager.is_enabled() else "not configured"
483             },
484             "react_builder": {
485                 "enabled": True,
486                 "status": "ready"
487             }
488         }
489     }
490
```

File 6: backend/vercel_deployer.pyBackend - Vercel Deployer

```
1      """
2      Vercel Deployment Manager
3      Handles automated deployment of React websites to Vercel
4      """
5
6      import os
7      import re
8      import json
9      import httpx
10     import tarfile
11     import io
12     import base64
13     from typing import Dict, Optional, List
14     from pathlib import Path
15     import asyncio
16
17     class VercelDeployer:
18         """Manages Vercel deployments for generated React websites"""
19
20         def __init__(self):
21             self.token = os.getenv("VERCEL_TOKEN")
22             self.team_id = os.getenv("VERCEL_TEAM_ID") # Optional
23             # Force deploy to personal account for public access (ignore team settings)
24             self.team_id = None # Always use personal account
25             self.base_url = "https://api.vercel.com"
26
27             if not self.token:
28                 print("[WARNING] Warning: VERCEL_TOKEN not configured")
29                 self.enabled = False
30             else:
31                 self.enabled = True
32                 print("[OK] Vercel deployer initialized (personal account mode)")
33
34         def is_enabled(self) -> bool:
35             """Check if Vercel is properly configured"""
36             return self.enabled
37
38         async def create_deployment(
39             self,
40             project_name: str,
41             files: Dict[str, str],
42             env_vars: Optional[Dict[str, str]] = None
43         ) -> Optional[Dict]:
44             """
45             Deploy a React website to Vercel
46
47             Args:
48                 project_name: Name for the Vercel project (will be slugified)
49                 files: Dictionary of {file_path: file_content}
50                     Example: {"package.json": "...", "src/App.jsx": "..."}
51                 env_vars: Optional environment variables
```

```
52
53     Returns:
54         Dict with deployment info including URL
55         {
56             "url": "https://project-abc123.vercel.app",
57             "id": "dpl_...",
58             "status": "ready",
59             "project_id": "prj_..."
60         }
61     """
62     if not self.enabled:
63         return None
64
65     try:
66         # Prepare files for deployment
67         deployment_files = []
68         for file_path, content in files.items():
69             deployment_files.append({
70                 "file": file_path,
71                 "data": content
72             })
73
74         # Prepare deployment payload
75         payload = {
76             "name": self._slugify(project_name),
77             "files": deployment_files,
78             "projectSettings": {
79                 "framework": "vite",
80                 "buildCommand": "npm run build",
81                 "outputDirectory": "dist",
82                 "installCommand": "npm install",
83                 "devCommand": "npm run dev"
84             },
85             "target": "production"
86             # Note: gitSource omitted - not needed for file-based deployments
87         }
88
89         # Add environment variables if provided
90         if env_vars:
91             payload["env"] = env_vars
92
93         # Don't add team ID - deploy to personal account for public access
94         # if self.team_id:
95         #     payload["teamId"] = self.team_id
96
97         # Make deployment request
98         headers = {
99             "Authorization": f"Bearer {self.token}",
100             "Content-Type": "application/json"
101         }
102
103         async with httpx.AsyncClient(timeout=120.0) as client:
104             response = await client.post(
105                 f"{self.base_url}/v13/deployments",
```

```
106         headers=headers,
107         json=payload
108     )
109
110     if response.status_code in [200, 201]:
111         data = response.json()
112
113         deployment_id = data.get('id')
114         project_id = data.get('projectId')
115
116         # Wait for deployment to be ready
117         await self._wait_for_deployment(deployment_id)
118
119         # Disable all protection on the project
120         if project_id:
121             await self.update_project_protection(project_id)
122
123         # Re-fetch deployment details AFTER it is ready ? the
124         # initial POST response does not yet have the production alias.
125         deployment_url = f"https://{data.get('url')}" # fallback
126         alias_list = []
127         async with httpx.AsyncClient(timeout=30.0) as fetch_client:
128             detail_resp = await fetch_client.get(
129                 f"{self.base_url}/v13/deployments/{deployment_id}",
130                 headers=headers
131             )
132             if detail_resp.status_code == 200:
133                 details = detail_resp.json()
134                 alias_list = details.get('alias', [])
135
136         if alias_list:
137             # Pick the production alias ? it has NO random hex hash suffix.
138             # Hash URLs look like: name-abc1def2.vercel.app
139             # Production URLs look like: name.vercel.app or
140             name-username.vercel.app
141
142             def _is_production(alias: str) -> bool:
143                 subdomain = alias.replace('.vercel.app', '')
144                 return not re.search(r'-[0-9a-f]{8}$', subdomain)
145
146             production = [a for a in alias_list if _is_production(a)]
147             chosen = production[0] if production else alias_list[0]
148             deployment_url = f"https://{chosen}"
149
150         print(f"[OK] Deployed successfully: {deployment_url}")
151         return {
152             "url": deployment_url,
153             "id": deployment_id,
154             "status": "ready",
155             "alias": alias_list,
156             "project_id": project_id
157         }
158     else:
159         error_msg = response.json() if response.text else response.text
160         print(f"[ERROR] Vercel deployment failed: {error_msg}")
```

```
159             return None
160
161         except Exception as e:
162             print(f"[ERROR] Error deploying to Vercel: {e}")
163             return None
164
165     async def _wait_for_deployment(self, deployment_id: str, max_attempts: int = 45) -> bool:
166         """Wait for deployment to be ready (poll status)"""
167         if not self.enabled:
168             return False
169
170         headers = {
171             "Authorization": f"Bearer {self.token}"
172         }
173
174         for attempt in range(max_attempts):
175             try:
176                 async with httpx.AsyncClient() as client:
177                     response = await client.get(
178                         f"{self.base_url}/v13/deployments/{deployment_id}",
179                         headers=headers
180                     )
181
182                     if response.status_code == 200:
183                         data = response.json()
184                         state = data.get('readyState', 'INITIALIZING')
185
186                         if state == 'READY':
187                             print(f"[OK] Deployment {deployment_id} is ready!")
188                             return True
189                         elif state == 'ERROR':
190                             print(f"[ERROR] Deployment {deployment_id} failed")
191                             return False
192
193                         # Still building, wait and retry
194                         await asyncio.sleep(2)
195                     else:
196                         print(f"[WARNING] Poll attempt {attempt+1}: status
197 {response.status_code}")
198                         await asyncio.sleep(2)
199             except Exception as e:
200                 print(f"[WARNING] Error checking deployment status: {e}")
201                 await asyncio.sleep(2)
202
203         print(f"[WARNING] Deployment {deployment_id} timed out after {max_attempts * 2}s")
204         return False
205
206     async def get_deployment(self, deployment_id: str) -> Optional[Dict]:
207         """Get deployment details"""
208         if not self.enabled:
209             return None
210
211         try:
```

```
212         headers = {
213             "Authorization": f"Bearer {self.token}"
214         }
215
216         async with httpx.AsyncClient() as client:
217             response = await client.get(
218                 f"{self.base_url}/v13/deployments/{deployment_id}",
219                 headers=headers
220             )
221
222             if response.status_code == 200:
223                 return response.json()
224             else:
225                 return None
226
227         except Exception as e:
228             print(f"[ERROR] Error getting deployment: {e}")
229             return None
230
231     async def list_deployments(self, project_id: Optional[str] = None) -> List[Dict]:
232         """List all deployments (optionally filtered by project)"""
233         if not self.enabled:
234             return []
235
236         try:
237             headers = {
238                 "Authorization": f"Bearer {self.token}"
239             }
240
241             params = {}
242             if project_id:
243                 params["projectId"] = project_id
244             if self.team_id:
245                 params["teamId"] = self.team_id
246
247             async with httpx.AsyncClient() as client:
248                 response = await client.get(
249                     f"{self.base_url}/v6/deployments",
250                     headers=headers,
251                     params=params
252                 )
253
254                 if response.status_code == 200:
255                     data = response.json()
256                     return data.get("deployments", [])
257                 else:
258                     return []
259
260         except Exception as e:
261             print(f"[ERROR] Error listing deployments: {e}")
262             return []
263
264     async def delete_deployment(self, deployment_id: str) -> bool:
265         """Delete a deployment"""
```

```
266         if not self.enabled:
267             return False
268
269         try:
270             headers = {
271                 "Authorization": f"Bearer {self.token}"
272             }
273
274             async with httpx.AsyncClient() as client:
275                 response = await client.delete(
276                     f"{self.base_url}/v13/deployments/{deployment_id}",
277                     headers=headers
278                 )
279
280                 return response.status_code == 200
281
282         except Exception as e:
283             print(f"[ERROR] Error deleting deployment: {e}")
284             return False
285
286     async def create_project(self, project_name: str, framework: str = "vite") ->
Optional[Dict]:
287         """Create a new Vercel project"""
288         if not self.enabled:
289             return None
290
291         try:
292             headers = {
293                 "Authorization": f"Bearer {self.token}",
294                 "Content-Type": "application/json"
295             }
296
297             payload = {
298                 "name": self._slugify(project_name),
299                 "framework": framework
300             }
301
302             if self.team_id:
303                 payload["teamId"] = self.team_id
304
305             async with httpx.AsyncClient() as client:
306                 response = await client.post(
307                     f"{self.base_url}/v9/projects",
308                     headers=headers,
309                     json=payload
310                 )
311
312                 if response.status_code in [200, 201]:
313                     return response.json()
314                 else:
315                     return None
316
317         except Exception as e:
318             print(f"[ERROR] Error creating project: {e}")
```

```
319         return None
320
321     async def update_project_protection(self, project_id: str) -> bool:
322         """Update project to disable password protection for public access"""
323         if not self.enabled:
324             return False
325
326         try:
327             headers = {
328                 "Authorization": f"Bearer {self.token}",
329                 "Content-Type": "application/json"
330             }
331
332             # Disable password protection to allow public access
333             payload = {
334                 "passwordProtection": None, # Disable password protection
335                 "ssoProtection": None # Disable SSO protection if applicable
336             }
337
338             url = f"{self.base_url}/v9/projects/{project_id}"
339             if self.team_id:
340                 url += f"?teamId={self.team_id}"
341
342             async with httpx.AsyncClient() as client:
343                 response = await client.patch(
344                     url,
345                     headers=headers,
346                     json=payload
347                 )
348
349                 if response.status_code in [200, 201]:
350                     print(f"[OK] Project {project_id} set to public access")
351                     return True
352                 else:
353                     print(f"[WARNING] Could not update project protection: {response.text}")
354                     return False
355
356         except Exception as e:
357             print(f"[ERROR] Error updating project protection: {e}")
358             return False
359
360     async def deploy_static_html(self, project_name: str, html_content: str) ->
Optional[Dict]:
361         """
362         Deploy a single self-contained HTML file as a static Vercel site.
363         This makes the deployed site look identical to the in-app preview.
364         """
365         if not self.enabled:
366             return None
367
368         try:
369             headers = {
370                 "Authorization": f"Bearer {self.token}",
371                 "Content-Type": "application/json"
```

```
372         }
373
374     payload = {
375         "name": self._slugify(project_name),
376         "files": [
377             {"file": "index.html", "data": html_content}
378         ],
379         "projectSettings": {
380             "framework": None,          # Static ? no build step needed
381             "buildCommand": None,
382             "outputDirectory": None,
383             "installCommand": None
384         },
385         "target": "production"
386     }
387
388     async with httpx.AsyncClient(timeout=120.0) as client:
389         response = await client.post(
390             f"{self.base_url}/v13/deployments",
391             headers=headers,
392             json=payload
393         )
394
395         if response.status_code in [200, 201]:
396             data = response.json()
397             deployment_id = data.get('id')
398             project_id = data.get('projectId')
399
400             ready = await self._wait_for_deployment(deployment_id)
401             if not ready:
402                 print(f"[ERROR] Deployment {deployment_id} did not reach READY state
? aborting")
403
404                 return None
405             if project_id:
406                 await self.update_project_protection(project_id)
407
408             deployment_url = f"https://{data.get('url')}"
409             alias_list = []
410             async with httpx.AsyncClient(timeout=30.0) as fc:
411                 dr = await fc.get(
412                     f"{self.base_url}/v13/deployments/{deployment_id}",
413                     headers=headers
414                 )
415                 if dr.status_code == 200:
416                     alias_list = dr.json().get('alias', [])
417
418             if alias_list:
419                 def _is_prod(a):
420                     return not re.search(r'-[0-9a-f]{8,}$', a.replace('.vercel.app',
''))
421
422                 prod = [a for a in alias_list if _is_prod(a)]
423                 deployment_url = f"https://{prod[0] if prod else alias_list[0]}"
424
425                 print(f"[OK] Static HTML deployed: {deployment_url}")
```

```
424         return {"url": deployment_url, "id": deployment_id, "status": "ready"}
425     else:
426         print(f"[ERROR] Static deploy failed: {response.json()}")
427         return None
428
429     except Exception as e:
430         print(f"[ERROR] deploy_static_html error: {e}")
431         return None
432
433     def _slugify(self, text: str) -> str:
434         """Convert text to URL-safe slug"""
435         import re
436         # Convert to lowercase
437         text = text.lower()
438         # Replace spaces and special chars with hyphens
439         text = re.sub(r'^a-z0-9+', '-', text)
440         # Remove leading/trailing hyphens
441         text = text.strip('-')
442         # Limit length
443         return text[:50]
444
445     def generate_vercel_config(self, framework: str = "vite") -> Dict:
446         """Generate vercel.json configuration"""
447         if framework == "vite":
448             return {
449                 "buildCommand": "npm run build",
450                 "outputDirectory": "dist",
451                 "devCommand": "npm run dev",
452                 "installCommand": "npm install",
453                 "framework": "vite"
454             }
455         elif framework == "nextjs":
456             return {
457                 "buildCommand": "npm run build",
458                 "outputDirectory": ".next",
459                 "devCommand": "npm run dev",
460                 "installCommand": "npm install",
461                 "framework": "nextjs"
462             }
463         else:
464             return {}
465
466     # Global Vercel deployer instance
467     vercel_deployer = VercelDeployer()
468
469
```

File 7: backend/professional_copywriter.pyBackend - Professional Copywriter

```
1      """
2      Professional Copywriter Module
3      Generates compelling, professional copy for websites.
4      Uses OpenRouter first, then Groq, then Gemini as fallbacks.
5      """
6
7      import os
8      import json
9      import google.generativeai as genai
10     import openrouter_helper
11     import groq_helper
12
13     GEMINI_API_KEY = os.getenv("GEMINI_API_KEY")
14     if GEMINI_API_KEY:
15         genai.configure(api_key=GEMINI_API_KEY)
16
17
18     class ProfessionalCopywriter:
19         """Generates professional, engaging copy for websites"""
20
21         def __init__(self):
22             self.model = genai.GenerativeModel('gemini-2.0-flash')
23
24         def generate_website_copy(
25             self,
26             business_name: str,
27             business_type: str,
28             business_description: str,
29             services: list,
30             location: str = "",
31             style_vibe: str = "modern",
32             service_categories: list = None,
33         ) -> dict:
34             """
35             Generate professional copy for all website sections
36
37             Returns:
38                 dict with hero_headline, hero_subtext, about_text, tagline, service_descriptions,
39                 cta_text
40
41             """
42
43             # Build enriched service lines with category/subcategory hints
44             cat_map = {}
45             for entry in (service_categories or []):
46                 name = entry.get("name", "").strip()
47                 if name:
48                     cat_map[name] = entry
49
50             service_lines = []
51             for svc in (services or []):
52                 entry = cat_map.get(svc, {})
```

```
51         cat = entry.get("category", "").strip()
52         sub = entry.get("subcategory", "").strip()
53         if sub:
54             service_lines.append(f"{svc} [Category: {cat} | Subcategory: {sub}]")
55         elif cat:
56             service_lines.append(f"{svc} [Category: {cat}]")
57         else:
58             service_lines.append(svc)
59         services_display = ', '.join(service_lines) if service_lines else 'Not specified'
60
61         prompt = f"""You are a world-class copywriter for a top web design agency. Create
compelling, professional copy for a website.
62
63 BUSINESS DETAILS:
64 - Name: {business_name}
65 - Type: {business_type}
66 - Description: {business_description}
67 - Services: {services_display}
68 - Location: {location if location else 'Not specified'}
69 - Style/Vibe: {style_vibe}
70
71 TASK: Generate professional, engaging copy that:
72 ? Captures the brand's personality
73 ? Speaks directly to the target audience
74 ? Is compelling and memorable (NOT generic!)
75 ? Uses emotional triggers and benefits
76 ? Avoids clichés like "best", "welcome to", "your one-stop"
77 ? Matches the {style_vibe} style
78
79 COPY TO GENERATE:
80
81 1. Hero Headline (5-10 words)
82     - Should be powerful, memorable, and unique
83     - Focus on the transformation/feeling, not just the service
84     - Examples:
85         * Coffee shop: "Where Every Cup Tells a Story"
86         * Gym: "Your Journey to Unstoppable Starts Here"
87         * Salon: "Confidence Crafted, Beauty Unveiled"
88
89 2. Hero Subtext (15-25 words)
90     - Expand on the headline with specific details
91     - Mention location if provided
92     - Create desire and urgency
93     - Example: "Artisan coffee and fresh pastries in the heart of Mumbai. Your daily escape
awaits."
94
95 3. About Section (40-60 words)
96     - Tell the story behind the business
97     - Use "we" or "our" voice (first person)
98     - Explain what makes them unique
99     - Connect emotionally with the reader
100     - Example: "Born from a passion for exceptional coffee, we bring together the finest beans
from around the world with a warm, inviting atmosphere that feels like home. Every visit is a
chance to pause, connect, and savor the moment."
```

```
101
102 4. **Tagline** (3-6 words)
103     - Brand slogan that's catchy and memorable
104     - Example: "Fuel Your Fire" (gym), "Crafted with Care" (cafe)
105
106 5. **Call-to-Action** (2-4 words)
107     - Action-oriented and specific
108     - Examples: "Book Your Spot", "Start Your Journey", "Reserve a Table", "Get Started Today"
109
110 6. **Service Descriptions** (for each service, 15-25 words each)
111     - Focus on benefits, not features
112     - Use sensory language and make it desirable
113     - Use the [Category] and [Subcategory] hints to make each description highly specific
114
115 Return ONLY valid JSON in this exact format:
116 {{
117     "hero_headline": "string",
118     "hero_subtext": "string",
119     "about_text": "string",
120     "tagline": "string",
121     "cta_text": "string",
122     "service_descriptions": {{
123         "service_name": "compelling description"
124     }}
125 }}
126
127 NO explanations, NO markdown, JUST the JSON.""
128
129     # ?? Try OpenRouter first (free, saves Gemini quota) ??????????????????????
130     if openrouter_helper.is_available():
131         print("[COPYWRITER] Using OpenRouter AI...")
132         result = openrouter_helper.generate_website_copy(
133             business_name=business_name,
134             business_type=business_type,
135             business_description=business_description,
136             services=services,
137             location=location,
138             style_vibe=style_vibe,
139             service_categories=service_categories,
140         )
141         if result:
142             print("[SUCCESS] OpenRouter copy generated!")
143             return result
144         print("[WARN] OpenRouter failed, trying Groq...")
145
146     # ?? Try Groq (fast LPU inference, OpenAI-compatible) ??????????????????????
147     if groq_helper.is_available():
148         print("[COPYWRITER] Using Groq AI...")
149         result = groq_helper.generate_website_copy(
150             business_name=business_name,
151             business_type=business_type,
152             business_description=business_description,
153             services=services,
154             location=location,
```

```
155         style_vibe=style_vibe,
156         service_categories=service_categories,
157     )
158     if result:
159         print("[SUCCESS] Groq copy generated!")
160         return result
161     print("[WARN] Groq failed, falling back to Gemini...")
162
163     try:
164         response = self.model.generate_content(prompt)
165         response_text = response.text.strip()
166
167         # Extract JSON from response (handle markdown code blocks)
168         if "```json" in response_text:
169             response_text = response_text.split("```json")[1].split("```")[0].strip()
170         elif "```" in response_text:
171             response_text = response_text.split("```")[1].split("```")[0].strip()
172
173         copy_data = json.loads(response_text)
174
175         print("[SUCCESS] Generated professional copy")
176         return copy_data
177
178     except Exception as e:
179         print("[WARNING] AI copywriting failed, using enhanced fallback")
180         return self._generate_enhanced_fallback(
181             business_name, business_type, business_description, services, location,
182             style_vibe
183         )
184
185     def _generate_enhanced_fallback(
186         self,
187         business_name: str,
188         business_type: str,
189         business_description: str,
190         services: list,
191         location: str,
192         style_vibe: str
193     ) -> dict:
194         """Generate enhanced fallback copy with better templates"""
195
196         # Business-type specific templates
197         templates = {
198             "cafe": {
199                 "hero_headline": f"Where Coffee Meets Community",
200                 "hero_subtext": f"Experience artisan coffee and warm hospitality at {business_name}. {f'Proudly serving {location}.' if location else 'Your daily ritual starts here.'}",
201                 "tagline": "Crafted with Passion",
202                 "cta_text": "Visit Us Today"
203             },
204             "restaurant": {
205                 "hero_headline": f"A Culinary Journey Awaits",
206                 "hero_subtext": f"Savor exceptional flavors and unforgettable moments at
```

```
    {business_name}. {f'Located in the heart of {location}.' if location else 'Every meal tells a
    story.'}",
206         "tagline": "Taste the Difference",
207         "cta_text": "Reserve a Table"
208     },
209     "gym": {
210         "hero_headline": f"Transform Your Body, Elevate Your Life",
211         "hero_subtext": f"Join {business_name} and discover what you're truly capable
    of. {f'Now in {location}.' if location else 'Your fitness journey starts now.'}",
212         "tagline": "Stronger Every Day",
213         "cta_text": "Start Your Journey"
214     },
215     "salon": {
216         "hero_headline": f"Where Beauty Meets Artistry",
217         "hero_subtext": f"Experience transformative beauty services at
    {business_name}. {f'Serving {location} with excellence.' if location else 'You deserve to
    shine.'}",
218         "tagline": "Unleash Your Glow",
219         "cta_text": "Book Your Appointment"
220     },
221     "yoga": {
222         "hero_headline": f"Find Your Balance, Transform Your Life",
223         "hero_subtext": f"Discover inner peace and physical strength at
    {business_name}. {f'Your sanctuary in {location}.' if location else 'Every breath brings you
    closer.'}",
224         "tagline": "Breathe, Flow, Transform",
225         "cta_text": "Join a Class"
226     },
227     "hotel": {
228         "hero_headline": f"Your Home Away From Home",
229         "hero_subtext": f"Luxury, comfort, and hospitality await at {business_name}.
    {f'Experience the best of {location}.' if location else 'Where memories are made.'}",
230         "tagline": "Stay Extraordinary",
231         "cta_text": "Book Your Stay"
232     },
233     "boutique": {
234         "hero_headline": f"Style That Speaks Your Language",
235         "hero_subtext": f"Curated collections and timeless pieces at {business_name}.
    {f'Your destination in {location}.' if location else 'Discover your signature look.'}",
236         "tagline": "Wear Your Story",
237         "cta_text": "Shop Now"
238     },
239     "clinic": {
240         "hero_headline": f"Your Health, Our Priority",
241         "hero_subtext": f"Compassionate care and expert treatment at {business_name}.
    {f'Serving {location} with dedication.' if location else 'Your wellness journey begins
    here.'}",
242         "tagline": "Caring for You",
243         "cta_text": "Book Appointment"
244     },
245     "real-estate": {
246         "hero_headline": f"Find the Home You've Always Dreamed Of",
247         "hero_subtext": f"Discover premium properties and expert guidance at
    {business_name}. {f'Your trusted real estate partner in {location}.' if location else 'Your
```

```
        perfect home is just a step away.'}]",
248         "tagline": "Keys to Your Future",
249         "cta_text": "Explore Properties"
250     },
251     "coaching": {
252         "hero_headline": f"Unlock Your Full Potential",
253         "hero_subtext": f"Expert guidance and proven strategies at {business_name}.
{f'Empowering learners in {location}.' if location else 'Your transformation starts
today.'}",
254         "tagline": "Learn. Grow. Succeed.",
255         "cta_text": "Join a Class"
256     },
257     "online-store": {
258         "hero_headline": f"Shop Smart, Live Better",
259         "hero_subtext": f"Curated collections and seamless shopping at
{business_name}. {f'Delivering to {location} and beyond.' if location else 'Quality products
delivered to your door.'}",
260         "tagline": "Style Meets Convenience",
261         "cta_text": "Shop Now"
262     },
263     "default": {
264         "hero_headline": f"Excellence in Every Detail",
265         "hero_subtext": f"Experience exceptional service at {business_name}.
{f'Proudly serving {location}.' if location else 'Where quality meets passion.'}",
266         "tagline": "Beyond Expectations",
267         "cta_text": "Get Started"
268     }
269 }
270
271 template = templates.get(business_type, templates["default"])
272
273 # Generate enhanced about text
274 about_templates = {
275     "cafe": f"At {business_name}, we believe coffee is more than a drink?it's an
experience. Our carefully sourced beans, expert baristas, and warm atmosphere create the
perfect space to connect, create, or simply unwind. Every cup is crafted with passion and
precision.",
276     "restaurant": f"Welcome to {business_name}, where culinary tradition meets modern
innovation. Our chefs craft each dish with premium ingredients and time-honored techniques,
creating flavors that delight and inspire. Join us for an unforgettable dining experience.",
277     "gym": f"{business_name} isn't just a gym?it's a community of driven individuals
committed to transformation. With state-of-the-art equipment, expert trainers, and proven
programs, we provide everything you need to reach your fitness goals and beyond.",
278     "salon": f"At {business_name}, we're passionate about bringing out your natural
beauty. Our talented stylists stay ahead of trends while honoring timeless techniques. From
cuts to color, we deliver results that make you feel confident and radiant.",
279     "real-estate": f"At {business_name}, we turn the dream of owning the perfect
property into reality. With deep local expertise, a trusted network of developers, and a
client-first approach, we guide you through every step of your property journey?from search
to possession. Your ideal home is closer than you think.",
280     "coaching": f"At {business_name}, we believe every learner has extraordinary
potential waiting to be unlocked. Our experienced mentors combine proven teaching methods
with personalized attention to help you achieve breakthroughs?in academics, skills, or life.
Your success story starts here.",
```

```
281         "online-store": f"{business_name} brings you a handpicked selection of quality
products designed to elevate your everyday life. From discovery to doorstep, we make shopping
effortless, enjoyable, and rewarding. Every order is packed with care and delivered with a
promise of satisfaction.",
282         "default": f"{business_name} is dedicated to providing exceptional
{business_type} services. We combine expertise, quality, and genuine care to ensure every
experience exceeds your expectations. Let us show you the difference true dedication makes."
283     }
284
285     about_text = about_templates.get(business_type, about_templates["default"])
286
287     # Add business description context if provided
288     if business_description and len(business_description) > 20:
289         about_text = f"{about_text} {business_description}"
290
291     return {
292         "hero_headline": template["hero_headline"],
293         "hero_subtext": template["hero_subtext"],
294         "about_text": about_text,
295         "tagline": template["tagline"],
296         "cta_text": template["cta_text"],
297         "service_descriptions": {}
298     }
299
300
301 # Singleton instance
302 copywriter = ProfessionalCopywriter()
303
```

File 8: backend/groq_helper.pyBackend - Groq Helper

```
1  """
2  Groq AI Helper
3  Fast AI text generation via Groq's LPU-powered API (OpenAI-compatible).
4  Set GROQ_API_KEY in your .env file to enable.
5
6  Recommended free models:
7      llama-3.3-70b-versatile    ? best quality, 70B, 128K ctx (default)
8      llama-3.1-8b-instant      ? fastest, lightweight tasks
9      gemma2-9b-it              ? Google Gemma, fast & accurate
10 """
11
12 import os
13 import json
14 import requests
15
16 GROQ_API_KEY = os.getenv("GROQ_API_KEY", "")
17 GROQ_MODEL = os.getenv("GROQ_MODEL", "llama-3.3-70b-versatile")
18 GROQ_BASE_URL = "https://api.groq.com/openai/v1/chat/completions"
19
20
21 def is_available() -> bool:
22     return bool(GROQ_API_KEY)
23
24
25 def generate_text(prompt: str, max_tokens: int = 1200, system: str = "") -> str:
26     """Call Groq and return the assistant response text."""
27     if not GROQ_API_KEY:
28         raise ValueError("GROQ_API_KEY is not set")
29
30     messages = []
31     if system:
32         messages.append({"role": "system", "content": system})
33     messages.append({"role": "user", "content": prompt})
34
35     response = requests.post(
36         GROQ_BASE_URL,
37         headers={
38             "Authorization": f"Bearer {GROQ_API_KEY}",
39             "Content-Type": "application/json",
40         },
41         json={
42             "model": GROQ_MODEL,
43             "messages": messages,
44             "max_tokens": max_tokens,
45             "temperature": 0.7,
46         },
47         timeout=30,
48     )
49     response.raise_for_status()
50     return response.json()["choices"][0]["message"]["content"].strip()
51
```

```
52
53 def _parse_json(text: str) -> dict:
54     """Extract and parse the first JSON object found in text."""
55     if "```json" in text:
56         text = text.split("```json")[1].split("```")[0].strip()
57     elif "```" in text:
58         text = text.split("```")[1].split("```")[0].strip()
59     j_start = text.find("{")
60     j_end = text.rfind("}") + 1
61     if j_start >= 0 and j_end > j_start:
62         return json.loads(text[j_start:j_end])
63     raise ValueError("No JSON object found in response")
64
65
66 def generate_website_copy(
67     business_name: str,
68     business_type: str,
69     business_description: str,
70     services: list,
71     location: str = "",
72     style_vibe: str = "modern",
73     service_categories: list = None,
74 ) -> dict:
75     """
76     Generate complete website copy using Groq.
77     Returns {} on failure so the caller can fall back to Gemini.
78     """
79     cat_map = {}
80     for entry in (service_categories or []):
81         name = entry.get("name", "").strip()
82         if name:
83             cat_map[name] = entry
84
85     service_lines = []
86     for svc in (services or []):
87         entry = cat_map.get(svc, {})
88         cat = entry.get("category", "").strip()
89         sub = entry.get("subcategory", "").strip()
90         if sub:
91             service_lines.append(f"{svc} [Category: {cat} | Subcategory: {sub}]")
92         elif cat:
93             service_lines.append(f"{svc} [Category: {cat}]")
94         else:
95             service_lines.append(svc)
96
97     services_str = ", ".join(service_lines) if service_lines else "Not specified"
98
99     prompt = (
100         "You are a world-class copywriter. Generate compelling website copy.\n\n"
101         f"Business: {business_name}\n"
102         f"Type: {business_type}\n"
103         f"Location: {location or 'Not specified'}\n"
104         f"Style: {style_vibe}\n"
105         f"Description: {business_description or 'Not specified'}\n"
```

```
106         f"Services/Products: {services_str}\n\n"
107         "Rules:\n"
108         "- Hero headline: 5-10 words, powerful and unique\n"
109         "- Avoid clichés: 'best', 'welcome to', 'your one-stop'\n"
110         "- Match the style vibe\n"
111         "- service_descriptions: one specific sentence per service (15-20 words)\n\n"
112         "Return ONLY valid JSON:\n"
113         "{\n"
114         '  "hero_headline": "powerful 5-10 word headline",\n'
115         '  "hero_subtext": "1-2 compelling sentences",\n'
116         '  "tagline": "3-5 word tagline",\n'
117         '  "about_text": "2-3 sentence brand story",\n'
118         '  "cta_text": "action button text (2-4 words)",\n'
119         '  "service_descriptions": {"service_name": "description"}\n'
120         "}"
121     )
122
123     try:
124         text = generate_text(prompt, max_tokens=600)
125         return _parse_json(text)
126     except Exception as e:
127         print(f"[GROQ] generate_website_copy failed: {e}")
128         return {}
129
130
131     def generate_service_descriptions(
132         services: list,
133         business_name: str,
134         business_type: str,
135         location: str = "",
136         service_categories: list = None,
137     ) -> dict:
138         """
139         Generate a short AI description for each service/product using Groq.
140         Returns fallback strings on failure.
141         """
142         if not services:
143             return {}
144
145         cat_map = {}
146         for entry in (service_categories or []):
147             name = entry.get("name", "").strip()
148             if name:
149                 cat_map[name] = entry
150
151         item_lines = []
152         for svc in services:
153             entry = cat_map.get(svc, {})
154             cat = entry.get("category", "").strip()
155             sub = entry.get("subcategory", "").strip()
156             if sub:
157                 item_lines.append(f"- {svc} [Category: {cat} | Subcategory: {sub}]\n")
158             elif cat:
159                 item_lines.append(f"- {svc} [Category: {cat}]\n")
```

```
160         else:
161             item_lines.append(f"- {svc}")
162
163         location_part = f" in {location}" if location else ""
164         prompt = (
165             f"You are an expert copywriter for {business_name}, "
166             f"a {business_type} business{location_part}.\n\n"
167             "Write one short, compelling sentence (max 18 words) for each item below.\n"
168             "Focus on the outcome, feeling, or specific benefit ? make it vivid and specific.\n"
169             "Do NOT use overused words like 'premium', 'experience our', 'crafted with care', or
'elevate'.\n\n"
170             "Items:\n" + "\n".join(item_lines) + "\n\n"
171             "Return ONLY valid JSON ? no markdown, no explanation:\n"
172             '{"item_name": "short specific description", ...}'
173         )
174
175         try:
176             text = generate_text(prompt, max_tokens=500)
177             return _parse_json(text)
178         except Exception as e:
179             print(f"[GR0Q] generate_service_descriptions failed: {e}")
180
181         # Fallback descriptions ? business-type-aware, varied by item hash
182         def _fallback_desc(svc: str, btype: str) -> str:
183             s = svc.strip()
184             sl = s.lower()
185             bt = (btype or "").lower()
186             if any(t in bt for t in ["restaurant", "cafe", "food", "dhaba", "kitchen",
187 "bistro"]):
188                 templates = [
189                     f"A rich and flavorful {sl}, crafted with fresh ingredients and authentic
190 spices.",
191                     f"Savor the irresistible taste of our {sl}, made fresh to order every time.",
192                     f"A classic {sl} bursting with bold flavors and time-honored recipes.",
193                     f"Indulge in our signature {sl}, prepared with the finest seasonal
194 ingredients.",
195                     f"A beloved {sl} that warms the soul ? perfectly spiced and beautifully
196 presented.",
197                 ]
198             elif any(t in bt for t in ["salon", "spa", "beauty", "parlour", "parlor"]):
199                 templates = [
200                     f"Rejuvenate with our expert {sl} treatment, tailored to bring out your
201 natural radiance.",
202                     f"Our professional {sl} service leaves you feeling refreshed, confident, and
203 glowing.",
204                     f"Our expert {sl} treatment uses top-quality products and skilled hands for
205 real results.",
206                 ]
207             elif any(t in bt for t in ["gym", "fitness", "yoga", "studio", "crossfit"]):
208                 templates = [
209                     f"Push your limits with {sl} ? expert-led, results-driven, and built for all
210 fitness levels.",
211                     f"Transform your body with our {sl} program ? structured, motivated, and
212 highly effective.",
```

```
204         f"Our {sl} sessions are designed to maximize results while keeping every
workout engaging.",
205     ]
206     elif any(t in bt for t in ["coaching", "tuition", "classes", "academy", "institute",
"tutor"]):
207         templates = [
208             f"Master {sl} with expert guidance, structured lessons, and proven results.",
209             f"Our {sl} program is designed to build deep understanding and exam-ready
confidence.",
210             f"Excel in {sl} with personalized mentoring and a focused, results-driven
curriculum.",
211             f"From basics to advanced ? our {sl} course prepares you for success at every
level.",
212         ]
213     elif bt == "online-store" or any(t in bt for t in ["clothing", "boutique",
"fashion"]):
214         templates = [
215             f"A stunning {sl} with rich fabric, sharp cuts, and a finish that turns
heads.",
216             f"A stunning {sl} that blends tradition with modern style ? designed to make
you stand out.",
217             f"Our {sl} collection redefines elegance with superior craftsmanship and rich
detailing.",
218         ]
219     elif any(t in bt for t in ["shop", "store"]):
220         templates = [
221             f"Discover our {sl} ? thoughtfully selected for quality, value, and everyday
satisfaction.",
222             f"Our {sl} brings you the best in its category ? reliable, affordable, and
built to impress.",
223             f"Experience the difference with our {sl} ? a customer favorite for good
reason.",
224         ]
225     elif any(t in bt for t in ["clinic", "hospital", "medical", "dental", "health"]):
226         templates = [
227             f"Expert {sl} care delivered with compassion, precision, and your wellbeing
in mind.",
228             f"Our {sl} service combines advanced techniques with personalized,
patient-first care.",
229         ]
230     elif any(t in bt for t in ["real-estate", "property", "realty", "real estate"]):
231         templates = [
232             f"Discover our {sl} ? thoughtfully designed for modern living and lasting
value.",
233             f"A well-designed {sl} offering the perfect blend of comfort, space, and
smart investment.",
234         ]
235     else:
236         templates = [
237             f"Our {sl} is delivered with genuine expertise and attention to every
detail.",
238             f"Our {sl} service is crafted to exceed your expectations every single
time.",
239             f"Quality you can count on ? our {sl} is built around your needs and
```

```
        satisfaction.",
240         ]
241         return templates[hash(sl) % len(templates)]
242
243     return {svc: _fallback_desc(svc, business_type) for svc in services}
244
```

File 9: backend/openrouter_helper.pyBackend - OpenRouter Helper

```
1  """
2  OpenRouter AI Helper
3  Provides AI text generation via OpenRouter API as an alternative to Gemini.
4  Set OPENROUTER_API_KEY in your .env file to enable.
5
6  Recommended free models (set OPENROUTER_MODEL to one of these):
7      meta-llama/llama-3.3-70b-instruct:free  ? best quality (default, 70B, 128K ctx)
8      google/gemma-3-27b-it:free           ? great quality, 27B, 131K ctx
9      mistralai/mistral-small-3.1-24b-instruct:free ? fast & precise, 24B, 128K ctx
10     deepseek/deepseek-r1-0528:free        ? reasoning model, 163K ctx
11     openrouter/free                       ? auto-picks any available free model
12  """
13
14  import os
15  import json
16  import requests
17
18  OPENROUTER_API_KEY = os.getenv("OPENROUTER_API_KEY", "")
19  OPENROUTER_MODEL   = os.getenv("OPENROUTER_MODEL", "openrouter/free")
20  OPENROUTER_BASE_URL = "https://openrouter.ai/api/v1/chat/completions"
21
22
23  def is_available() -> bool:
24      return bool(OPENROUTER_API_KEY)
25
26
27  def generate_text(prompt: str, max_tokens: int = 1200, system: str = "") -> str:
28      """Call OpenRouter and return the raw assistant response text."""
29      if not OPENROUTER_API_KEY:
30          raise ValueError("OPENROUTER_API_KEY is not set")
31
32      messages = []
33      if system:
34          messages.append({"role": "system", "content": system})
35      messages.append({"role": "user", "content": prompt})
36
37      response = requests.post(
38          OPENROUTER_BASE_URL,
39          headers={
40              "Authorization": f"Bearer {OPENROUTER_API_KEY}",
41              "Content-Type": "application/json",
42              "HTTP-Referer": "https://sitekraft.app",
43              "X-Title": "Sitekraft",
44          },
45          json={
46              "model": OPENROUTER_MODEL,
47              "messages": messages,
48              "max_tokens": max_tokens,
49          },
50          timeout=30,
51      )
```

```
52     response.raise_for_status()
53     msg = response.json()["choices"][0]["message"]
54     # Some reasoning models put output in 'reasoning' when content is empty
55     content = msg.get("content") or msg.get("reasoning") or ""
56     return content.strip()
57
58
59 def _parse_json(text: str) -> dict:
60     """Extract and parse the first JSON object found in text."""
61     # Strip markdown fences if present
62     if "` ` `json" in text:
63         text = text.split("` ` `json")[1].split("` ` `")[0].strip()
64     elif "```" in text:
65         text = text.split("```")[1].split("```")[0].strip()
66     j_start = text.find("{")
67     j_end = text.rfind("}") + 1
68     if j_start >= 0 and j_end > j_start:
69         return json.loads(text[j_start:j_end])
70     raise ValueError("No JSON object found in response")
71
72
73 def generate_service_descriptions(
74     services: list,
75     business_name: str,
76     business_type: str,
77     location: str = "",
78     service_categories: list = None,
79 ) -> dict:
80     """
81     Generate a short AI description for each service/product.
82     service_categories: list of {name, category, subcategory} dicts (optional).
83     Returns: {service_name: "one-sentence description"}
84     Falls back to template strings if API call fails.
85     """
86     if not services:
87         return {}
88
89     # Build a lookup for category/subcategory context per item
90     cat_map = {}
91     for entry in (service_categories or []):
92         name = entry.get("name", "").strip()
93         if name:
94             cat_map[name] = entry
95
96     # Format each item with its category context for the prompt
97     item_lines = []
98     for svc in services:
99         entry = cat_map.get(svc, {})
100         cat = entry.get("category", "").strip()
101         sub = entry.get("subcategory", "").strip()
102         if sub:
103             item_lines.append(f"- {svc} [Category: {cat} | Subcategory: {sub}]")
104         elif cat:
105             item_lines.append(f"- {svc} [Category: {cat}]")
```

```
106         else:
107             item_lines.append(f"- {svc}")
108
109         location_part = f" in {location}" if location else ""
110         prompt = (
111             f"You are an expert copywriter for {business_name}, "
112             f"a {business_type} business{location_part}.\n\n"
113             "Write one short, compelling sentence (max 18 words) for each item below.\n"
114             "Use the category and subcategory hints to make each description highly specific.\n"
115             "Focus on the outcome, feeling, or specific benefit ? make it vivid and concrete.\n"
116             "Do NOT use overused words like 'premium', 'experience our', 'crafted with care', or
'elevate'.\n\n"
117             "Items:\n" + "\n".join(item_lines) + "\n\n"
118             "Return ONLY valid JSON ? no markdown, no explanation:\n"
119             '{"item_name": "short specific description", ...}'
120         )
121
122         try:
123             text = generate_text(prompt, max_tokens=500)
124             return _parse_json(text)
125         except Exception as e:
126             print(f"[OPENROUTER] generate_service_descriptions failed: {e}")
127
128         # Fallback descriptions ? business-type-aware, varied by item hash
129         def _fallback_desc(svc: str, btype: str) -> str:
130             s = svc.strip()
131             sl = s.lower()
132             bt = (btype or "").lower()
133             if any(t in bt for t in ["restaurant", "cafe", "food", "dhaba", "kitchen",
134 "bistro"]):
135                 templates = [
136                     f"A rich and flavorful {sl}, crafted with fresh ingredients and authentic
spices.",
137                     f"Savor the irresistible taste of our {sl}, made fresh to order every time.",
138                     f"A classic {sl} bursting with bold flavors and time-honored recipes.",
139                     f"Indulge in our signature {sl}, prepared with the finest seasonal
ingredients.",
140                     f"A beloved {sl} that warms the soul ? perfectly spiced and beautifully
presented.",
141                 ]
142                 return templates[hash(sl) % len(templates)]
143             elif any(t in bt for t in ["salon", "spa", "beauty", "parlour", "parlor"]):
144                 templates = [
145                     f"Rejuvenate with our expert {sl} treatment, tailored to bring out your
natural radiance.",
146                     f"Our professional {sl} service leaves you feeling refreshed, confident, and
glowing.",
147                     f"Our expert {sl} treatment uses top-quality products and skilled hands for
real results.",
148                 ]
149                 return templates[hash(sl) % len(templates)]
150             elif any(t in bt for t in ["gym", "fitness", "yoga", "studio", "crossfit"]):
151                 templates = [
152                     f"Push your limits with {sl} ? expert-led, results-driven, and built for all
```

```
        fitness levels.",
152            f"Transform your body with our {sl} program ? structured, motivated, and
        highly effective.",
153            f"Our {sl} sessions are designed to maximize results while keeping every
        workout engaging.",
154        ]
155        return templates[hash(sl) % len(templates)]
156    elif any(t in bt for t in ["coaching", "tuition", "classes", "academy", "institute",
        "tutor"]):
157        templates = [
158            f"Master {sl} with expert guidance, structured lessons, and proven results.",
159            f"Our {sl} program is designed to build deep understanding and exam-ready
        confidence.",
160            f"Excel in {sl} with personalized mentoring and a focused, results-driven
        curriculum.",
161            f"From basics to advanced ? our {sl} course prepares you for success at every
        level.",
162        ]
163        return templates[hash(sl) % len(templates)]
164    elif bt == "online-store" or any(t in bt for t in ["clothing", "boutique",
        "fashion"]):
165        templates = [
166            f"A stunning {sl} with rich fabric, sharp cuts, and a finish that turns
        heads.",
167            f"A stunning {sl} that blends tradition with modern style ? designed to make
        you stand out.",
168            f"Our {sl} collection redefines elegance with superior craftsmanship and rich
        detailing.",
169        ]
170        return templates[hash(sl) % len(templates)]
171    elif any(t in bt for t in ["shop", "store"]):
172        templates = [
173            f"Discover our {sl} ? thoughtfully selected for quality, value, and everyday
        satisfaction.",
174            f"Our {sl} brings you the best in its category ? reliable, affordable, and
        built to impress.",
175            f"Experience the difference with our {sl} ? a customer favorite for good
        reason.",
176        ]
177        return templates[hash(sl) % len(templates)]
178    elif any(t in bt for t in ["clinic", "hospital", "medical", "dental", "health"]):
179        templates = [
180            f"Expert {sl} care delivered with compassion, precision, and your wellbeing
        in mind.",
181            f"Our {sl} service combines advanced techniques with personalized,
        patient-first care.",
182        ]
183        return templates[hash(sl) % len(templates)]
184    elif any(t in bt for t in ["real-estate", "property", "realty", "real estate"]):
185        templates = [
186            f"Discover our {sl} ? thoughtfully designed for modern living and lasting
        value.",
187            f"A well-designed {sl} offering the perfect blend of comfort, space, and
        smart investment.",
```

```
188         ]
189         return templates[hash(sl) % len(templates)]
190     else:
191         templates = [
192             f"Our {sl} is delivered with genuine expertise and attention to every
detail.",
193             f"Our {sl} service is crafted to exceed your expectations every single
time.",
194             f"Quality you can count on ? our {sl} is built around your needs and
satisfaction.",
195         ]
196         return templates[hash(sl) % len(templates)]
197
198     return {svc: _fallback_desc(svc, business_type) for svc in services}
199
200
201 def generate_website_copy(
202     business_name: str,
203     business_type: str,
204     business_description: str,
205     services: list,
206     location: str = "",
207     style_vibe: str = "modern",
208     service_categories: list = None,
209 ) -> dict:
210     """
211     Generate complete website copy ? same return shape as ProfessionalCopywriter.
212     service_categories: list of {name, category, subcategory} dicts (optional).
213     Returns {} on failure so the caller can fall back to Gemini.
214     """
215     # Build enriched service lines with category/subcategory context
216     cat_map = {}
217     for entry in (service_categories or []):
218         name = entry.get("name", "").strip()
219         if name:
220             cat_map[name] = entry
221
222     service_lines = []
223     for svc in (services or []):
224         entry = cat_map.get(svc, {})
225         cat = entry.get("category", "").strip()
226         sub = entry.get("subcategory", "").strip()
227         if sub:
228             service_lines.append(f"{svc} [Category: {cat} | Subcategory: {sub}]")
229         elif cat:
230             service_lines.append(f"{svc} [Category: {cat}]")
231         else:
232             service_lines.append(svc)
233
234     services_str = ", ".join(service_lines) if service_lines else "Not specified"
235
236     prompt = (
237         "You are a world-class copywriter. Generate compelling website copy.\n\n"
238         f"Business: {business_name}\n"
```

```
239         f"Type: {business_type}\n"
240         f"Location: {location or 'Not specified'}\n"
241         f"Style: {style_vibe}\n"
242         f"Services/Products: {services_str}\n\n"
243         "Rules:\n"
244         "- Hero headline: 5-10 words, powerful and unique\n"
245         "- Avoid clichés: 'best', 'welcome to', 'your one-stop'\n"
246         "- Match the style vibe\n"
247         "- service_descriptions: one specific sentence per service (15-20 words); "
248         "use the category/subcategory hints to make each description highly relevant\n\n"
249         "Return ONLY valid JSON:\n"
250         "{\n"
251         '  "hero_headline": "powerful 5-10 word headline",\n'
252         '  "hero_subtext": "1-2 compelling sentences",\n'
253         '  "tagline": "3-5 word tagline",\n'
254         '  "about_text": "2-3 sentence brand story",\n'
255         '  "cta_text": "action button text (2-4 words)",\n'
256         '  "service_descriptions": {"service_name": "description"}\n'
257         "}"
258     )
259
260     try:
261         text = generate_text(prompt, max_tokens=600)
262         return _parse_json(text)
263     except Exception as e:
264         print(f"[OPENROUTER] generate_website_copy failed: {e}")
265         return {}
266
```

File 10: backend/pexels_helper.pyBackend - Pexels Helper

```
1      """
2      Pexels Image Search Helper
3      Fetches high-quality realistic stock photos via Pexels API.
4      """
5
6      import os
7      import random
8      import requests
9      from typing import Optional
10
11      PEXELS_API_KEY = os.getenv("PEXELS_API_KEY")
12      PEXELS_SEARCH_URL = "https://api.pexels.com/v1/search"
13
14
15      def search_pexels_image(query: str, orientation: str = "landscape") -> Optional[str]:
16          """
17          Search for a single photo on Pexels.
18
19          Args:
20              query: Search terms (e.g., "yoga fitness studio")
21              orientation: "landscape", "portrait", or "square"
22
23          Returns:
24              Direct image URL (large2x quality) or None on failure.
25          """
26          results = search_pexels_images(query, count=1, orientation=orientation)
27          return results[0] if results else None
28
29
30      def search_pexels_images(query: str, count: int = 1, orientation: str = "landscape") -> list:
31          """
32          Search for multiple unique photos on Pexels.
33
34          Args:
35              query: Search terms (e.g., "yoga fitness studio")
36              count: Number of unique image URLs to return
37              orientation: "landscape", "portrait", or "square"
38
39          Returns:
40              List of image URLs (large2x quality). Empty list on failure.
41          """
42          if not PEXELS_API_KEY:
43              print("[Pexels] WARNING: PEXELS_API_KEY not set")
44              return []
45
46          try:
47              headers = {"Authorization": PEXELS_API_KEY}
48              per_page = min(max(count * 3, 15), 80) # fetch extras for variety
49              params = {"query": query, "per_page": per_page}
50              if orientation:
51                  params["orientation"] = orientation
```

```
52         response = requests.get(PEXELS_SEARCH_URL, headers=headers, params=params,
    timeout=10)
53         response.raise_for_status()
54
55         data = response.json()
56         photos = data.get("photos", [])
57
58         if not photos:
59             print(f"[Pexels] No results for '{query}'")
60             return []
61
62         # Shuffle and pick `count` unique images
63         pool = photos[:min(len(photos), per_page)]
64         random.shuffle(pool)
65         urls = []
66         for photo in pool:
67             src = photo["src"]
68             url = src.get("large2x") or src.get("large") or src.get("original")
69             if url:
70                 urls.append(url)
71                 print(f"[Pexels] Found image for '{query}' (photo id: {photo['id']})")
72             if len(urls) >= count:
73                 break
74
75         return urls
76
77     except Exception as e:
78         print(f"[Pexels] Error: {e}")
79         return []
80
```

File 11: backend/pixabay_helper.pyBackend - Pixabay Helper

```
1      """
2      Pixabay Image Search Helper
3      Fetches relevant images for services dynamically
4      """
5
6      import os
7      import requests
8      from typing import List, Dict
9
10     PIXABAY_API_KEY = os.getenv("PIXABAY_API_KEY")
11     PIXABAY_API_URL = "https://pixabay.com/api/"
12
13
14     def search_image(query: str, business_type: str = "") -> str:
15         """
16         Search for a single image on Pixabay
17
18         Args:
19             query: Service name (e.g., "lehenga", "pizza", "espresso")
20             business_type: Business category for better context
21
22         Returns:
23             Image URL (webformatURL) or fallback Unsplash image
24         """
25         if not PIXABAY_API_KEY:
26             print("[WARNING] Pixabay API key not found, using fallback images")
27             return get_fallback_image(business_type)
28
29         try:
30             # Combine query with business type for better results
31             search_query = f"{query} {business_type}".strip()
32
33             params = {
34                 "key": PIXABAY_API_KEY,
35                 "q": search_query,
36                 "image_type": "photo",
37                 "per_page": 3,
38                 "safesearch": "true"
39             }
40
41             response = requests.get(PIXABAY_API_URL, params=params, timeout=5)
42             response.raise_for_status()
43
44             data = response.json()
45
46             if data.get("hits") and len(data["hits"]) > 0:
47                 hit = data["hits"][0]
48
49                 # Pixabay has multiple URL formats - try them in order of preference
50                 # Use imageURL (full size, user_id format) or previewURL (stable CDN)
51                 image_url = None
```

```
52
53         # Try to construct a stable Pixabay CDN URL
54         image_id = hit.get("id")
55         user_id = hit.get("user_id")
56
57         if image_id and user_id:
58             # Try the user-based URL format which may be more stable
59             # Format:
https://cdn.pixabay.com/photo/{upload_date}/{user_id}/{image_id}_640.jpg
60             # Since we don't have upload date, try the preview URL which uses CDN
61             preview_url = hit.get("previewURL") # This uses CDN and is stable
62             if preview_url and "cdn.pixabay.com" in preview_url:
63                 # Modify preview URL to get higher resolution
64                 # Preview is 150px, we want 640px
65                 image_url = preview_url.replace("_150.", "_640.")
66                 print("[SUCCESS] Using modified Pixabay CDN URL")
67             else:
68                 # Fallback to webformatURL
69                 image_url = hit.get("webformatURL")
70                 print("[SUCCESS] Using webformatURL")
71         else:
72             # If ID/user_id not available, use webformatURL
73             image_url = hit.get("webformatURL")
74             print("[SUCCESS] Using webformatURL")
75
76         return image_url if image_url else get_fallback_image(business_type)
77     else:
78         print("[WARNING] No Pixabay results, using fallback")
79         return get_fallback_image(business_type)
80
81     except Exception as e:
82         print("[ERROR] Pixabay API error, using fallback")
83         return get_fallback_image(business_type)
84
85
86 def get_service_images(services: List[str], business_type: str = "") -> Dict[str, str]:
87     """
88     Get images for multiple services
89
90     Args:
91         services: List of service names
92         business_type: Business category
93
94     Returns:
95         Dict mapping service name to image URL
96     """
97     image_map = {}
98
99     for service in services[:6]: # Limit to 6 services
100         service_name = service.strip()
101         if service_name:
102             image_url = search_image(service_name, business_type)
103             # If fallback was returned, use the service name as unique seed
104             # so each service card shows a different image
```

```
105         if "seed/business" in image_url:
106             seed = service_name.lower().replace(' ', '').replace("'", '')
107             image_url = f"https://picsum.photos/seed/{seed}/640/400"
108             image_map[service_name] = image_url
109
110     return image_map
111
112
113 def get_fallback_image(business_type: str) -> str:
114     """
115     Get a fallback Unsplash image based on business type
116     """
117     fallback_images = {
118         "boutique": "https://picsum.photos/seed/boutique/640/400",
119         "cafe": "https://picsum.photos/seed/cafe/640/400",
120         "restaurant": "https://picsum.photos/seed/restaurant/640/400",
121         "gym": "https://picsum.photos/seed/gym/640/400",
122         "salon": "https://picsum.photos/seed/salon/640/400",
123         "hotel": "https://picsum.photos/seed/hotel/640/400",
124         "yoga": "https://picsum.photos/seed/yoga/640/400",
125         "bakery": "https://picsum.photos/seed/bakery/640/400",
126     }
127
128     return fallback_images.get(business_type, "https://picsum.photos/seed/business/640/400")
129
130
131 # Test function
132 if __name__ == "__main__":
133     # Test with different services
134     test_services = ["Lehenga", "Saree", "Kurti"]
135     print("Testing Pixabay image search...")
136     images = get_service_images(test_services, "boutique")
137
138     for service, url in images.items():
139         print("Image fetched successfully")
140
```

File 12: backend/image_proxy.pyBackend - Image Proxy

```
1      """
2      Image Proxy - Download and cache images from Pixabay
3      Converts expiring Pixabay URLs to permanent base64 data URLs
4      """
5
6      import requests
7      import base64
8      from typing import Dict, List
9
10     def download_and_encode_image(image_url: str) -> str:
11         """
12         Download image from URL and convert to base64 data URL
13
14         Args:
15             image_url: Original image URL (e.g., Pixabay URL)
16
17         Returns:
18             Base64 data URL (permanent, doesn't expire)
19         """
20         try:
21             response = requests.get(image_url, timeout=10)
22             response.raise_for_status()
23
24             # Get content type
25             content_type = response.headers.get('Content-Type', 'image/jpeg')
26
27             # Convert to base64
28             image_data = base64.b64encode(response.content).decode('utf-8')
29
30             # Return as data URL
31             data_url = f"data:{content_type};base64,{image_data}"
32
33             print("[SUCCESS] Downloaded and encoded image")
34             return data_url
35
36         except Exception as e:
37             print("[ERROR] Failed to download image")
38             return None
39
40
41     def proxy_service_images(service_images: Dict[str, str]) -> Dict[str, str]:
42         """
43         Download all service images and convert to base64 data URLs
44
45         Args:
46             service_images: Dict mapping service name to Pixabay URL
47
48         Returns:
49             Dict mapping service name to base64 data URL
50         """
51         proxied_images = {}
```

```
52
53     for service_name, image_url in service_images.items():
54         # Download and encode the image
55         data_url = download_and_encode_image(image_url)
56
57         if data_url:
58             proxied_images[service_name] = data_url
59         else:
60             # If download fails, keep original URL (will fallback on frontend)
61             proxied_images[service_name] = image_url
62
63     return proxied_images
64
```

File 13: backend/supabase_config.pyBackend - Supabase Config

```
1      """
2      Supabase Configuration and Helper Functions
3      Handles database, auth, storage, and real-time features
4      """
5
6      import os
7      from typing import Optional, Dict, List
8      from supabase import create_client, Client
9      from datetime import datetime
10     import json
11
12     class SupabaseManager:
13         """Manages Supabase operations for the AI Website Builder"""
14
15         def __init__(self):
16             self.url = os.getenv("SUPABASE_URL")
17             self.key = os.getenv("SUPABASE_KEY")
18             self.service_key = os.getenv("SUPABASE_SERVICE_KEY")
19
20             if not self.url or not self.key:
21                 print("[WARNING] Supabase credentials not configured")
22                 self.client = None
23                 self.enabled = False
24             else:
25                 try:
26                     self.client: Client = create_client(self.url, self.key)
27                     self.enabled = True
28                     print("[OK] Supabase connected successfully")
29                 except Exception as e:
30                     print(f"[WARNING] Supabase connection failed: {e}")
31                     print("[WARNING] Continuing without Supabase...")
32                     self.client = None
33                     self.enabled = False
34
35         def is_enabled(self) -> bool:
36             """Check if Supabase is properly configured"""
37             return self.enabled
38
39         # =====
40         # WEBSITE METADATA OPERATIONS
41         # =====
42
43         async def save_website_metadata(self, website_data: Dict) -> Optional[Dict]:
44             """
45             Save generated website metadata to Supabase
46
47             Args:
48                 website_data: {
49                     'session_id': str,
50                     'title': str,
51                     'description': str,
```

```
52         'deployment_url': str,
53         'framework': str (e.g., 'react-vite'),
54         'created_at': str,
55         'last_modified': str,
56         'version': int,
57         'features': list,
58         'status': str ('draft', 'deployed', 'archived')
59     }
60     """
61     if not self.enabled:
62         return None
63
64     try:
65         data = {
66             "session_id": website_data.get("session_id"),
67             "title": website_data.get("title"),
68             "description": website_data.get("description"),
69             "deployment_url": website_data.get("deployment_url"),
70             "framework": website_data.get("framework", "react-vite"),
71             "created_at": website_data.get("created_at", datetime.now().isoformat()),
72             "last_modified": datetime.now().isoformat(),
73             "version": website_data.get("version", 1),
74             "features": json.dumps(website_data.get("features", [])),
75             "status": website_data.get("status", "draft"),
76             "metadata": json.dumps(website_data.get("metadata", {}))
77         }
78
79         response = self.client.table("websites").insert(data).execute()
80         return response.data[0] if response.data else None
81
82     except Exception as e:
83         print(f"[ERROR] Error saving website metadata: {e}")
84         return None
85
86     async def get_website_metadata(self, session_id: str) -> Optional[Dict]:
87         """Get website metadata by session ID"""
88         if not self.enabled:
89             return None
90
91         try:
92             response = self.client.table("websites").select("*").eq("session_id",
session_id).execute()
93             return response.data[0] if response.data else None
94         except Exception as e:
95             print(f"[ERROR] Error fetching website metadata: {e}")
96             return None
97
98     async def update_website_metadata(self, session_id: str, updates: Dict) ->
Optional[Dict]:
99         """Update website metadata"""
100         if not self.enabled:
101             return None
102
103         try:
```

```
104         updates["last_modified"] = datetime.now().isoformat()
105         response = self.client.table("websites").update(updates).eq("session_id",
session_id).execute()
106         return response.data[0] if response.data else None
107     except Exception as e:
108         print(f"[ERROR] Error updating website metadata: {e}")
109         return None
110
111     # =====
112     # BOOKING OPERATIONS
113     # =====
114
115     async def save_booking(self, session_id: str, booking_data: Dict) -> Optional[Dict]:
116         """Save a booking submission"""
117         if not self.enabled:
118             return None
119
120         try:
121             data = {
122                 "session_id": session_id,
123                 "name": booking_data.get("name"),
124                 "email": booking_data.get("email"),
125                 "phone": booking_data.get("phone"),
126                 "service": booking_data.get("service"),
127                 "date": booking_data.get("date"),
128                 "time": booking_data.get("time"),
129                 "notes": booking_data.get("notes", ""),
130                 "status": "pending",
131                 "created_at": datetime.now().isoformat()
132             }
133
134             response = self.client.table("bookings").insert(data).execute()
135             return response.data[0] if response.data else None
136
137         except Exception as e:
138             print(f"[ERROR] Error saving booking: {e}")
139             return None
140
141     async def get_bookings(self, session_id: str) -> List[Dict]:
142         """Get all bookings for a website"""
143         if not self.enabled:
144             return []
145
146         try:
147             response = self.client.table("bookings").select("*").eq("session_id",
session_id).order("created_at", desc=True).execute()
148             return response.data or []
149         except Exception as e:
150             print(f"[ERROR] Error fetching bookings: {e}")
151             return []
152
153     # =====
154     # ORDER OPERATIONS
155     # =====
```

```
156
157     async def save_order(self, session_id: str, order_data: Dict) -> Optional[Dict]:
158         """Save an order submission"""
159         if not self.enabled:
160             return None
161
162         try:
163             data = {
164                 "session_id": session_id,
165                 "name": order_data.get("name"),
166                 "email": order_data.get("email"),
167                 "phone": order_data.get("phone"),
168                 "address": order_data.get("address"),
169                 "items": json.dumps(order_data.get("items", [])),
170                 "total": sum(item.get("price", 0) * item.get("qty", 0) for item in
order_data.get("items", [])),
171                 "notes": order_data.get("notes", ""),
172                 "status": "pending",
173                 "created_at": datetime.now().isoformat()
174             }
175
176             response = self.client.table("orders").insert(data).execute()
177             return response.data[0] if response.data else None
178
179         except Exception as e:
180             print(f"[ERROR] Error saving order: {e}")
181             return None
182
183     async def get_orders(self, session_id: str) -> List[Dict]:
184         """Get all orders for a website"""
185         if not self.enabled:
186             return []
187
188         try:
189             response = self.client.table("orders").select("").eq("session_id",
session_id).order("created_at", desc=True).execute()
190             return response.data or []
191         except Exception as e:
192             print(f"[ERROR] Error fetching orders: {e}")
193             return []
194
195     # =====
196     # CONTACT FORM OPERATIONS
197     # =====
198
199     async def save_contact(self, session_id: str, contact_data: Dict) -> Optional[Dict]:
200         """Save a contact form submission"""
201         if not self.enabled:
202             return None
203
204         try:
205             data = {
206                 "session_id": session_id,
207                 "name": contact_data.get("name"),
```

```
208         "email": contact_data.get("email"),
209         "phone": contact_data.get("phone", ""),
210         "message": contact_data.get("message"),
211         "status": "unread",
212         "created_at": datetime.now().isoformat()
213     }
214
215     response = self.client.table("contacts").insert(data).execute()
216     return response.data[0] if response.data else None
217
218 except Exception as e:
219     print(f"[ERROR] Error saving contact: {e}")
220     return None
221
222 async def get_contacts(self, session_id: str) -> List[Dict]:
223     """Get all contact submissions for a website"""
224     if not self.enabled:
225         return []
226
227     try:
228         response = self.client.table("contacts").select("*").eq("session_id",
229 session_id).order("created_at", desc=True).execute()
230         return response.data or []
231     except Exception as e:
232         print(f"[ERROR] Error fetching contacts: {e}")
233         return []
234
235 # =====
236 # FILE STORAGE OPERATIONS
237 # =====
238
239 async def upload_file(self, bucket_name: str, file_path: str, file_data: bytes) ->
Optional[str]:
240     """
241     Upload a file to Supabase Storage
242
243     Returns: Public URL of the uploaded file
244     """
245     if not self.enabled:
246         return None
247
248     try:
249         response = self.client.storage.from_(bucket_name).upload(file_path, file_data)
250
251         # Get public URL
252         public_url = self.client.storage.from_(bucket_name).get_public_url(file_path)
253         return public_url
254
255     except Exception as e:
256         print(f"[ERROR] Error uploading file: {e}")
257         return None
258
259 async def delete_file(self, bucket_name: str, file_path: str) -> bool:
260     """Delete a file from Supabase Storage"""
```

```
260         if not self.enabled:
261             return False
262
263         try:
264             self.client.storage.from_(bucket_name).remove([file_path])
265             return True
266         except Exception as e:
267             print(f"[ERROR] Error deleting file: {e}")
268             return False
269
270         # =====
271         # AUTHENTICATION OPERATIONS
272         # =====
273
274     async def sign_up_user(self, email: str, password: str) -> Optional[Dict]:
275         """Sign up a new user"""
276         if not self.enabled:
277             return None
278
279         try:
280             response = self.client.auth.sign_up({
281                 "email": email,
282                 "password": password
283             })
284             return response
285         except Exception as e:
286             print(f"[ERROR] Error signing up user: {e}")
287             return None
288
289     async def sign_in_user(self, email: str, password: str) -> Optional[Dict]:
290         """Sign in a user"""
291         if not self.enabled:
292             return None
293
294         try:
295             response = self.client.auth.sign_in_with_password({
296                 "email": email,
297                 "password": password
298             })
299             return response
300         except Exception as e:
301             print(f"[ERROR] Error signing in user: {e}")
302             return None
303
304
305     # Global Supabase manager instance
306     supabase_manager = SupabaseManager()
307
```

File 14: backend/requirements.txt

Backend - Requirements

```
1     fastapi==0.115.0
2     uvicorn[standard]==0.30.6
3     pydantic==2.9.2
4     python-multipart==0.0.12
5     google-generativeai==0.8.3
6     python-dotenv==1.0.1
7     supabase==2.9.1
8     requests==2.32.3
9     httpx==0.27.2
10
```

File 15: backend/Dockerfile

Backend - Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
# ?? Python dependencies ??????????????????????????????????????????????????????????????
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
# ?? Copy backend code ??????????????????????????????????????????????????????????????
COPY . .
# ?? Create runtime directories ??????????????????????????????????????????????????????????
RUN mkdir -p generated_websites website_states static/uploads site_data
EXPOSE 8000
CMD uvicorn autonomous_agent:app --host 0.0.0.0 --port ${PORT:-8000}
```

File 16: backend/render.yaml

Backend - Render Config

```
1  services:
2    - type: web
3      name: ai-website-builder-api
4      runtime: python
5      buildCommand: pip install -r requirements.txt
6      startCommand: uvicorn autonomous_agent:app --host 0.0.0.0 --port $PORT
7      healthCheckPath: /
8      envVars:
9        - key: PYTHON_VERSION
10          value: 3.12.0
11        - key: GEMINI_API_KEY
12          sync: false
13        - key: OPENROUTER_API_KEY
14          sync: false
15        - key: OPENROUTER_MODEL
16          value: google/gemma-3-27b-it:free
17        - key: GROQ_API_KEY
18          sync: false
19        - key: GROQ_MODEL
20          value: llama-3.3-70b-versatile
21        - key: PEXELS_API_KEY
22          sync: false
23        - key: SUPABASE_URL
24          sync: false
25        - key: SUPABASE_KEY
26          sync: false
27        - key: SUPABASE_SERVICE_KEY
28          sync: false
29        - key: VERCEL_TOKEN
30          sync: false
31        - key: PYTHONIOENCODING
32          value: utf-8
33
```

File 17: backend/runtime.txt

Backend - Runtime

```
1    python-3.12.0
2
```

File 18: frontend/src/main.jsx

Frontend - main.jsx

```
1   import { StrictMode } from 'react'
2   import { createRoot } from 'react-dom/client'
3   import './index.css'
4   import App from './App.jsx'
5
6   createRoot(document.getElementById('root')).render(
7     <StrictMode>
8       <App />
9     </StrictMode>,
10  )
11
```

File 19: frontend/src/App.jsx

Frontend - App.jsx

```

1    import { useState, useEffect } from 'react'
2    import axios from 'axios'
3    import JSZip from 'jszip'
4    import Onboarding from './Onboarding'
5    import { useTheme } from './useTheme'
6    import './App.css'
7
8    const API = import.meta.env.VITE_API_URL || ''
9
10   // ??? Result View (shown after website is built) ??????????????????????????????
11
12   function ResultView({ result, userData, onStartOver }) {
13     const [theme, toggleTheme] = useTheme()
14     const [deployedUrl, setDeployedUrl] = useState(result.deployment_url || null)
15     const [deploying, setDeploying] = useState(false)
16     const [downloading, setDownloading] = useState(false)
17     const [previewOpen, setPreviewOpen] = useState(false)
18
19     const generatePreviewHTML = () => {
20       const btype = userData?.businessType || 'business'
21       const bname = userData?.businessName || 'Your Business'
22       const blocation = userData?.location || 'your city'
23       const items = userData?.items || []
24       // Default brand colors chosen to feel natural for each business type
25       const typeDefaultColors = {
26         salon: '#b5838d', // dusty rose ? feminine, luxurious
27         gym: '#f97316', // vivid orange ? energetic, powerful
28         restaurant: '#c2622d', // warm terracotta ? cozy, appetizing
29         clothing: '#6b21a8', // deep purple ? fashion-forward
30         'real-estate': '#1d4ed8', // strong blue ? trustworthy, professional
31         coaching: '#0d9488', // teal ? progressive, educational
32         'online-store': '#059669', // emerald ? fresh, modern ecommerce
33         other: '#6366f1', // indigo default
34       }
35       const color = userData?.aiColor
36         ? (typeDefaultColors[btype] || '#6366f1')
37         : (userData?.brandColor || typeDefaultColors[btype] || '#6366f1')
38       const isProduct = userData?.sellingType === 'products'
39       const phone = userData?.phone || ''
40       const email = userData?.email || ''
41       const instagram = userData?.instagram || ''
42
43       // Derive a light tint from the brand color for backgrounds
44       const colorLight = color + '18'
45
46       // ?? Per-business-type design themes ??????????????????????????????????
47       const typeThemes = {
48         salon: {
49           fonts: 'Playfair+Display:ital,wght@0,700;1,400;1,700&family=Lato:wght@300;400;700',
50           sectionLabel: 'Treatments',
51

```

CSS:

```

`body{font-family: 'Lato', sans-serif;background:#ffffb9}.logo,.section-title{font-family: 'Play
fair
Display', serif;font-style:italic}.hero
h1{font-style:italic;letter-spacing:1px}.nav-links
a{letter-spacing:2px;text-transform:uppercase;font-size:11px}.item-card{border:1px
solid
#f5e6e8;border-radius:28px}.btn{border-radius:50px}.section{background:#ffffb9}.about-bg{back
ground:#fdf3f0!important}.stats-bar{background:#fff6f3}`,
52 },
53 gym: {
54 fonts: 'Oswald:wght@500;600;700&family=Roboto:wght@300;400;700',
55 sectionLabel: 'Programs',
56
css:
`body{background:#080808;color:#e0e0e0;font-family: 'Roboto', sans-serif}nav{background:rgba(8,
8,8,0.98)!important;border-bottom:1px
solid
#1a1a1a}.logo{font-family: 'Oswald', sans-serif;text-transform:uppercase;letter-spacing:3px;col
or:${color}!important}.nav-links
a{color:#888!important;text-transform:uppercase;letter-spacing:2px;font-size:11px}.nav-links
a:hover{color:${color}!important}h2.section-title{font-family: 'Oswald', sans-serif;text-transf
orm:uppercase;letter-spacing:3px;color:#fff}.section-sub{color:#888}.section{background:#0d0d
0d}.about-bg{background:#080808!important}.stats-bar{background:#050505;border-top:3px solid
${color}}.stat-num{color:${color};font-size:40px}.stat-label{color:#666}.item-card{background
:#141414;border:1px
solid
#252525;border-radius:6px}.item-card-body
h3{color:#fff;font-family: 'Oswald', sans-serif;text-transform:uppercase;letter-spacing:1px;font-size:15px}.item-card-body
p{color:#888}.section-tag{background:${color}20;color:${color}}.btn{border-radius:4px;text-tr
ansform:uppercase;letter-spacing:2px;font-size:12px;font-family: 'Oswald', sans-serif}footer{ba
ckground:#030303}.footer-copy,.footer-built{color:#444!important}.footer-desc{color:#444!impo
rtant}.footer-links
ul
li
a{color:#666}.footer-logo{color:${color}}.why-item{color:#aaa}.contact-info-text
strong{color:#fff}.contact-info-text
span{color:#888}.testi-card{background:rgba(255,255,255,0.04);border-color:#222}.hero-badge{b
ackground:rgba(255,255,255,0.08);border-color:rgba(255,255,255,0.15)}`,
57 },
58 restaurant: {
59 fonts: 'Playfair+Display:ital,wght@0,700;1,400;1,700&family=Lato:wght@300;400;700',
60 sectionLabel: 'Menu',
61
css:
`body{background:#fffd5f;font-family: 'Lato', sans-serif}.logo,.section-title{font-family: 'Play
fair
Display', serif;font-style:italic}.hero
h1{font-style:italic}.section{background:#fffd5f}.about-bg{background:#fff8e8!important}.item
-card{border:1px
solid
#f5e8d0;background:#fffef9;border-radius:18px}.item-card-body
h3{font-family: 'Playfair
Display', serif}.hero-badge{background:rgba(245,180,50,0.2);border-color:rgba(245,180,50,0.5)}
.stats-bar{background:#fff9ef}footer{background:#1a0a00}`,
62 },
63 clothing: {
64
css:
65 fonts:
'Cormorant+Garamond:ital,wght@0,400;0,600;1,400&family=Inter:wght@300;400;500',
66 sectionLabel: 'Collection',
css:
`body{background:#fafafa;font-family: 'Inter', sans-serif}.logo{font-family: 'Cormorant
Garamond', serif;font-size:26px;letter-spacing:6px;text-transform:uppercase;font-weight:400}.n
av-links
a{letter-spacing:3px;text-transform:uppercase;font-size:10px;font-weight:400}h2.section-title
{font-family: 'Cormorant

```

```

    Garamond', serif; font-size: 2.6rem !important; font-weight: 400; letter-spacing: 2px; font-style: italic}.section-tag{letter-spacing: 4px; font-size: 10px; text-transform: uppercase}.item-card, .btn, .item-btn{border-radius: 0}.item-card
img{height: 280px}.item-btn{letter-spacing: 2px; text-transform: uppercase; font-size: 11px}.btn{letter-spacing: 2px; text-transform: uppercase; font-size: 12px}footer{background: #0a0a0a}`,
67     },
68     'online-store': {
69         fonts: 'Nunito:wght@400;600;700;800;900',
70         sectionLabel: 'Products',
71
72         css: `body{font-family: 'Nunito', sans-serif}.logo{font-weight: 900; font-size: 24px}.item-card{border-radius: 20px}.item-price{font-size: 20px; font-weight: 900}.item-btn{border-radius: 10px; font-weight: 700}.section-title{font-weight: 900}`,
73     },
74     'real-estate': {
75         fonts: 'Raleway:wght@400;600;700;800&family=Open+Sans:wght@300;400;600',
76         sectionLabel: 'Properties',
77
78         css: `body{font-family: 'Open Sans', sans-serif}.logo{font-family: 'Raleway', sans-serif; font-weight: 800; letter-spacing: 1px}h2.section-title{font-family: 'Raleway', sans-serif; font-weight: 800}.hero-badge{letter-spacing: 2px; text-transform: uppercase; font-size: 10px}.item-card
img{height: 240px}.section-tag{letter-spacing: 2px; text-transform: uppercase}`,
79     },
80     coaching: {
81         fonts: 'Poppins:wght@300;400;500;600;700;800',
82         sectionLabel: 'Courses',
83
84         css: `body{font-family: 'Poppins', sans-serif}.logo, .section-title, .hero h1{font-weight: 800}.item-card{border-radius: 20px}`,
85     },
86     }
87
88     const bTheme = typeThemes[btype] || typeThemes.coaching
89     const bFonts = bTheme.fonts || 'Inter:wght@400;500;600;700;800'
90     const bThemeCSS = bTheme.css || ''
91     const bSectionLabel = isProduct ? 'Products' : (bTheme.sectionLabel || 'Services')
92
93     // Category filtering: detect if any items have categories set
94     const hasCategories = items.some(i => i.category && i.category.trim())
95     const uniqueCategories = hasCategories
96       ? [...new Set(items.filter(i => i.category && i.category.trim()).map(i => i.category.trim()))]
97       : []
98
99     // All images come from Pexels via the backend API ? no Unsplash fallbacks
100     const heroImg = userData?.heroImage || null
101     const aboutImg = userData?.aboutImage || null
102
103     // Hero background ? gradient only if no Pexels image
104     const heroBg = heroImg
105       ? `linear-gradient(135deg, rgba(0,0,0,0.6) 0%, rgba(0,0,0,0.35) 100%), url('${heroImg}')
```

```

106     ? ``
107     : `<div style="width:100%;height:460px;background:linear-gradient(135deg,${colorLight}
0%,${color}44
100%);display:flex;align-items:center;justify-content:center;font-size:64px;border-radius:0;"
>?</div>`
108
109     const whyPoints = {
110         restaurant: ['Fresh ingredients daily','Authentic recipes','Warm ambience','Expert
chefs'],
111         clothing: ['Premium fabrics','Latest trends','Perfect fit guarantee','Wide
collection'],
112         salon: ['Certified professionals','Premium products','Relaxing
experience','Latest techniques'],
113         gym: ['Expert trainers','Modern equipment','Personalised plans','Supportive
community'],
114         coaching: ['Experienced mentors','Proven curriculum','Small batch
sessions','Career support'],
115         'real-estate': ['Verified properties','Expert guidance','Best deals','Trusted
network'],
116         'online-store':['Fast delivery','Secure payments','Easy returns','Wide selection'],
117         other: ['Professional team','Quality service','Customer first','Best results'],
118         default: ['Professional team','Quality service','Customer first','Best results'],
119     }
120     const whys = whyPoints[btype] || whyPoints.default
121
122     const testimonials = [
123         { name: 'Priya S.', text: `${bname} completely exceeded my expectations. Highly
recommend!`, stars: 5 },
124         { name: 'Rahul M.', text: `Best ${btype} in ${blocation}. The quality is unmatched.`,
stars: 5 },
125         { name: 'Anjali K.', text: `I've been a loyal customer for years. Always a wonderful
experience.`, stars: 5 },
126     ]
127
128     return `<!DOCTYPE html>
129     <html lang="en">
130     <head>
131         <meta charset="UTF-8">
132         <meta name="viewport" content="width=device-width, initial-scale=1.0">
133         <title>${bname}</title>
134         <link rel="icon" href="data:,">
135         <link href="https://fonts.googleapis.com/css2?family=${bFonts}&display=swap"
rel="stylesheet">
136     <style>
137         *{margin:0;padding:0;box-sizing:border-box}
138         html{scroll-behavior:smooth}
139         body{font-family:'Inter',sans-serif;color:#0f172a;background:#fff}
140
141         /* NAV */
142         nav{background:rgba(255,255,255,0.95);backdrop-filter:blur(10px);padding:0
5%;height:68px;display:flex;justify-content:space-between;align-items:center;position:sticky;
top:0;z-index:100;border-bottom:1px solid #f1f5f9;box-shadow:0 1px 20px rgba(0,0,0,0.06)}
143         .logo{font-size:22px;font-weight:800;color:${color};letter-spacing:-0.5px}
144         .nav-links{display:flex;gap:32px;align-items:center}

```

```

145 .nav-links
a{text-decoration:none;color:#475569;font-weight:500;font-size:14px;transition:color .2s}
146 .nav-links a:hover{color:${color}}
147 .nav-cta{background:${color};color:#fff !important;padding:10px
22px;border-radius:8px;font-weight:600 !important;font-size:13px !important}
148
.hamburger{display:none;flex-direction:column;gap:5px;cursor:pointer;background:none;border:none;padding:4px;z-index:101}
149 .hamburger
span{display:block;width:24px;height:2px;background:#475569;border-radius:2px;transition:all
.3s}
150
151 /* HERO */
152
.hero{min-height:520px;display:flex;align-items:center;justify-content:center;text-align:center;background:${heroBg};position:relative}
153 .hero-badge{display:inline-block;background:rgba(255,255,255,0.15);border:1px solid
rgba(255,255,255,0.3);color:#fff;padding:6px
18px;border-radius:50px;font-size:13px;font-weight:600;margin-bottom:20px;backdrop-filter:blur(4px)}
154 .hero-inner{max-width:760px;padding:0 24px;color:#fff}
155 .hero
h1{font-size:clamp(2.2rem,5.5vw,4rem);font-weight:800;line-height:1.1;margin-bottom:20px;letter-spacing:-1px}
156 .hero h1 span{color:${color}}
157 .hero
p{font-size:18px;opacity:.88;margin-bottom:36px;line-height:1.7;max-width:560px;margin-left:auto;margin-right:auto}
158 .hero-btns{display:flex;gap:14px;justify-content:center;flex-wrap:wrap}
159 .btn{display:inline-flex;align-items:center;gap:8px;padding:14px
32px;border-radius:10px;font-weight:700;font-size:15px;text-decoration:none;cursor:pointer;border:none;transition:all .2s;font-family:inherit}
160 .btn-primary{background:${color};color:#fff;box-shadow:0 4px 20px ${color}55}
161 .btn-primary:hover{transform:translateY(-2px);box-shadow:0 8px 28px ${color}77}
162 .btn-outline{background:transparent;color:#fff;border:2px solid rgba(255,255,255,0.7)}
163 .btn-outline:hover{background:rgba(255,255,255,0.1)}
164
165 /* STATS BAR */
166 .stats-bar{background:#fff;padding:28px
5%;display:flex;justify-content:center;gap:60px;box-shadow:0 4px 24px
rgba(0,0,0,0.08);flex-wrap:wrap}
167 .stat{text-align:center}
168 .stat-num{font-size:28px;font-weight:800;color:${color}}
169 .stat-label{font-size:13px;color:#64748b;font-weight:500;margin-top:2px}
170
171 /* SECTIONS */
172 .section{padding:88px 5%}
173 .section-inner{max-width:1200px;margin:0 auto}
174
.section-tag{display:inline-block;background:${colorLight};color:${color};font-size:12px;font-weight:700;padding:5px
14px;border-radius:50px;text-transform:uppercase;letter-spacing:1px;margin-bottom:14px}
175
.section-title{font-size:clamp(1.8rem,3.5vw,2.6rem);font-weight:800;line-height:1.15;margin-b

```

```

    ottom:16px; letter-spacing:-0.5px}
176     .section-sub{font-size:16px;color:#64748b;line-height:1.7;max-width:580px}
177     .section-head{margin-bottom:56px}
178     .section-head.center{text-align:center}
179     .section-head.center .section-sub{margin:0 auto}
180
181     /* ITEMS GRID */
182
183     .items-grid{display:grid;grid-template-columns:repeat(auto-fill,minmax(280px,1fr));gap:28px}
184         .item-card{border-radius:20px;overflow:hidden;box-shadow:0 2px 20px
185         rgba(0,0,0,0.07);background:#fff;transition:transform .25s,box-shadow .25s;border:1px solid
186         #f1f5f9}
187         .item-card:hover{transform:translateY(-6px);box-shadow:0 12px 40px rgba(0,0,0,0.12)}
188         .item-card img{width:100%;height:210px;object-fit:cover}
189         .item-card-body{padding:22px}
190         .item-card-body h3{font-size:17px;font-weight:700;margin-bottom:8px}
191         .item-card-body p{font-size:14px;color:#64748b;line-height:1.6}
192
193         .item-card-footer{display:flex;justify-content:space-between;align-items:center;margin-top:16
194         px;padding-top:16px;border-top:1px solid #f1f5f9}
195         .item-price{font-weight:800;font-size:18px;color:${color}}
196         .item-btn{background:${colorLight};color:${color};border:none;padding:8px
197         18px;border-radius:8px;font-size:13px;font-weight:600;cursor:pointer;font-family:inherit;tran
198         sition:background .2s}
199         .item-btn:hover{background:${color};color:#fff}
200
201     /* ABOUT */
202     .about-bg{background:#f8fafc}
203     .about-grid{display:grid;grid-template-columns:1fr 1fr;gap:64px;align-items:center}
204     .about-img{border-radius:24px;overflow:hidden;position:relative}
205
206         .about-img
207         img{width:100%;height:460px;object-fit:contain;display:block;background:#f8f9fa}
208
209         .about-img-badge{position:absolute;bottom:24px;left:24px;background:#fff;border-radius:14px;p
210         adding:16px 20px;box-shadow:0 8px 32px rgba(0,0,0,0.15)}
211         .about-img-badge strong{display:block;font-size:26px;font-weight:800;color:${color}}
212         .about-img-badge span{font-size:12px;color:#64748b;font-weight:500}
213         .why-list{display:grid;grid-template-columns:1fr 1fr;gap:14px;margin-top:32px}
214
215         .why-item{display:flex;align-items:center;gap:10px;font-size:14px;font-weight:500;color:#3741
216         51}
217
218         .why-check{width:22px;height:22px;border-radius:6px;background:${colorLight};display:flex;ali
219         gn-items:center;justify-content:center;flex-shrink:0;color:${color};font-size:13px;font-weigh
220         t:700}
221
222     /* TESTIMONIALS */
223     .testimonials-bg{background:linear-gradient(135deg,#0f172a 0%,#1e293b 100%) !important}
224     .testi-grid{display:grid;grid-template-columns:repeat(3,1fr);gap:24px}
225         .testi-card{background:rgba(255,255,255,0.06);border:1px solid
226         rgba(255,255,255,0.1);border-radius:20px;padding:28px;color:#fff}
227         .testi-stars{color:#f59e0b;font-size:16px;margin-bottom:14px}
228         .testi-text{font-size:15px;line-height:1.7;opacity:.85;margin-bottom:20px}
229         .testi-author{display:flex;align-items:center;gap:12px}

```

```

213     .testi-avatar{width:42px;height:42px;border-radius:50%;background:${color};display:flex;align
-items:center;justify-content:center;font-weight:700;font-size:16px;color:#fff;flex-shrink:0}
214     .testi-name{font-weight:600;font-size:14px}
215     .testi-role{font-size:12px;opacity:.6;margin-top:2px}
216
217     /* CONTACT */
218     .contact-grid{display:grid;grid-template-columns:1fr 1fr;gap:64px;align-items:start}
219     .contact-info-list{display:flex;flex-direction:column;gap:20px;margin-top:32px}
220     .contact-info-item{display:flex;align-items:center;gap:14px}
221
222     .contact-icon{width:44px;height:44px;border-radius:12px;background:${colorLight};display:flex
;align-items:center;justify-content:center;font-size:20px;flex-shrink:0}
223     .contact-info-text strong{display:block;font-size:14px;font-weight:700;margin-bottom:2px}
224     .contact-info-text span{font-size:13px;color:#64748b}
225     .contact-form{background:#f8fafc;border-radius:20px;padding:36px}
226     .form-group{margin-bottom:18px}
227     .form-group label{display:block;font-size:13px;font-weight:600;color:#374151;margin-bottom:7px}
228     .form-group input,.form-group textarea{width:100%;padding:12px 16px;border:1.5px solid
#e2e8f0;border-radius:10px;font-size:14px;font-family:inherit;color:#0f172a;outline:none;tran
sition:border-color .2s;background:#fff}
229     .form-group input:focus,.form-group textarea:focus{border-color:${color}}
230     .form-group textarea{height:110px;resize:none}
231     .form-submit{width:100%;padding:14px;background:${color};color:#fff;border:none;border-radius
:10px;font-size:15px;font-weight:700;cursor:pointer;font-family:inherit;transition:opacity
.2s}
232     .form-submit:hover{opacity:.9}
233
234     /* FOOTER */
235     footer{background:#0f172a;color:#fff;padding:48px 5% 28px}
236     .footer-inner{max-width:1200px;margin:0 auto}
237
238     .footer-top{display:flex;justify-content:space-between;align-items:flex-start;gap:40px;flex-w
rap:wrap;padding-bottom:40px;border-bottom:1px solid rgba(255,255,255,.1)}
239     .footer-brand{max-width:300px}
240     .footer-logo{font-size:22px;font-weight:800;color:${color};margin-bottom:12px}
241     .footer-desc{font-size:14px;opacity:.6;line-height:1.7}
242     .footer-links
h4{font-size:13px;font-weight:700;text-transform:uppercase;letter-spacing:.8px;opacity:.5;margin-bottom:16px}
243     .footer-links ul{list-style:none;display:flex;flex-direction:column;gap:10px}
244     .footer-links ul li
a{text-decoration:none;color:#fff;opacity:.7;font-size:14px;transition:opacity .2s}
245     .footer-links ul li a:hover{opacity:1}
246
247     .footer-bottom{padding-top:24px;display:flex;justify-content:space-between;align-items:center
;flex-wrap:wrap;gap:12px}
248     .footer-copy{font-size:13px;opacity:.5}
249     .footer-built{font-size:12px;opacity:.4}
250
251     /* PREVIEW BANNER */
252     .preview-banner{background:#fef3c7;border-bottom:3px solid #f59e0b;padding:10px

```

```

20px;text-align:center;font-size:13px;font-weight:600;color:#92400e;display:flex;align-items:
center;justify-content:center;gap:8px}

250
251     /* RESPONSIVE */
252     @media(max-width:900px){
253         .about-grid,.contact-grid{grid-template-columns:1fr}
254         .testi-grid{grid-template-columns:1fr}
255         .stats-bar{gap:32px}
256         .why-list{grid-template-columns:1fr}
257         .hamburger{display:flex}
258
259         .nav-links{display:none;position:absolute;top:68px;left:0;right:0;background:rgba(255,255,255
260 ,0.98);flex-direction:column;padding:16px 5%;gap:0;box-shadow:0 8px 24px
261 rgba(0,0,0,0.1);border-top:1px solid #f1f5f9;z-index:99}
262         .nav-links.open{display:flex}
263         .nav-links a{padding:14px 0;border-bottom:1px solid #f1f5f9;font-size:15px}
264         .nav-links a:last-child{border-bottom:none}
265         .nav-cta{margin-top:8px;text-align:center;border-radius:8px;padding:12px !important}
266     }
267     @media(max-width:600px){
268         .hero h1{font-size:2rem}
269         .section{padding:56px 5%}
270         .items-grid{grid-template-columns:1fr}
271     }
272
273     /* FILTER TABS */
274     .filter-tabs-wrap{margin-bottom:36px;text-align:center}
275
276     .filter-tabs{display:flex;flex-wrap:wrap;gap:10px;justify-content:center;margin-bottom:14px}
277     .filter-tab{padding:9px 22px;border-radius:50px;border:1.5px solid
278 #e2e8f0;background:#fff;color:#64748b;font-size:13px;font-weight:600;cursor:pointer;transitio
279 n:all .2s;font-family:inherit}
280     .filter-tab.active{background:${color};color:#fff;border-color:${color};box-shadow:0 4px
281 14px ${color}44}
282     .filter-tab:hover:not(.active){border-color:${color};color:${color}}
283     .sub-tabs{display:flex;flex-wrap:wrap;gap:8px;justify-content:center}
284     .sub-tab{padding:6px 16px;border-radius:50px;border:1.5px solid
285 #e2e8f0;background:transparent;color:#64748b;font-size:12px;font-weight:600;cursor:pointer;tr
286 ansition:all .2s;font-family:inherit}
287     .sub-tab.active{background:${color}22;color:${color};border-color:${color}88}
288     .sub-tab:hover:not(.active){border-color:${color}66;color:${color}}
289     .item-card.hidden{display:none !important}
290
291     /* ?? ANIMATIONS ?????????????????????????????????????????????????????????????? */
292     @keyframes ob-fadeUp
293     {from{opacity:0;transform:translateY(48px)}to{opacity:1;transform:translateY(0)}}
294     @keyframes ob-fadeIn {from{opacity:0}to{opacity:1}}
295     @keyframes ob-slideL
296     {from{opacity:0;transform:translateX(-64px)}to{opacity:1;transform:translateX(0)}}
297     @keyframes ob-slideR
298     {from{opacity:0;transform:translateX(64px)}to{opacity:1;transform:translateX(0)}}
299     @keyframes ob-scaleUp{from{opacity:0;transform:scale(0.82)
300 translateY(24px)}to{opacity:1;transform:scale(1) translateY(0)}}
301     @keyframes ob-float {0%,100%{transform:translateY(0)}50%{transform:translateY(-10px)}}

```

```

289     @keyframes ob-pulse {0%,100%{box-shadow:0 4px 20px ${color}55}50%{box-shadow:0 8px 40px
    ${color}99}}
290     @keyframes hero-rise {from{opacity:0;transform:translateY(36px)}
    to{opacity:1;transform:translateY(0)}}
291     @keyframes hero-scaleUp {from{opacity:0;transform:scale(.88) translateY(20px)}
    to{opacity:1;transform:scale(1) translateY(0)}}
292     /* Hero ? per-business-type entrance */
293     ${
294         salon:      `.hero-badge{animation:hero-rise .9s cubic-bezier(.25,.46,.45,.94) .05s
    both}.hero h1{animation:hero-rise 1.1s cubic-bezier(.25,.46,.45,.94) .22s both}.hero
    p{animation:hero-rise 1.1s cubic-bezier(.25,.46,.45,.94) .42s
    both}.hero-btns{animation:hero-rise 1.1s cubic-bezier(.25,.46,.45,.94) .6s both}`,
295         gym:        `.hero-badge{animation:hero-scaleUp .45s cubic-bezier(.22,1,.36,1) .0s
    both}.hero h1{animation:hero-scaleUp .5s cubic-bezier(.22,1,.36,1) .12s both}.hero
    p{animation:hero-rise .5s ease .28s both}.hero-btns{animation:hero-scaleUp .5s
    cubic-bezier(.34,1.56,.64,1) .4s both}`,
296         restaurant: `.hero-badge{animation:hero-rise .8s cubic-bezier(.25,.46,.45,.94) .1s
    both}.hero h1{animation:hero-rise .95s cubic-bezier(.25,.46,.45,.94) .25s both}.hero
    p{animation:hero-rise .95s cubic-bezier(.25,.46,.45,.94) .44s
    both}.hero-btns{animation:hero-rise .95s cubic-bezier(.25,.46,.45,.94) .62s both}`,
297         clothing:   `.hero-badge{animation:hero-rise .7s cubic-bezier(.16,1,.3,1) .0s
    both}.hero h1{animation:hero-rise .8s cubic-bezier(.16,1,.3,1) .18s both}.hero
    p{animation:hero-rise .8s cubic-bezier(.16,1,.3,1) .36s both}.hero-btns{animation:hero-rise
    .8s cubic-bezier(.16,1,.3,1) .52s both}`,
298         'real-estate':`.hero-badge{animation:hero-rise 1s cubic-bezier(.25,.46,.45,.94) .05s
    both}.hero h1{animation:hero-rise 1.1s cubic-bezier(.25,.46,.45,.94) .2s both}.hero
    p{animation:hero-rise 1.1s cubic-bezier(.25,.46,.45,.94) .4s
    both}.hero-btns{animation:hero-rise 1.1s cubic-bezier(.25,.46,.45,.94) .58s both}`,
299         'online-store':`.hero-badge{animation:hero-scaleUp .5s cubic-bezier(.34,1.56,.64,1)
    .0s both}.hero h1{animation:hero-scaleUp .55s cubic-bezier(.34,1.56,.64,1) .15s both}.hero
    p{animation:hero-rise .6s ease .32s both}.hero-btns{animation:hero-scaleUp .55s
    cubic-bezier(.34,1.56,.64,1) .46s both}`,
300     ][btype] || `.hero-badge{animation:hero-rise .8s cubic-bezier(.25,.46,.45,.94) .05s
    both}.hero h1{animation:hero-rise .9s cubic-bezier(.25,.46,.45,.94) .2s both}.hero
    p{animation:hero-rise .9s cubic-bezier(.25,.46,.45,.94) .38s
    both}.hero-btns{animation:hero-rise .9s cubic-bezier(.25,.46,.45,.94) .55s both}`
301     /* Scroll-triggered elements ? JS hides them before observing */
302     [data-anim].ob-done{opacity:1}
303     /* Always-on ambient */
304     .about-img-badge{animation:ob-float 3.8s ease-in-out infinite !important}
305     .btn-primary{animation:ob-pulse 2.8s ease-in-out infinite}
306     /* ?? Business-type overrides ?? */
307     ${bThemeCSS}
308     </style>
309     </head>
310     <body>
311
312     <!-- NAV -->
313     <nav>
314         ${userData?.logoImage
315             ? `![${bname}](${userData.logoImage})

```

```

318         <a href="#"
onclick="event.preventDefault();closeMenu();document.getElementById('home').scrollIntoView({b
ehavior:'smooth'})">Home</a>
319         <a href="#"
onclick="event.preventDefault();closeMenu();document.getElementById('services').scrollIntoVie
w({behavior:'smooth'})">${bSectionLabel}</a>
320         <a href="#"
onclick="event.preventDefault();closeMenu();document.getElementById('about').scrollIntoView({
behavior:'smooth'})">About</a>
321         <a href="#"
onclick="event.preventDefault();closeMenu();document.getElementById('contact').scrollIntoView
({behavior:'smooth'})">Contact</a>
322         <a href="#" class="nav-cta"
onclick="event.preventDefault();closeMenu();document.getElementById('contact').scrollIntoView
({behavior:'smooth'})">Get Started</a>
323     </div>
324     <button class="hamburger" id="hamburger-btn" onclick="toggleMenu()" aria-label="Toggle
navigation">
325         <span></span><span></span><span></span>
326     </button>
327 </nav>
328 <script>
329     function toggleMenu(){
330         var menu=document.getElementById('nav-menu');
331         var btn=document.getElementById('hamburger-btn');
332         menu.classList.toggle('open');
333         btn.classList.toggle('active');
334     }
335     function closeMenu(){
336         document.getElementById('nav-menu').classList.remove('open');
337         document.getElementById('hamburger-btn').classList.remove('active');
338     }
339 </script>
340 <script>
341 (function(){
342     /*
343     * Per-business-type animation personality:
344     * salon      ? slow, silky fade-up (luxury spa feel)
345     * gym        ? fast, punchy scale-up (high energy)
346     * restaurant ? warm gentle fade-up (cozy, inviting)
347     * clothing   ? crisp slide-up with overshoot (editorial fashion)
348     * real-estate ? steady slow fade-up (trustworthy, premium)
349     * coaching   ? clean fade-up (clear, professional)
350     * online-store? quick scale-pop (product excitement)
351     * other      ? smooth fade-up
352     */
353     var cfg = {
354         restaurant: {anim:'ob-fadeUp', card:'ob-fadeUp', dur:'0.8s',
ease:'cubic-bezier(.25,.46,.45,.94)'},
355         clothing: {anim:'ob-fadeUp', card:'ob-scaleUp', dur:'0.65s',
ease:'cubic-bezier(.16,1,.3,1)'},
356         salon: {anim:'ob-fadeUp', card:'ob-fadeUp', dur:'1.0s',
ease:'cubic-bezier(.25,.46,.45,.94)'},
357         gym: {anim:'ob-scaleUp', card:'ob-scaleUp', dur:'0.48s',

```

```
ease:'cubic-bezier(.22,1,.36,1)'}},
358     coaching: {anim:'ob-fadeUp', card:'ob-fadeUp', dur:'0.72s',
ease:'cubic-bezier(.25,.46,.45,.94)'}},
359     'real-estate': {anim:'ob-fadeUp', card:'ob-fadeUp', dur:'0.9s',
ease:'cubic-bezier(.25,.46,.45,.94)'}},
360     'online-store':{anim:'ob-scaleUp', card:'ob-scaleUp', dur:'0.55s',
ease:'cubic-bezier(.34,1.56,.64,1)'}},
361     other: {anim:'ob-fadeUp', card:'ob-fadeUp', dur:'0.7s',
ease:'cubic-bezier(.25,.46,.45,.94)'}},
362     }['${btype}'] || {anim:'ob-fadeUp', card:'ob-fadeUp', dur:'0.7s',
ease:'cubic-bezier(.25,.46,.45,.94)'};
363
364     /* Named overrides */
365     var named = {slideL:'ob-slideL', slideR:'ob-slideR', fadeUp:'ob-fadeUp',
fadeIn:'ob-fadeIn', card: cfg.card};
366
367     var io = new IntersectionObserver(function(entries){
368     entries.forEach(function(e){
369     if(!e.isIntersecting) return;
370     var el = e.target;
371     var anim = named[el.dataset.anim] || cfg.anim;
372     var delay= el.dataset.delay || '0s';
373     el.style.animation = anim+' '+cfg.dur+' '+cfg.ease+' '+delay+' both';
374     el.classList.add('ob-done');
375     io.unobserve(el);
376     });
377     },{threshold:0.08, rootMargin:'0px 0px -30px 0px'}));
378
379     /* Hide via JS ? so elements are visible if JS/observer fails */
380     var observed = [];
381     document.querySelectorAll('[data-anim]').forEach(function(el){
382     el.style.opacity = '0';
383     observed.push(el);
384     io.observe(el);
385     });
386
387     /* Stagger card children automatically if parent has data-stagger */
388     document.querySelectorAll('[data-stagger]').forEach(function(parent){
389     var step = parseFloat(parent.dataset.stagger) || 0.1;
390     Array.from(parent.children).forEach(function(child, i){
391     child.dataset.anim = child.dataset.anim || 'card';
392     child.dataset.delay = (i * step).toFixed(2) + 's';
393     child.style.opacity = '0';
394     observed.push(child);
395     io.observe(child);
396     });
397     });
398
399     /* Safety fallback: after 1.8s show anything the observer missed */
400     setTimeout(function(){
401     observed.forEach(function(el){
402     if(!el.classList.contains('ob-done')){
403     el.style.transition = 'opacity 0.6s ease, transform 0.6s ease';
404     el.style.opacity = '1';
```

```
405         el.style.transform = 'none';
406     }
407     });
408     }, 1800);
409     })();
410 </script>
411
412 <!-- HERO -->
413 <section class="hero" id="home">
414     <div class="hero-inner">
415         <div class="hero-badge">? ${blocation}</div>
416         <h1>${result.hero_headline || `Welcome to <span>${bname}</span>`}</h1>
417         <p>${result.hero_subtext || `Your trusted ${btype} in ${blocation}. Quality, passion, and
excellence in every detail.`}</p>
418         <div class="hero-btns">
419             <button class="btn btn-primary">Get Started ?</button>
420             <button class="btn btn-outline">Learn More</button>
421         </div>
422     </div>
423 </section>
424
425 <!-- STATS BAR -->
426 <div class="stats-bar">
427     <div class="stat" data-anim="fadeUp" data-delay="0s"><div class="stat-num">500+</div><div
class="stat-label">Happy Customers</div></div>
428         <div class="stat" data-anim="fadeUp" data-delay="0.12s"><div
class="stat-num">${items.length || 6}</div><div
class="stat-label">${bSectionLabel}</div></div>
429         <div class="stat" data-anim="fadeUp" data-delay="0.24s"><div
class="stat-num">4.9?</div><div class="stat-label">Average Rating</div></div>
430         <div class="stat" data-anim="fadeUp" data-delay="0.36s"><div class="stat-num">5+</div><div
class="stat-label">Years Experience</div></div>
431     </div>
432
433 <!-- PRODUCTS / SERVICES -->
434 <section class="section" id="services">
435     <div class="section-inner">
436         <div class="section-head center" data-anim="fadeUp">
437             <span class="section-tag">Our ${bSectionLabel}</span>
438             <h2 class="section-title">What We ${isProduct ? 'Offer' : 'Provide'}</h2>
439             <p class="section-sub">Handpicked ${bSectionLabel.toLowerCase()} crafted with care and
expertise for the best experience.</p>
440         </div>
441         ${hasCategories ? `
442         <div class="filter-tabs-wrap" data-anim="fadeUp" data-delay="0.1s">
443             <div class="filter-tabs" id="cat-tabs">
444                 <button class="filter-tab active" onclick="setCatTab(this,'all')">All</button>
445                 ${uniqueCategories.map(cat => `<button class="filter-tab"
onclick="setCatTab(this,'${cat.replace(/'/g, '\\')}')">${cat}</button>`).join('')}
446             </div>
447             <div class="sub-tabs" id="sub-tabs-wrap" style="display:none"></div>
448         </div>
449         ` : ''}
450         <div class="items-grid" ${!hasCategories ? 'data-stagger="0.1"' : ''}>
```

```

451     ${items.slice(0, hasCategories ? 30 : 6).map((item, idx) => `
452         <div class="item-card" data-anim="card" data-delay="${hasCategories ? '0s' : (idx *
0.1) + 's'}"${item.category} && item.category.trim() ? `
data-cat="${item.category.trim().replace(/"/g, '&quot;')}"` : ''`${item.subcategory} &&
item.subcategory.trim() ? ` data-sub="${item.subcategory.trim().replace(/"/g, '&quot;')}"` :
''>
453         ${(() => {
454             const imgSrc = item.fetchedImage
455             || (result.service_images && result.service_images[item.name])
456             || item.imagePreview
457             || null
458             return imgSrc
459             ? ``
460             : `<div
style="width:100%;height:210px;background:linear-gradient(135deg,${colorLight} 0%,${color}44
100%);display:flex;align-items:center;justify-content:center;font-size:40px;">??</div>`
461         })()}
462         <div class="item-card-body">
463             <h3>${item.name}</h3>
464             <p>${item.description} || (result.service_descriptions &&
(result.service_descriptions[item.name] ||
result.service_descriptions[item.name.toLowerCase()]) ||
Object.entries(result.service_descriptions).find(([k]) => k.toLowerCase() ===
item.name.toLowerCase())?.[1]) || ''</p>
465             <div class="item-card-footer">
466                 ${item.price ? `<span class="item-price">${item.price}</span>` : `<span
class="item-price" style="color:#94a3b8;font-size:14px">Contact for price</span>`}
467                 <button class="item-btn">${isProduct ? 'Order Now' : 'Book Now'}</button>
468             </div>
469         </div>
470     </div>
471     `).join('')}
472 </div>
473 ${hasCategories ? `
474 <script>
475 (function(){
476     var allCards = document.querySelectorAll('#services .item-card');
477     var subWrap = document.getElementById('sub-tabs-wrap');
478     var subMap = {};
479     allCards.forEach(function(el){
480         var cat = el.dataset.cat;
481         var sub = el.dataset.sub;
482         if (!cat) return;
483         if (!subMap[cat]) subMap[cat] = [];
484         if (sub && subMap[cat].indexOf(sub) === -1) subMap[cat].push(sub);
485     });
486     var activeCat = 'all', activeSub = 'all';
487     function filterItems(){
488         allCards.forEach(function(el){
489             var cat = el.dataset.cat || '';
490             var sub = el.dataset.sub || '';
491             var okCat = activeCat === 'all' || cat === activeCat;
492             var okSub = activeSub === 'all' || sub === activeSub;

```

```

493         el.classList.toggle('hidden', !(okCat && okSub));
494     });
495 }
496 function renderSubTabs(cat){
497     if (!subWrap) return;
498     var subs = (cat !== 'all' && subMap[cat]) ? subMap[cat] : [];
499     if (subs.length === 0){ subWrap.style.display = 'none'; return; }
500     var h = '<button class="sub-tab active"
onclick="setSubTab(this,\'all\')">All</button>';
501     subs.forEach(function(s){
502         h += '<button class="sub-tab" onclick="setSubTab(this,\'\' +
s.replace(/'/g, '\\\\') + '\\')">' + s + '</button>';
503     });
504     subWrap.innerHTML = h;
505     subWrap.style.display = 'flex';
506     activeSub = 'all';
507 }
508 window.setCatTab = function(btn, cat){
509     document.querySelectorAll('#cat-tabs .filter-tab').forEach(function(t){
510         t.classList.remove('active'); });
511     btn.classList.add('active');
512     activeCat = cat; activeSub = 'all';
513     renderSubTabs(cat); filterItems();
514 }
515 window.setSubTab = function(btn, sub){
516     document.querySelectorAll('#sub-tabs-wrap .sub-tab').forEach(function(t){
517         t.classList.remove('active'); });
518     btn.classList.add('active');
519     activeSub = sub; filterItems();
520 }
521 </script>
522 ` : ''}
523 </div>
524 </section>
525
526 <!-- ABOUT -->
527 <section class="section about-bg" id="about">
528     <div class="section-inner">
529         <div class="about-grid">
530             <div class="about-img" data-anim="fadeUp">
531                 ${aboutImgHtml}
532                 <div class="about-img-badge">
533                     <strong>5+</strong>
534                     <span>Years of Excellence</span>
535                 </div>
536             </div>
537             <div data-anim="fadeUp" data-delay="0.18s">
538                 <span class="section-tag">Our Story</span>
539                 <h2 class="section-title">Why Choose ${bname}?</h2>
540                 <p class="section-sub">We are passionate about delivering the best ${btype}
experience in ${blocation}. Every detail matters to us ? from quality to customer
satisfaction.</p>

```

```
541     <div class="why-list">
542         ${whys.map(w => `
543             <div class="why-item">
544                 <div class="why-check">?</div>
545                 <span>${w}</span>
546             </div>
547         `).join('')}
548     </div>
549     <div style="margin-top:32px">
550         <button class="btn btn-primary" style="font-size:14px">Learn More About Us</button>
551     </div>
552 </div>
553 </div>
554 </div>
555 </section>
556
557 <!-- TESTIMONIALS -->
558 <section class="section testimonials-bg">
559     <div class="section-inner">
560         <div class="section-head center" data-anim="fadeUp" style="color:#fff">
561             <span class="section-tag"
562 style="background:rgba(255,255,255,0.1);color:#fff">Reviews</span>
563         <h2 class="section-title" style="color:#fff">What Our Customers Say</h2>
564         <p class="section-sub" style="color:rgba(255,255,255,0.6)">Real experiences from our
565 happy customers.</p>
566     </div>
567     <div class="testi-grid">
568         ${testimonials.map((t, i) => `
569             <div class="testi-card" data-anim="card" data-delay="${i * 0.15}s">
570                 <div class="testi-stars">${t.stars}</div>
571                 <p class="testi-text">${t.text}</p>
572                 <div class="testi-author">
573                     <div class="testi-avatar">${t.name.charAt(0)}</div>
574                     <div>
575                         <div class="testi-name">${t.name}</div>
576                         <div class="testi-role">Verified Customer</div>
577                     </div>
578                 </div>
579             </div>
580         `).join('')}
581     </div>
582 </section>
583
584 <!-- CONTACT -->
585 <section class="section" id="contact">
586     <div class="section-inner">
587         <div class="contact-grid">
588             <div data-anim="fadeUp">
589                 <span class="section-tag">Contact Us</span>
590                 <h2 class="section-title">Get In Touch</h2>
591                 <p class="section-sub">We'd love to hear from you. Reach out and we'll respond as
592 soon as possible.</p>
593                 <div class="contact-info-list">
```

```

592         ${phone} ? `<div class="contact-info-item"><div class="contact-icon">?</div><div
class="contact-info-text"><strong>Phone</strong><span>${phone}</span></div></div>` : ''}
593         ${email} ? `<div class="contact-info-item"><div class="contact-icon">??</div><div
class="contact-info-text"><strong>Email</strong><span>${email}</span></div></div>` : ''}
594         ${instagram?} `<div class="contact-info-item"><div class="contact-icon">?</div><div
class="contact-info-text"><strong>Instagram</strong><span>${instagram}</span></div></div>` :
''}
595         <div class="contact-info-item"><div class="contact-icon">?</div><div
class="contact-info-text"><strong>Location</strong><span>${blocation}</span></div></div>
596     </div>
597 </div>
598 <div class="contact-form" data-anim="fadeUp" data-delay="0.18s">
599     <h3 style="font-size:18px;font-weight:700;margin-bottom:24px">Send a Message</h3>
600     <div class="form-group"><label>Your Name</label><input type="text" placeholder="Enter
your name"></div>
601         <div class="form-group"><label>Email Address</label><input type="email"
placeholder="Enter your email"></div>
602         <div class="form-group"><label>Message</label><textarea placeholder="How can we help
you?"></div>
603         <button class="form-submit">Send Message ?</button>
604     </div>
605 </div>
606 </div>
607 </section>
608
609 <!-- FOOTER -->
610 <footer>
611     <div class="footer-inner">
612         <div class="footer-top">
613             <div class="footer-brand" data-anim="fadeUp">
614                 ${userData?.logoImage}
615                 ? ``
616                 : `<div class="footer-logo">${bname}</div>`
617             <p class="footer-desc">Your trusted ${btype} in ${blocation}. Quality and excellence
in everything we do.</p>
618         </div>
619         <div class="footer-links" data-anim="fadeUp" data-delay="0.15s">
620             <h4>Quick Links</h4>
621             <ul>
622                 <li><a href="#home">Home</a></li>
623                 <li><a href="#services">${bSectionLabel}</a></li>
624                 <li><a href="#about">About Us</a></li>
625                 <li><a href="#contact">Contact</a></li>
626             </ul>
627         </div>
628         <div class="footer-links" data-anim="fadeUp" data-delay="0.3s">
629             <h4>Contact</h4>
630             <ul>
631                 ${phone ? `<li><a href="#">${phone}</a></li>` : ''}
632                 ${email ? `<li><a href="#">${email}</a></li>` : ''}
633                 <li><a href="#">${blocation}</a></li>
634             </ul>
635         </div>

```

```
636     </div>
637     <div class="footer-bottom" data-anim="fadeUp" data-delay="0.1s">
638       <span class="footer-copy">© 2025 ${bname}. All rights reserved.</span>
639       <span class="footer-built">Built with Sitekraft</span>
640     </div>
641   </div>
642 </footer>
643
644 </body>
645 </html>`
646   }
647
648   // Deploy the preview HTML to Vercel as a static site on mount
649   useEffect(() => {
650     if (deployedUrl) return // already deployed (from backend)
651     const projectName = userData?.businessName || 'my-website'
652     setDeploying(true)
653     const html = generatePreviewHTML()
654     axios.post(`${API}/api/react/deploy-html`, {
655       project_name: projectName,
656       html_content: html
657     })
658       .then(res => {
659         if (res.data?.deployment_url) setDeployedUrl(res.data.deployment_url)
660       })
661       .catch(err => console.error('[deploy-html]', err))
662       .finally(() => setDeploying(false))
663   }, []) // eslint-disable-line react-hooks/exhaustive-deps
664
665   const openPreview = () => {
666     if (window.innerWidth <= 768) {
667       setPreviewOpen(true)
668     } else {
669       const html = generatePreviewHTML()
670       const blob = new Blob([html], { type: 'text/html' })
671       window.open(URL.createObjectURL(blob), '_blank')
672     }
673   }
674
675   const downloadFiles = async () => {
676     if (!result.files || downloading) return
677     setDownloading(true)
678     try {
679       const zip = new JSZip()
680       const projectName = userData?.businessName
681         ? userData.businessName.toLowerCase().replace(/\s+/g, '-')
682         : 'sitekraft-website'
683
684       // ?? Step 1: collect all external image URLs from generated files ???????
685       const isImageUrl = (url) => {
686         if (!url || !url.startsWith('http')) return false
687         return (
688           /images\.pexels\.com\/.test(url) ||
689           /cdn\.pixabay\.com\/.test(url) ||
```

```
690         /pixabay\.com\/get\/.test(url) ||
691         /picsum\.photos\/.test(url) ||
692         /\.(?:jpg|jpeg|png|webp|gif|svg)(\?|$)/i.test(url)
693     )
694 }
695 const urlRegex = /https?:\/\/[^\s"'`\\,\\)]+\/g
696 const urlToLocal = new Map() // externalURL ? "public/images/image-N.ext"
697 let imgIdx = 0
698
699 Object.values(result.files).forEach(content => {
700     const matches = content.match(urlRegex) || []
701     matches.forEach(raw => {
702         const url = raw.replace(/["'`]+$/, '') // strip trailing quotes/commas
703         if (isImageUrl(url) && !urlToLocal.has(url)) {
704             const extMatch = url.match(/\.(jpg|jpeg|png|webp|gif|svg)/i)
705             const ext = extMatch ? extMatch[1].toLowerCase() : 'jpg'
706             urlToLocal.set(url, `public/images/image-${imgIdx++}.${ext}`)
707         }
708     })
709 })
710
711 // ?? Step 2: fetch images in parallel, add to ZIP ??????????????????????
712 await Promise.allSettled(
713     Array.from(urlToLocal.entries()).map(async ([url, localPath]) => {
714         try {
715             const res = await fetch(url)
716             if (!res.ok) { urlToLocal.delete(url); return }
717             const blob = await res.blob()
718             zip.file(localPath, blob)
719         } catch {
720             urlToLocal.delete(url) // keep original URL in code if fetch fails
721         }
722     })
723 )
724
725 // ?? Step 3: add code files, rewriting fetched image URLs to local paths ??
726 Object.entries(result.files).forEach(([name, content]) => {
727     let modified = content
728     urlToLocal.forEach((localPath, url) => {
729         // Replace external URL with Vite public-asset path
730         modified = modified.split(url).join(`/${localPath.replace('public/', '')}`)
731     })
732     zip.file(name, modified)
733 })
734
735 // ?? Step 4: generate ZIP and trigger download ??????????????????????
736 const blob = await zip.generateAsync({ type: 'blob' })
737 const a = document.createElement('a')
738 a.href = URL.createObjectURL(blob)
739 a.download = `${projectName}.zip`
740 document.body.appendChild(a)
741 a.click()
742 document.body.removeChild(a)
743 URL.revokeObjectURL(a.href)
```

```
744     } finally {
745         setDownloading(false)
746     }
747 }
748
749 return (
750     <div className="result-root">
751         <header className="result-header">
752             <div className="result-brand">? Sitekraft</div>
753             <div className="result-header-right">
754                 <button
755                     className="result-theme-btn"
756                     onClick={toggleTheme}
757                     title={theme === 'dark' ? 'Switch to light mode' : 'Switch to dark mode'}
758                 >
759                     {theme === 'dark' ? '??' : '?'}
760                 </button>
761                 <button className="result-new-btn" onClick={onStartOver}>+ New Website</button>
762             </div>
763         </header>
764
765         <div className="result-body">
766             {/* Left panel */}
767             <div className="result-left">
768                 <div className="result-success-card">
769                     <div className="result-tick">?</div>
770                     <h2>Your website is ready!</h2>
771                     <p>{userData?.businessName} · {userData?.businessType}</p>
772                 </div>
773
774
775                 {deploying && (
776                     <div style={{background:'#eff6ff',border:'1px solid
777 #bfdbfe',borderRadius:'10px',padding:'12px 16px',fontSize:'13px',color:'#1d4ed8',display:'flex',alignItems:'center',gap:'8px'}}>
778                         <span style={{display:'inline-block',width:'14px',height:'14px',border:'2px solid #3b82f6',borderTopColor:'transparent',borderRadius:'50%',animation:'spin 0.8s linear infinite'}} />
779                         Deploying your website to Vercel?
780                     </div>
781                 )}
782
783                 {deployedUrl && !deploying && (
784                     <div style={{display:'flex',flexDirection:'column',gap:'8px'}}>
785                         <button
786                             className="result-live-link"
787                             style={{cursor:'pointer',border:'none',textAlign:'center'}}
788                             onClick={() => window.open(deployedUrl, '_blank')}
789                         >
790                             ? Open Live Website ?
791                         </button>
792
793                         <div style={{background:'#f1f5f9',borderRadius:'8px',padding:'10px 14px',fontSize:'12px',wordBreak:'break-all',color:'#475569'}}>
794                             <span style={{fontWeight:600,display:'block',marginBottom:'4px'}}>Your
```

```

    shareable link:</span>
793         <span style={{userSelect:'all',cursor:'text'}}>{deployedUrl}</span>
794     </div>
795 </div>
796 }}
797
798 <div className="result-actions">
799     <button className="result-btn result-btn-primary" onClick={openPreview}>
800         ? Open Preview
801     </button>
802     {result.files && (
803         <button
804             className="result-btn result-btn-secondary"
805             onClick={downloadFiles}
806             disabled={downloading}
807             style={downloading ? { opacity: 0.7, cursor: 'wait' } : {}}
808         >
809             {downloading ? '? Preparing ZIP...' : '? Download ZIP'}
810         </button>
811     )}
812 </div>
813 </div>
814
815 {/* Right panel ? inline preview (desktop only) */}
816 <div className="result-right">
817     <div className="result-preview-bar">
818         <span className="result-preview-dot red" />
819         <span className="result-preview-dot yellow" />
820         <span className="result-preview-dot green" />
821
822         <span
823             className="result-preview-url">{userData?.businessName?.toLowerCase().replace(/\s+/g, '-')}vercel.app</span>
824     </div>
825     <iframe
826         className="result-iframe"
827         srcDoc={generatePreviewHTML()}
828         title="Website Preview"
829         sandbox="allow-scripts allow-same-origin"
830     />
831 </div>
832
833 {/* Fullscreen preview modal ? mobile only */}
834 {previewOpen && (
835     <div className="result-preview-modal">
836         <div className="result-preview-modal-bar">
837             <div style={{display:'flex',alignItems:'center',gap:'6px',flex:1,minWidth:0}}>
838                 <span className="result-preview-dot red" />
839                 <span className="result-preview-dot yellow" />
840                 <span className="result-preview-dot green" />
841                 <span className="result-preview-url" style={{marginLeft:'8px'}}>
842                     {userData?.businessName?.toLowerCase().replace(/\s+/g, '-')}vercel.app
843                 </span>
844             </div>
845         </div>
846     </div>
847 )}

```

```
844         <button className="result-preview-modal-close" onClick={() =>
      setPreviewOpen(false)}></button>
845     </div>
846     <iframe
847         className="result-iframe"
848         srcDoc={generatePreviewHTML()}
849         title="Website Preview"
850         sandbox="allow-scripts allow-same-origin"
851     />
852 </div>
853 )}
854 </div>
855 )
856 }
857
858 // ??? App Root ?????????????????????????????????????????????????????????????
859
860 export default function App() {
861     const [result, setResult] = useState(null)
862     const [onboardingData, setOnboardingData] = useState(null)
863
864     const handleComplete = (apiResult, userData) => {
865         setResult(apiResult)
866         setOnboardingData(userData)
867     }
868
869     if (!result) {
870         return <Onboarding onComplete={handleComplete} />
871     }
872
873     return (
874         <ResultView
875             result={result}
876             userData={onboardingData}
877             onStartOver={() => { setResult(null); setOnboardingData(null) }}
878         />
879     )
880 }
881
```

File 20: frontend/src/Onboarding.jsx

Frontend - Onboarding.jsx

```

1    import { useState, useEffect, useRef } from 'react'
2    import axios from 'axios'
3    import { useTheme } from './useTheme'
4    import './Onboarding.css'
5
6    // ??? Image Utilities ?????????????????????????????????????????????????????????????
7
8    /**
9     * Convert a File/Blob to a compressed base64 JPEG data URL.
10    * Resizes to maxW×maxH keeping aspect ratio, then compresses to JPEG.
11    * Returns the data URL string (permanent ? survives deployment).
12    */
13    function toCompressedBase64(file, maxW = 1200, maxH = 900, quality = 0.78) {
14      return new Promise((resolve, reject) => {
15        const reader = new FileReader()
16        reader.onerror = reject
17        reader.onload = e => {
18          const img = new Image()
19          img.onerror = reject
20          img.onload = () => {
21            let { width: w, height: h } = img
22            const ratio = Math.min(maxW / w, maxH / h, 1) // never upscale
23            w = Math.round(w * ratio)
24            h = Math.round(h * ratio)
25            const canvas = document.createElement('canvas')
26            canvas.width = w
27            canvas.height = h
28            canvas.getContext('2d').drawImage(img, 0, 0, w, h)
29            resolve(canvas.toDataURL('image/jpeg', quality))
30          }
31          img.src = e.target.result
32        }
33        reader.readAsDataURL(file)
34      })
35    }
36
37    // ??? Constants ?????????????????????????????????????????????????????????????
38
39    const BUSINESS_TYPES = [
40      { value: 'restaurant', label: 'Restaurant / Café', emoji: '??', sells: 'services' },
41      { value: 'clothing', label: 'Clothing Store', emoji: '?', sells: 'products' },
42      { value: 'salon', label: 'Salon / Spa', emoji: '?', sells: 'services' },
43      { value: 'gym', label: 'Gym / Fitness', emoji: '?', sells: 'services' },
44      { value: 'real-estate', label: 'Real Estate', emoji: '?', sells: 'services' },
45      { value: 'coaching', label: 'Coaching / Classes', emoji: '?', sells: 'services' },
46      { value: 'online-store', label: 'Online Store', emoji: '?', sells: 'products' },
47      { value: 'other', label: 'Other Business', emoji: '?', sells: 'both' },
48    ]
49
50    const STYLE_OPTIONS = [
51      { value: 'modern', label: 'Modern & Minimal', desc: 'Clean, spacious, professional',

```

```

    bg: '#f8fafc', text: '#1e293b' },
52   { value: 'bold',    label: 'Bold & Colorful',    desc: 'Vibrant, energetic, eye-catching',
    bg: '#fbbf24', text: '#1e293b' },
53   { value: 'elegant', label: 'Elegant & Premium', desc: 'Luxury feel, sophisticated',
    bg: '#1e293b', text: '#f8fafc' },
54   { value: 'simple',   label: 'Simple & Clean',     desc: 'Easy to navigate, friendly',
    bg: '#dbeafe', text: '#1e293b' },
55   { value: 'dark',    label: 'Dark Theme',        desc: 'Sleek, tech-forward, modern',
    bg: '#0f172a', text: '#e2e8f0' },
56 ]
57
58 const QUICK_COLORS =
  ['#6366f1', '#0ea5e9', '#10b981', '#f59e0b', '#ef4444', '#8b5cf6', '#ec4899', '#1e293b']
59
60 const TOTAL_STEPS = 8
61 // In production (Railway), frontend is served by the same FastAPI server,
62 // so relative URLs work. In local dev, VITE_API_URL points to localhost:8000.
63 const API = import.meta.env.VITE_API_URL || ''
64
65 // ??? Shared UI Atoms ?????????????????????????????????????????????????????????
66
67 function StepWrapper({ children, title, subtitle, onBack }) {
68   return (
69     <div className="ob-step">
70       <div className="ob-back">
71         <button className="ob-back" onClick={onBack} type="button">
72           ? Back
73         </button>
74       </div>
75       <div className="ob-step-head">
76         <h2 className="ob-title">{title}</h2>
77         <p className="ob-subtitle">{subtitle}</p>
78       </div>
79       {children}
80     </div>
81   )
82 }
83
84 function PrimaryBtn({ onClick, disabled, children }) {
85   return (
86     <button className="ob-primary-btn" onClick={onClick} disabled={disabled} type="button">
87       {children}
88     </button>
89   )
90 }
91
92 // ??? Step 1 · Business Name ?????????????????????????????????????????????????????
93
94 function Step1({ data, update, next, back }) {
95   const ok = data.businessName.trim().length > 0
96   return (
97     <StepWrapper
98       title="What's your business called? ?"
99       subtitle="Don't worry ? you can always change this later."

```

```
100     onBack={back}
101   >
102     <input
103       className="ob-input ob-input-xl"
104       type="text"
105       placeholder="e.g. Raj Bakery"
106       value={data.businessName}
107       onChange={e => update('businessName', e.target.value)}
108       onKeyDown={e => e.key === 'Enter' && ok && next()}
109       autoFocus
110     />
111     <PrimaryBtn onClick={next} disabled={!ok}>Continue ?</PrimaryBtn>
112   </StepWrapper>
113 )
114 }
115
116 // ??? Step 2 · Business Type + selling-type confirmation ?????????????????????
117
118 function Step2({ data, update, next, back, onTypeSelect }) {
119   // Sub-state A: no type chosen yet
120   if (!data.businessType) {
121     return (
122       <StepWrapper
123         title="What type of business is this?"
124         subtitle="Pick the one that fits best ? you can fine-tune after."
125         onBack={back}
126       >
127         <div className="ob-type-grid">
128           {BUSINESS_TYPES.map(bt => (
129             <button
130               key={bt.value}
131               className="ob-type-card"
132               onClick={() => onTypeSelect(bt.value, bt.sells)}
133               type="button"
134             >
135               <span className="ob-type-emoji">{bt.emoji}</span>
136               <span className="ob-type-label">{bt.label}</span>
137             </button>
138           ))}
139         </div>
140       </StepWrapper>
141     )
142   }
143
144   // Sub-state B: type chosen, awaiting selling-type confirmation
145   const sellingLabels = { products: 'Products', services: 'Services', both: 'Products &
Services' }
146   return (
147     <StepWrapper
148       title={`Looks like you sell ${sellingLabels[data.sellingType]} ?`}
149       subtitle={`Based on your business type, we auto-selected
150       "${sellingLabels[data.sellingType]}. Is this right?`}
151       onBack={() => { update('businessType', ''); update('sellingType', '') }}
152     >
```

```
152     <div className="ob-confirm-list">
153       <button className="ob-confirm-yes" onClick={() => next()} type="button">
154         ? Yes, that's right
155       </button>
156       {[
157         { v: 'products', icon: '?', label: 'I sell Products' },
158         { v: 'services', icon: '??', label: 'I sell Services' },
159         { v: 'both', icon: '?', label: 'I sell Both' },
160       ]}.filter(o => o.v !== data.sellingType).map(o => (
161         <button
162           key={o.v}
163           className="ob-confirm-alt"
164           onClick={() => { update('sellingType', o.v); next() }}
165           type="button"
166         >
167           {o.icon} {o.label}
168         </button>
169       )))
170     </div>
171   </StepWrapper>
172 )
173 }
174
175 // ??? Step 3 · Location ?????????????????????????????????????????????????????????
176
177 function Step3({ data, update, next, back }) {
178   const ok = data.location.trim().length > 0
179   return (
180     <StepWrapper
181       title="Where is your business located? ?"
182       subtitle="Just city and country is enough."
183       onBack={back}
184     >
185       <input
186         className="ob-input ob-input-xl"
187         type="text"
188         placeholder="e.g. Mumbai, India"
189         value={data.location}
190         onChange={e => update('location', e.target.value)}
191         onKeyDown={e => e.key === 'Enter' && ok && next()}
192         autoFocus
193       />
194       <PrimaryBtn onClick={next} disabled={!ok}>Continue ?</PrimaryBtn>
195     </StepWrapper>
196   )
197 }
198
199 // ??? Step 4 · Style ?????????????????????????????????????????????????????????
200
201 function Step4({ data, update, next, back }) {
202   return (
203     <StepWrapper
204       title="What style do you prefer? ?"
205       subtitle="Sets the look and feel of your entire website."
```

```
206     onBack={back}
207   >
208     <div className="ob-style-list">
209       {STYLE_OPTIONS.map(opt => (
210         <button
211           key={opt.value}
212           className={`ob-style-card${data.style === opt.value ? ' ob-style-selected' :
213 ''}}`}
214           onClick={() => { update('style', opt.value); setTimeout(next, 180) }}
215           type="button"
216         >
217           <div className="ob-style-swatch" style={{ background: opt.bg }}>
218             <span style={{ color: opt.text, fontWeight: 800, fontSize: 12 }}>Aa</span>
219           </div>
220           <div className="ob-style-text">
221             <span className="ob-style-name">{opt.label}</span>
222             <span className="ob-style-desc">{opt.desc}</span>
223           </div>
224         </button>
225       )]}
226     </div>
227   </StepWrapper>
228 )
229 }
230 // ??? Step 5 · Brand Color ?????????????????????????????????????????????????????????
231
232 function Step5({ data, update, next, back }) {
233   return (
234     <StepWrapper
235       title="Choose your main brand color ??"
236       subtitle="Used for buttons, highlights, and accents."
237       onBack={back}
238     >
239       <div className="ob-color-wrap">
240
241         {!data.aiColor && (
242           <div className="ob-color-picker-row">
243             <input
244               type="color"
245               className="ob-color-input"
246               value={data.brandColor}
247               onChange={e => update('brandColor', e.target.value)}
248             />
249             <span className="ob-color-hex">{data.brandColor.toUpperCase()}</span>
250           </div>
251         )}
252
253         <div className="ob-quick-colors">
254           {QUICK_COLORS.map(c => (
255             <button
256               key={c}
257               className={`ob-dot${!data.aiColor && data.brandColor === c ? ' ob-dot-active' :
258 ''}}`}
```

```

258         style={{ background: c }}
259         onClick={() => { update('brandColor', c); update('aiColor', false) }}
260         type="button"
261     />
262     )}}
263 </div>
264
265     <button
266       className={`ob-ai-color-btn${data.aiColor ? ' ob-ai-active' : ''}`}
267       onClick={() => update('aiColor', !data.aiColor)}
268       type="button"
269     >
270       ? {data.aiColor ? 'AI will pick the perfect color' : 'Let AI decide for me'}
271     </button>
272 </div>
273
274     <PrimaryBtn onClick={next}>Continue ?</PrimaryBtn>
275 </StepWrapper>
276 )
277 }
278
279 // ??? Step 6 · Products / Services ?????????????????????????????????????????????
280
281 // Business-type-specific placeholder hints shown inside the input fields
282 const BUSINESS_HINTS = {
283   restaurant: {
284     name: 'e.g. Dal Makhni',
285     price: 'e.g. ?240',
286     description: 'e.g. Slow-cooked black lentils in rich buttery tomato gravy',
287     category: 'e.g. Main Course / Starters',
288     subcategory: 'e.g. North Indian / South Indian',
289   },
290   clothing: {
291     name: 'e.g. Floral Kurti',
292     price: 'e.g. ?1,299',
293     description: 'e.g. Lightweight cotton kurti with floral embroidery',
294     category: 'e.g. Kurtis / Lehengas',
295     subcategory: 'e.g. Casual Wear / Party Wear',
296   },
297   salon: {
298     name: 'e.g. Hair Cut & Styling',
299     price: 'e.g. ?500 onwards',
300     description: 'e.g. Professional cut and blow-dry for all hair types',
301     category: 'e.g. Hair Services / Skin Care',
302     subcategory: 'e.g. Women\'s Cut / Men\'s Cut',
303   },
304   gym: {
305     name: 'e.g. Personal Training',
306     price: 'e.g. ?2,000/month',
307     description: 'e.g. One-on-one sessions with a certified fitness coach',
308     category: 'e.g. Fitness / Yoga',
309     subcategory: 'e.g. Weight Loss / Muscle Gain',
310   },
311   'real-estate': {

```

```
312     name:      'e.g. 2BHK Apartment',
313     price:      'e.g. ₹45 Lakh',
314     description: 'e.g. Spacious 2BHK with modern amenities in prime location',
315     category:    'e.g. Residential / Commercial',
316     subcategory: 'e.g. Apartment / Villa',
317   },
318   coaching: {
319     name:      'e.g. Math Coaching',
320     price:      'e.g. ₹1,500/month',
321     description: 'e.g. Comprehensive coaching for board exams with weekly tests',
322     category:    'e.g. Academics / Competitive Exams',
323     subcategory: 'e.g. Class 10 / Class 12',
324   },
325   'online-store': {
326     name:      'e.g. Wireless Earbuds',
327     price:      'e.g. ₹1,999',
328     description: 'e.g. True wireless earbuds with 24-hour battery life',
329     category:    'e.g. Electronics / Accessories',
330     subcategory: 'e.g. Audio / Wearables',
331   },
332   other: {
333     name:      'e.g. Consultation',
334     price:      'e.g. ₹500',
335     description: 'e.g. Brief description of what this service includes',
336     category:    'e.g. Services / Packages',
337     subcategory: 'e.g. Basic / Premium',
338   },
339 }
340
341 function Step6({ data, update, next, back }) {
342   const isProduct = data.sellingType === 'products'
343   const label      = isProduct ? 'Product' : 'Service'
344   const hints      = BUSINESS_HINTS[data.businessType] || BUSINESS_HINTS.other
345
346   const lastCardRef = useRef(null)
347   const prevLengthRef = useRef(data.items.length)
348
349   const addItem = () =>
350     update('items', [...data.items, { name: '', price: '', description: '', image: null,
351     imagePreview: null, category: '', subcategory: '' }])
352
353   const updateItem = (idx, field, val) =>
354     update('items', prevItems => prevItems.map((it, i) => i === idx ? { ...it, [field]: val }
355     : it))
356
357   const removeItem = idx =>
358     update('items', data.items.filter((_, i) => i !== idx))
359
360   // Auto-add first row on mount
361   useEffect(() => {
362     if (data.items.length === 0) {
363       update('items', [{ name: '', price: '', description: '', image: null, imagePreview:
364       null, category: '', subcategory: '' }])
365     }
366   })
367 }
```

```
363     }, []) // eslint-disable-line react-hooks/exhaustive-deps
364
365     // Auto-scroll + focus when a new item is added
366     useEffect(() => {
367       if (data.items.length > prevLengthRef.current) {
368         lastCardRef.current?.scrollIntoView({ behavior: 'smooth', block: 'nearest' })
369         lastCardRef.current?.querySelector('input')?.focus()
370       }
371       prevLengthRef.current = data.items.length
372     }, [data.items.length])
373
374     const canNext = data.items.length > 0 && data.items.every(i => i.name.trim())
375
376     return (
377       <StepWrapper
378         title={`Tell us about your ${label.toLowerCase()}s`}
379         subtitle={isProduct
380           ? 'Add the products you want to showcase on your site.'
381           : 'What services do you offer? Add them below.'}
382         onBack={back}
383       >
384         <div className="ob-items-scroll">
385           {data.items.map((item, idx) => (
386             <div key={idx} className="ob-item-card" ref={idx === data.items.length - 1 ?
lastCardRef : null}>
387               <div className="ob-item-row-head">
388                 <span className="ob-item-badge">{label} {idx + 1}</span>
389                 {data.items.length > 1 && (
390                   <button className="ob-remove" onClick={() => removeItem(idx)}
type="button">?</button>
391                 )}
392               </div>
393               <input
394                 className="ob-input"
395                 placeholder={`{label} name * ? ${hints.name}`}
396                 value={item.name}
397                 onChange={e => updateItem(idx, 'name', e.target.value)}
398               />
399               <input
400                 className="ob-input"
401                 placeholder={`Price ? optional ${hints.price}`}
402                 value={item.price}
403                 onChange={e => updateItem(idx, 'price', e.target.value)}
404               />
405               <textarea
406                 className="ob-textarea"
407                 placeholder={`Short description ? optional ${hints.description}`}
408                 rows={2}
409                 value={item.description}
410                 onChange={e => updateItem(idx, 'description', e.target.value)}
411               />
412               <input
413                 className="ob-input"
414                 placeholder={`Category ? optional ${hints.category}`}
```

```
415         value={item.category}
416         onChange={e => updateItem(idx, 'category', e.target.value)}
417     />
418     <input
419         className="ob-input"
420         placeholder={`Subcategory ? optional ${hints.subcategory}`}
421         value={item.subcategory}
422         onChange={e => updateItem(idx, 'subcategory', e.target.value)}
423     />
424
425     {/* Per-item photo ? uploaded right next to the correct item */}
426     <div className="ob-item-img-row">
427         {item.imagePreview ? (
428             <div className="ob-item-thumb-wrap">
429                 <img className="ob-item-thumb" src={item.imagePreview} alt="preview" />
430                 <button
431                     className="ob-item-thumb-remove"
432                     type="button"
433                     onClick={() => { updateItem(idx, 'image', null); updateItem(idx,
'imageUrl', null); updateItem(idx, 'uploadedDataUrl', null) }}
434                 >?</button>
435                 <span className="ob-item-uploaded-badge">? Uploaded</span>
436             </div>
437         ) : (
438             <label className="ob-item-upload-btn">
439                 ? Add photo ? optional
440                 <input
441                     type="file"
442                     accept="image/*"
443                     style={{ display: 'none' }}
444                     onChange={async e => {
445                         const file = e.target.files[0]
446                         if (!file) return
447                         updateItem(idx, 'image', file)
448                         updateItem(idx, 'imagePreview', URL.createObjectURL(file))
449                         // Also convert to permanent base64 for backend/deployment
450                         try {
451                             const b64 = await toCompressedBase64(file)
452                             updateItem(idx, 'uploadedDataUrl', b64)
453                         } catch { /* non-fatal ? preview still works */ }
454                     }}
455                 />
456             </label>
457         )}
458     </div>
459 </div>
460 )}}
461 </div>
462
463 <button className="ob-add-btn" onClick={addItem} type="button">
464     + Add Another {label}
465 </button>
466
467 <PrimaryBtn onClick={next} disabled={!canNext}>Continue ?</PrimaryBtn>
```

```
468     </StepWrapper>
469   )
470 }
471
472 // ??? Step 7 · Photos ?????????????????????????????????????????????????????????????
473
474 function Step7({ data, update, next, back }) {
475
476   if (data.useOwnPhotos === null) {
477     return (
478       <StepWrapper
479         title="Do you have business photos to upload? ?"
480         subtitle="Great photos make websites shine ? but we can also find them for you."
481         onBack={back}
482       >
483         <div className="ob-choice-list">
484           <button
485             className="ob-choice-card"
486             onClick={() => update('useOwnPhotos', true)}
487             type="button"
488           >
489             <span className="ob-choice-icon">?</span>
490             <div className="ob-choice-text">
491               <span className="ob-choice-title">Yes, I'll upload my own photos</span>
492               <span className="ob-choice-sub">Logo, storefront, team</span>
493             </div>
494           </button>
495           <button
496             className="ob-choice-card"
497             onClick={() => { update('useOwnPhotos', false); next() }}
498             type="button"
499           >
500             <span className="ob-choice-icon">?</span>
501             <div className="ob-choice-text">
502               <span className="ob-choice-title">Use AI-generated + stock images</span>
503               <span className="ob-choice-sub">We'll find the best visuals
504                 automatically</span>
505             </div>
506           </button>
507         </div>
508       </StepWrapper>
509     )
510   }
511
512   const UPLOAD_FIELDS = [
513     { key: 'logo', label: 'Logo', hint: 'PNG or SVG preferred', multiple:
514       false },
515     { key: 'storefront', label: 'Store / Office Photo', hint: 'Your physical space',
516       multiple: false },
517     { key: 'team', label: 'Team Photo', hint: 'Optional',
518       multiple: false },
519   ]
520
521   return (
```

```
518     <StepWrapper
519       title="Upload your photos ?"
520       subtitle="All uploads are optional ? skip any you don't have."
521       onBack={() => update('useOwnPhotos', null)}
522     >
523     <div className="ob-upload-list">
524       {UPLOAD_FIELDS.map(({ key, label, hint, multiple }) => {
525         const hasFile = data.uploadedPhotos[key] &&
526           (Array.isArray(data.uploadedPhotos[key])
527             ? data.uploadedPhotos[key].length > 0
528             : true)
529         return (
530           <div key={key} className="ob-upload-row">
531             <div className="ob-upload-info">
532               <span className="ob-upload-label">{label}</span>
533               <span className="ob-upload-hint">{hint}</span>
534             </div>
535             <label className={`ob-upload-btn${hasFile ? ' ob-uploaded' : ''}`>
536               {hasFile ? '? Uploaded' : 'Choose file'}
537               <input
538                 type="file"
539                 accept="image/*"
540                 multiple={multiple}
541                 style={{ display: 'none' }}
542                 onChange={e => {
543                   const val = multiple ? Array.from(e.target.files) : e.target.files[0]
544                   update('uploadedPhotos', { ...data.uploadedPhotos, [key]: val })
545                 }}
546               />
547             </label>
548           </div>
549         )
550       })}
551     </div>
552     <PrimaryBtn onClick={next}>Continue ?</PrimaryBtn>
553   </StepWrapper>
554 )
555 }
556
557 // ??? Step 8 · Contact Info ?????????????????????????????????????????????????????????
558
559 function Step8({ data, update, back, onSubmit, loading }) {
560   const canSubmit = data.phone.trim() && data.email.trim()
561
562   return (
563     <StepWrapper
564       title="Almost done! How can customers reach you? ?"
565       subtitle="Phone and email are required. Everything else is optional."
566       onBack={back}
567     >
568     <div className="ob-contact-grid">
569       {[
570         { key: 'phone',      label: 'Phone number *',      type: 'tel',      ph: '+91
98765 43210'
          },
```

```

571         { key: 'email',      label: 'Email address *',      type: 'email',  ph:
    'hello@yourbusiness.com'    },
572         { key: 'whatsapp',   label: 'WhatsApp',             type: 'tel',    ph: '+91
    98765 43210'                },
573         { key: 'instagram',  label: 'Instagram',            type: 'text',   ph:
    '@yourbusiness'             },
574         { key: 'facebook',   label: 'Facebook',             type: 'text',   ph:
    'facebook.com/yourbusiness' },
575     ].map(({ key, label, type, ph }) => (
576         <div key={key} className="ob-contact-field">
577             <label className="ob-field-label">
578                 {label.replace(' *', '')}
579                 {label.includes('*')}
580                 ? <span className="ob-required"> *</span>
581                 : <span className="ob-opt"> (optional)</span>
582             </label>
583             <input
584                 className="ob-input"
585                 type={type}
586                 placeholder={ph}
587                 value={data[key]}
588                 onChange={e => update(key, e.target.value)}
589             />
590         </div>
591     )))
592 </div>
593
594 <button
595     className={`ob-submit-btn${loading ? ' ob-submitting' : ''}`}
596     onClick={onSubmit}
597     disabled={!canSubmit || loading}
598     type="button"
599 >
600     {loading
601         ? <><span className="ob-spinner" /> Building your website?</>
602         : '? Build My Website'}
603 </button>
604 </StepWrapper>
605 )
606 }
607
608 // ??? Root Component ?????????????????????????????????????????????????????????
609
610 export default function Onboarding({ onComplete }) {
611     const [theme, toggleTheme] = useTheme()
612     const [step, setStep]      = useState(1)
613     const [loading, setLoading] = useState(false)
614     const [data, setData]      = useState({
615         businessName: '',
616         businessType: '',
617         sellingType:  '',
618         location:     '',
619         style:        '',
620         brandColor:   '#6366f1',

```

```
621     aiColor:      false,
622     items:        [],
623     useOwnPhotos: null,
624     uploadedPhotos: { logo: null, storefront: null, products: [], team: null },
625     phone:        '',
626     whatsapp:     '',
627     email:        '',
628     instagram:    '',
629     facebook:     '',
630   })
631
632   const update = (key, val) => setData(prev => ({ ...prev, [key]: typeof val === 'function' ?
val(prev[key]) : val })))
633   const next    = ()          => setStep(s => Math.min(s + 1, TOTAL_STEPS))
634   const back    = ()          => setStep(s => Math.max(s - 1, 1))
635
636   const onTypeSelect = (type, sells) => {
637     update('businessType', type)
638     update('sellingType', sells)
639   }
640
641   const handleSubmit = async () => {
642     setLoading(true)
643     try {
644       // Hero queries: striking full-width background ? show the business at its best
645       const heroSearchContext = {
646         clothing:    'indian ethnic fashion model traditional outfit editorial photoshoot',
647         restaurant:  'restaurant dining table food evening ambiance warm candlelight',
648         salon:       'luxury beauty salon elegant interior professional hair styling',
649         gym:         'modern gym fitness interior athlete workout equipment',
650         coaching:    'bright classroom students learning education professional',
651         'real-estate': 'luxury house exterior architecture modern property driveway',
652         'online-store': 'fashion clothing products lifestyle stylish flat lay editorial',
653         other:       'modern professional business office interior clean',
654       }
655       // About queries: warm, human, behind-the-scenes ? team or owner feel
656       const aboutSearchContext = {
657         clothing:    'fashion boutique owner woman ethnic clothing stylish smiling warm',
658         restaurant:  'chef kitchen cooking team culinary professional smiling',
659         salon:       'hairstylist salon professional woman smiling beauty treatment',
660         gym:         'fitness trainer coach gym motivation smiling team',
661         coaching:    'teacher mentor student whiteboard classroom smiling',
662         'real-estate': 'real estate agent handshake keys house property professional
smiling',
663         'online-store': 'entrepreneur woman business fashion packaging professional smiling',
664         other:       'professional business team office smiling warm friendly',
665       }
666       // Gallery queries: 4 varied visuals per business type for the gallery section
667       const gallerySearchContext = {
668         restaurant:  ['indian food dish plated presentation', 'restaurant interior ambiance
dining', 'dessert sweet food presentation', 'restaurant kitchen cooking chef'],
669         clothing:    ['indian ethnic saree woman fashion', 'kurti ethnic clothing woman',
'lehenga traditional indian dress', 'fashion clothing boutique display'],
670         salon:       ['hair styling salon professional', 'facial skin care beauty spa',
```

```

        'nail art manicure beauty', 'makeup beauty professional woman'],
671         gym:          ['gym workout weights exercise', 'cardio running fitness treadmill',
        'yoga stretch fitness', 'muscle training gym athlete'],
672         coaching:     ['students studying classroom education', 'online learning laptop
        course', 'whiteboard teaching professional', 'graduation certificate achievement'],
673         'real-estate': ['luxury living room interior modern', 'bedroom interior elegant
        design', 'modern kitchen interior house', 'house exterior garden pool'],
674         'online-store':['clothing fashion product display', 'online shopping packaging
        delivery', 'fashion accessories stylish', 'ecommerce product flat lay'],
675         other:        ['business office interior professional', 'team meeting professional',
        'customer service professional', 'product service quality'],
676     }
677     const heroQuery  = heroSearchContext[data.businessType] || 'modern professional
        business interior'
678     const aboutQuery = aboutSearchContext[data.businessType] || 'professional business team
        smiling'
679     const galleryQueries = gallerySearchContext[data.businessType] ||
        gallerySearchContext.other
680     const [heroRes, aboutRes, itemsWithImages, galleryResults] = await Promise.all([
681         axios.get(`${API}/api/image-search`, { params: { query: heroQuery, orientation:
        'landscape' } }).catch(() => ({ data: { url: null } })),
682         axios.get(`${API}/api/image-search`, { params: { query: aboutQuery } }).catch(() =>
        ({ data: { url: null } })),
683         Promise.all(
684             data.items.map(async item => {
685                 // User uploaded a photo ? use their base64 data URL directly (works in deployed
        site too)
686                 if (item.uploadedDataUrl) return { ...item, fetchedImage: item.uploadedDataUrl }
687                 if (item.imagePreview && !item.uploadedDataUrl) return item // base64 still
        converting, preview only
688                 try {
689                     // Add business-type context so Pexels returns the served product, not the raw
        ingredient
690                     const itemContextSuffix = {
691                         restaurant: 'plated dish',
692                         clothing:    'fashion clothing wear outfit',
693                         salon:       'beauty salon spa professional',
694                         gym:         'fitness workout exercise',
695                         coaching:    'education learning class study',
696                         'real-estate': 'property interior home',
697                         'online-store': '', // rely on item name + category; generic suffix caused
        wrong images
698                         other:      '',
699                     }
700                     // Per-service overrides for ambiguous salon terms that return wrong images
701                     const salonQueryOverrides = {
702                         'waxing':      'body waxing beauty salon treatment woman',
703                         'wax':         'body waxing beauty salon treatment woman',
704                         'manicure':    'nail manicure beauty salon nails',
705                         'pedicure':    'nail pedicure foot spa treatment',
706                         'facial':      'facial skin care beauty spa treatment',
707                         'face treatment': 'facial skin care beauty spa treatment',
708                         'bleaching':   'hair bleach salon professional treatment',
709                         'hair color':  'hair coloring salon professional',

```

```
710         'hair colour':    'hair coloring salon professional',
711         'threading':      'eyebrow threading beauty salon',
712     }
713     // Clothing overrides: correct spelling + visual context for Indian traditional
wear
714     const clothingQueryOverrides = {
715         // Indian Ethnic - Women
716         'lehnga':          'lehenga indian bridal traditional dress woman',
717         'lehenga':          'lehenga indian bridal traditional dress woman',
718         'bridal lehnga':    'bridal lehenga indian wedding dress woman',
719         'kurti':            'women kurti indian ethnic kurta fashion model',
720         'kurtis':           'women kurti indian ethnic kurta fashion model',
721         'saree':            'saree indian traditional sari woman elegant',
722         'sari':             'saree indian traditional sari woman elegant',
723         'salwar':           'salwar kameez indian ethnic suit woman',
724         'salwar kameez':    'salwar kameez indian ethnic suit woman',
725         'churidar':         'churidar indian ethnic wear woman fashion',
726         'dupatta':          'dupatta indian ethnic scarf woman fashion',
727         'anarkali':         'anarkali suit indian ethnic dress woman',
728         'sharara':          'sharara indian ethnic wear woman fashion',
729         'palazzo suit':     'palazzo suit indian ethnic fashion woman',
730         'indo western':     'indo western fusion outfit woman fashion',
731         'indo-western':     'indo western fusion outfit woman fashion',
732         // Indian Ethnic - Men
733         'sherwani':         'sherwani men indian ethnic wedding fashion',
734         'kurta':            'kurta men indian ethnic wear fashion',
735         'nehru jacket':     'nehru jacket men indian ethnic fashion',
736         // Western - Women
737         'skirts':           'women skirt fashion model stylish outfit',
738         'skirt':            'women skirt fashion model stylish outfit',
739         'ruffle skirt':     'women ruffle skirt fashion model stylish',
740         'mini skirt':       'women mini skirt fashion model stylish',
741         'maxi skirt':       'women maxi skirt fashion model stylish',
742         'tops':             'women top blouse fashion model stylish outfit',
743         'top':              'women top blouse fashion model stylish outfit',
744         'peplum':           'women peplum top fashion model stylish',
745         'crop top':         'women crop top fashion model stylish',
746         'dresses':          'women dress fashion model stylish elegant',
747         'dress':            'women dress fashion model stylish elegant',
748         'maxi dress':       'women maxi dress fashion model elegant',
749         'mini dress':       'women mini dress fashion model stylish',
750         'jeans':            'women jeans denim fashion model stylish',
751         'denim':            'women denim jeans fashion model stylish',
752         'palazzo':          'women palazzo pants wide leg fashion model',
753         'trousers':         'women trousers formal pants fashion model',
754         'shorts':           'women shorts casual fashion model stylish',
755         'co-ord':           'women co-ord matching set fashion model stylish',
756         'coord':            'women co-ord matching set fashion model stylish',
757         'co ord':           'women co-ord matching set fashion model stylish',
758         'jacket':           'women jacket fashion model stylish outerwear',
759         'jackets':          'women jacket fashion model stylish outerwear',
760         'blazer':           'women blazer fashion model stylish formal',
761         'sweater':          'women sweater knitwear fashion model cozy',
762         'sweaters':         'women sweater knitwear fashion model cozy',
```

```

763         'hoodie':          'women hoodie casual fashion model stylish',
764         'activewear':      'women activewear sportswear gym fashion model',
765         'swimwear':        'women swimwear beach fashion model stylish',
766         // Western - Men
767         'shirts':          'men shirt fashion model formal casual',
768         'shirt':           'men shirt fashion model formal casual',
769         't-shirt':         'men t-shirt casual fashion model streetwear',
770         't-shirts':        'men t-shirt casual fashion model streetwear',
771         'men jeans':       'men jeans denim fashion model stylish',
772         'formal wear':     'men formal suit fashion model professional',
773         // Accessories
774         'handbags':        'women handbag purse fashion stylish leather',
775         'handbag':         'women handbag purse fashion stylish leather',
776         'bags':            'women bag purse fashion stylish collection',
777         'bag':             'women bag purse fashion stylish',
778         'clutch':          'women clutch bag evening fashion stylish',
779         'necklace':        'women necklace jewelry fashion gold silver',
780         'earrings':        'women earrings jewelry fashion gold silver',
781         'bangles':         'bangles jewelry indian fashion gold',
782         'jewelry':         'women jewelry fashion gold silver elegant',
783         'watches':         'watch wristwatch fashion luxury stylish',
784         'watch':           'watch wristwatch fashion luxury stylish',
785         'sunglasses':      'sunglasses eyewear fashion model trendy',
786         // Footwear
787         'heels':           'women heels shoes fashion model elegant',
788         'sandals':         'women sandals shoes fashion summer stylish',
789         'boots':           'women boots shoes fashion model stylish',
790         'sneakers':        'sneakers shoes fashion model casual sporty',
791         'footwear':        'women footwear shoes fashion collection stylish',
792         'shoes':           'women shoes fashion model stylish collection',
793     }
794     // Gym overrides: specific fitness class/activity visuals
795     const gymQueryOverrides = {
796         'zumba':            'zumba dance fitness class women energetic colourful',
797         'yoga':             'yoga class studio women meditation calm',
798         'boxing':           'boxing training gym punching bag gloves athlete',
799         'kickboxing':       'kickboxing martial arts training gym athlete',
800         'crossfit':         'crossfit gym workout athlete functional fitness',
801         'hiit':             'hiit high intensity interval training workout gym',
802         'pilates':          'pilates class studio reformer woman fitness',
803         'cardio':           'cardio running treadmill gym fitness athlete',
804         'swimming':         'swimming pool lane athlete training water',
805         'cycling':          'indoor cycling spin class gym fitness',
806         'spin':             'spin class indoor cycling bike gym',
807         'aerobics':         'aerobics fitness class women gym energetic',
808         'strength':         'strength training weight lifting gym dumbbell',
809         'weight training':  'weight training gym dumbbell barbell athlete',
810         'personal training': 'personal trainer client gym coaching fitness',
811         'martial arts':     'martial arts karate training dojo athlete',
812         'dance':            'dance fitness class studio women energetic',
813         'functional':       'functional fitness training gym athlete',
814         'muscle':           'muscle building gym bodybuilder dumbbell training',
815     }
816     // Coaching overrides: subject/exam specific study visuals

```

```
817         const coachingQueryOverrides = {
818             'neet': 'medical entrance exam preparation students studying books',
819             'jee': 'engineering entrance exam preparation students studying',
820             'iit': 'iit jee engineering exam preparation students notes',
821             'upsc': 'upsc civil services exam preparation student books',
822             'ias': 'upsc ias exam preparation notes student studying',
823             'cat': 'mba cat exam preparation students studying notes',
824             'gmat': 'gmat mba exam preparation student studying notes',
825             'ielts': 'ielts english language test preparation student',
826             'toefl': 'toefl english language test preparation student',
827             'spoken english': 'english speaking class conversation students confident',
828             'english speaking': 'english speaking class conversation students confident',
829             'english': 'english language learning class students whiteboard',
830             'mathematics': 'mathematics tutor student whiteboard equations classroom',
831             'maths': 'mathematics tutor student whiteboard equations classroom',
832             'physics': 'physics experiment lab students science classroom',
833             'chemistry': 'chemistry lab experiment students science',
834             'biology': 'biology students microscope lab science classroom',
835             'science': 'science students lab experiment classroom learning',
836             'computer': 'computer programming coding class students laptop',
837             'coding': 'coding programming class students laptop technology',
838             'python': 'python programming coding class students laptop',
839             'accounts': 'accounting finance class students professional study',
840             'commerce': 'commerce students classroom education books notes',
841             'drawing': 'art drawing class students creative sketch',
842             'music': 'music class guitar piano students learning',
843             'dance': 'dance class studio students learning performance',
844             'abacus': 'abacus mental math kids classroom learning',
845         }
846         // Restaurant overrides: match specific Indian/popular dishes to vivid food
photography
847         const restaurantQueryOverrides = {
848             'biryani': 'biryani rice indian food plated restaurant',
849             'chicken biryani': 'chicken biryani rice indian food plated',
850             'mutton biryani': 'mutton biryani rice indian food plated',
851             'dal makhani': 'dal makhani indian food lentil curry plated',
852             'butter chicken': 'butter chicken curry indian food plated restaurant',
853             'paneer': 'paneer indian food curry plated restaurant',
854             'naan': 'naan bread indian food restaurant baked',
855             'roti': 'roti chapati indian bread food restaurant',
856             'paratha': 'paratha indian flatbread food plated restaurant',
857             'dosa': 'dosa south indian food crepe plated restaurant',
858             'idli': 'idli sambar south indian food plated breakfast',
859             'thali': 'indian thali platter assorted food restaurant',
860             'pizza': 'pizza italian food plated restaurant cheesy',
861             'pasta': 'pasta italian food plated restaurant creamy',
862             'burger': 'burger sandwich food plated restaurant bun',
863             'sandwich': 'sandwich food plated restaurant fresh',
864             'momos': 'momos dumplings food steamed plated street food',
865             'sushi': 'sushi japanese food plated restaurant fresh',
866             'steak': 'steak grilled meat food plated restaurant',
867             'soup': 'soup bowl food plated restaurant warm',
868             'salad': 'salad fresh bowl food plated restaurant healthy',
869             'dessert': 'dessert sweet food plated restaurant elegant',
```

```

870         'cake':          'cake slice dessert food plated bakery',
871         'ice cream':      'ice cream scoop dessert food sweet colorful',
872         'coffee':        'coffee cup latte art cafe professional',
873         'smoothie':       'smoothie fruit drink colorful glass fresh',
874         'juice':          'fresh juice glass fruit colorful healthy',
875         'rolls':          'rolls wrap food plated street food fresh',
876         'kebab':          'kebab grilled meat skewer food plated restaurant',
877         'tandoori':       'tandoori grilled chicken food plated restaurant',
878         'curry':          'curry indian food bowl plated restaurant spicy',
879         'fried rice':     'fried rice chinese food plated restaurant wok',
880         'noodles':        'noodles chinese food plated restaurant wok',
881         'chowmein':       'chowmein chinese noodles food plated restaurant',
882         'manchurian':     'manchurian chinese food plated restaurant sauce',
883         'pav bhaji':      'pav bhaji indian street food plated restaurant',
884         'chole':          'chole bhature indian food plated restaurant',
885         'samosa':         'samosa indian snack food fried crispy plated',
886         'gulab jamun':    'gulab jamun indian sweet dessert plated restaurant',
887         'rasgulla':       'rasgulla indian sweet dessert plated restaurant',
888         'halwa':          'halwa indian sweet dessert plated restaurant',
889         'lassi':          'lassi indian yogurt drink glass restaurant',
890         'chai':           'masala chai tea cup indian drink restaurant',
891         'tea':            'tea cup hot drink saucer professional',
892     }
893     // Real-estate overrides: map property types to vivid interior/exterior Pexels
queries
894     const realEstateQueryOverrides = {
895         'studio apartment': 'studio apartment interior modern minimal cozy',
896         'studio':          'studio apartment interior modern compact',
897         '1bhk':            '1bhk apartment interior living room modern',
898         '1 bhk':           '1bhk apartment interior living room modern',
899         '1bhk apartment':  '1bhk apartment interior living room modern',
900         '2bhk':            '2bhk apartment interior spacious living room',
901         '2 bhk':           '2bhk apartment interior spacious living room',
902         '2bhk apartment':  '2bhk apartment interior spacious living room',
903         '3bhk':            '3bhk apartment interior luxury living room',
904         '3 bhk':           '3bhk apartment interior luxury living room',
905         '3bhk apartment':  '3bhk apartment interior luxury living room',
906         '4bhk':            '4bhk apartment interior premium spacious',
907         '4 bhk':           '4bhk apartment interior premium spacious',
908         'villa':           'luxury villa exterior pool garden modern',
909         'bungalow':        'bungalow house exterior modern architecture',
910         'penthouse':       'penthouse apartment interior luxury city view',
911         'duplex':          'duplex house interior elegant modern staircase',
912         'rowhouse':        'row house exterior modern residential',
913         'row house':       'row house exterior modern residential',
914         'plot':            'residential plot land green neighbourhood',
915         'commercial':      'commercial office space interior modern professional',
916         'shop':            'retail shop commercial space interior modern',
917         'office space':    'modern office interior professional workspace',
918         'warehouse':       'industrial warehouse interior storage large',
919     }
920     const nameLower = item.name.trim().toLowerCase()
921     // Strip marketing prefixes (Premium, Special, etc.) so Pexels finds the actual
food/item

```

```

922                                     const      cleanName      =
item.name.trim().replace(/^(premium|special|classic|deluxe|signature|house|royal|chef'?s?|fre
sh|homemade)\s+/i, '')
923          // Find best override: check if item name CONTAINS any key (longest key first
for specificity)
924          const findOverride = (overrides, name) => {
925              const key = Object.keys(overrides).sort((a, b) => b.length - a.length).find(k
=> name.includes(k))
926              return key ? overrides[key] : null
927          }
928          const salonOverride      = data.businessType === 'salon'
              ? findOverride(salonQueryOverrides, nameLower)      : null
929          const clothingOverride    = (data.businessType === 'clothing' ||
data.businessType === 'online-store') ? findOverride(clothingQueryOverrides, nameLower)    :
null
930          const gymOverride         = data.businessType === 'gym'
              ? findOverride(gymQueryOverrides, nameLower)         : null
931          const coachingOverride    = data.businessType === 'coaching'
              ? findOverride(coachingQueryOverrides, nameLower)    : null
932          const restaurantOverride = data.businessType === 'restaurant'
              ? findOverride(restaurantQueryOverrides, nameLower)  : null
933          const realEstateOverride = data.businessType === 'real-estate'
              ? findOverride(realEstateQueryOverrides, nameLower)  : null
934          const suffix = itemContextSuffix[data.businessType] || ''
935          const subcatCtx = item.subcategory && item.subcategory.trim()
936          const catCtx    = item.category && item.category.trim()
937          // Organisational/dietary labels that add noise to image search ? skip them
938          const NON_VISUAL = new Set(['veg', 'non-veg', 'vegetarian', 'non-vegetarian',
'jain',
939          'main course', 'starter', 'starters', 'side', 'sides', 'add-on',
940          'ready to move', 'under construction', 'resale', 'new launch', 'pre-launch',
941          'affordable', 'luxury', 'budget', 'premium', 'semi-furnished', 'furnished',
'unfurnished'])
942          const subcatVisual = subcatCtx && !NON_VISUAL.has(subcatCtx.toLowerCase())
943          const catVisual    = catCtx && !NON_VISUAL.has(catCtx.toLowerCase())
944          // Query strategy:
945          // 1. Specific override (salon/clothing/gym/coaching/restaurant/real-estate)
946          // 2. cleanName + visual subcategory + suffix (most context)
947          // 3. cleanName + suffix (when subcategory is non-visual or absent)
948          // 4. cleanName alone (no suffix defined for this business type)
949          const specificOverride = salonOverride || clothingOverride || gymOverride ||
coachingOverride || restaurantOverride || realEstateOverride
950          const q = specificOverride
951              ? specificOverride
952              : suffix
953              ? (subcatVisual
954                  ? `${cleanName} ${subcatCtx} ${suffix}`
955                  : catVisual
956                  ? `${cleanName} ${catCtx} ${suffix}`
957                  : `${cleanName} ${suffix}`)
958              : subcatVisual
959                  ? `${cleanName} ${subcatCtx}`
960                  : catVisual
961                  ? `${cleanName} ${catCtx}`

```

```
962             : cleanName
963             const imgRes = await axios.get(`${API}/api/image-search`, { params: { query: q
    } })
964             return { ...item, fetchedImage: imgRes.data.url || null }
965         } catch {
966             return item
967         }
968     })
969 },
970 // Fetch 4 dedicated gallery images in parallel
971 Promise.all(
972     galleryQueries.map(q =>
973         axios.get(`${API}/api/image-search`, { params: { query: q } }).catch(() => ({
data: { url: null } })))
974     )
975 ),
976 ])
977
978 // Convert uploaded storefront/logo/team photos to base64 (permanent, works in deployed
site)
979 const storefrontFile = data.uploadedPhotos?.storefront
980 const logoFile       = data.uploadedPhotos?.logo
981 const teamFile       = data.uploadedPhotos?.team
982
983 const [storefrontB64, logoB64] = await Promise.all([
984     storefrontFile ? toCompressedBase64(storefrontFile) : Promise.resolve(null),
985     logoFile       ? toCompressedBase64(logoFile, 400, 400, 0.9) : Promise.resolve(null),
986 ])
987
988 // About image: prefer user's storefront upload (base64), fall back to Pexels
989 const aboutImage = storefrontB64 || aboutRes.data.url || null
990
991 const enrichedData = { ...data, items: itemsWithImages, heroImage: heroRes.data.url ||
null, aboutImage, logoImage: logoB64 || null }
992
993 // Collect permanent image URLs for each service (base64 if user uploaded, else Pexels
URL)
994 const serviceImageUrls = itemsWithImages.map(it => it.fetchedImage || null)
995 // About image URL ? base64 if user uploaded storefront, else Pexels
996 const aboutImageUrl = storefrontB64 || aboutRes.data.url || null
997 // Gallery images (4 dedicated Pexels shots for the gallery section)
998 const galleryImageUrls = galleryResults.map(r => r.data?.url || null).filter(Boolean)
999
1000 const res = await axios.post(`${API}/api/react/build`, {
1001     session_id:      `session-${Date.now()}`,
1002     business_name:    data.businessName,
1003     business_type:    data.businessType,
1004     business_description: `${data.businessName} ? a ${data.businessType} business in
`${data.location}.`,
1005     services:         data.items.map(i => i.name).filter(Boolean),
1006     service_categories: data.items
1007         .filter(i => i.name.trim())
1008         .map(i => ({ name: i.name.trim(), category: i.category || '', subcategory:
i.subcategory || '', price: i.price || '' })),
```

```
1009         location:           data.location,
1010         style_vibe:          data.style || 'modern',
1011         features:             ['contact', 'gallery', data.sellingType === 'products' ?
'ordering' : 'booking'],
1012         deploy:               false,    // HTML deployed from frontend via
/api/react/deploy-html
1013         brand_color:          data.aiColor ? null : data.brandColor,
1014         use_own_photos:       data.useOwnPhotos,
1015         // Images: Pexels URLs or base64 data URLs (user uploads)
1016         hero_image:           heroRes.data.url || null,
1017         about_image:          aboutImageUrl,
1018         logo_image:           logoB64 || null,
1019         service_images:       serviceImageUrls,
1020         gallery_images:       galleryImageUrls,
1021         contact: {
1022             phone: data.phone, whatsapp: data.whatsapp,
1023             email: data.email, instagram: data.instagram, facebook: data.facebook,
1024         },
1025     })
1026     onComplete(res.data, enrichedData)
1027 } catch (err) {
1028     alert(err.response?.data?.detail || 'Something went wrong. Please try again.')
1029 } finally {
1030     setLoading(false)
1031 }
1032 }
1033
1034 const progress = ((step - 1) / (TOTAL_STEPS - 1)) * 100
1035 const props    = { data, update, next, back }
1036
1037 return (
1038     <div className="ob-root">
1039
1040         {/* ?? Header ?? */}
1041         <header className="ob-header">
1042             <div className="ob-brand">? Sitekraft</div>
1043             <div className="ob-header-right">
1044                 <span className="ob-step-counter">Step {step} of {TOTAL_STEPS}</span>
1045                 <button
1046                     className="ob-theme-btn"
1047                     onClick={toggleTheme}
1048                     title={theme === 'dark' ? 'Switch to light mode' : 'Switch to dark mode'}
1049                 >
1050                     {theme === 'dark' ? '??' : '?'}
1051                 </button>
1052             </div>
1053         </header>
1054
1055         {/* ?? Progress bar ?? */}
1056         <div className="ob-progress-track">
1057             <div className="ob-progress-fill" style={{ width: `${progress}%` }} />
1058         </div>
1059
1060         {/* ?? Step card ?? */}
```

```
1061     <main className="ob-main">
1062       <div className="ob-card" key={step}>
1063         {step === 1 && <Step2 {...props} back={undefined} onTypeSelect={onTypeSelect} />}
1064         {step === 2 && <Step1 {...props} />}
1065         {step === 3 && <Step3 {...props} />}
1066         {step === 4 && <Step4 {...props} />}
1067         {step === 5 && <Step5 {...props} />}
1068         {step === 6 && <Step6 {...props} />}
1069         {step === 7 && <Step7 {...props} />}
1070         {step === 8 && <Step8 {...props} onSubmit={handleSubmit} loading={loading} />}
1071       </div>
1072     </main>
1073
1074   </div>
1075 )
1076 }
1077
```

File 21: frontend/src/useTheme.js

Frontend - useTheme.js

```
1   import { useState, useEffect } from 'react'
2
3   export function useTheme() {
4     const [theme, setTheme] = useState(() => localStorage.getItem('sk-theme') || 'light')
5     useEffect(() => {
6       document.documentElement.setAttribute('data-theme', theme)
7       localStorage.setItem('sk-theme', theme)
8     }, [theme])
9     const toggle = () => setTheme(t => t === 'dark' ? 'light' : 'dark')
10    return [theme, toggle]
11  }
12
```

File 22: frontend/src/App.cssFrontend - App.css

```
1      /* ?? Result View ? Light + Moonlight dark ?????????????????????? */
2
3      .result-root {
4          min-height: 100vh;
5          display: flex;
6          flex-direction: column;
7          background: var(--bg-page);
8          font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Inter', Roboto, sans-serif;
9          color: var(--text-primary);
10         transition: background 0.3s ease;
11     }
12
13     .result-header {
14         display: flex;
15         justify-content: space-between;
16         align-items: center;
17         padding: 14px 28px;
18         background: var(--header-bg);
19         backdrop-filter: blur(12px);
20         -webkit-backdrop-filter: blur(12px);
21         border-bottom: 1px solid var(--border);
22         transition: background 0.3s ease, border-color 0.3s ease;
23     }
24     .result-brand {
25         font-size: 18px;
26         font-weight: 800;
27         background: linear-gradient(135deg, var(--accent), var(--accent-2));
28         -webkit-background-clip: text;
29         -webkit-text-fill-color: transparent;
30         background-clip: text;
31         letter-spacing: -0.5px;
32     }
33     .result-header-right {
34         display: flex;
35         align-items: center;
36         gap: 10px;
37     }
38     .result-theme-btn {
39         width: 36px;
40         height: 36px;
41         border-radius: 10px;
42         border: 1.5px solid var(--border);
43         background: var(--bg-subtle);
44         color: var(--text-secondary);
45         font-size: 16px;
46         cursor: pointer;
47         display: flex;
48         align-items: center;
49         justify-content: center;
50         transition: all 0.2s ease;
51     }
```

```
52 .result-theme-btn:hover {
53   border-color: var(--accent);
54   color: var(--accent);
55   background: var(--accent-light);
56   transform: rotate(15deg);
57 }
58 .result-new-btn {
59   padding: 8px 18px;
60   border-radius: 9px;
61   background: linear-gradient(135deg, var(--accent), var(--accent-2));
62   color: #fff;
63   font-size: 13px;
64   font-weight: 600;
65   border: none;
66   cursor: pointer;
67   transition: opacity 0.15s, box-shadow 0.15s;
68 }
69 .result-new-btn:hover {
70   opacity: 0.9;
71   box-shadow: var(--shadow-btn);
72 }
73
74 .result-body {
75   flex: 1;
76   display: grid;
77   grid-template-columns: 320px 1fr;
78   overflow: hidden;
79   height: calc(100vh - 53px);
80 }
81
82 /* Left panel */
83 .result-left {
84   display: flex;
85   flex-direction: column;
86   gap: 14px;
87   padding: 24px 20px;
88   background: var(--bg-card);
89   border-right: 1px solid var(--border);
90   overflow-y: auto;
91   transition: background 0.3s ease, border-color 0.3s ease;
92 }
93 .result-left::-webkit-scrollbar { width: 4px; }
94 .result-left::-webkit-scrollbar-track { background: transparent; }
95 .result-left::-webkit-scrollbar-thumb { background: var(--border-accent); border-radius: 4px; }
96
97 .result-success-card {
98   background: linear-gradient(135deg, var(--accent), var(--accent-2));
99   border-radius: 18px;
100  padding: 22px;
101  color: #fff;
102  text-align: center;
103  position: relative;
104  overflow: hidden;
```

```
105 }
106 .result-success-card::before {
107     content: '';
108     position: absolute;
109     top: -30px; right: -30px;
110     width: 100px; height: 100px;
111     border-radius: 50%;
112     background: rgba(255,255,255,0.08);
113 }
114 .result-tick {
115     width: 44px; height: 44px;
116     border-radius: 50%;
117     background: rgba(255,255,255,0.2);
118     display: flex; align-items: center; justify-content: center;
119     font-size: 20px; font-weight: 700;
120     margin: 0 auto 12px;
121 }
122 .result-success-card h2 { font-size: 18px; font-weight: 700; margin-bottom: 4px; }
123 .result-success-card p { font-size: 13px; opacity: .75; }
124
125 .result-live-link {
126     display: block; padding: 12px 18px; border-radius: 12px;
127     background: var(--success-bg); color: var(--success-color);
128     font-size: 14px; font-weight: 600;
129     text-decoration: none; text-align: center;
130     border: 1.5px solid var(--success-border);
131     transition: opacity 0.15s, transform 0.15s;
132 }
133 .result-live-link:hover { opacity: 0.85; transform: translateY(-1px); }
134
135 .result-actions { display: flex; flex-direction: column; gap: 10px; }
136 .result-btn {
137     width: 100%; padding: 12px; border-radius: 12px;
138     font-size: 14px; font-weight: 600; border: none;
139     cursor: pointer; font-family: inherit;
140     transition: all 0.18s;
141 }
142 .result-btn-primary {
143     background: linear-gradient(135deg, var(--accent), var(--accent-2));
144     color: #fff;
145 }
146 .result-btn-primary:hover { opacity: 0.9; box-shadow: var(--shadow-btn); transform: translateY(-1px); }
147 .result-btn-secondary {
148     background: var(--bg-input);
149     color: var(--text-primary);
150     border: 1.5px solid var(--border);
151 }
152 .result-btn-secondary:hover { border-color: var(--accent); color: var(--accent); background: var(--accent-light); }
153
154 /* Deploy section */
155 .result-deploy-box {
156     padding: 14px 16px;
```

```
157     background: var(--bg-input);
158     border: 1.5px solid var(--border);
159     border-radius: 13px;
160     display: flex;
161     flex-direction: column;
162     gap: 10px;
163     transition: background 0.3s ease, border-color 0.3s ease;
164 }
165 .result-deploy-label {
166     font-size: 12px;
167     font-weight: 700;
168     color: var(--text-muted);
169     text-transform: uppercase;
170     letter-spacing: 0.5px;
171 }
172
173 /* Right panel ? browser frame */
174 .result-right {
175     display: flex;
176     flex-direction: column;
177     background: var(--bg-page);
178     overflow: hidden;
179     transition: background 0.3s ease;
180 }
181 .result-preview-bar {
182     display: flex; align-items: center; gap: 6px;
183     padding: 10px 16px;
184     background: var(--bg-card);
185     border-bottom: 1px solid var(--border);
186     flex-shrink: 0;
187     transition: background 0.3s ease, border-color 0.3s ease;
188 }
189 .result-preview-dot { width: 12px; height: 12px; border-radius: 50%; }
190 .result-preview-dot.red { background: #ef4444; }
191 .result-preview-dot.yellow { background: #f59e0b; }
192 .result-preview-dot.green { background: #10b981; }
193 .result-preview-url {
194     flex: 1; margin-left: 10px;
195     background: var(--bg-input);
196     border: 1px solid var(--border);
197     border-radius: 6px; padding: 5px 12px;
198     font-size: 12px; color: var(--text-muted); font-family: monospace;
199     transition: background 0.3s ease, border-color 0.3s ease;
200 }
201 .result-iframe { flex: 1; width: 100%; border: none; background: #fff; }
202
203 @media (max-width: 768px) {
204     .result-body { grid-template-columns: 1fr; height: auto; }
205     .result-right { display: none; }
206 }
207
208 /* Fullscreen preview modal ? appears on mobile when Open Preview is tapped */
209 .result-preview-modal {
210     position: fixed;
```

```
211     inset: 0;
212     z-index: 1000;
213     display: flex;
214     flex-direction: column;
215     background: #fff;
216 }
217 .result-preview-modal-bar {
218     display: flex;
219     align-items: center;
220     padding: 10px 14px;
221     background: var(--bg-card);
222     border-bottom: 1px solid var(--border);
223     flex-shrink: 0;
224     gap: 8px;
225 }
226 .result-preview-modal-close {
227     flex-shrink: 0;
228     width: 34px;
229     height: 34px;
230     border-radius: 8px;
231     border: 1.5px solid var(--border);
232     background: var(--bg-subtle);
233     color: var(--text-secondary);
234     font-size: 15px;
235     font-weight: 700;
236     cursor: pointer;
237     display: flex;
238     align-items: center;
239     justify-content: center;
240     transition: all 0.15s;
241 }
242 .result-preview-modal-close:hover {
243     border-color: #ef4444;
244     color: #ef4444;
245     background: #fef2f2;
246 }
247
```

File 23: frontend/src/Onboarding.css

Frontend - Onboarding.css

```
1  /* ????????????????????????????????????????????????????????????? */
2      Onboarding ? Sitekraft AI wizard · Light + Moonlight dark
3      ????????????????????????????????????????????????????????????? */
4
5  /* ?? Root ??????????????????????????????????????????????????????? */
6  .ob-root {
7      min-height: 100vh;
8      background: var(--bg-page);
9      display: flex;
10     flex-direction: column;
11     font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Inter', Roboto, sans-serif;
12     color: var(--text-primary);
13     transition: background 0.3s ease, color 0.3s ease;
14 }
15
16 /* ?? Moonlight star background (dark only) ?????????????????????? */
17 [data-theme="dark"] .ob-root::before {
18     content: '';
19     position: fixed;
20     inset: 0;
21     pointer-events: none;
22     background:
23         radial-gradient(ellipse 80% 50% at 20% 10%, rgba(139,92,246,0.12) 0%, transparent 60%),
24         radial-gradient(ellipse 60% 40% at 80% 80%, rgba(99,102,241,0.08) 0%, transparent 60%);
25     z-index: 0;
26 }
27 [data-theme="dark"] .ob-header,
28 [data-theme="dark"] .ob-progress-track,
29 [data-theme="dark"] .ob-main {
30     position: relative;
31     z-index: 1;
32 }
33
34 /* ?? Header ?????????????????????????????????????????????????????? */
35 .ob-header {
36     display: flex;
37     justify-content: space-between;
38     align-items: center;
39     padding: 14px 28px;
40     background: var(--header-bg);
41     backdrop-filter: blur(12px);
42     -webkit-backdrop-filter: blur(12px);
43     border-bottom: 1px solid var(--border);
44     position: sticky;
45     top: 0;
46     z-index: 100;
47     transition: background 0.3s ease, border-color 0.3s ease;
48 }
49 .ob-brand {
50     font-size: 18px;
51     font-weight: 800;
```

```
52     background: linear-gradient(135deg, var(--accent), var(--accent-2));
53     -webkit-background-clip: text;
54     -webkit-text-fill-color: transparent;
55     background-clip: text;
56     letter-spacing: -0.5px;
57 }
58 .ob-header-right {
59     display: flex;
60     align-items: center;
61     gap: 14px;
62 }
63 .ob-step-counter {
64     font-size: 13px;
65     color: var(--text-muted);
66     font-weight: 500;
67 }
68
69 /* ?? Theme toggle button ?????????????????????????????????????? */
70 .ob-theme-btn {
71     width: 36px;
72     height: 36px;
73     border-radius: 10px;
74     border: 1.5px solid var(--border);
75     background: var(--bg-subtle);
76     color: var(--text-secondary);
77     font-size: 16px;
78     cursor: pointer;
79     display: flex;
80     align-items: center;
81     justify-content: center;
82     transition: all 0.2s ease;
83     flex-shrink: 0;
84 }
85 .ob-theme-btn:hover {
86     border-color: var(--accent);
87     color: var(--accent);
88     background: var(--accent-light);
89     transform: rotate(15deg);
90 }
91
92 /* ?? Progress bar ?????????????????????????????????????????????? */
93 .ob-progress-track {
94     height: 3px;
95     background: var(--progress-bg);
96     transition: background 0.3s ease;
97 }
98 .ob-progress-fill {
99     height: 100%;
100    background: linear-gradient(90deg, var(--accent), var(--accent-2));
101    transition: width 0.45s cubic-bezier(0.4, 0, 0.2, 1);
102    box-shadow: 0 0 8px var(--glow-strong);
103 }
104
105 /* ?? Main / Card ?????????????????????????????????????????????? */
```

```
106 .ob-main {
107     flex: 1;
108     display: flex;
109     justify-content: center;
110     align-items: flex-start;
111     padding: 48px 20px 80px;
112 }
113 .ob-card {
114     background: var(--bg-card);
115     border-radius: 22px;
116     padding: 40px 44px;
117     width: 100%;
118     max-width: 540px;
119     box-shadow: var(--shadow-card);
120     border: 1px solid var(--border);
121     animation: ob-fadein 0.28s ease;
122     transition: background 0.3s ease, border-color 0.3s ease, box-shadow 0.3s ease;
123 }
124 [data-theme="dark"] .ob-card {
125     box-shadow: var(--shadow-card), 0 0 0 1px rgba(139,92,246,0.08);
126 }
127 @keyframes ob-fadein {
128     from { opacity: 0; transform: translateY(12px); }
129     to   { opacity: 1; transform: translateY(0); }
130 }
131
132 /* ?? Step inner layout ?????????????????????????????????????????? */
133 .ob-step {
134     display: flex;
135     flex-direction: column;
136     gap: 18px;
137 }
138 .ob-step-head { margin-bottom: 2px; }
139 .ob-title {
140     font-size: 22px;
141     font-weight: 700;
142     color: var(--text-primary);
143     margin: 0 0 8px;
144     line-height: 1.3;
145 }
146 .ob-subtitle {
147     font-size: 14px;
148     color: var(--text-secondary);
149     margin: 0;
150     line-height: 1.55;
151 }
152 .ob-back {
153     display: inline-flex;
154     align-items: center;
155     gap: 4px;
156     background: none;
157     border: none;
158     color: var(--text-muted);
159     font-size: 13px;
```

```
160     font-weight: 500;
161     cursor: pointer;
162     padding: 0;
163     transition: color 0.15s;
164 }
165 .ob-back:hover { color: var(--accent); }
166
167 /* ?? Inputs ????????????????????????????????????????????????????????? */
168 .ob-input {
169     width: 100%;
170     padding: 12px 14px;
171     border: 1.5px solid var(--border);
172     border-radius: 11px;
173     font-size: 15px;
174     color: var(--text-primary);
175     background: var(--bg-input);
176     outline: none;
177     transition: border-color 0.18s, box-shadow 0.18s, background 0.18s;
178     box-sizing: border-box;
179     font-family: inherit;
180 }
181 .ob-input:focus {
182     border-color: var(--border-focus);
183     background: var(--bg-input-focus);
184     box-shadow: 0 0 0 3px var(--glow);
185 }
186 .ob-input::placeholder { color: var(--text-muted); }
187 .ob-input-xl {
188     font-size: 18px;
189     padding: 15px 18px;
190 }
191 .ob-textarea {
192     width: 100%;
193     padding: 11px 14px;
194     border: 1.5px solid var(--border);
195     border-radius: 11px;
196     font-size: 14px;
197     color: var(--text-primary);
198     background: var(--bg-input);
199     outline: none;
200     resize: vertical;
201     font-family: inherit;
202     transition: border-color 0.18s, box-shadow 0.18s, background 0.18s;
203     box-sizing: border-box;
204 }
205 .ob-textarea:focus {
206     border-color: var(--border-focus);
207     background: var(--bg-input-focus);
208     box-shadow: 0 0 0 3px var(--glow);
209 }
210
211 /* ?? Primary nav button ????????????????????????????????????????????? */
212 .ob-primary-btn {
213     width: 100%;
```

```
214     padding: 14px;
215     border-radius: 12px;
216     background: linear-gradient(135deg, var(--accent), var(--accent-2));
217     color: #fff;
218     font-size: 15px;
219     font-weight: 600;
220     border: none;
221     cursor: pointer;
222     transition: transform 0.18s, box-shadow 0.18s, opacity 0.18s;
223     font-family: inherit;
224     position: relative;
225     overflow: hidden;
226 }
227 .ob-primary-btn::after {
228     content: '';
229     position: absolute;
230     inset: 0;
231     background: linear-gradient(135deg, rgba(255,255,255,0.1), transparent);
232     opacity: 0;
233     transition: opacity 0.18s;
234 }
235 .ob-primary-btn:hover:not(:disabled)::after { opacity: 1; }
236 .ob-primary-btn:hover:not(:disabled) {
237     transform: translateY(-1px);
238     box-shadow: var(--shadow-btn);
239 }
240 .ob-primary-btn:disabled {
241     background: var(--border);
242     color: var(--text-muted);
243     cursor: not-allowed;
244     transform: none;
245     box-shadow: none;
246 }
247
248 /* ?? Step 2 · Business type grid ?????????????????????????????? */
249 .ob-type-grid {
250     display: grid;
251     grid-template-columns: 1fr 1fr;
252     gap: 10px;
253 }
254 .ob-type-card {
255     display: flex;
256     flex-direction: column;
257     align-items: center;
258     gap: 8px;
259     padding: 18px 10px;
260     border: 1.5px solid var(--border);
261     border-radius: 14px;
262     background: var(--bg-input);
263     cursor: pointer;
264     font-family: inherit;
265     transition: border-color 0.17s, background 0.17s, transform 0.17s, box-shadow 0.17s;
266 }
267 .ob-type-card:hover {
```

```
268     border-color: var(--accent);
269     background: var(--accent-light);
270     transform: translateY(-2px);
271     box-shadow: 0 4px 16px var(--glow);
272 }
273 .ob-type-emoji { font-size: 26px; }
274 .ob-type-label {
275     font-size: 13px;
276     font-weight: 600;
277     color: var(--text-secondary);
278     text-align: center;
279     line-height: 1.3;
280 }
281
282 /* Step 2 · selling-type confirmation */
283 .ob-confirm-list {
284     display: flex;
285     flex-direction: column;
286     gap: 9px;
287 }
288 .ob-confirm-yes {
289     padding: 14px 18px;
290     border-radius: 12px;
291     background: linear-gradient(135deg, var(--accent), var(--accent-2));
292     color: #fff;
293     font-size: 15px;
294     font-weight: 600;
295     border: none;
296     cursor: pointer;
297     font-family: inherit;
298     text-align: left;
299     transition: opacity 0.15s, box-shadow 0.15s;
300 }
301 .ob-confirm-yes:hover { opacity: 0.9; box-shadow: var(--shadow-btn); }
302 .ob-confirm-alt {
303     padding: 13px 18px;
304     border: 1.5px solid var(--border);
305     border-radius: 12px;
306     background: var(--bg-input);
307     font-size: 14px;
308     font-weight: 500;
309     color: var(--text-primary);
310     cursor: pointer;
311     text-align: left;
312     font-family: inherit;
313     transition: border-color 0.15s, background 0.15s;
314 }
315 .ob-confirm-alt:hover {
316     border-color: var(--accent);
317     background: var(--accent-light);
318 }
319
320 /* ?? Step 4 · Style selector ?????????????????????????????????? */
321 .ob-style-list {
```

```
322     display: flex;
323     flex-direction: column;
324     gap: 9px;
325 }
326 .ob-style-card {
327     display: flex;
328     align-items: center;
329     gap: 14px;
330     padding: 13px 14px;
331     border: 1.5px solid var(--border);
332     border-radius: 13px;
333     background: var(--bg-input);
334     cursor: pointer;
335     font-family: inherit;
336     text-align: left;
337     transition: border-color 0.17s, background 0.17s, box-shadow 0.17s;
338 }
339 .ob-style-card:hover, .ob-style-selected {
340     border-color: var(--accent);
341     background: var(--accent-light);
342 }
343 .ob-style-selected {
344     box-shadow: 0 0 0 2px var(--glow-strong);
345 }
346 .ob-style-swatch {
347     width: 50px;
348     height: 38px;
349     border-radius: 8px;
350     flex-shrink: 0;
351     display: flex;
352     align-items: center;
353     justify-content: center;
354     border: 1px solid rgba(0,0,0,0.08);
355 }
356 .ob-style-text { display: flex; flex-direction: column; gap: 2px; }
357 .ob-style-name { font-size: 14px; font-weight: 600; color: var(--text-primary); }
358 .ob-style-desc { font-size: 12px; color: var(--text-secondary); }
359
360 /* ?? Step 5 · Color picker ?????????????????????????????????????? */
361 .ob-color-wrap {
362     display: flex;
363     flex-direction: column;
364     gap: 14px;
365 }
366 .ob-color-picker-row {
367     display: flex;
368     align-items: center;
369     gap: 12px;
370 }
371 .ob-color-input {
372     width: 54px;
373     height: 54px;
374     border: none;
375     border-radius: 12px;
```

```
376     cursor: pointer;
377     padding: 3px;
378     background: none;
379 }
380 .ob-color-hex {
381     font-size: 16px;
382     font-weight: 600;
383     color: var(--text-primary);
384     font-family: 'SF Mono', 'Fira Code', monospace;
385 }
386 .ob-quick-colors {
387     display: flex;
388     gap: 9px;
389     flex-wrap: wrap;
390 }
391 .ob-dot {
392     width: 32px;
393     height: 32px;
394     border-radius: 50%;
395     border: 2.5px solid transparent;
396     cursor: pointer;
397     transition: transform 0.15s, border-color 0.15s;
398 }
399 .ob-dot:hover, .ob-dot-active {
400     border-color: var(--text-primary);
401     transform: scale(1.18);
402 }
403 .ob-ai-color-btn {
404     padding: 12px 18px;
405     border: 1.5px dashed var(--border-accent);
406     border-radius: 12px;
407     background: var(--accent-light);
408     color: var(--accent);
409     font-size: 14px;
410     font-weight: 600;
411     cursor: pointer;
412     font-family: inherit;
413     transition: all 0.18s;
414 }
415 .ob-ai-color-btn:hover, .ob-ai-active {
416     background: linear-gradient(135deg, var(--accent), var(--accent-2));
417     color: #fff;
418     border-color: transparent;
419     box-shadow: var(--shadow-btn);
420 }
421
422 /* ?? Step 6 · Products / Services ?????????????????????????????? */
423 .ob-items-scroll {
424     display: flex;
425     flex-direction: column;
426     gap: 12px;
427     max-height: 360px;
428     overflow-y: auto;
429     padding-right: 4px;
```

```
430 }
431 .ob-items-scroll::-webkit-scrollbar { width: 4px; }
432 .ob-items-scroll::-webkit-scrollbar-track { background: transparent; }
433 .ob-items-scroll::-webkit-scrollbar-thumb {
434     background: var(--border-accent);
435     border-radius: 4px;
436 }
437
438 .ob-item-card {
439     display: flex;
440     flex-direction: column;
441     gap: 9px;
442     padding: 14px;
443     border: 1.5px solid var(--border);
444     border-radius: 13px;
445     background: var(--bg-input);
446     transition: border-color 0.2s, box-shadow 0.2s;
447 }
448 .ob-item-card:focus-within {
449     border-color: var(--border-focus);
450     box-shadow: 0 0 0 3px var(--glow);
451 }
452 .ob-item-row-head {
453     display: flex;
454     justify-content: space-between;
455     align-items: center;
456 }
457 .ob-item-badge {
458     font-size: 11px;
459     font-weight: 700;
460     color: var(--accent);
461     text-transform: uppercase;
462     letter-spacing: 0.6px;
463 }
464 .ob-remove {
465     background: none;
466     border: none;
467     color: var(--text-muted);
468     cursor: pointer;
469     font-size: 14px;
470     padding: 2px 6px;
471     border-radius: 6px;
472     transition: background 0.15s, color 0.15s;
473 }
474 .ob-remove:hover { background: var(--danger-bg); color: var(--danger); }
475 .ob-add-btn {
476     width: 100%;
477     padding: 11px;
478     border: 1.5px dashed var(--border-accent);
479     border-radius: 12px;
480     background: var(--accent-light);
481     color: var(--accent);
482     font-size: 14px;
483     font-weight: 600;
```

```
484     cursor: pointer;
485     font-family: inherit;
486     transition: background 0.15s, border-color 0.15s;
487 }
488 .ob-add-btn:hover {
489     background: var(--bg-subtle);
490     border-color: var(--accent);
491 }
492
493 /* ?? Step 6 · Per-item image upload ?????????????????????????? */
494 .ob-item-img-row { margin-top: 2px; }
495 .ob-item-upload-btn {
496     display: inline-flex;
497     align-items: center;
498     gap: 6px;
499     padding: 8px 14px;
500     border: 1.5px dashed var(--border-accent);
501     border-radius: 9px;
502     background: var(--accent-light);
503     color: var(--accent);
504     font-size: 13px;
505     font-weight: 600;
506     cursor: pointer;
507     transition: background 0.15s, border-color 0.15s;
508 }
509 .ob-item-upload-btn:hover {
510     background: var(--bg-subtle);
511     border-color: var(--accent);
512 }
513 .ob-item-thumb-wrap {
514     position: relative;
515     display: inline-block;
516 }
517 .ob-item-thumb {
518     width: 80px;
519     height: 60px;
520     object-fit: cover;
521     border-radius: 8px;
522     border: 1.5px solid var(--border);
523     display: block;
524 }
525 .ob-item-uploaded-badge {
526     display: block;
527     font-size: 11px;
528     font-weight: 700;
529     color: var(--success-color);
530     text-align: center;
531     margin-top: 4px;
532     letter-spacing: 0.01em;
533 }
534 .ob-item-thumb-remove {
535     position: absolute;
536     top: -6px;
537     right: -6px;
```

```

538     width: 20px;
539     height: 20px;
540     border-radius: 50%;
541     background: var(--danger);
542     color: #fff;
543     border: none;
544     font-size: 10px;
545     font-weight: 700;
546     cursor: pointer;
547     display: flex;
548     align-items: center;
549     justify-content: center;
550     line-height: 1;
551 }
552
553 /* ?? Step 7 · Photos ?????????????????????????????????????????????????????????????? */
554 .ob-choice-list {
555     display: flex;
556     flex-direction: column;
557     gap: 10px;
558 }
559 .ob-choice-card {
560     display: flex;
561     align-items: center;
562     gap: 16px;
563     padding: 18px;
564     border: 1.5px solid var(--border);
565     border-radius: 14px;
566     background: var(--bg-input);
567     cursor: pointer;
568     font-family: inherit;
569     text-align: left;
570     transition: border-color 0.17s, background 0.17s, transform 0.17s, box-shadow 0.17s;
571 }
572 .ob-choice-card:hover {
573     border-color: var(--accent);
574     background: var(--accent-light);
575     transform: translateY(-1px);
576     box-shadow: 0 4px 16px var(--glow);
577 }
578 .ob-choice-icon { font-size: 26px; flex-shrink: 0; }
579 .ob-choice-text { display: flex; flex-direction: column; gap: 3px; }
580 .ob-choice-title { font-size: 15px; font-weight: 600; color: var(--text-primary); }
581 .ob-choice-sub { font-size: 13px; color: var(--text-secondary); }
582
583 .ob-upload-list { display: flex; flex-direction: column; gap: 10px; }
584 .ob-upload-row {
585     display: flex;
586     justify-content: space-between;
587     align-items: center;
588     padding: 13px 15px;
589     border: 1.5px solid var(--border);
590     border-radius: 12px;
591     background: var(--bg-input);

```

```
592     gap: 12px;
593   }
594   .ob-upload-info { display: flex; flex-direction: column; gap: 2px; }
595   .ob-upload-label { font-size: 14px; font-weight: 600; color: var(--text-primary); }
596   .ob-upload-hint { font-size: 12px; color: var(--text-muted); }
597   .ob-upload-btn {
598     padding: 8px 14px;
599     border-radius: 9px;
600     background: var(--accent-light);
601     color: var(--accent);
602     font-size: 13px;
603     font-weight: 600;
604     cursor: pointer;
605     border: 1.5px solid var(--border-accent);
606     white-space: nowrap;
607     transition: background 0.15s, color 0.15s, border-color 0.15s;
608     flex-shrink: 0;
609   }
610   .ob-upload-btn:hover { background: var(--accent); color: #fff; border-color: var(--accent); }
611   .ob-uploaded { background: var(--success-bg); color: var(--success-color); border-color:
var(--success-border); }
612   .ob-uploaded:hover { background: var(--success-color); color: #fff; border-color:
var(--success-color); }
613
614   /* ?? Step 8 · Contact ????????????????????????????????????????????????????????????? */
615   .ob-contact-grid { display: flex; flex-direction: column; gap: 12px; }
616   .ob-contact-field { display: flex; flex-direction: column; gap: 5px; }
617   .ob-field-label { font-size: 13px; font-weight: 600; color: var(--text-secondary); }
618   .ob-required { color: var(--danger); }
619   .ob-opt { font-weight: 400; color: var(--text-muted); }
620
621   /* ?? Submit button ????????????????????????????????????????????????????????????? */
622   .ob-submit-btn {
623     width: 100%;
624     padding: 15px;
625     border-radius: 13px;
626     background: linear-gradient(135deg, var(--accent), var(--accent-2));
627     color: #fff;
628     font-size: 16px;
629     font-weight: 700;
630     border: none;
631     cursor: pointer;
632     display: flex;
633     align-items: center;
634     justify-content: center;
635     gap: 10px;
636     font-family: inherit;
637     transition: transform 0.18s, box-shadow 0.18s, opacity 0.18s;
638     margin-top: 4px;
639     position: relative;
640     overflow: hidden;
641   }
642   .ob-submit-btn::after {
643     content: '';
```

```
644     position: absolute;
645     inset: 0;
646     background: linear-gradient(135deg, rgba(255,255,255,0.12), transparent);
647     opacity: 0;
648     transition: opacity 0.18s;
649 }
650 .ob-submit-btn:hover:not(:disabled)::after { opacity: 1; }
651 .ob-submit-btn:hover:not(:disabled) {
652     transform: translateY(-2px);
653     box-shadow: var(--shadow-btn);
654 }
655 .ob-submit-btn:disabled {
656     opacity: 0.5;
657     cursor: not-allowed;
658     transform: none;
659 }
660 .ob-submitting { opacity: 0.85; }
661
662 /* Spinner */
663 .ob-spinner {
664     width: 17px;
665     height: 17px;
666     border: 2.5px solid rgba(255,255,255,0.35);
667     border-top-color: #fff;
668     border-radius: 50%;
669     display: inline-block;
670     animation: ob-spin 0.65s linear infinite;
671 }
672 @keyframes ob-spin { to { transform: rotate(360deg); } }
673
674 /* ?? Mobile enhancements (?768px) ?????????????????????????? */
675 @media (max-width: 768px) {
676
677     /* Subtle top + bottom blobs ? lighter than desktop so they don't overpower content */
678     .ob-root {
679         background:
680             radial-gradient(ellipse 80% 35% at 50% 0%,    rgba(99,102,241,0.10) 0%, transparent
681 70%),
682             radial-gradient(ellipse 70% 30% at 50% 100%,   rgba(139,92,246,0.08) 0%, transparent
683 70%),
684             var(--bg-page);
685     }
686     [data-theme="dark"] .ob-root {
687         background:
688             radial-gradient(ellipse 80% 35% at 50% 0%,    rgba(139,92,246,0.22) 0%, transparent
689 70%),
690             radial-gradient(ellipse 70% 30% at 50% 100%,   rgba(99,102,241,0.14) 0%, transparent
691 70%),
692             var(--bg-page);
693     }
694
695     /* Card fills width nicely with a soft shadow */
696     .ob-card {
697         border-radius: 20px;
```

```
694     box-shadow:
695         0 2px 8px rgba(0,0,0,0.06),
696         0 8px 32px rgba(99,102,241,0.10);
697 }
698 [data-theme="dark"] .ob-card {
699     box-shadow:
700         0 2px 8px rgba(0,0,0,0.4),
701         0 8px 32px rgba(139,92,246,0.18),
702         0 0 0 1px rgba(139,92,246,0.10);
703 }
704
705 /* Compact but polished header */
706 .ob-header {
707     padding: 12px 18px;
708 }
709 .ob-brand {
710     font-size: 17px;
711 }
712 .ob-theme-btn {
713     width: 34px;
714     height: 34px;
715     font-size: 15px;
716 }
717
718 /* Tighter main padding */
719 .ob-main {
720     padding: 24px 14px 56px;
721     align-items: flex-start;
722 }
723
724 /* Slightly smaller title so it fits on one line */
725 .ob-title { font-size: 19px; }
726 .ob-card { padding: 26px 20px; }
727
728 /* Single column business type grid */
729 .ob-type-grid { grid-template-columns: 1fr 1fr; }
730
731 /* Progress bar a tiny bit thicker on mobile ? easier to see */
732 .ob-progress-track { height: 4px; }
733
734 /* Input font comfortable on mobile ? prevents iOS auto-zoom */
735 .ob-input { font-size: 16px; }
736 .ob-textarea { font-size: 16px; }
737 }
738
739 /* ?? Desktop enhancements (?769px) ? mobile completely unchanged ?? */
740 @media (min-width: 769px) {
741
742     /* Rich gradient background so the page never looks empty */
743     .ob-root {
744         background:
745             radial-gradient(ellipse 55% 60% at 8% 15%,    rgba(99,102,241,0.09) 0%, transparent
746             65%),
747             radial-gradient(ellipse 45% 55% at 92% 85%,    rgba(139,92,246,0.07) 0%, transparent
```

```
        65%),
747      radial-gradient(ellipse 35% 40% at 50% 50%, rgba(168,85,247,0.04) 0%, transparent
55%),
748      var(--bg-page);
749    }
750
751    /* Dark mode gets deeper, more dramatic blobs */
752    [data-theme="dark"] .ob-root {
753      background:
754        radial-gradient(ellipse 55% 60% at 8% 15%, rgba(139,92,246,0.18) 0%, transparent 65%),
755        radial-gradient(ellipse 45% 55% at 92% 85%, rgba(99,102,241,0.12) 0%, transparent
65%),
756        radial-gradient(ellipse 30% 35% at 60% 30%, rgba(167,139,250,0.08) 0%, transparent
50%),
757        var(--bg-page);
758    }
759
760    /* Decorative floating orbs ? purely visual, pointer-events off */
761    .ob-main {
762      position: relative;
763      align-items: center;
764      padding-top: 40px;
765      padding-bottom: 60px;
766    }
767    .ob-main::before,
768    .ob-main::after {
769      content: '';
770      position: fixed;
771      border-radius: 50%;
772      pointer-events: none;
773      z-index: 0;
774      animation: ob-float 8s ease-in-out infinite;
775    }
776    .ob-main::before {
777      width: 340px; height: 340px;
778      top: 8%; left: -80px;
779      background: radial-gradient(circle, rgba(99,102,241,0.10) 0%, transparent 70%);
780      animation-delay: 0s;
781    }
782    .ob-main::after {
783      width: 280px; height: 280px;
784      bottom: 10%; right: -60px;
785      background: radial-gradient(circle, rgba(139,92,246,0.09) 0%, transparent 70%);
786      animation-delay: -4s;
787    }
788    [data-theme="dark"] .ob-main::before {
789      background: radial-gradient(circle, rgba(139,92,246,0.20) 0%, transparent 70%);
790    }
791    [data-theme="dark"] .ob-main::after {
792      background: radial-gradient(circle, rgba(99,102,241,0.16) 0%, transparent 70%);
793    }
794    @keyframes ob-float {
795      0%, 100% { transform: translateY(0px) scale(1); }
796      50%      { transform: translateY(-24px) scale(1.04); }
```

```
797     }
798
799     /* Card sits above the orbs */
800     .ob-card {
801         position: relative;
802         z-index: 1;
803         max-width: 560px;
804         box-shadow:
805             var(--shadow-card),
806             0 20px 60px rgba(99,102,241,0.08);
807     }
808     [data-theme="dark"] .ob-card {
809         box-shadow:
810             var(--shadow-card),
811             0 20px 60px rgba(139,92,246,0.15),
812             0 0 0 1px rgba(139,92,246,0.10);
813     }
814
815     /* Bigger, more visible brand name */
816     .ob-brand { font-size: 21px; }
817
818     /* Bigger theme toggle so it's easy to spot */
819     .ob-theme-btn {
820         width: 40px;
821         height: 40px;
822         font-size: 18px;
823         border-radius: 12px;
824     }
825 }
826
```

File 24: frontend/src/index.css

Frontend - index.css

```

1  /* ?? Design tokens ? Light (default) ?????????????????????????? */
2  :root {
3      --bg-page:          #f1f5f9;
4      --bg-card:          #ffffff;
5      --bg-input:         #f8fafc;
6      --bg-input-focus:   #ffffff;
7      --bg-subtle:        #eef2ff;
8      --border:           #e2e8f0;
9      --border-focus:     #6366f1;
10     --border-accent:     #c7d2fe;
11     --text-primary:      #0f172a;
12     --text-secondary:    #64748b;
13     --text-muted:        #94a3b8;
14     --accent:            #6366f1;
15     --accent-2:          #8b5cf6;
16     --accent-light:      #eef2ff;
17     --glow:              rgba(99,102,241,0.12);
18     --glow-strong:       rgba(99,102,241,0.28);
19     --shadow-card:       0 1px 3px rgba(0,0,0,0.06), 0 4px 16px rgba(0,0,0,0.07);
20     --shadow-btn:        0 6px 18px rgba(99,102,241,0.35);
21     --header-bg:         rgba(255,255,255,0.92);
22     --progress-bg:       #e2e8f0;
23     --success-bg:        #d1fae5;
24     --success-color:     #059669;
25     --success-border:    #6ee7b7;
26     --danger:            #ef4444;
27     --danger-bg:         #fee2e2;
28 }
29
30 /* ?? Moonlight Dark theme ?????????????????????????????????? */
31 [data-theme="dark"] {
32     --bg-page:          #0d1117;
33     --bg-card:          #161b2e;
34     --bg-input:         #1e2640;
35     --bg-input-focus:   #232d4a;
36     --bg-subtle:        #1a2035;
37     --border:           #2a3550;
38     --border-focus:     #a78bfa;
39     --border-accent:    #4c3d8f;
40     --text-primary:     #e2e8f0;
41     --text-secondary:   #8892a4;
42     --text-muted:       #5a6580;
43     --accent:           #8b5cf6;
44     --accent-2:         #a78bfa;
45     --accent-light:     #1e1a3a;
46     --glow:             rgba(139,92,246,0.15);
47     --glow-strong:      rgba(139,92,246,0.35);
48     --shadow-card:      0 1px 3px rgba(0,0,0,0.4), 0 4px 24px rgba(0,0,0,0.5);
49     --shadow-btn:       0 6px 24px rgba(139,92,246,0.45);
50     --header-bg:        rgba(13,17,23,0.92);
51     --progress-bg:      #1e2640;

```

```
52     --success-bg:      #0d2a1f;
53     --success-color:   #34d399;
54     --success-border:  #065f46;
55     --danger:          #f87171;
56     --danger-bg:       #2a1515;
57 }
58
59 *, ::before, ::after { box-sizing: border-box; margin: 0; padding: 0; }
60
61 body {
62     min-height: 100vh;
63     font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', 'Inter', Roboto, sans-serif;
64     -webkit-font-smoothing: antialiased;
65     background: var(--bg-page);
66     color: var(--text-primary);
67     transition: background 0.3s ease, color 0.3s ease;
68 }
69
70 #root { min-height: 100vh; }
71
```

File 25: frontend/index.html

Frontend - index.html

```
1    <!doctype html>
2    <html lang="en">
3      <head>
4        <meta charset="UTF-8" />
5        <link rel="icon" type="image/svg+xml" href="/vite.svg" />
6        <meta name="viewport" content="width=device-width, initial-scale=1.0" />
7        <title>Sitekraft</title>
8      </head>
9      <body>
10       <div id="root"></div>
11       <script type="module" src="/src/main.jsx"></script>
12     </body>
13   </html>
14
```

File 26: frontend/vite.config.js

Frontend - vite.config.js

```
1  import { defineConfig } from 'vite'
2  import react from '@vitejs/plugin-react'
3
4  // https://vite.dev/config/
5  export default defineConfig({
6    plugins: [react()],
7    server: {
8      proxy: {
9        '/api': {
10          target: 'http://localhost:8000',
11          changeOrigin: true,
12        }
13      }
14    }
15  })
16
```

File 27: frontend/eslint.config.js

Frontend - eslint.config.js

```
1   import js from '@eslint/js'
2   import globals from 'globals'
3   import reactHooks from 'eslint-plugin-react-hooks'
4   import reactRefresh from 'eslint-plugin-react-refresh'
5   import { defineConfig, globalIgnores } from 'eslint/config'
6
7   export default defineConfig([
8     globalIgnores(['dist']),
9     {
10      files: ['**/*.js', '**/*.jsx'],
11      extends: [
12        js.configs.recommended,
13        reactHooks.configs.flat.recommended,
14        reactRefresh.configs.vite,
15      ],
16      languageOptions: {
17        ecmaVersion: 2020,
18        globals: globals.browser,
19        parserOptions: {
20          ecmaVersion: 'latest',
21          ecmaFeatures: { jsx: true },
22          sourceType: 'module',
23        },
24      },
25      rules: {
26        'no-unused-vars': ['error', { varsIgnorePattern: '^[_A-Z_]' }],
27      },
28    },
29  ])
30
```

File 28: frontend/vercel.json

Frontend - vercel.json

```
1  {
2    "rewrites": [
3      { "source": "/(.*)", "destination": "/index.html" }
4    ]
5  }
6
```

File 29: frontend/package.json

Frontend - package.json

```
1  {
2    "name": "frontend",
3    "private": true,
4    "version": "0.0.0",
5    "type": "module",
6    "scripts": {
7      "dev": "vite",
8      "build": "vite build",
9      "lint": "eslint .",
10     "preview": "vite preview"
11  },
12  "dependencies": {
13    "axios": "^1.13.5",
14    "jszip": "^3.10.1",
15    "react": "^19.2.0",
16    "react-dom": "^19.2.0"
17  },
18  "devDependencies": {
19    "@eslint/js": "^9.39.1",
20    "@types/react": "^19.2.7",
21    "@types/react-dom": "^19.2.3",
22    "@vitejs/plugin-react": "^5.1.1",
23    "eslint": "^9.39.1",
24    "eslint-plugin-react-hooks": "^7.0.1",
25    "eslint-plugin-react-refresh": "^0.4.24",
26    "globals": "^16.5.0",
27    "vite": "^7.3.1"
28  }
29 }
30
```